

MONOPOLY.LK

A Sri Lanka themed Monopoly simulation in C

Line-by-Line Code Explanation

Every function from every source file, explained line by line

Generated from the actual source code, with original line numbers

Table of Contents

types.h.....	5
What is inside types.h.....	5
A. The #define constants.....	5
B. The enums.....	7
C. The structs.....	10
functions.h.....	15
Function prototypes grouped by implementation file.....	15
board.c.....	19
3.1 initializeBoard().....	19
3.2 setProperty().....	22
3.3 setRailway().....	24
3.4 setUtility().....	26
3.5 setSpecialSquare().....	28
players.c.....	30
4.1 initializePlayers().....	30
4.2 shouldBuyProperty().....	32
4.3 shouldConstruct().....	34
4.4 wantsLoan().....	36
4.5 wantsToRepayLoan().....	37
4.6 wantsIncreaseLoan().....	38
4.7 wantsExtendLoan().....	39
4.8 wantsRefinance().....	40
4.9 desiredInsurance().....	41
4.10 shouldRenovateAgeDepreciation().....	42
4.11 shouldMaintain().....	43
4.12 willingToBid().....	44
4.13 shouldMortgage().....	45
4.14 shouldRedeemMortgage().....	46
4.15 shouldPayBail().....	47
game.c.....	49
5.1 startGame().....	49
5.2 playGame().....	50
5.3 playTurn().....	55
5.4 movePlayer().....	59
5.5 rollDice().....	61
5.6 handleJail().....	61
5.7 resolveGroup().....	64
5.8 determineTurnOrder().....	67
5.9 countHouses().....	68
5.10 countHotels().....	69
5.11 displayRoundSummary().....	70
5.12 countSolventPlayers().....	72
5.13 determineWinner().....	72
5.14 displayFinalResults().....	74
finance.c.....	78
6.1 stripOwnership().....	78
6.2 resetLoan().....	79
6.3 adjustOwnedCount().....	80
6.4 groupOf().....	80
6.5 receiveMoney().....	81

6.6	payMoney()	82
6.7	liquidateBankruptAssets()	83
6.8	payTax()	85
6.9	payCommunityFundTax()	86
6.10	findPropertyToMortgage()	88
6.11	findMortgagedProperty()	89
6.12	mortgageProperty()	90
6.13	redeemMortgage()	91
6.14	handleMortgageDecisions()	92
6.15	calculateRent()	93
6.16	calculateRailwayRent()	95
6.17	calculateUtilityRent()	97
6.18	groupSize()	98
6.19	ownsMonopoly()	99
6.20	developGroup()	100
6.21	constructBuildings()	105
6.22	calculatePropertyValue()	106
6.23	calculateBuildingValue()	107
6.24	calculateNetWorth()	108
6.25	countUndevelopedProperties()	109
6.26	wouldCompleteMonopoly()	110
6.27	sellLowValueProperty()	111
6.28	sellDecliningProperties()	113
6.29	sellUndevelopedPropertyToAuction()	116
6.30	forceSellExcess()	118
6.31	enforceAntiSpeculation()	119
6.32	buyProperty()	121
6.33	payRent()	124
bank.c.		128
7.1	groupBasePrice()	128
7.2	totalEligibleCollateral()	129
7.3	calculateMaxLoan()	131
7.4	obtainLoan()	131
7.5	repayLoan()	133
7.6	handleBankVisit()	135
7.7	increaseLoan()	137
7.8	extendLoan()	140
7.9	refinanceLoan()	141
7.10	demolishBuildingsOn()	142
7.11	foreclose()	142
7.12	processLoans()	144
auction.c.		147
8.1	getAskingValue()	147
8.2	runAuction()	148
events.c.		155
9.1	clampLoanInterest()	155
9.2	adjustLoanInterest()	155
9.3	executeEvent()	156
9.4	triggerEconomicEvent()	164
9.5	triggerGovernmentRegulation()	168
9.6	decrementEventTimers()	172
economy.c.		173

10.1	initEconomy()	173
10.2	applyRate()	174
10.3	addSourcedModifier()	175
10.4	addModifier()	176
10.5	modifierMultiplier()	176
10.6	isModifierActive()	178
10.7	currentTaxRatePercent()	179
10.8	decrementModifiers()	180
10.9	formatLKR()	181
10.10	applyInflation()	182
10.11	currentMarketValue()	184
10.12	ageProperties()	186
10.13	tryRenovateAgeDepreciation()	188
10.14	rentConditionPercent()	189
10.15	ageBuildings()	190
10.16	performMaintenance()	192
10.17	renovateStructuralDamage()	195
insurance.c		198
11.1	propertyValue()	198
11.2	repairCost()	198
11.3	isCovered()	199
11.4	compensationPercent()	200
11.5	pickDisaster()	200
11.6	purchaseInsurance()	202
11.7	findPropertyToInsure()	204
11.8	findPropertyToRenew()	205
11.9	handleInsuranceVisit()	206
11.10	tryAutoRepair()	208
11.11	disasterName()	209
11.12	payCompensation()	210
11.13	triggerDisaster()	212
11.14	processInsuranceExpiry()	215
market.c		217
12.1	initMarket()	217
12.2	groupName()	218
12.3	pickEligibleGroup()	219
12.4	reviewPropertyMarket()	220
12.5	drawRegionalCard()	222
12.6	printSectionHeader()	226
12.7	printBoomDecline()	226
12.8	describeRegionalCondition()	227
12.9	printRegionalSection()	229
12.10	describeOtherCondition()	230
12.11	printOtherConditions()	232
12.12	displayMarketConditions()	234
main.c		237
13.1	main()	237

Part 1 types.h

Shared header: every compile-time constant (#define), every enum and every struct used by all modules.

What is inside types.h

Everything here is shared state. The #define values are the rule numbers made concrete, the enums give readable names to numbers, and the structs are the actual variables the game mutates. Read the structs carefully: every function in every .c file works by reading and writing exactly these fields.

A. The #define constants

Every compile-time constant used across the game, with its meaning and the rule it belongs to. These values are what the viva examiner will pick from: know why each number exists.

Line	Code	Explanation
2	#define TYPES_H	Preprocessor directive.
4	#define MAX_PLAYERS 4	Defines macro MAX_PLAYERS = 4 (Number of players in the game.).
5	#define BOARD_SIZE 40	Defines macro BOARD_SIZE = 40 (Number of squares on the board.).
6	#define MAX_NAME_LENGTH 50	Defines macro MAX_NAME_LENGTH = 50 (Buffer size for any name (player, square, group).).
7	#define MAX_ROUNDS 500	Defines macro MAX_ROUNDS = 500 (Hard cap on game rounds; if reached the game ends with a net-worth comparison.).
8	#define START_MONEY 30000	Defines macro START_MONEY = 30000 (Each player starts with LKR 30,000.).
10	#define GO_MONEY 2000	Defines macro GO_MONEY = 2000 (Money collected each time a player passes GO.).
11	#define JAIL_BAIL 300	Defines macro JAIL_BAIL = 300 (Cost of paying bail to leave Jail.).
13	#define INCOME_TAX_RATE 15 /* base rate at game start, drifts with market */	Defines macro INCOME_TAX_RATE = 15 /* base rate at game start, drifts with market */ (Base Income Tax rate (%) at game start; drifts with market.).
14	#define COMMUNITY_FUND_TAX_RATE 10 /* base rate, drifts with market */	Defines macro COMMUNITY_FUND_TAX_RATE = 10 /* base rate, drifts with market */ (Base Community Development Fund rate (%) at game start.).
16	#define MAX_HOUSES 4	Defines macro MAX_HOUSES = 4 (Maximum houses allowed on one property.).
17	#define MAX_HOTELS 1	Defines macro MAX_HOTELS = 1 (Maximum hotels allowed on one property.).
19	#define LOAN_DURATION_ROUNDS 20	Defines macro LOAN_DURATION_ROUNDS = 20 (Initial length of a loan in rounds.).
20	#define LOAN_INTEREST_RATE 8 /* starting rate, drifts with inflation */	Defines macro LOAN_INTEREST_RATE = 8 /* starting rate, drifts with inflation */ (Starting loan interest rate (%), drifts with inflation.).
21	#define LOAN_COLLATERAL_PERCENT 75 /* Rule-LK 2: max loan is 75% of collateral */	Defines macro LOAN_COLLATERAL_PERCENT = 75 /* Rule-LK 2: max loan is 75% of collateral */ (Rule-LK 2: max loan is 75% of eligible collateral.).
22	#define LOAN_EXTEND_ROUNDS 10 /* Rule-LK 5: extension adds 10 rounds */	Defines macro LOAN_EXTEND_ROUNDS = 10 /* Rule-LK 5: extension adds 10 rounds */ (Rule-LK 5: extending a loan adds 10 rounds.).
23	#define MORTGAGE_REPAY_PERCENT 110 /* redeeming a mortgage costs 110% of its value */	Defines macro MORTGAGE_REPAY_PERCENT = 110 /* redeeming a mortgage costs 110% of its value */ (Redeeming a mortgage costs 110% of its value.).
26	#define LOAN_INTEREST_MIN 1 /* interest can never go below this */	Defines macro LOAN_INTEREST_MIN = 1 /* interest can never go below this */ (Loan interest can never go below this.).
27	#define LOAN_INTEREST_MAX 25 /* ...or above this */	Defines macro LOAN_INTEREST_MAX = 25 /* ...or above this */ (Loan interest can never exceed this.).
28	#define LOAN_INTEREST_STEP 2 /* cards/regulations move it by 2 points */	Defines macro LOAN_INTEREST_STEP = 2 /* cards/regulations move it by 2 points */ (Cards/regulations move loan interest by 2 points.).
29	#define TAX_RATE_MAX 25	Defines macro TAX_RATE_MAX = 25 (Cap on the income tax rate after a regulation raises it.).
30	#define HOTEL_LUXURY_TAX_PERCENT 25 /* luxury tax: 25% of a hotel's value */	Defines macro HOTEL_LUXURY_TAX_PERCENT = 25 /* luxury tax: 25% of a hotel's value */ (Luxury tax: 25% of a hotel's value.).

33	#define MIN_CASH_SAFETY 1000 /* reserve before spending / borrowing */	Defines macro MIN_CASH_SAFETY = 1000 /* reserve before spending / borrowing */ (Reserve a player keeps before spending / borrowing.).
34	#define MIN_CASH_FLOOR 500 /* risk taker's floor before mortgaging */	Defines macro MIN_CASH_FLOOR = 500 /* risk taker's floor before mortgaging */ (Risk Taker's floor before mortgaging.).
35	#define TRADER_CASH_FLOOR 1200 /* opportunistic trader's mortgage floor */	Defines macro TRADER_CASH_FLOOR = 1200 /* opportunistic trader's mortgage floor */ (Opportunistic Trader's mortgage floor.).
36	#define COMFORTABLE_CASH 3000 /* conservative banker's comfort line */	Defines macro COMFORTABLE_CASH = 3000 /* conservative banker's comfort line */ (Conservative Banker's comfort line.).
37	#define BAIL_CASH_SPARE 1500 /* conservative banker's bail threshold */	Defines macro BAIL_CASH_SPARE = 1500 /* conservative banker's bail threshold */ (Conservative Banker's bail threshold.).
38	#define RICH_CASH_LINE 5000 /* rich-player threshold */	Defines macro RICH_CASH_LINE = 5000 /* rich-player threshold */ (Rich-player threshold for certain decisions.).
39	#define EXPENSIVE_PROP_PRICE 6000 /* price above which a trader insures */	Defines macro EXPENSIVE_PROP_PRICE = 6000 /* price above which a trader insures */ (Price above which the Trader insures a property.).
42	#define RAILWAY_RENT_1 250	Defines macro RAILWAY_RENT_1 = 250 (Railway rent when owning 1 station (Table 7).).
43	#define RAILWAY_RENT_2 500	Defines macro RAILWAY_RENT_2 = 500 (Railway rent when owning 2 stations.).
44	#define RAILWAY_RENT_3 1000	Defines macro RAILWAY_RENT_3 = 1000 (Railway rent when owning 3 stations.).
45	#define RAILWAY_RENT_4 2000	Defines macro RAILWAY_RENT_4 = 2000 (Railway rent when owning all 4 stations.).
46	#define UTILITY_RENT_ONE 4 /* rent = 4 x dice */	Defines macro UTILITY_RENT_ONE = 4 /* rent = 4 x dice */ (Utility rent = 4 x dice when owning 1 utility (Table 8).).
47	#define UTILITY_RENT_TWO 10 /* rent = 10 x dice when both owned */	Defines macro UTILITY_RENT_TWO = 10 /* rent = 10 x dice when both owned */ (Utility rent = 10 x dice when owning both utilities.).
50	#define AUCTION_OPENING_DIVISOR 2 /* opening bid = asking value / 2 */	Defines macro AUCTION_OPENING_DIVISOR = 2 /* opening bid = asking value / 2 */ (Rule-LK 19: opening bid = asking value / 2.).
51	#define AUCTION_BID_INCREMENT 250 /* every bid must raise the price by this */	Defines macro AUCTION_BID_INCREMENT = 250 /* every bid must raise the price by this */ (Every bid must raise the price by this many LKR.).
52	#define AUCTION_MAX_ROUNDS 200 /* safety limit so bidding always ends */	Defines macro AUCTION_MAX_ROUNDS = 200 /* safety limit so bidding always ends */ (Safety limit so bidding always terminates.).
55	#define MARKET_REVIEW_EVERY 10 /* review happens every 10 rounds */	Defines macro MARKET_REVIEW_EVERY = 10 /* review happens every 10 rounds */ (A property market review happens every 10 rounds.).
56	#define MARKET_COOLDOWN_ROUNDS 30 /* a group can't re-trigger for 30 rounds */	Defines macro MARKET_COOLDOWN_ROUNDS = 30 /* a group can't re-trigger for 30 rounds */ (A group can't re-trigger boom/decline for 30 rounds.).
59	#define PROPERTY_AGE_LIMIT 50 /* value starts dropping past this age */	Defines macro PROPERTY_AGE_LIMIT = 50 /* value starts dropping past this age */ (Value starts dropping once age passes this (rule-LK 15).).
60	#define DEPRECIATION_STEP 5 /* every 5 rounds past the limit */	Defines macro DEPRECIATION_STEP = 5 /* every 5 rounds past the limit */ (Depreciation ticks up every 5 rounds past the age limit.).
61	#define MAX_DEPRECIATION 30	Defines macro MAX_DEPRECIATION = 30 (Depreciation is capped at 30%).).
62	#define RENOVATION_COST_PERCENT 10 /* renovation costs 10% of current value */	Defines macro RENOVATION_COST_PERCENT = 10 /* renovation costs 10% of current value */ (Renovation costs 10% of current value (rule-LK 17).).
63	#define RENOVATION_RENT_BOOST 5 /* renovated base rent rises 5% */	Defines macro RENOVATION_RENT_BOOST = 5 /* renovated base rent rises 5% */ (Renovated base rent rises 5%).).
64	#define NEGLECT_ROUNDS 20 /* no maintenance this long = structural damage */	Defines macro NEGLECT_ROUNDS = 20 /* no maintenance this long = structural damage */ (No maintenance for 20 rounds causes structural damage.).
65	#define STRUCTURAL_VALUE_LOSS 15 /* damaged building loses 15% of its value */	Defines macro STRUCTURAL_VALUE_LOSS = 15 /* damaged building loses 15% of its value */ (Damaged building loses 15% of its value.).
66	#define STRUCTURAL_RENT_LOSS 25 /* ...and 25% of its rent */	Defines macro STRUCTURAL_RENT_LOSS = 25 /* ...and 25% of its rent */ (Damaged building loses 25% of its rent.).
67	#define DAMAGE_REPAIR_MULTIPLIER 150 /* fixing structural damage costs 150% */	Defines macro DAMAGE_REPAIR_MULTIPLIER = 150 /* fixing structural damage costs 150% */ (Fixing structural damage costs 150% of replacement value.).

70	#define REPAIR_COST_PERCENT 30 /* repairs cost 30% of a property's value */	Defines macro REPAIR_COST_PERCENT = 30 /* repairs cost 30% of a property's value */ (Disaster repairs cost 30% of a property's value.).
71	#define PREMIUM_BASIC_PERCENT 5	Defines macro PREMIUM_BASIC_PERCENT = 5 (Basic insurance premium = 5% of value.).
72	#define PREMIUM_COMPREHENSIVE_PERCENT 10	Defines macro PREMIUM_COMPREHENSIVE_PERCENT = 10 (Comprehensive premium = 10% of value.).
73	#define PREMIUM_INTERRUPTON_PERCENT 15	Defines macro PREMIUM_INTERRUPTON_PERCENT = 15 (Business Interruption premium = 15% of value.).
74	#define INSURANCE_DURATION_ROUNDS 20	Defines macro INSURANCE_DURATION_ROUNDS = 20 (A policy lasts 20 rounds.).
75	#define INSURANCE_RENEW_AT 10 /* renew when 10 or fewer rounds remain */	Defines macro INSURANCE_RENEW_AT = 10 /* renew when 10 or fewer rounds remain */ (Renew when 10 or fewer rounds remain.).
76	#define INSURANCE_EXPIRY_WARNING 3 /* warn 3 rounds before a policy expires */	Defines macro INSURANCE_EXPIRY_WARNING = 3 /* warn 3 rounds before a policy expires */ (Warn 3 rounds before a policy expires.).
77	#define DISASTER_WEIGHT 20 /* every disaster is equally likely at base */	Defines macro DISASTER_WEIGHT = 20 /* every disaster is equally likely at base */ (Every disaster is equally likely at base weight 20.).
78	#define DISASTER_BOOST 30 /* weather/unrest makes one type +30 likely */	Defines macro DISASTER_BOOST = 30 /* weather/unrest makes one type +30 likely */ (Weather/unrest makes one disaster type +30 likely.).
79	#define BI_LOST_INCOME_ROUNDS 5 /* Business Interruption: no rent for 5 rounds */	Defines macro BI_LOST_INCOME_ROUNDS = 5 /* Business Interruption: no rent for 5 rounds */ (Business Interruption: no rent for 5 rounds.).
82	#define JAIL_SQUARE 10	Defines macro JAIL_SQUARE = 10 (Board index of the Jail square.).
83	#define JAIL_MAX_TURNS 3	Defines macro JAIL_MAX_TURNS = 3 (After this many failed turns in Jail, bail is forced.).
86	#define EVENT_CARD_COUNT 20	Defines macro EVENT_CARD_COUNT = 20 (Number of National Event Cards (0..19).).
87	#define ECONOMIC_EVENT_COUNT 8	Defines macro ECONOMIC_EVENT_COUNT = 8 (Number of random economic events.).
88	#define REGULATION_COUNT 8	Defines macro REGULATION_COUNT = 8 (Number of government regulations.).
89	#define REGIONAL_CARD_COUNT 12	Defines macro REGIONAL_CARD_COUNT = 12 (Number of Regional Development Cards (Table 4).).
92	#define ECONOMY_CYCLE 10 /* disasters, inflation, market review */	Defines macro ECONOMY_CYCLE = 10 /* disasters, inflation, market review */ (Disasters, inflation and market review every 10 rounds.).
93	#define EVENT_CYCLE 15 /* economic events and regional cards */	Defines macro EVENT_CYCLE = 15 /* economic events and regional cards */ (Economic events + regional cards every 15 rounds.).
94	#define REGULATION_CYCLE 20 /* government regulations */	Defines macro REGULATION_CYCLE = 20 /* government regulations */ (Government regulations every 20 rounds.).
97	#define MAX_UNDEVELOPED_PROPS 3 /* at most this many undeveloped properties */	Defines macro MAX_UNDEVELOPED_PROPS = 3 /* at most this many undeveloped properties */ (Anti-Speculation Act: at most 3 undeveloped properties.).
98	#define ANTI_SPEC_TRIGGER_ROUNDS 5 /* enforced after 5 consecutive rounds */	Defines macro ANTI_SPEC_TRIGGER_ROUNDS = 5 /* enforced after 5 consecutive rounds */ (Forced sale after 5 consecutive rounds over the limit.).
318	#define MAX_MODIFIERS 48	Defines macro MAX_MODIFIERS = 48 (Capacity of the modifiers[] array in Economy.).

B. The enums

enum SquareType

Original code:

```

101     typedef enum
102     {
103         GO,
104         PROPERTY,
105         RAILWAY,
106         UTILITY,
107         TAX,
108         COMMUNITY_FUND,
109         EVENT,

```

```

110     BANK,
111     INSURANCE,
112     JAIL,
113     FREE_PARKING,
114     GO_TO_JAIL

116 } SquareType;

```

Value	Meaning
GO	Starting square; passing it pays GO_MONEY.
PROPERTY	An ownable colour property (developable).
RAILWAY	An ownable railway; rent depends on number owned.
UTILITY	An ownable utility; rent depends on dice roll.
TAX	Income Tax square.
COMMUNITY_FUND	Community Development Fund square.
EVENT	National Event Card square.
BANK	Bank of Ceylon square (loans).
INSURANCE	Insurance square (policies).
JAIL	Jail / Just Visiting.
FREE_PARKING	Do nothing.
GO_TO_JAIL	Sent to Jail.

enum PropertyGroup

Original code:

```

120     typedef enum
121     {
122         BROWN,
123         LIGHT_BLUE,
124         PINK,
125         ORANGE,
126         RED,
127         YELLOW,
128         GREEN,
129         DARK_BLUE,
130         NO_GROUP

132     } PropertyGroup;

```

Value	Meaning
BROWN	Brown properties (Pettah, Maradana).
LIGHT_BLUE	Light Blue (Bambalapitiya, Wellawatte, Mount Lavinia).
PINK	Pink (Nugegoda, Maharagama, Kottawa).
ORANGE	Orange (Negombo, Katunayake, Ja-Ela).
RED	Red (Kandy City, Peradeniya, Katugastota).
YELLOW	Yellow (Galle Fort, Unawatuna, Hikkaduwa).
GREEN	Green (Jaffna Town, Nallur, Trincomalee).
DARK_BLUE	Dark Blue (Nuwara Eliya, Galle Face).
NO_GROUP	Sentinel = number of real groups; also used as an array bound.

enum PlayerType

Original code:

```

136     typedef enum
137     {
138         AGGRESSIVE_INVESTOR,
139         CONSERVATIVE_BANKER,
140         RISK_TAKER,
141         OPPORTUNISTIC_TRADER

143     } PlayerType;

```

Value	Meaning
AGGRESSIVE_INVESTOR	Player 1. Buys to complete monopolies.
CONSERVATIVE_BANKER	Player 2. Avoids risk and debt.

RISK_TAKER	Player 3. Spends everything; insures only after a loss.
OPPORTUNISTIC_TRADER	Player 4. Buys cheap, sells into declines.

enum InsuranceType

Original code:

```

147     typedef enum
148     {
149         NO_INSURANCE,
150         BASIC_INSURANCE,
151         COMPREHENSIVE_INSURANCE,
152         BUSINESS_INTERRUPTION
153     };
154 }
```

Value	Meaning
NO_INSURANCE	No policy.
BASIC_INSURANCE	Covers Fire and Flood; pays 80%.
COMPREHENSIVE_INSURANCE	Covers all disasters; pays 100%.
BUSINESS_INTERRUPTION	Covers all; 100% + 5 rounds of lost income.

enum DisasterType

Original code:

```

158     typedef enum
159     {
160         FIRE,
161         FLOOD,
162         RIOT,
163         BUILDING_COLLAPSE,
164         ELECTRICAL_FAILURE
165     };
166 }
```

Value	Meaning
FIRE	Fire damage.
FLOOD	Flood damage.
RIOT	Riot damage.
BUILDING_COLLAPSE	Structural collapse.
ELECTRICAL_FAILURE	Electrical failure.

enum ModifierType

Original code:

```

269     typedef enum
270     {
271         MOD_VALUE_GLOBAL,           /* all property values */
272         MOD_GROUP_VALUE,           /* one colour group's property values */
273         MOD_INDEX_VALUE,           /* a single square's value (any type) */
274         MOD_RAIL_VALUE,            /* all railway station values */
275         MOD_PURCHASE_PRICE,        /* direct purchase prices (boom +15%) */
276         MOD_RENT_GLOBAL,           /* all property/rail/utility rent */
277         MOD_GROUP_RENT,            /* one colour group's rent */
278         MOD_HOTEL_RENT,            /* hotel rent */
279         MOD_RAIL_RENT,             /* railway rent */
280         MOD_UTIL_RENT,             /* utility rent */
281         MOD_CONSTRUCTION,          /* house/hotel construction costs */
282         MOD_INSURANCE,             /* insurance premiums */
283         MOD_MARKET_MORTGAGE,        /* mortgage values (boom +15% / decline -10%) */
284         MOD_AUCTION_PRICE,         /* auction starting prices (decline -25%) */
285         MOD_FLOOD_RISK,            /* Heavy Monsoon - flood more likely */
286         MOD_RIOT_RISK,             /* Political Unrest - riot more likely */
287         MOD_BI_CLAIMS,             /* Political Unrest - business interruption claims */
288         MOD RECESSION              /* Economic Recession - strategies may react */
289     };
290 }
```

Value	Meaning
-------	---------

MOD_VALUE_GLOBAL	All property values.
MOD_GROUP_VALUE	One colour group's property values.
MOD_INDEX_VALUE	A single square's value (any type).
MOD_RAIL_VALUE	All railway values.
MOD_PURCHASE_PRICE	Direct purchase prices.
MOD_RENT_GLOBAL	All rent.
MOD_GROUP_RENT	One colour group's rent.
MOD_HOTEL_RENT	Hotel rent.
MOD_RAIL_RENT	Railway rent.
MOD_UTIL_RENT	Utility rent.
MOD_CONSTRUCTION	House/hotel construction costs.
MOD_INSURANCE	Insurance premiums.
MOD_MARKET_MORTGAGE	Mortgage values.
MOD_AUCTION_PRICE	Auction starting prices.
MOD_FLOOD_RISK	Flood more likely (Heavy Monsoon).
MOD_RIOT_RISK	Riot more likely (Political Unrest).
MOD_BI_CLAIMS	BI claims inflated (Political Unrest).
MOD RECESSION	Strategy flag for Economic Recession.

enum ModifierSource

Original code:

```

296     typedef enum
297     {
298         SRC_GENERAL,    /* events, national cards, government regulations */
299         SRC_BOOM,       /* Property Market Review - boom group */
300         SRC_DECLINE,    /* Property Market Review - decline group */
301         SRC_REGIONAL    /* Regional Development Card */
302     } ModifierSource;

```

Value	Meaning
SRC_GENERAL	Events, national cards, government regulations.
SRC_BOOM	Property Market Review - booming group.
SRC_DECLINE	Property Market Review - declining group.
SRC_REGIONAL	Regional Development Card.

C. The structs

struct Loan

Original code:

```

169     typedef struct
170     {
171         int active;
172         int amount;
173         int interestRate;
174         int remainingRounds;
175     } Loan;

```

Field	Type	Purpose
active	int	1 when the player has an outstanding loan (only one loan at a time).
amount	int	Current outstanding balance in LKR (grows with compound interest).
interestRate	int	Annual/round interest rate the loan is charged at.
remainingRounds	int	Rounds left before foreclosure.

struct Property

Original code:

```

179     typedef struct
180     {
181         char name[MAX_NAME_LENGTH];
182     } PropertyGroup;

```

```

185     int purchasePrice;
186     int mortgageValue;
187     int baseRent;

189     int houseCost;
190     int hotelCost;

192     int owner;          /* -1 means unowned */

194     int houses;
195     int hotel;

197     int mortgaged;
198     int loanLocked;     /* pledged as loan collateral */

200     InsuranceType insurance;
201     int insuranceRoundsLeft;

203     int damaged;        /* disaster damage - no rent until repaired */
204     int repairCostOwed;
205     int lostIncomeRoundsLeft; /* Business Interruption: no rent for 5 rounds */

207     int age;
208     int depreciation;

210     int condition;      /* building condition, Rule-LK 25-27 */
211     int roundsSinceMaintenance;
212     int structurallyDamaged;
213     int preDamagePurchasePrice; /* remembered so renovation can restore it */
214     int preDamageBaseRent;
215     int maintenanceCostMultiplierPercent;

217 } Property;

```

Field	Type	Purpose
name	char[]	Property display name (echoed into the Square name too).
group	PropertyGroup	Colour group (BROWN..DARK_BLUE).
purchasePrice	int	Base price to buy (inflation and modifiers can move this).
mortgageValue	int	Value received when mortgaging (redeem costs +10%).
baseRent	int	Rent with zero houses (Table multipliers on top).
houseCost	int	Cost of building one house.
hotelCost	int	Cost of building a hotel.
owner	int	Player index who owns it; -1 means unowned.
houses	int	Number of houses (0..4).
hotel	int	1 if a hotel is built (replaces the 4 houses).
mortgaged	int	1 if the property is mortgaged (no rent until redeemed).
loanLocked	int	1 if the property is pledged as loan collateral (cannot be sold).
insurance	InsuranceType	Currently held policy type.
insuranceRoundsLeft	int	Rounds remaining on the current policy.
damaged	int	1 after a disaster until repaired (no rent).
repairCostOwed	int	Remaining repair cost after insurance compensation.
lostIncomeRoundsLeft	int	Business Interruption: rounds with no rent.
age	int	Rounds since purchase/last renovation.
depreciation	int	Percent value loss from age (0..30%).
condition	int	Building condition 0..100 (Table 3 rent mapping).
roundsSinceMaintenance	int	Rounds since last maintenance.
structurallyDamaged	int	1 when neglect (20+ rounds) caused structural damage.
preDamagePurchasePrice	int	Purchase price before structural damage (for restore).
preDamageBaseRent	int	Base rent before structural damage (for restore).
maintenanceCostMultiplierPercent	int	Cost multiplier (150% after damage, else 100%).

struct Square

Original code:

```
220     typedef struct
221     {
222         int index;

224         SquareType type;

226         char name[MAX_NAME_LENGTH];

228         Property property;

230     } Square;
```

Field	Type	Purpose
index	int	Board position 0..39.
type	SquareType	What this square is (GO, PROPERTY, ...).
name	char []	Square display name.
property	Property	The embedded property data used by all square kinds.

struct Player

Original code:

```
233     typedef struct
234     {
235         char name[MAX_NAME_LENGTH];

237         PlayerType strategy;

239         int position;

241         int cash;

243         int inJail;

245         int jailTurns;

247         int bankrupt;

249         int propertiesOwned;

251         int railwaysOwned;

253         int utilitiesOwned;

255         Loan loan;

257         int sufferedLoss; /* Risk Taker only insures after a loss */

259         int antiSpecRounds; /* rounds in a row above the 3-undeveloped limit */

261     } Player;
```

Field	Type	Purpose
name	char []	Strategy name (Aggressive Investor, etc.).
strategy	PlayerType	Which strategy drives every decision.
position	int	Current board position 0..39.
cash	int	Cash balance in LKR.
inJail	int	1 while in Jail.
jailTurns	int	Consecutive jail turns while failing to roll doubles.
bankrupt	int	1 once the player is out of the game.
propertiesOwned	int	Count of colour properties owned.
railwaysOwned	int	Count of railways owned.
utilitiesOwned	int	Count of utilities owned.
loan	Loan	The player's (single) loan.

sufferedLoss	int	1 after an uninsured disaster (Risk Taker now insures).
antiSpecRounds	int	Consecutive rounds over the 3-undeveloped limit.

struct ActiveModifier

Original code:

```

306     typedef struct
307     {
308         ModifierType type;
309         int group;      /* group for GROUP modifiers, else -1 */
310         int index;      /* square index for INDEX modifiers, else -1 */
311         int percent;    /* 100 means no change */
312         int roundsLeft;
313         ModifierSource source;
314
315     } ActiveModifier;

```

Field	Type	Purpose
type	ModifierType	What the modifier affects (value, rent, ...).
group	int	Colour group if GROUP-scoped, else -1.
index	int	Square index if INDEX-scoped, else -1.
percent	int	Multiplier percent (100 = no change, 120 = +20%).
roundsLeft	int	Rounds until the modifier expires.
source	ModifierSource	Origin used by Rule-LK 36 reporting (boom/decline/regional/general).

struct Economy

Original code:

```

323     typedef struct
324     {
325         int inflationRate;
326         int loanInterestRate;
327
328         int constructionSuspendedRoundsLeft;
329
330         int closedPropertyIndex;
331         int closedPropertyRoundsLeft;
332
333         int incomeTaxRate;
334
335         int antiSpeculationActive;
336
337         int currentCardIndex;  /* position in the National Event Card deck */
338
339         int groupCooldownUntilRound[NO_GROUP];
340
341         int lastBoomGroup;
342         int lastDeclineGroup;
343
344         ActiveModifier modifiers[MAX_MODIFIERS];
345         int modifierCount;
346
347     } Economy;

```

Field	Type	Purpose
inflationRate	int	Current inflation percent (-3..12) applied every 10 rounds.
loanInterestRate	int	Current economy loan interest rate (1..25).
constructionSuspendedRoundsLeft	int	Labour Strike blocks building while > 0.
closedPropertyIndex	int	Square closed by Political Rally (-1 none).
closedPropertyRoundsLeft	int	Rounds the closed property stays closed.
incomeTaxRate	int	Current income tax rate (base 15, can rise via regulation).
antiSpeculationActive	int	1 once the Anti-Speculation regulation fires.
currentCardIndex	int	Position in the National Event Card deck (0..19).
groupCooldownUntilRound	int[]	Per group: earliest round it may boom/decline again.

lastBoomGroup	int	Group boosted by the most recent market review.
lastDeclineGroup	int	Group declined by the most recent market review.
modifiers	ActiveModifie r[]	Array of up to 48 active timed modifiers.
modifierCount	int	How many modifiers are currently active.

struct GameState

Original code:

```

353     typedef struct
354     {
355         Square board[BOARD_SIZE];
356         Player players[MAX_PLAYERS];
357         Economy economy;
358
359     } GameState;

```

Field	Type	Purpose
board	Square[]	The 40 squares of the board.
players	Player[]	The 4 players.
economy	Economy	Shared economy state (inflation, modifiers, decks).

Part 2 functions.h

Project API header: one prototype for every function, grouped by the .c file that implements it.

Function prototypes grouped by implementation file

functions.h declares the API. Each block below is one source file. Read the grouping to see the module boundaries of the project - this is exactly the structure the sections of this document follow.

Line	Code	Explanation
8	<code>void initializeBoard(GameState game[]);</code>	Declared in functions.h; implemented in board.c.c - see that section.
10	<code>void setProperty(GameState game[],</code>	Declared in functions.h; implemented in board.c.c - see that section.
11	<code>int index,</code>	Declared in functions.h; implemented in board.c.c - see that section.
12	<code>char name[],</code>	Declared in functions.h; implemented in board.c.c - see that section.
13	<code>PropertyGroup group,</code>	Declared in functions.h; implemented in board.c.c - see that section.
14	<code>int purchasePrice,</code>	Declared in functions.h; implemented in board.c.c - see that section.
15	<code>int baseRent,</code>	Declared in functions.h; implemented in board.c.c - see that section.
16	<code>int mortgageValue,</code>	Declared in functions.h; implemented in board.c.c - see that section.
17	<code>int houseCost,</code>	Declared in functions.h; implemented in board.c.c - see that section.
18	<code>int hotelCost);</code>	Declared in functions.h; implemented in board.c.c - see that section.
20	<code>void setRailway(GameState game[], int index, char</code> <code>name[], int purchasePrice);</code>	Declared in functions.h; implemented in board.c.c - see that section.
21	<code>void setUtility(GameState game[], int index, char</code> <code>name[], int purchasePrice);</code>	Declared in functions.h; implemented in board.c.c - see that section.
23	<code>void setSpecialSquare(GameState game[],</code>	Declared in functions.h; implemented in board.c.c - see that section.
24	<code>int index,</code>	Declared in functions.h; implemented in board.c.c - see that section.
25	<code>SquareType type,</code>	Declared in functions.h; implemented in board.c.c - see that section.
26	<code>char name[]);</code>	Declared in functions.h; implemented in board.c.c - see that section.
30	<code>void initializePlayers(GameState game[]);</code>	Declared in functions.h; implemented in players.c.c - see that section.
32	<code>int shouldBuyProperty(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
33	<code>int shouldConstruct(GameState game[], int</code> <code>playerIndex, int cost, int isHotel);</code>	Declared in functions.h; implemented in players.c.c - see that section.
34	<code>int wantsLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
35	<code>int wantsToRepayLoan(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
36	<code>int wantsIncreaseLoan(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
37	<code>int wantsExtendLoan(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
38	<code>int wantsRefinance(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
39	<code>InsuranceType desiredInsurance(GameState game[], int</code> <code>playerIndex, int propIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
40	<code>int shouldRenovateAgeDepreciation(GameState game[],</code> <code>int playerIndex, int depreciationPercent);</code>	Declared in functions.h; implemented in players.c.c - see that section.
41	<code>int shouldMaintain(GameState game[], int</code> <code>playerIndex, int condition, int cost);</code>	Declared in functions.h; implemented in players.c.c - see that section.
42	<code>int willingToBid(GameState game[], int playerIndex,</code> <code>int propIndex, int candidateBid);</code>	Declared in functions.h; implemented in players.c.c - see that section.
43	<code>int shouldMortgage(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
44	<code>int shouldRedeemMortgage(GameState game[], int</code> <code>playerIndex, int redeemCost);</code>	Declared in functions.h; implemented in players.c.c - see that section.
45	<code>int shouldPayBail(GameState game[], int</code> <code>playerIndex);</code>	Declared in functions.h; implemented in players.c.c - see that section.
49	<code>void stripOwnership(GameState game[], int index);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
50	<code>void resetLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.

51	<code>void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
52	<code>PropertyGroup groupOf(GameState game[], int index);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
54	<code>void buyProperty(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
55	<code>void payRent(GameState game[], int playerIndex, int diceValue);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
56	<code>void payTax(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
57	<code>void payCommunityFundTax(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
58	<code>void payMoney(GameState game[], int playerIndex, int amount);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
59	<code>void liquidateBankruptAssets(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
60	<code>void receiveMoney(GameState game[], int playerIndex, int amount);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
62	<code>int calculatePropertyValue(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
63	<code>int calculateBuildingValue(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
64	<code>int calculateNetWorth(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
66	<code>int findPropertyToMortgage(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
67	<code>int findMortgagedProperty(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
68	<code>void mortgageProperty(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
69	<code>void redeemMortgage(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
70	<code>void handleMortgageDecisions(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
72	<code>int calculateRent(GameState game[], int position);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
73	<code>int calculateRailwayRent(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
74	<code>int calculateUtilityRent(GameState game[], int playerIndex, int diceValue);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
75	<code>int countUndevelopedProperties(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
77	<code>int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
78	<code>void sellLowValueProperty(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
79	<code>void sellDecliningProperties(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
80	<code>void enforceAntiSpeculation(GameState game[]);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
81	<code>void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
83	<code>int groupSize(GameState game[], PropertyGroup group);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
84	<code>int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
85	<code>int developGroup(GameState game[], int playerIndex, PropertyGroup group);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
86	<code>void constructBuildings(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in finance.c.c - see that section.
90	<code>void startGame(GameState game[]);</code>	Declared in functions.h; implemented in game.c.c - see that section.
91	<code>void playGame(GameState game[], int turnOrder[]);</code>	Declared in functions.h; implemented in game.c.c - see that section.
92	<code>int playTurn(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in game.c.c - see that section.

93	<code>int movePlayer(GameState game[], int playerIndex, int dice);</code>	Declared in functions.h; implemented in game.c.c - see that section.
94	<code>int rollDice(void);</code>	Declared in functions.h; implemented in game.c.c - see that section.
95	<code>int handleJail(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in game.c.c - see that section.
97	<code>int resolveGroup(GameState game[], int groupPlayers[], int groupSize, int output[], int startIndex);</code>	Declared in functions.h; implemented in game.c.c - see that section.
98	<code>int output[], int startIndex);</code>	Declared in functions.h; implemented in game.c.c - see that section.
100	<code>void determineTurnOrder(GameState game[], int turnOrder[]);</code>	Declared in functions.h; implemented in game.c.c - see that section.
101	<code>void displayRoundSummary(GameState game[], int round);</code>	Declared in functions.h; implemented in game.c.c - see that section.
102	<code>int countSolventPlayers(GameState game[]);</code>	Declared in functions.h; implemented in game.c.c - see that section.
103	<code>int countHouses(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in game.c.c - see that section.
104	<code>int countHotels(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in game.c.c - see that section.
105	<code>int determineWinner(GameState game[]);</code>	Declared in functions.h; implemented in game.c.c - see that section.
106	<code>void displayFinalResults(GameState game[]);</code>	Declared in functions.h; implemented in game.c.c - see that section.
110	<code>void clampLoanInterest(GameState game[]);</code>	Declared in functions.h; implemented in events.c.c - see that section.
111	<code>void adjustLoanInterest(GameState game[], int deltaPercent);</code>	Declared in functions.h; implemented in events.c.c - see that section.
112	<code>void executeEvent(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in events.c.c - see that section.
113	<code>void triggerEconomicEvent(GameState game[]);</code>	Declared in functions.h; implemented in events.c.c - see that section.
114	<code>void triggerGovernmentRegulation(GameState game[]);</code>	Declared in functions.h; implemented in events.c.c - see that section.
115	<code>void decrementEventTimers(GameState game[]);</code>	Declared in functions.h; implemented in events.c.c - see that section.
119	<code>void initMarket(GameState game[]);</code>	Declared in functions.h; implemented in market.c.c - see that section.
120	<code>void groupName(PropertyGroup group, char buffer[]);</code>	Declared in functions.h; implemented in market.c.c - see that section.
121	<code>PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2);</code>	Declared in functions.h; implemented in market.c.c - see that section.
122	<code>void reviewPropertyMarket(GameState game[], int currentRound);</code>	Declared in functions.h; implemented in market.c.c - see that section.
123	<code>void drawRegionalCard(GameState game[]);</code>	Declared in functions.h; implemented in market.c.c - see that section.
124	<code>void displayMarketConditions(GameState game[]);</code>	Declared in functions.h; implemented in market.c.c - see that section.
128	<code>int groupBasePrice(GameState game[], PropertyGroup group);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
129	<code>int totalEligibleCollateral(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
130	<code>int calculateMaxLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
131	<code>void obtainLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
132	<code>void repayLoan(GameState game[], int playerIndex, int amount);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
133	<code>void handleBankVisit(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
134	<code>void increaseLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
135	<code>void extendLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
136	<code>void refinanceLoan(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
137	<code>void demolishBuildingsOn(GameState game[], int index);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
138	<code>void foreclose(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
139	<code>void processLoans(GameState game[]);</code>	Declared in functions.h; implemented in bank.c.c - see that section.
143	<code>int propertyValue(GameState game[], int propIndex);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
144	<code>int repairCost(GameState game[], int propIndex);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
145	<code>DisasterType pickDisaster(GameState game[]);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
146	<code>int isCovered(InsuranceType policy, DisasterType disaster);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
147	<code>int compensationPercent(InsuranceType policy);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.

148	<code>void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
149	<code>int findPropertyToInsure(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
150	<code>int findPropertyToRenew(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
151	<code>void handleInsuranceVisit(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
152	<code>void tryAutoRepair(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
153	<code>void triggerDisaster(GameState game[]);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
154	<code>void processInsuranceExpiry(GameState game[]);</code>	Declared in functions.h; implemented in insurance.c.c - see that section.
158	<code>int applyRate(int oldValue, int ratePercent);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
159	<code>void initEconomy(GameState game[]);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
160	<code>void applyInflation(GameState game[]);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
161	<code>void addModifier(GameState game[], ModifierType type, int group, int index, int percent, int roundsLeft);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
162		Declared in functions.h; implemented in economy.c.c - see that section.
163	<code>void addSourcedModifier(GameState game[], ModifierType type, int group, int index, int percent, int roundsLeft, ModifierSource source);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
164		Declared in functions.h; implemented in economy.c.c - see that section.
165		Declared in functions.h; implemented in economy.c.c - see that section.
166	<code>int modifierMultiplier(GameState game[], ModifierType type, int group, int index);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
167	<code>int isModifierActive(GameState game[], ModifierType type, int group, int index);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
168	<code>void decrementModifiers(GameState game[]);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
169	<code>void formatLKR(int amount, char out[]);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
170	<code>int currentMarketValue(GameState game[], int propIndex);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
171	<code>int currentTaxRatePercent(GameState game[], int baseRate);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
172	<code>void ageProperties(GameState game[]);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
173	<code>void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
174	<code>int rentConditionPercent(int condition);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
175	<code>void ageBuildings(GameState game[]);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
176	<code>void performMaintenance(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
177	<code>void renovateStructuralDamage(GameState game[], int playerIndex);</code>	Declared in functions.h; implemented in economy.c.c - see that section.
181	<code>int getAskingValue(GameState game[], int propIndex);</code>	Declared in functions.h; implemented in auction.c.c - see that section.
182	<code>void runAuction(GameState game[], int propIndex, int sellerIndex);</code>	Declared in functions.h; implemented in auction.c.c - see that section.

Builds the 40-square Sri Lanka board: special squares, railways, utilities and the 22 colour-group properties.

The sub-topics below are the 5 functions implemented in board.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in board.c

3.1	initializeBoard()	(source line 145)	p. 19
3.2	setProperty()	(source line 6)	p. 22
3.3	setRailway()	(source line 55)	p. 24
3.4	setUtility()	(source line 91)	p. 26
3.5	setSpecialSquare()	(source line 125)	p. 28

3.1 initializeBoard()

```
void initializeBoard(GameState game[])
```

Parameters: none | Returns: void - the board is written into game[0].board.

The function, verbatim from the source (starts at line 145)

```
145 void initializeBoard(GameState game[])
146 {
147     setSpecialSquare(game, 0, GO, "GO");
148     setSpecialSquare(game, 2, COMMUNITY_FUND, "Community Development Fund");
149     setSpecialSquare(game, 4, TAX, "Income Tax");
150     setSpecialSquare(game, 7, EVENT, "National Event Card");
151     setSpecialSquare(game, 10, JAIL, "Jail / Just Visiting");
152     setSpecialSquare(game, 17, INSURANCE, "Sri Lanka Insurance");
153     setSpecialSquare(game, 20, FREE_PARKING, "Free Parking");
154     setSpecialSquare(game, 22, EVENT, "National Event Card");
155     setSpecialSquare(game, 30, GO_TO_JAIL, "Go To Jail");
156     setSpecialSquare(game, 33, INSURANCE, "Ceylinco Insurance");
157     setSpecialSquare(game, 36, EVENT, "National Event Card");
158     setSpecialSquare(game, 38, BANK, "Bank of Ceylon");

160     setRailway(game, 5, "Colombo Fort Railway Station", 1500);
161     setRailway(game, 15, "Kandy Railway Station", 1500);
162     setRailway(game, 25, "Galle Railway Station", 1500);
163     setRailway(game, 35, "Jaffna Railway Station", 1500);

165     setUtility(game, 12, "Ceylon Electricity Board", 1500);
166     setUtility(game, 28, "National Water Supply and Drainage Board", 1500);

168     /* Brown */
169     setProperty(game, 1, "Pettah", BROWN, 1500, 100, 750, 500, 2000);
170     setProperty(game, 3, "Maradana", BROWN, 1800, 120, 900, 500, 2000);

172     /* Light Blue */
173     setProperty(game, 6, "Bambalapitiya", LIGHT_BLUE, 2500, 180, 1250, 750, 3000);
174     setProperty(game, 8, "Wellawatte", LIGHT_BLUE, 2700, 200, 1350, 750, 3000);
175     setProperty(game, 9, "Mount Lavinia", LIGHT_BLUE, 3000, 220, 1500, 750, 3000);

177     /* Pink */
178     setProperty(game, 11, "Nugegoda", PINK, 3500, 260, 1750, 1000, 4000);
179     setProperty(game, 13, "Maharagama", PINK, 3800, 280, 1900, 1000, 4000);
180     setProperty(game, 14, "Kottawa", PINK, 4000, 300, 2000, 1000, 4000);

182     /* Orange */
183     setProperty(game, 16, "Negombo", ORANGE, 4500, 350, 2250, 1250, 5000);
184     setProperty(game, 18, "Katunayake", ORANGE, 4700, 370, 2350, 1250, 5000);
185     setProperty(game, 19, "Ja-Ela", ORANGE, 5000, 400, 2500, 1250, 5000);

187     /* Red */
188     setProperty(game, 21, "Kandy City", RED, 5500, 450, 2750, 1500, 6000);
189     setProperty(game, 23, "Peradeniya", RED, 5800, 480, 2900, 1500, 6000);
190     setProperty(game, 24, "Katugastota", RED, 6000, 500, 3000, 1500, 6000);
```

```

192      /* Yellow */
193      setProperty(game, 26, "Galle Fort", YELLOW, 6500, 600, 3250, 2000, 8000);
194      setProperty(game, 27, "Unawatuna", YELLOW, 6800, 620, 3400, 2000, 8000);
195      setProperty(game, 29, "Hikkaduwa", YELLOW, 7000, 650, 3500, 2000, 8000);

197      /* Green */
198      setProperty(game, 31, "Jaffna Town", GREEN, 8000, 750, 4000, 2500, 10000);
199      setProperty(game, 32, "Nallur", GREEN, 8300, 780, 4150, 2500, 10000);
200      setProperty(game, 34, "Trincomalee", GREEN, 8500, 800, 4250, 2500, 10000);

202      /* Dark Blue */
203      setProperty(game, 37, "Nuwara Eliya", DARK_BLUE, 10000, 1000, 5000, 3000, 12000);
204      setProperty(game, 39, "Galle Face", DARK_BLUE, 12000, 1200, 6000, 3000, 12000);
205  }

```

Line-by-line explanation

Line	Code	Explanation
145	<code>void initializeBoard(GameState game[])</code>	<code>void initializeBoard(GameState game[])</code>
146	<code>{</code>	Opens a new block.
147	<code>setSpecialSquare(game, 0, GO, "GO");</code>	Calls <code>setSpecialSquare(game, 0, GO, "GO")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
148	<code>setSpecialSquare(game, 2, COMMUNITY_FUND, "Community Development Fund");</code>	Calls <code>setSpecialSquare(game, 2, COMMUNITY_FUND, "Community Development Fund")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
149	<code>setSpecialSquare(game, 4, TAX, "Income Tax");</code>	Calls <code>setSpecialSquare(game, 4, TAX, "Income Tax")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
150	<code>setSpecialSquare(game, 7, EVENT, "National Event Card");</code>	Calls <code>setSpecialSquare(game, 7, EVENT, "National Event Card")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
151	<code>setSpecialSquare(game, 10, JAIL, "Jail / Just Visiting");</code>	Calls <code>setSpecialSquare(game, 10, JAIL, "Jail / Just Visiting")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
152	<code>setSpecialSquare(game, 17, INSURANCE, "Sri Lanka Insurance");</code>	Calls <code>setSpecialSquare(game, 17, INSURANCE, "Sri Lanka Insurance")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
153	<code>setSpecialSquare(game, 20, FREE_PARKING, "Free Parking");</code>	Calls <code>setSpecialSquare(game, 20, FREE_PARKING, "Free Parking")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
154	<code>setSpecialSquare(game, 22, EVENT, "National Event Card");</code>	Calls <code>setSpecialSquare(game, 22, EVENT, "National Event Card")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
155	<code>setSpecialSquare(game, 30, GO_TO_JAIL, "Go To Jail");</code>	Calls <code>setSpecialSquare(game, 30, GO_TO_JAIL, "Go To Jail")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
156	<code>setSpecialSquare(game, 33, INSURANCE, "Ceylinco Insurance");</code>	Calls <code>setSpecialSquare(game, 33, INSURANCE, "Ceylinco Insurance")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
157	<code>setSpecialSquare(game, 36, EVENT, "National Event Card");</code>	Calls <code>setSpecialSquare(game, 36, EVENT, "National Event Card")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
158	<code>setSpecialSquare(game, 38, BANK, "Bank of Ceylon");</code>	Calls <code>setSpecialSquare(game, 38, BANK, "Bank of Ceylon")</code> . (Creates a non-ownable square (GO, Tax, Jail,)
159		Blank line.
160	<code>setRailway(game, 5, "Colombo Fort Railway Station", 1500);</code>	Calls <code>setRailway(game, 5, "Colombo Fort Railway Station", 1500)</code> . (Creates a railway square)
161	<code>setRailway(game, 15, "Kandy Railway Station", 1500);</code>	Calls <code>setRailway(game, 15, "Kandy Railway Station", 1500)</code> . (Creates a railway square)
162	<code>setRailway(game, 25, "Galle Railway Station", 1500);</code>	Calls <code>setRailway(game, 25, "Galle Railway Station", 1500)</code> . (Creates a railway square)
163	<code>setRailway(game, 35, "Jaffna Railway Station", 1500);</code>	Calls <code>setRailway(game, 35, "Jaffna Railway Station", 1500)</code> . (Creates a railway square)
164		Blank line.
165	<code>setUtility(game, 12, "Ceylon Electricity Board", 1500);</code>	Calls <code>setUtility(game, 12, "Ceylon Electricity Board", 1500)</code> . (Creates a utility square (mortgage = half price), same zeroing discipline as railways)
166	<code>setUtility(game, 28, "National Water Supply and Drainage Board", 1500);</code>	Calls <code>setUtility(game, 28, "National Water Supply and Drainage Board", 1500)</code> . (Creates a utility square (mortgage = half price), same zeroing discipline as railways)
167		Blank line.

168	<code>/* Brown */</code>	Comment: <code>/* Brown */</code>
169	<code>setProperty(game, 1, "Pettah", BROWN, 1500, 100, 750, 500, 2000);</code>	Calls <code>setProperty(game, 1, "Pettah", BROWN, 1500, 100, 750, 500, 2000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
170	<code>setProperty(game, 3, "Maradana", BROWN, 1800, 120, 900, 500, 2000);</code>	Calls <code>setProperty(game, 3, "Maradana", BROWN, 1800, 120, 900, 500, 2000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
171		Blank line.
172	<code>/* Light Blue */</code>	Comment: <code>/* Light Blue */</code>
173	<code>setProperty(game, 6, "Bambalapitiya", LIGHT_BLUE, 2500, 180, 1250, 750, 3000);</code>	Calls <code>setProperty(game, 6, "Bambalapitiya", LIGHT_BLUE, 2500, 180, 1250, 750, 3000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
174	<code>setProperty(game, 8, "Wellawatte", LIGHT_BLUE, 2700, 200, 1350, 750, 3000);</code>	Calls <code>setProperty(game, 8, "Wellawatte", LIGHT_BLUE, 2700, 200, 1350, 750, 3000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
175	<code>setProperty(game, 9, "Mount Lavinia", LIGHT_BLUE, 3000, 220, 1500, 750, 3000);</code>	Calls <code>setProperty(game, 9, "Mount Lavinia", LIGHT_BLUE, 3000, 220, 1500, 750, 3000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
176		Blank line.
177	<code>/* Pink */</code>	Comment: <code>/* Pink */</code>
178	<code>setProperty(game, 11, "Nugegoda", PINK, 3500, 260, 1750, 1000, 4000);</code>	Calls <code>setProperty(game, 11, "Nugegoda", PINK, 3500, 260, 1750, 1000, 4000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
179	<code>setProperty(game, 13, "Maharagama", PINK, 3800, 280, 1900, 1000, 4000);</code>	Calls <code>setProperty(game, 13, "Maharagama", PINK, 3800, 280, 1900, 1000, 4000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
180	<code>setProperty(game, 14, "Kottawa", PINK, 4000, 300, 2000, 1000, 4000);</code>	Calls <code>setProperty(game, 14, "Kottawa", PINK, 4000, 300, 2000, 1000, 4000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
181		Blank line.
182	<code>/* Orange */</code>	Comment: <code>/* Orange */</code>
183	<code>setProperty(game, 16, "Negombo", ORANGE, 4500, 350, 2250, 1250, 5000);</code>	Calls <code>setProperty(game, 16, "Negombo", ORANGE, 4500, 350, 2250, 1250, 5000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
184	<code>setProperty(game, 18, "Katunayake", ORANGE, 4700, 370, 2350, 1250, 5000);</code>	Calls <code>setProperty(game, 18, "Katunayake", ORANGE, 4700, 370, 2350, 1250, 5000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
185	<code>setProperty(game, 19, "Ja-Ela", ORANGE, 5000, 400, 2500, 1250, 5000);</code>	Calls <code>setProperty(game, 19, "Ja-Ela", ORANGE, 5000, 400, 2500, 1250, 5000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
186		Blank line.
187	<code>/* Red */</code>	Comment: <code>/* Red */</code>
188	<code>setProperty(game, 21, "Kandy City", RED, 5500, 450, 2750, 1500, 6000);</code>	Calls <code>setProperty(game, 21, "Kandy City", RED, 5500, 450, 2750, 1500, 6000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
189	<code>setProperty(game, 23, "Peradeniya", RED, 5800, 480, 2900, 1500, 6000);</code>	Calls <code>setProperty(game, 23, "Peradeniya", RED, 5800, 480, 2900, 1500, 6000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
190	<code>setProperty(game, 24, "Katugastota", RED, 6000, 500, 3000, 1500, 6000);</code>	Calls <code>setProperty(game, 24, "Katugastota", RED, 6000, 500, 3000, 1500, 6000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
191		Blank line.
192	<code>/* Yellow */</code>	Comment: <code>/* Yellow */</code>
193	<code>setProperty(game, 26, "Galle Fort", YELLOW, 6500, 600, 3250, 2000, 8000);</code>	Calls <code>setProperty(game, 26, "Galle Fort", YELLOW, 6500, 600, 3250, 2000, 8000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)

194	<code>setProperty(game, 27, "Unawatuna", YELLOW, 6800, 620, 3400, 2000, 8000);</code>	Calls <code>setProperty(game, 27, "Unawatuna", YELLOW, 6800, 620, 3400, 2000, 8000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
195	<code>setProperty(game, 29, "Hikkaduwa", YELLOW, 7000, 650, 3500, 2000, 8000);</code>	Calls <code>setProperty(game, 29, "Hikkaduwa", YELLOW, 7000, 650, 3500, 2000, 8000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
196		Blank line.
197	<code>/* Green */</code>	Comment: <code>/* Green */</code>
198	<code>setProperty(game, 31, "Jaffna Town", GREEN, 8000, 750, 4000, 2500, 10000);</code>	Calls <code>setProperty(game, 31, "Jaffna Town", GREEN, 8000, 750, 4000, 2500, 10000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
199	<code>setProperty(game, 32, "Nallur", GREEN, 8300, 780, 4150, 2500, 10000);</code>	Calls <code>setProperty(game, 32, "Nallur", GREEN, 8300, 780, 4150, 2500, 10000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
200	<code>setProperty(game, 34, "Trincomalee", GREEN, 8500, 800, 4250, 2500, 10000);</code>	Calls <code>setProperty(game, 34, "Trincomalee", GREEN, 8500, 800, 4250, 2500, 10000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
201		Blank line.
202	<code>/* Dark Blue */</code>	Comment: <code>/* Dark Blue */</code>
203	<code>setProperty(game, 37, "Nuwara Eliya", DARK_BLUE, 10000, 1000, 5000, 3000, 12000);</code>	Calls <code>setProperty(game, 37, "Nuwara Eliya", DARK_BLUE, 10000, 1000, 5000, 3000, 12000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
204	<code>setProperty(game, 39, "Galle Face", DARK_BLUE, 12000, 1200, 6000, 3000, 12000);</code>	Calls <code>setProperty(game, 39, "Galle Face", DARK_BLUE, 12000, 1200, 6000, 3000, 12000)</code> . (Fills in one colour property square with its trade economics and resets ownership/build state)
205	<code>}</code>	Closes the current block.

Nested functions inside `initializeBoard()` - study these

- `initializeBoard()` (Builds the complete 40-square board: special squares, 4 railways, 2 utilities and every co)
- `setSpecialSquare()` (Creates a non-ownable square (GO, Tax, Jail, ...).)
- `setRailway()` (Creates a railway square. Mortgage value is fixed at half the purchase price.)
- `setUtility()` (Creates a utility square (mortgage = half price), same zeroing discipline as railways.)
- `setProperty()` (Fills in one colour property square with its trade economics and resets ownership/build st)

MAIN PURPOSE

Builds the complete 40-square board: special squares, 4 railways, 2 utilities and every colour property, using the helper setters.

3.2 `setProperty()`

```
void setProperty(GameState game[], int index, char name[], PropertyGroup group, int purchasePrice, int baseRent, int mortgageValue, int houseCost, int hotelCost)
```

Parameters: `index`, `name`, `group`, `purchasePrice`, `baseRent`, `mortgageValue`, `houseCost`, `hotelCost` | Returns: `void`.

The function, verbatim from the source (starts at line 6)

```
6 void setProperty(GameState game[],
7                 int index,
8                 char name[],
9                 PropertyGroup group,
10                int purchasePrice,
11                int baseRent,
12                int mortgageValue,
13                int houseCost,
14                int hotelCost)
15 {
16     game[0].board[index].index = index;
17     game[0].board[index].type = PROPERTY;
18
19     strcpy(game[0].board[index].name, name);
20     strcpy(game[0].board[index].property.name, name);
21
22     game[0].board[index].property.group = group;
```

```

23     game[0].board[index].property.purchasePrice = purchasePrice;
24     game[0].board[index].property.baseRent = baseRent;
25     game[0].board[index].property.mortgageValue = mortgageValue;
26     game[0].board[index].property.houseCost = houseCost;
27     game[0].board[index].property.hotelCost = hotelCost;

29     /* nobody owns it yet */
30     game[0].board[index].property.owner = -1;

32     game[0].board[index].property.houses = 0;
33     game[0].board[index].property.hotel = 0;
34     game[0].board[index].property.mortgaged = 0;
35     game[0].board[index].property.loanLocked = 0;

37     game[0].board[index].property.insurance = NO_INSURANCE;
38     game[0].board[index].property.insuranceRoundsLeft = 0;

40     game[0].board[index].property.damaged = 0;
41     game[0].board[index].property.repairCostOwed = 0;
42     game[0].board[index].property.lostIncomeRoundsLeft = 0;

44     game[0].board[index].property.age = 0;
45     game[0].board[index].property.depreciation = 0;
46     game[0].board[index].property.condition = 100;

48     game[0].board[index].property.roundsSinceMaintenance = 0;
49     game[0].board[index].property.structurallyDamaged = 0;
50     game[0].board[index].property.preDamagePurchasePrice = 0;
51     game[0].board[index].property.preDamageBaseRent = 0;
52     game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
53 }

```

Line-by-line explanation

Line	Code	Explanation
6	void setProperty(GameState game[],	void setProperty(GameState game[],
7	int index,	int index,
8	char name[],	char name[],
9	PropertyGroup group,	PropertyGroup group,
10	int purchasePrice,	int purchasePrice,
11	int baseRent,	int baseRent,
12	int mortgageValue,	int mortgageValue,
13	int houseCost,	int houseCost,
14	int hotelCost)	int hotelCost)
15	{	Opens a new block.
16	game[0].board[index].index = index;	Sets the field index of the square on the board (i.e. Square index if INDEX-scoped, else -1.) = index.
17	game[0].board[index].type = PROPERTY;	Sets the field type of the square on the board (i.e. What the modifier affects (value, rent, ...).) = PROPERTY.
18		Blank line.
19	strcpy(game[0].board[index].name, name);	Calls strcpy(game[0].board[index].name, name).
20	strcpy(game[0].board[index].property.name, name);	Calls strcpy(game[0].board[index].property.name, name).
21		Blank line.
22	game[0].board[index].property.group = group;	Sets the field group of the property on the board (i.e. Colour group if GROUP-scoped, else -1.) = group.
23	game[0].board[index].property.purchasePrice = purchasePrice;	Sets the field purchasePrice of the property on the board (i.e. Base price to buy (inflation and modifiers can move this).) = purchasePrice.
24	game[0].board[index].property.baseRent = baseRent;	Sets the field baseRent of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = baseRent.
25	game[0].board[index].property.mortgageValue = mortgageValue;	Sets the field mortgageValue of the property on the board (i.e. Value received when mortgaging (redeem costs +10%).) = mortgageValue.
26	game[0].board[index].property.houseCost = houseCost;	Sets the field houseCost of the property on the board (i.e. Cost of building one house.) = houseCost.

27	<code>game[0].board[index].property.hotelCost = hotelCost;</code>	Sets the field <code>hotelCost</code> of the property on the board (i.e. Cost of building a hotel.) = <code>hotelCost</code> .
28		Blank line.
29	<code>/* nobody owns it yet */</code>	Comment: <code>/* nobody owns it yet */</code>
30	<code>game[0].board[index].property.owner = -1;</code>	Sets the field <code>owner</code> of the property on the board (i.e. Player index who owns it; -1 means unowned.) = -1.
31		Blank line.
32	<code>game[0].board[index].property.houses = 0;</code>	Sets the field <code>houses</code> of the property on the board (i.e. Number of houses (0..4).) = 0.
33	<code>game[0].board[index].property.hotel = 0;</code>	Sets the field <code>hotel</code> of the property on the board (i.e. 1 if a hotel is built (replaces the 4 houses).) = 0.
34	<code>game[0].board[index].property.mortgaged = 0;</code>	Sets the field <code>mortgaged</code> of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed).) = 0.
35	<code>game[0].board[index].property.loanLocked = 0;</code>	Sets the field <code>loanLocked</code> of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 0.
36		Blank line.
37	<code>game[0].board[index].property.insurance = NO_INSURANCE;</code>	Sets the field <code>insurance</code> of the property on the board (i.e. Currently held policy type.) = <code>NO_INSURANCE</code> .
38	<code>game[0].board[index].property.insuranceRoundsLeft = 0;</code>	Sets the field <code>insuranceRoundsLeft</code> of the property on the board (i.e. Rounds remaining on the current policy.) = 0.
39		Blank line.
40	<code>game[0].board[index].property.damaged = 0;</code>	Sets the field <code>damaged</code> of the property on the board (i.e. 1 after a disaster until repaired (no rent).) = 0.
41	<code>game[0].board[index].property.repairCostOwed = 0;</code>	Sets the field <code>repairCostOwed</code> of the property on the board (i.e. Remaining repair cost after insurance compensation.) = 0.
42	<code>game[0].board[index].property.lostIncomeRoundsLeft = 0;</code>	Sets the field <code>lostIncomeRoundsLeft</code> of the property on the board (i.e. Business Interruption: rounds with no rent.) = 0.
43		Blank line.
44	<code>game[0].board[index].property.age = 0;</code>	Sets the field <code>age</code> of the property on the board (i.e. Rounds since purchase/last renovation.) = 0.
45	<code>game[0].board[index].property.depreciation = 0;</code>	Sets the field <code>depreciation</code> of the property on the board (i.e. Percent value loss from age (0..30%).) = 0.
46	<code>game[0].board[index].property.condition = 100;</code>	Sets the field <code>condition</code> of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 100.
47		Blank line.
48	<code>game[0].board[index].property.roundsSinceMaintenance = 0;</code>	Sets the field <code>roundsSinceMaintenance</code> of the property on the board (i.e. Rounds since last maintenance.) = 0.
49	<code>game[0].board[index].property.structurallyDamaged = 0;</code>	Sets the field <code>structurallyDamaged</code> of the property on the board (i.e. 1 when neglect (20+ rounds) caused structural damage.) = 0.
50	<code>game[0].board[index].property.preDamagePurchasePrice = 0;</code>	Sets the field <code>preDamagePurchasePrice</code> of the property on the board (i.e. Purchase price before structural damage (for restore).) = 0.
51	<code>game[0].board[index].property.preDamageBaseRent = 0;</code>	Sets the field <code>preDamageBaseRent</code> of the property on the board (i.e. Base rent before structural damage (for restore).) = 0.
52	<code>game[0].board[index].property.maintenanceCostMultiplierPercent = 100;</code>	Sets the field <code>maintenanceCostMultiplierPercent</code> of the property on the board (i.e. Cost multiplier (150% after damage, else 100%).) = 100.
53	<code>}</code>	Closes the current block.

Nested functions inside `setProperty()` - study these

- `setProperty()` (Fills in one colour property square with its trade economics and resets ownership/build st)

MAIN PURPOSE

Fills in one colour property square with its trade economics and resets ownership/build state.

3.3 `setRailway()`

```
void setRailway(GameState game[], int index, char name[], int purchasePrice)
```

Parameters: `index`, `name`, `purchasePrice` | Returns: `void`.

The function, verbatim from the source (starts at line 55)

```
55 void setRailway(GameState game[], int index, char name[], int purchasePrice)
56 {
57     game[0].board[index].index = index;
58     game[0].board[index].type = RAILWAY;

60     strcpy(game[0].board[index].name, name);
61     strcpy(game[0].board[index].property.name, name);

63     game[0].board[index].property.purchasePrice = purchasePrice;
64     game[0].board[index].property.mortgageValue = purchasePrice / 2;
65     game[0].board[index].property.owner = -1;
66     game[0].board[index].property.mortgaged = 0;
67     game[0].board[index].property.loanLocked = 0;

69     /* railways can't have houses/hotels/insurance, but we still zero
70        these out since GameState is local now and won't do it for us */
71     game[0].board[index].property.houses = 0;
72     game[0].board[index].property.hotel = 0;
73     game[0].board[index].property.insurance = NO_INSURANCE;
74     game[0].board[index].property.insuranceRoundsLeft = 0;
75     game[0].board[index].property.damaged = 0;
76     game[0].board[index].property.repairCostOwed = 0;
77     game[0].board[index].property.lostIncomeRoundsLeft = 0;
78     game[0].board[index].property.age = 0;
79     game[0].board[index].property.depreciation = 0;
80     game[0].board[index].property.condition = 100;
81     game[0].board[index].property.roundsSinceMaintenance = 0;
82     game[0].board[index].property.structurallyDamaged = 0;
83     game[0].board[index].property.preDamagePurchasePrice = 0;
84     game[0].board[index].property.preDamageBaseRent = 0;
85     game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
86     game[0].board[index].property.baseRent = 0;
87     game[0].board[index].property.houseCost = 0;
88     game[0].board[index].property.hotelCost = 0;
89 }
```

Line-by-line explanation

Line	Code	Explanation
55	void setRailway(GameState game[], int index, char name[], int purchasePrice)	void setRailway(GameState game[], int index, char name[], int purchasePrice)
56	{	Opens a new block.
57	game[0].board[index].index = index;	Sets the field index of the square on the board (i.e. Square index if INDEX-scoped, else -1.) = index.
58	game[0].board[index].type = RAILWAY;	Sets the field type of the square on the board (i.e. What the modifier affects (value, rent, ...)) = RAILWAY.
59		Blank line.
60	strcpy(game[0].board[index].name, name);	Calls strcpy(game[0].board[index].name, name).
61	strcpy(game[0].board[index].property.name, name);	Calls strcpy(game[0].board[index].property.name, name).
62		Blank line.
63	game[0].board[index].property.purchasePrice = purchasePrice;	Sets the field purchasePrice of the property on the board (i.e. Base price to buy (inflation and modifiers can move this).) = purchasePrice.
64	game[0].board[index].property.mortgageValue = purchasePrice / 2;	Sets the field mortgageValue of the property on the board (i.e. Value received when mortgaging (redeem costs +10%).) = purchasePrice / 2.
65	game[0].board[index].property.owner = -1;	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = -1.
66	game[0].board[index].property.mortgaged = 0;	Sets the field mortgaged of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed).) = 0.
67	game[0].board[index].property.loanLocked = 0;	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 0.
68		Blank line.
69	/* railways can't have houses/hotels/insurance, but we still zero	Comment: /* railways can't have houses/hotels/insurance, but we still zero

70	these out since GameState is local now and won't do it for us */	these out since GameState is local now and won't do it for us */
71	game[0].board[index].property.houses = 0;	Sets the field houses of the property on the board (i.e. Number of houses (0..4).) = 0.
72	game[0].board[index].property.hotel = 0;	Sets the field hotel of the property on the board (i.e. 1 if a hotel is built (replaces the 4 houses).) = 0.
73	game[0].board[index].property.insurance = NO_INSURANCE;	Sets the field insurance of the property on the board (i.e. Currently held policy type.) = NO_INSURANCE.
74	game[0].board[index].property.insuranceRoundsLeft = 0;	Sets the field insuranceRoundsLeft of the property on the board (i.e. Rounds remaining on the current policy.) = 0.
75	game[0].board[index].property.damaged = 0;	Sets the field damaged of the property on the board (i.e. 1 after a disaster until repaired (no rent).) = 0.
76	game[0].board[index].property.repairCostOwed = 0;	Sets the field repairCostOwed of the property on the board (i.e. Remaining repair cost after insurance compensation.) = 0.
77	game[0].board[index].property.lostIncomeRoundsLeft = 0;	Sets the field lostIncomeRoundsLeft of the property on the board (i.e. Business Interruption: rounds with no rent.) = 0.
78	game[0].board[index].property.age = 0;	Sets the field age of the property on the board (i.e. Rounds since purchase/last renovation.) = 0.
79	game[0].board[index].property.depreciation = 0;	Sets the field depreciation of the property on the board (i.e. Percent value loss from age (0..30%).) = 0.
80	game[0].board[index].property.condition = 100;	Sets the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 100.
81	game[0].board[index].property.roundsSinceMaintenance = 0;	Sets the field roundsSinceMaintenance of the property on the board (i.e. Rounds since last maintenance.) = 0.
82	game[0].board[index].property.structurallyDamaged = 0;	Sets the field structurallyDamaged of the property on the board (i.e. 1 when neglect (20+ rounds) caused structural damage.) = 0.
83	game[0].board[index].property.preDamagePurchasePrice = 0;	Sets the field preDamagePurchasePrice of the property on the board (i.e. Purchase price before structural damage (for restore).) = 0.
84	game[0].board[index].property.preDamageBaseRent = 0;	Sets the field preDamageBaseRent of the property on the board (i.e. Base rent before structural damage (for restore).) = 0.
85	game[0].board[index].property.maintenanceCostMultiplierPercent = 100;	Sets the field maintenanceCostMultiplierPercent of the property on the board (i.e. Cost multiplier (150% after damage, else 100%).) = 100.
86	game[0].board[index].property.baseRent = 0;	Sets the field baseRent of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = 0.
87	game[0].board[index].property.houseCost = 0;	Sets the field houseCost of the property on the board (i.e. Cost of building one house.) = 0.
88	game[0].board[index].property.hotelCost = 0;	Sets the field hotelCost of the property on the board (i.e. Cost of building a hotel.) = 0.
89	}	Closes the current block.

Nested functions inside setRailway() - study these

- setRailway() (Creates a railway square. Mortgage value is fixed at half the purchase price.)

MAIN PURPOSE

Creates a railway square. Mortgage value is fixed at half the purchase price.

3.4 setUtility()

```
void setUtility(GameState game[], int index, char name[], int purchasePrice)
```

Parameters: index, name, purchasePrice | Returns: void.

The function, verbatim from the source (starts at line 91)

```
91 void setUtility(GameState game[], int index, char name[], int purchasePrice)
92 {
93     game[0].board[index].index = index;
94     game[0].board[index].type = UTILITY;

96     strcpy(game[0].board[index].name, name);
97     strcpy(game[0].board[index].property.name, name);
```

```

99     game[0].board[index].property.purchasePrice = purchasePrice;
100    game[0].board[index].property.mortgageValue = purchasePrice / 2;
101    game[0].board[index].property.owner = -1;
102    game[0].board[index].property.mortgaged = 0;
103    game[0].board[index].property.loanLocked = 0;

105    game[0].board[index].property.houses = 0;
106    game[0].board[index].property.hotel = 0;
107    game[0].board[index].property.insurance = NO_INSURANCE;
108    game[0].board[index].property.insuranceRoundsLeft = 0;
109    game[0].board[index].property.damaged = 0;
110    game[0].board[index].property.repairCostOwed = 0;
111    game[0].board[index].property.lostIncomeRoundsLeft = 0;
112    game[0].board[index].property.age = 0;
113    game[0].board[index].property.depreciation = 0;
114    game[0].board[index].property.condition = 100;
115    game[0].board[index].property.roundsSinceMaintenance = 0;
116    game[0].board[index].property.structurallyDamaged = 0;
117    game[0].board[index].property.preDamagePurchasePrice = 0;
118    game[0].board[index].property.preDamageBaseRent = 0;
119    game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
120    game[0].board[index].property.baseRent = 0;
121    game[0].board[index].property.houseCost = 0;
122    game[0].board[index].property.hotelCost = 0;
123 }

```

Line-by-line explanation

Line	Code	Explanation
91	<code>void setUtility(GameState game[], int index, char name[], int purchasePrice)</code>	<code>void setUtility(GameState game[], int index, char name[], int purchasePrice)</code>
92	<code>{</code>	Opens a new block.
93	<code>game[0].board[index].index = index;</code>	Sets the field index of the square on the board (i.e. Square index if INDEX-scoped, else -1.) = index.
94	<code>game[0].board[index].type = UTILITY;</code>	Sets the field type of the square on the board (i.e. What the modifier affects (value, rent, ...)) = UTILITY.
95		Blank line.
96	<code>strcpy(game[0].board[index].name, name);</code>	Calls <code>strcpy(game[0].board[index].name, name)</code> .
97	<code>strcpy(game[0].board[index].property.name, name);</code>	Calls <code>strcpy(game[0].board[index].property.name, name)</code> .
98		Blank line.
99	<code>game[0].board[index].property.purchasePrice = purchasePrice;</code>	Sets the field purchasePrice of the property on the board (i.e. Base price to buy (inflation and modifiers can move this)) = purchasePrice.
100	<code>game[0].board[index].property.mortgageValue = purchasePrice / 2;</code>	Sets the field mortgageValue of the property on the board (i.e. Value received when mortgaging (redeem costs +10%)) = purchasePrice / 2.
101	<code>game[0].board[index].property.owner = -1;</code>	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = -1.
102	<code>game[0].board[index].property.mortgaged = 0;</code>	Sets the field mortgaged of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed)) = 0.
103	<code>game[0].board[index].property.loanLocked = 0;</code>	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold)) = 0.
104		Blank line.
105	<code>game[0].board[index].property.houses = 0;</code>	Sets the field houses of the property on the board (i.e. Number of houses (0.4.)) = 0.
106	<code>game[0].board[index].property.hotel = 0;</code>	Sets the field hotel of the property on the board (i.e. 1 if a hotel is built (replaces the 4 houses.)) = 0.
107	<code>game[0].board[index].property.insurance = NO_INSURANCE;</code>	Sets the field insurance of the property on the board (i.e. Currently held policy type.) = NO_INSURANCE.
108	<code>game[0].board[index].property.insuranceRoundsLeft = 0;</code>	Sets the field insuranceRoundsLeft of the property on the board (i.e. Rounds remaining on the current policy.) = 0.
109	<code>game[0].board[index].property.damaged = 0;</code>	Sets the field damaged of the property on the board (i.e. 1 after a disaster until repaired (no rent.)) = 0.
110	<code>game[0].board[index].property.repairCostOwed = 0;</code>	Sets the field repairCostOwed of the property on the board (i.e. Remaining repair cost after insurance compensation.) = 0.

111	game[0].board[index].property.lostIncomeRoundsLeft = 0;	Sets the field lostIncomeRoundsLeft of the property on the board (i.e. Business Interruption: rounds with no rent.) = 0.
112	game[0].board[index].property.age = 0;	Sets the field age of the property on the board (i.e. Rounds since purchase/last renovation.) = 0.
113	game[0].board[index].property.depreciation = 0;	Sets the field depreciation of the property on the board (i.e. Percent value loss from age (0..30%).) = 0.
114	game[0].board[index].property.condition = 100;	Sets the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 100.
115	game[0].board[index].property.roundsSinceMaintenance = 0;	Sets the field roundsSinceMaintenance of the property on the board (i.e. Rounds since last maintenance.) = 0.
116	game[0].board[index].property.structurallyDamaged = 0;	Sets the field structurallyDamaged of the property on the board (i.e. 1 when neglect (20+ rounds) caused structural damage.) = 0.
117	game[0].board[index].property.preDamagePurchasePrice = 0;	Sets the field preDamagePurchasePrice of the property on the board (i.e. Purchase price before structural damage (for restore).) = 0.
118	game[0].board[index].property.preDamageBaseRent = 0;	Sets the field preDamageBaseRent of the property on the board (i.e. Base rent before structural damage (for restore).) = 0.
119	game[0].board[index].property.maintenanceCostMultiplierPercent = 100;	Sets the field maintenanceCostMultiplierPercent of the property on the board (i.e. Cost multiplier (150% after damage, else 100%).) = 100.
120	game[0].board[index].property.baseRent = 0;	Sets the field baseRent of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = 0.
121	game[0].board[index].property.houseCost = 0;	Sets the field houseCost of the property on the board (i.e. Cost of building one house.) = 0.
122	game[0].board[index].property.hotelCost = 0;	Sets the field hotelCost of the property on the board (i.e. Cost of building a hotel.) = 0.
123	}	Closes the current block.

Nested functions inside setUtility() - study these

- setUtility() (Creates a utility square (mortgage = half price), same zeroing discipline as railways.)

MAIN PURPOSE

Creates a utility square (mortgage = half price), same zeroing discipline as railways.

3.5 setSpecialSquare()

```
void setSpecialSquare(GameState game[], int index, SquareType type, char name[])
```

Parameters: index, type, name | Returns: void.

The function, verbatim from the source (starts at line 125)

```
125 void setSpecialSquare(GameState game[],
126                        int index,
127                        SquareType type,
128                        char name[])
129 {
130     game[0].board[index].index = index;
131     game[0].board[index].type = type;
132
133     strcpy(game[0].board[index].name, name);
134
135     /* Owner must be -1 (unowned), not the default 0, otherwise this
136        square would look like it belongs to Player 0. */
137     game[0].board[index].property.owner = -1;
138     game[0].board[index].property.mortgaged = 0;
139     game[0].board[index].property.loanLocked = 0;
140     game[0].board[index].property.houses = 0;
141     game[0].board[index].property.hotel = 0;
142     game[0].board[index].property.insurance = NO_INSURANCE;
143 }
```

Line-by-line explanation

Line	Code	Explanation
125	void setSpecialSquare(GameState game[],	void setSpecialSquare(GameState game[],
126	int index,	int index,

127	SquareType type,	SquareType type,
128	char name[])	char name[])
129	{	Opens a new block.
130	game[0].board[index].index = index;	Sets the field index of the square on the board (i.e. Square index if INDEX-scoped, else -1.) = index.
131	game[0].board[index].type = type;	Sets the field type of the square on the board (i.e. What the modifier affects (value, rent, ...)) = type.
132		Blank line.
133	strcpy(game[0].board[index].name, name);	Calls strcpy(game[0].board[index].name, name).
134		Blank line.
135	/* Owner must be -1 (unowned), not the default 0, otherwise this	Comment: /* Owner must be -1 (unowned), not the default 0, otherwise this
136	square would look like it belongs to Player 0. */	square would look like it belongs to Player 0. */
137	game[0].board[index].property.owner = -1;	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = -1.
138	game[0].board[index].property.mortgaged = 0;	Sets the field mortgaged of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed).) = 0.
139	game[0].board[index].property.loanLocked = 0;	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 0.
140	game[0].board[index].property.houses = 0;	Sets the field houses of the property on the board (i.e. Number of houses (0..4).) = 0.
141	game[0].board[index].property.hotel = 0;	Sets the field hotel of the property on the board (i.e. 1 if a hotel is built (replaces the 4 houses).) = 0.
142	game[0].board[index].property.insurance = NO_INSURANCE;	Sets the field insurance of the property on the board (i.e. Currently held policy type.) = NO_INSURANCE.
143	}	Closes the current block.

Nested functions inside setSpecialSquare() - study these

- setSpecialSquare() (Creates a non-ownable square (GO, Tax, Jail, ...).)

MAIN PURPOSE

Creates a non-ownable square (GO, Tax, Jail, ...).

Part 4 players.c

The four AI strategies (Aggressive Investor, Risk Taker, Opportunistic Trader, Conservative Banker) and every 'should / wants' decision.

The sub-topics below are the 15 functions implemented in players.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in players.c

4.1	initializePlayers()	(source line 6)	p. 30
4.2	shouldBuyProperty()	(source line 42)	p. 32
4.3	shouldConstruct()	(source line 95)	p. 34
4.4	wantsLoan()	(source line 134)	p. 36
4.5	wantsToRepayLoan()	(source line 160)	p. 37
4.6	wantsIncreaseLoan()	(source line 187)	p. 38
4.7	wantsExtendLoan()	(source line 214)	p. 39
4.8	wantsRefinance()	(source line 245)	p. 40
4.9	desiredInsurance()	(source line 253)	p. 41
4.10	shouldRenovateAgeDepreciation()	(source line 290)	p. 42
4.11	shouldMaintain()	(source line 308)	p. 43
4.12	willingToBid()	(source line 323)	p. 44
4.13	shouldMortgage()	(source line 351)	p. 45
4.14	shouldRedeemMortgage()	(source line 376)	p. 46
4.15	shouldPayBail()	(source line 380)	p. 47

4.1 initializePlayers()

```
void initializePlayers(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 6)

```
6    void initializePlayers(GameState game[])
7    {
8        int i;

10        strcpy(game[0].players[0].name, "Aggressive Investor");
11        game[0].players[0].strategy = AGGRESSIVE_INVESTOR;

13        strcpy(game[0].players[1].name, "Conservative Banker");
14        game[0].players[1].strategy = CONSERVATIVE_BANKER;

16        strcpy(game[0].players[2].name, "Risk Taker");
17        game[0].players[2].strategy = RISK_TAKER;

19        strcpy(game[0].players[3].name, "Opportunistic Trader");
20        game[0].players[3].strategy = OPPORTUNISTIC_TRADER;

22        for(i = 0; i < MAX_PLAYERS; i++)
23        {
24            game[0].players[i].position = 0;
25            game[0].players[i].cash = START_MONEY;

27            game[0].players[i].inJail = 0;
28            game[0].players[i].jailTurns = 0;

30            game[0].players[i].bankrupt = 0;

32            game[0].players[i].propertiesOwned = 0;
33            game[0].players[i].railwaysOwned = 0;
34            game[0].players[i].utilitiesOwned = 0;

36            resetLoan(game, i);
```

```

38         game[0].players[i].sufferedLoss = 0;
39         game[0].players[i].antiSpecRounds = 0;
40     }
41 }

```

Line-by-line explanation

Line	Code	Explanation
6	<code>void initializePlayers(GameState game[])</code>	<code>void initializePlayers(GameState game[])</code>
7	<code>{</code>	Opens a new block.
8	<code>int i;</code>	Declares <code>int i</code> .
9		Blank line.
10	<code>strcpy(game[0].players[0].name, "Aggressive Investor");</code>	Calls <code>strcpy(game[0].players[0].name, "Aggressive Investor")</code> .
11	<code>game[0].players[0].strategy = AGGRESSIVE_INVESTOR;</code>	Sets the field strategy of the player (i.e. Which strategy drives every decision.) = <code>AGGRESSIVE_INVESTOR</code> .
12		Blank line.
13	<code>strcpy(game[0].players[1].name, "Conservative Banker");</code>	Calls <code>strcpy(game[0].players[1].name, "Conservative Banker")</code> .
14	<code>game[0].players[1].strategy = CONSERVATIVE_BANKER;</code>	Sets the field strategy of the player (i.e. Which strategy drives every decision.) = <code>CONSERVATIVE_BANKER</code> .
15		Blank line.
16	<code>strcpy(game[0].players[2].name, "Risk Taker");</code>	Calls <code>strcpy(game[0].players[2].name, "Risk Taker")</code> .
17	<code>game[0].players[2].strategy = RISK_TAKER;</code>	Sets the field strategy of the player (i.e. Which strategy drives every decision.) = <code>RISK_TAKER</code> .
18		Blank line.
19	<code>strcpy(game[0].players[3].name, "Opportunistic Trader");</code>	Calls <code>strcpy(game[0].players[3].name, "Opportunistic Trader")</code> .
20	<code>game[0].players[3].strategy = OPPORTUNISTIC_TRADER;</code>	Sets the field strategy of the player (i.e. Which strategy drives every decision.) = <code>OPPORTUNISTIC_TRADER</code> .
21		Blank line.
22	<code>for(i = 0; i < MAX_PLAYERS; i++)</code>	Loop over all entries: init <code>i = 0</code> ; run while <code>i < MAX_PLAYERS</code> ; every loop <code>i++</code> .
23	<code>{</code>	Opens a new block.
24	<code>game[0].players[i].position = 0;</code>	Sets the field position of the player (i.e. Current board position 0..39.) = 0.
25	<code>game[0].players[i].cash = START_MONEY;</code>	Sets the field cash of the player (i.e. Cash balance in LKR.) = <code>START_MONEY</code> .
26		Blank line.
27	<code>game[0].players[i].inJail = 0;</code>	Sets the field <code>inJail</code> of the player (i.e. 1 while in Jail.) = 0.
28	<code>game[0].players[i].jailTurns = 0;</code>	Sets the field <code>jailTurns</code> of the player (i.e. Consecutive jail turns while failing to roll doubles.) = 0.
29		Blank line.
30	<code>game[0].players[i].bankrupt = 0;</code>	Sets the field <code>bankrupt</code> of the player (i.e. 1 once the player is out of the game.) = 0.
31		Blank line.
32	<code>game[0].players[i].propertiesOwned = 0;</code>	Sets the field <code>propertiesOwned</code> of the player (i.e. Count of colour properties owned.) = 0.
33	<code>game[0].players[i].railwaysOwned = 0;</code>	Sets the field <code>railwaysOwned</code> of the player (i.e. Count of railways owned.) = 0.
34	<code>game[0].players[i].utilitiesOwned = 0;</code>	Sets the field <code>utilitiesOwned</code> of the player (i.e. Count of utilities owned.) = 0.
35		Blank line.
36	<code>resetLoan(game, i);</code>	Calls <code>resetLoan(game, i)</code> . (Wipes a player's loan (active, amount, rate, rounds))
37		Blank line.
38	<code>game[0].players[i].sufferedLoss = 0;</code>	Sets the field <code>sufferedLoss</code> of the player (i.e. 1 after an uninsured disaster (Risk Taker now insures).) = 0.
39	<code>game[0].players[i].antiSpecRounds = 0;</code>	Sets the field <code>antiSpecRounds</code> of the player (i.e. Consecutive rounds over the 3-undeveloped limit.) = 0.

40	}	Closes the current block.
41	}	Closes the current block.

Nested functions inside initializePlayers() - study these

- initializePlayers() (Names the four players and gives their starting state.)
- resetLoan() (Wipes a player's loan (active, amount, rate, rounds).)

MAIN PURPOSE

Names the four players and gives their starting state.

4.2 shouldBuyProperty()

```
int shouldBuyProperty(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *1 = buy, 0 = pass.*

The function, verbatim from the source (starts at line 42)

```

43  /* Decides whether a player wants to buy the property they just
44     landed on. Each strategy has its own rule, from Section 3. */
45  int shouldBuyProperty(GameState game[], int playerIndex)
46  {
47      int price;
48      int position;

50      position = game[0].players[playerIndex].position;
51      price = game[0].board[position].property.purchasePrice;

53      /* match the price buyProperty() actually charges (Rule-LK 31/34) */
54      if(game[0].board[position].type == PROPERTY)
55      {
56          price = (currentMarketValue(game, position) *
57                  modifierMultiplier(game, MOD_PURCHASE_PRICE,
58                                     game[0].board[position].property.group, -1)) / 100;
59      }

61      switch(game[0].players[playerIndex].strategy)
62      {
63          case AGGRESSIVE_INVESTOR:
64              /* buys to complete a monopoly group even if it dips below
65                 the usual 1000 reserve (Section 3.1) */
66              if(wouldCompleteMonopoly(game, playerIndex, position))
67                  return 1;
68              return (game[0].players[playerIndex].cash - price >= 1000);

70          case CONSERVATIVE_BANKER:
71              /* avoids property purchases entirely during a recession,
72                 but will still buy rail/utility income (Section 3.2) */
73              if(game[0].board[position].type == PROPERTY &&
74                 isModifierActive(game, MOD_RECESSION, -1, -1))
75                  return 0;
76              return (game[0].players[playerIndex].cash - price >=
77                      game[0].players[playerIndex].cash / 2);

79          case RISK_TAKER:
80              return (game[0].players[playerIndex].cash >= price);

82          case OPPORTUNISTIC_TRADER:
83              /* never buys into a group that is currently declining */
84              if(game[0].board[position].type == PROPERTY &&
85                 modifierMultiplier(game, MOD_GROUP_VALUE,
86                                    game[0].board[position].property.group, -1) < 100)
87                  return 0;
88              /* only buys cheaper properties */
89              return (price <= game[0].players[playerIndex].cash * 40 / 100);

91          default:
92              return 0;

```

```

93     }
94 }

```

Line-by-line explanation

Line	Code	Explanation
42		Blank line.
43	<code>/* Decides whether a player wants to buy the property they just</code>	Comment: <code>/* Decides whether a player wants to buy the property they just</code>
44	<code>landed on. Each strategy has its own rule, from</code>	landed on. Each strategy has its own rule, from Section 3. <code>*/</code>
45	<code>int shouldBuyProperty(GameState game[], int</code>	<code>int shouldBuyProperty(GameState game[], int playerIndex)</code>
46	<code>{</code>	Opens a new block.
47	<code>int price;</code>	Declares <code>int price</code> .
48	<code>int position;</code>	Declares <code>int position</code> .
49		Blank line.
50	<code>position = game[0].players[playerIndex].position;</code>	<code>position = game[0].players[playerIndex].position;</code>
51	<code>price =</code>	<code>price = game[0].board[position].property.purchasePrice;</code>
52		Blank line.
53	<code>/* match the price buyProperty() actually charges</code>	Comment: <code>/* match the price buyProperty() actually charges (Rule-LK</code>
54	<code>(Rule-LK 31/34) */</code>	<code>31/34) */</code>
55	<code>if(game[0].board[position].type == PROPERTY)</code>	Condition: <code>game[0].board[position].type == PROPERTY</code> -> run the next block if true.
56	<code>{</code>	Opens a new block.
57	<code>price = (currentMarketValue(game, position) *</code>	<code>price = (currentMarketValue(game, position) *</code>
58	<code>modifierMultiplier(game, MOD_PURCHASE_PRICE,</code>	<code>modifierMultiplier(game, MOD_PURCHASE_PRICE,</code>
59	<code>game[0].board[position].property.group, -1)) / 100;</code>	<code>game[0].board[position].property.group, -1)) / 100;</code>
60	<code>}</code>	Closes the current block.
61	<code>switch(game[0].players[playerIndex].strategy)</code>	Chooses a branch based on the value of the current <code>player.strategy</code> .
62	<code>{</code>	Opens a new block.
63	<code>case AGGRESSIVE_INVESTOR:</code>	This case of the switch runs when its constant matches.
64	<code>/* buys to complete a monopoly group even if it dips</code>	Comment: <code>/* buys to complete a monopoly group even if it dips below</code>
65	<code>below</code>	
66	<code>the usual 1000 reserve (Section 3.1) */</code>	the usual 1000 reserve (Section 3.1) <code>*/</code>
67	<code>if(wouldCompleteMonopoly(game, playerIndex,</code>	Condition: <code>wouldCompleteMonopoly(game, playerIndex, position)</code> -> run the next block if true.
68	<code>position))</code>	
69	<code>return 1;</code>	Returns 1 immediately.
70	<code>return (game[0].players[playerIndex].cash - price >=</code>	Returns <code>(game[0].players[playerIndex].cash - price >= 1000)</code> immediately.
71	<code>1000);</code>	
72		Blank line.
73	<code>case CONSERVATIVE_BANKER:</code>	This case of the switch runs when its constant matches.
74	<code>/* avoids property purchases entirely during a</code>	Comment: <code>/* avoids property purchases entirely during a recession,</code>
75	<code>recession,</code>	
76	<code>but will still buy rail/utility income (Section 3.2)</code>	but will still buy rail/utility income (Section 3.2) <code>*/</code>
77	<code>*/</code>	
78	<code>if(game[0].board[position].type == PROPERTY &&</code>	Condition: -> run the next block if true.
79	<code>isModifierActive(game, MOD_RECESSION, -1, -1))</code>	Calls <code>isModifierActive(game, MOD_RECESSION, -1, -1)</code> . (Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active)
80	<code>return 0;</code>	Returns 0 immediately.
81	<code>return (game[0].players[playerIndex].cash - price >=</code>	Returns <code>(game[0].players[playerIndex].cash - price >= immediately.</code>
82	<code>game[0].players[playerIndex].cash / 2);</code>	<code>game[0].players[playerIndex].cash / 2);</code>
83		Blank line.
84	<code>case RISK_TAKER:</code>	This case of the switch runs when its constant matches.
85	<code>return (game[0].players[playerIndex].cash >= price);</code>	Returns <code>(game[0].players[playerIndex].cash >= price)</code> immediately.
86		Blank line.

82	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
83	/* never buys into a group that is currently declining */	Comment: /* never buys into a group that is currently declining */
84	if(game[0].board[position].type == PROPERTY &&	Condition: -> run the next block if true.
85	modifierMultiplier(game, MOD_GROUP_VALUE,	modifierMultiplier(game, MOD_GROUP_VALUE,
86	game[0].board[position].property.group, -1) < 100)	game[0].board[position].property.group, -1) < 100)
87	return 0;	Returns 0 immediately.
88	/* only buys cheaper properties */	Comment: /* only buys cheaper properties */
89	return (price <= game[0].players[playerIndex].cash * 40 / 100);	Returns (price <= game[0].players[playerIndex].cash * 40 / 100) immediately.
90		Blank line.
91	default:	The fallback switch branch when no case matched.
92	return 0;	Returns 0 immediately.
93	}	Closes the current block.
94	}	Closes the current block.

Nested functions inside shouldBuyProperty() - study these

- shouldBuyProperty() (Decides whether the player buys the square they just landed on.)
- buyProperty() (Handles landing on an unowned square: prices it, asks the strategy, buys or sends to aucti)
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- wouldCompleteMonopoly() (True if buying the square at pos would complete a whole colour group.)
- isModifierActive() (Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is ac)

MAIN PURPOSE

Decides whether the player buys the square they just landed on.

4.3 shouldConstruct()

```
int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
```

Parameters: *playerIndex, cost, isHotel* | Returns: *1 = build, 0 = don't.*

The function, verbatim from the source (starts at line 95)

```

96  /* Decides whether a player is willing to spend on a house/hotel. */
97  int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
98  {
99      int cash;

101     cash = game[0].players[playerIndex].cash;

103     switch(game[0].players[playerIndex].strategy)
104     {
105         case AGGRESSIVE_INVESTOR:
106             return (cash - cost >= MIN_CASH_SAFETY);

108         case CONSERVATIVE_BANKER:
109             /* never builds hotels while a loan is active - houses are
110              still fine (Section 3.2) */
111             if(game[0].players[playerIndex].loan.active && isHotel)
112                 return 0;
113             return (cash - cost >= cash / 2);

115         case RISK_TAKER:
116             return (cash >= cost);

118         case OPPORTUNISTIC_TRADER:
119             /* builds eagerly while a construction subsidy is active */
120             if(modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100)
121                 return (cash - cost >= cash * 30 / 100);

123             /* delays building while inflation pushes costs up */
124             if(game[0].economy.inflationRate > 0)

```

```

125         return (cash - cost >= cash * 60 / 100);

127         return (cash - cost >= cash * 40 / 100);

129     default:
130         return 0;
131     }
132 }

```

Line-by-line explanation

Line	Code	Explanation
95		Blank line.
96	<code>/* Decides whether a player is willing to spend on a house/hotel. */</code>	Comment: <code>/* Decides whether a player is willing to spend on a house/hotel. */</code>
97	<code>int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)</code>	<code>int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)</code>
98	<code>{</code>	Opens a new block.
99	<code>int cash;</code>	Declares <code>int cash</code> .
100		Blank line.
101	<code>cash = game[0].players[playerIndex].cash;</code>	<code>cash = game[0].players[playerIndex].cash;</code>
102		Blank line.
103	<code>switch(game[0].players[playerIndex].strategy)</code>	Chooses a branch based on the value of the current <code>player.strategy</code> .
104	<code>{</code>	Opens a new block.
105	<code>case AGGRESSIVE_INVESTOR:</code>	This case of the switch runs when its constant matches.
106	<code>return (cash - cost >= MIN_CASH_SAFETY);</code>	Returns <code>(cash - cost >= MIN_CASH_SAFETY)</code> immediately.
107		Blank line.
108	<code>case CONSERVATIVE_BANKER:</code>	This case of the switch runs when its constant matches.
109	<code>/* never builds hotels while a loan is active - houses are</code>	Comment: <code>/* never builds hotels while a loan is active - houses are</code>
110	<code>still fine (Section 3.2) */</code>	<code>still fine (Section 3.2) */</code>
111	<code>if(game[0].players[playerIndex].loan.active && isHotel)</code>	Condition: the current <code>player.loan.active && isHotel</code> -> run the next block if true.
112	<code>return 0;</code>	Returns 0 immediately.
113	<code>return (cash - cost >= cash / 2);</code>	Returns <code>(cash - cost >= cash / 2)</code> immediately.
114		Blank line.
115	<code>case RISK_TAKER:</code>	This case of the switch runs when its constant matches.
116	<code>return (cash >= cost);</code>	Returns <code>(cash >= cost)</code> immediately.
117		Blank line.
118	<code>case OPPORTUNISTIC_TRADER:</code>	This case of the switch runs when its constant matches.
119	<code>/* builds eagerly while a construction subsidy is active */</code>	Comment: <code>/* builds eagerly while a construction subsidy is active */</code>
120	<code>if(modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100)</code>	Condition: <code>modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100</code> -> run the next block if true.
121	<code>return (cash - cost >= cash * 30 / 100);</code>	Returns <code>(cash - cost >= cash * 30 / 100)</code> immediately.
122		Blank line.
123	<code>/* delays building while inflation pushes costs up */</code>	Comment: <code>/* delays building while inflation pushes costs up */</code>
124	<code>if(game[0].economy.inflationRate > 0)</code>	Condition: current <code>inflation > 0</code> -> run the next block if true.
125	<code>return (cash - cost >= cash * 60 / 100);</code>	Returns <code>(cash - cost >= cash * 60 / 100)</code> immediately.
126		Blank line.
127	<code>return (cash - cost >= cash * 40 / 100);</code>	Returns <code>(cash - cost >= cash * 40 / 100)</code> immediately.
128		Blank line.
129	<code>default:</code>	The fallback switch branch when no case matched.
130	<code>return 0;</code>	Returns 0 immediately.
131	<code>}</code>	Closes the current block.
132	<code>}</code>	Closes the current block.

Nested functions inside shouldConstruct() - study these

- shouldConstruct() (Decides whether the player spends on a house/hotel now.)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

Decides whether the player spends on a house/hotel now.

4.4 wantsLoan()

```
int wantsLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: 1 = wants a loan, 0 = doesn't.

The function, verbatim from the source (starts at line 134)

```
134 int wantsLoan(GameState game[], int playerIndex)
135 {
136     int cash;

138     cash = game[0].players[playerIndex].cash;

140     switch(game[0].players[playerIndex].strategy)
141     {
142         case AGGRESSIVE_INVESTOR:
143             return 1;

145         case CONSERVATIVE_BANKER:
146             /* only borrows if bankruptcy is close */
147             return (cash < MIN_CASH_SAFETY);

149         case RISK_TAKER:
150             return 1;

152         case OPPORTUNISTIC_TRADER:
153             return (cash < RICH_CASH_LINE);

155         default:
156             return 0;
157     }
158 }
```

Line-by-line explanation

Line	Code	Explanation
134	int wantsLoan(GameState game[], int playerIndex)	int wantsLoan(GameState game[], int playerIndex)
135	{	Opens a new block.
136	int cash;	Declares int cash.
137		Blank line.
138	cash = game[0].players[playerIndex].cash;	cash = game[0].players[playerIndex].cash;
139		Blank line.
140	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
141	{	Opens a new block.
142	case AGGRESSIVE_INVESTOR:	This case of the switch runs when its constant matches.
143	return 1;	Returns 1 immediately.
144		Blank line.
145	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.
146	/* only borrows if bankruptcy is close */	Comment: /* only borrows if bankruptcy is close */
147	return (cash < MIN_CASH_SAFETY);	Returns (cash < MIN_CASH_SAFETY) immediately.
148		Blank line.
149	case RISK_TAKER:	This case of the switch runs when its constant matches.
150	return 1;	Returns 1 immediately.
151		Blank line.
152	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
153	return (cash < RICH_CASH_LINE);	Returns (cash < RICH_CASH_LINE) immediately.
154		Blank line.

155	default:	The fallback switch branch when no case matched.
156	return 0;	Returns 0 immediately.
157	}	Closes the current block.
158	}	Closes the current block.

Nested functions inside wantsLoan() - study these

- wantsLoan() (Whether the player wants to take out a new loan (Bank visit, no current loan).)

MAIN PURPOSE

Whether the player wants to take out a new loan (Bank visit, no current loan).

4.5 wantsToRepayLoan()

```
int wantsToRepayLoan(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: 1 = repay in full, 0 = keep the loan.

The function, verbatim from the source (starts at line 160)

```
160 int wantsToRepayLoan(GameState game[], int playerIndex)
161 {
162     int cash;
163     int loanAmount;

164
165     cash = game[0].players[playerIndex].cash;
166     loanAmount = game[0].players[playerIndex].loan.amount;

167
168     switch(game[0].players[playerIndex].strategy)
169     {
170         case AGGRESSIVE_INVESTOR:
171             return (cash >= 2 * loanAmount);

172
173         case CONSERVATIVE_BANKER:
174             return (cash >= loanAmount);

175
176         case RISK_TAKER:
177             /* rarely repays - prefers to keep using the cash */
178             return (cash >= 5 * loanAmount);

179
180         case OPPORTUNISTIC_TRADER:
181             return (cash >= (loanAmount * 3) / 2);

182
183         default:
184             return 0;
185     }
186 }
```

Line-by-line explanation

Line	Code	Explanation
160	int wantsToRepayLoan(GameState game[], int playerIndex)	int wantsToRepayLoan(GameState game[], int playerIndex)
161	{	Opens a new block.
162	int cash;	Declares int cash.
163	int loanAmount;	Declares int loanAmount.
164		Blank line.
165	cash = game[0].players[playerIndex].cash;	cash = game[0].players[playerIndex].cash;
166	loanAmount = game[0].players[playerIndex].loan.amount;	loanAmount = game[0].players[playerIndex].loan.amount;
167		Blank line.
168	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
169	{	Opens a new block.
170	case AGGRESSIVE_INVESTOR:	This case of the switch runs when its constant matches.
171	return (cash >= 2 * loanAmount);	Returns (cash >= 2 * loanAmount) immediately.
172		Blank line.
173	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.

174	return (cash >= loanAmount);	Returns (cash >= loanAmount) immediately.
175		Blank line.
176	case RISK_TAKER:	This case of the switch runs when its constant matches.
177	/* rarely repays - prefers to keep using the cash */	Comment: /* rarely repays - prefers to keep using the cash */
178	return (cash >= 5 * loanAmount);	Returns (cash >= 5 * loanAmount) immediately.
179		Blank line.
180	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
181	return (cash >= (loanAmount * 3) / 2);	Returns (cash >= (loanAmount * 3) / 2) immediately.
182		Blank line.
183	default:	The fallback switch branch when no case matched.
184	return 0;	Returns 0 immediately.
185	}	Closes the current block.
186	}	Closes the current block.

Nested functions inside wantsToRepayLoan() - study these

- wantsToRepayLoan() (Whether the player repays the loan (fully) during a Bank visit.)

MAIN PURPOSE

Whether the player repays the loan (fully) during a Bank visit.

4.6 wantsIncreaseLoan()

```
int wantsIncreaseLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *1 = increase the loan, 0 = don't.*

The function, verbatim from the source (starts at line 187)

```
188  /* Rule-LK 5: whether a player wants to borrow more against new
189     collateral during a Bank visit. */
190  int wantsIncreaseLoan(GameState game[], int playerIndex)
191  {
192      int cash;

194      cash = game[0].players[playerIndex].cash;

196      switch(game[0].players[playerIndex].strategy)
197      {
198          case AGGRESSIVE_INVESTOR:
199              return 1;

201          case CONSERVATIVE_BANKER:
202              return (cash < COMFORTABLE_CASH);

204          case RISK_TAKER:
205              return 1;

207          case OPPORTUNISTIC_TRADER:
208              return (cash < RICH_CASH_LINE);

210          default:
211              return 0;
212      }
213  }
```

Line-by-line explanation

Line	Code	Explanation
187		Blank line.
188	/* Rule-LK 5: whether a player wants to borrow more against new	Comment: /* Rule-LK 5: whether a player wants to borrow more against new
189	collateral during a Bank visit. */	collateral during a Bank visit. */
190	int wantsIncreaseLoan(GameState game[], int playerIndex)	int wantsIncreaseLoan(GameState game[], int playerIndex)
191	{	Opens a new block.

192	int cash;	Declares int cash.
193		Blank line.
194	cash = game[0].players[playerIndex].cash;	cash = game[0].players[playerIndex].cash;
195		Blank line.
196	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
197	{	Opens a new block.
198	case AGGRESSIVE_INVESTOR:	This case of the switch runs when its constant matches.
199	return 1;	Returns 1 immediately.
200		Blank line.
201	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.
202	return (cash < COMFORTABLE_CASH);	Returns (cash < COMFORTABLE_CASH) immediately.
203		Blank line.
204	case RISK_TAKER:	This case of the switch runs when its constant matches.
205	return 1;	Returns 1 immediately.
206		Blank line.
207	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
208	return (cash < RICH_CASH_LINE);	Returns (cash < RICH_CASH_LINE) immediately.
209		Blank line.
210	default:	The fallback switch branch when no case matched.
211	return 0;	Returns 0 immediately.
212	}	Closes the current block.
213	}	Closes the current block.

Nested functions inside wantsIncreaseLoan() - study these

- wantsIncreaseLoan() (Rule-LK 5: borrow more against newly acquired collateral during a Bank visit.)

MAIN PURPOSE

Rule-LK 5: borrow more against newly acquired collateral during a Bank visit.

4.7 wantsExtendLoan()

```
int wantsExtendLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *1 = extend, 0 = don't.*

The function, verbatim from the source (starts at line 214)

```

215  /* Rule-LK 5: whether a player wants to extend the loan term. */
216  int wantsExtendLoan(GameState game[], int playerIndex)
217  {
218      int remainingRounds;
219      int cash;
220      int loanAmount;

222      remainingRounds = game[0].players[playerIndex].loan.remainingRounds;
223      cash = game[0].players[playerIndex].cash;
224      loanAmount = game[0].players[playerIndex].loan.amount;

226      switch(game[0].players[playerIndex].strategy)
227      {
228          case AGGRESSIVE_INVESTOR:
229              /* prefers to borrow more rather than extend */
230              return 0;

232          case CONSERVATIVE_BANKER:
233              return (remainingRounds <= 5 && cash < loanAmount);

235          case RISK_TAKER:
236              return (remainingRounds <= 10);

238          case OPPORTUNISTIC_TRADER:
239              return (remainingRounds <= 5);

241          default:

```

```

242         return 0;
243     }
244 }

```

Line-by-line explanation

Line	Code	Explanation
214		Blank line.
215	<code>/* Rule-LK 5: whether a player wants to extend the loan term. */</code>	Comment: <code>/* Rule-LK 5: whether a player wants to extend the loan term. */</code>
216	<code>int wantsExtendLoan(GameState game[], int playerIndex)</code>	<code>int wantsExtendLoan(GameState game[], int playerIndex)</code>
217	<code>{</code>	Opens a new block.
218	<code>int remainingRounds;</code>	Declares <code>int remainingRounds</code> .
219	<code>int cash;</code>	Declares <code>int cash</code> .
220	<code>int loanAmount;</code>	Declares <code>int loanAmount</code> .
221		Blank line.
222	<code>remainingRounds = game[0].players[playerIndex].loan.remainingRounds;</code>	<code>remainingRounds = game[0].players[playerIndex].loan.remainingRounds;</code>
223	<code>cash = game[0].players[playerIndex].cash;</code>	<code>cash = game[0].players[playerIndex].cash;</code>
224	<code>loanAmount = game[0].players[playerIndex].loan.amount;</code>	<code>loanAmount = game[0].players[playerIndex].loan.amount;</code>
225		Blank line.
226	<code>switch(game[0].players[playerIndex].strategy)</code>	Chooses a branch based on the value of the current player.strategy.
227	<code>{</code>	Opens a new block.
228	<code>case AGGRESSIVE_INVESTOR:</code>	This case of the switch runs when its constant matches.
229	<code>/* prefers to borrow more rather than extend */</code>	Comment: <code>/* prefers to borrow more rather than extend */</code>
230	<code>return 0;</code>	Returns 0 immediately.
231		Blank line.
232	<code>case CONSERVATIVE_BANKER:</code>	This case of the switch runs when its constant matches.
233	<code>return (remainingRounds <= 5 && cash < loanAmount);</code>	Returns <code>(remainingRounds <= 5 && cash < loanAmount)</code> immediately.
234		Blank line.
235	<code>case RISK_TAKER:</code>	This case of the switch runs when its constant matches.
236	<code>return (remainingRounds <= 10);</code>	Returns <code>(remainingRounds <= 10)</code> immediately.
237		Blank line.
238	<code>case OPPORTUNISTIC_TRADER:</code>	This case of the switch runs when its constant matches.
239	<code>return (remainingRounds <= 5);</code>	Returns <code>(remainingRounds <= 5)</code> immediately.
240		Blank line.
241	<code>default:</code>	The fallback switch branch when no case matched.
242	<code>return 0;</code>	Returns 0 immediately.
243	<code>}</code>	Closes the current block.
244	<code>}</code>	Closes the current block.

Nested functions inside `wantsExtendLoan()` - study these

- `wantsExtendLoan()` (Rule-LK 5: extend the loan term by 10 rounds.)

MAIN PURPOSE

Rule-LK 5: extend the loan term by 10 rounds.

4.8 `wantsRefinance()`

```
int wantsRefinance(GameState game[], int playerIndex)
```

Parameters: `playerIndex` | Returns: 1 = refinance, 0 = don't.

The function, verbatim from the source (starts at line 245)

```

246     /* Rule-LK 5: whether a player wants to refinance at the current rate
247        (only makes sense when the current economy rate is lower). */
248     int wantsRefinance(GameState game[], int playerIndex)
249     {

```

```

250         return (game[0].economy.loanInterestRate <
251                 game[0].players[playerIndex].loan.interestRate);
252     }

```

Line-by-line explanation

Line	Code	Explanation
245		Blank line.
246	<code>/* Rule-LK 5: whether a player wants to refinance at the current rate</code>	Comment: <code>/* Rule-LK 5: whether a player wants to refinance at the current rate</code>
247	<code>(only makes sense when the current economy rate is lower). */</code>	<code>(only makes sense when the current economy rate is lower). */</code>
248	<code>int wantsRefinance(GameState game[], int playerIndex)</code>	<code>int wantsRefinance(GameState game[], int playerIndex)</code>
249	<code>{</code>	Opens a new block.
250	<code>return (game[0].economy.loanInterestRate <</code>	Returns <code>(game[0].economy.loanInterestRate <</code> immediately.
251	<code>game[0].players[playerIndex].loan.interestRate);</code>	<code>game[0].players[playerIndex].loan.interestRate);</code>
252	<code>}</code>	Closes the current block.

Nested functions inside wantsRefinance() - study these

- wantsRefinance() (Rule-LK 5: refinance at the economy's current (lower) rate.)

MAIN PURPOSE

Rule-LK 5: refinance at the economy's current (lower) rate.

4.9 desiredInsurance()

```
InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
```

Parameters: *playerIndex, propIndex* | Returns: *InsuranceType* for the policy.

The function, verbatim from the source (starts at line 253)

```

254     /* Which insurance policy (if any) a player wants for a property. */
255     InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
256     {
257         int hasHotel;
258         int purchasePrice;
259
260         hasHotel = game[0].board[propIndex].property.hotel;
261         purchasePrice = game[0].board[propIndex].property.purchasePrice;
262
263         switch(game[0].players[playerIndex].strategy)
264         {
265             case AGGRESSIVE_INVESTOR:
266                 if(hasHotel)
267                     return COMPREHENSIVE_INSURANCE;
268                 return BASIC_INSURANCE;
269
270             case CONSERVATIVE_BANKER:
271                 return COMPREHENSIVE_INSURANCE;
272
273             case RISK_TAKER:
274                 /* only insures after already suffering a loss */
275                 if(!game[0].players[playerIndex].sufferedLoss)
276                     return NO_INSURANCE;
277                 if(hasHotel)
278                     return BUSINESS_INTERRUPTION;
279                 return BASIC_INSURANCE;
280
281             case OPPORTUNISTIC_TRADER:
282                 if(purchasePrice >= EXPENSIVE_PROP_PRICE)
283                     return COMPREHENSIVE_INSURANCE;
284                 return NO_INSURANCE;
285
286             default:

```

```

287         return NO_INSURANCE;
288     }
289 }

```

Line-by-line explanation

Line	Code	Explanation
253		Blank line.
254	<code>/* Which insurance policy (if any) a player wants for a property. */</code>	Comment: <code>/* Which insurance policy (if any) a player wants for a property. */</code>
255	<code>InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)</code>	<code>InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)</code>
256	<code>{</code>	Opens a new block.
257	<code>int hasHotel;</code>	Declares <code>int hasHotel</code> .
258	<code>int purchasePrice;</code>	Declares <code>int purchasePrice</code> .
259		Blank line.
260	<code>hasHotel = game[0].board[propIndex].property.hotel;</code>	<code>hasHotel = game[0].board[propIndex].property.hotel;</code>
261	<code>purchasePrice = game[0].board[propIndex].property.purchasePrice;</code>	<code>purchasePrice = game[0].board[propIndex].property.purchasePrice;</code>
262		Blank line.
263	<code>switch(game[0].players[playerIndex].strategy)</code>	Chooses a branch based on the value of the current <code>player.strategy</code> .
264	<code>{</code>	Opens a new block.
265	<code>case AGGRESSIVE_INVESTOR:</code>	This case of the switch runs when its constant matches.
266	<code>if(hasHotel)</code>	Condition: <code>hasHotel</code> -> run the next block if true.
267	<code>return COMPREHENSIVE_INSURANCE;</code>	Returns <code>COMPREHENSIVE_INSURANCE</code> immediately.
268	<code>return BASIC_INSURANCE;</code>	Returns <code>BASIC_INSURANCE</code> immediately.
269		Blank line.
270	<code>case CONSERVATIVE_BANKER:</code>	This case of the switch runs when its constant matches.
271	<code>return COMPREHENSIVE_INSURANCE;</code>	Returns <code>COMPREHENSIVE_INSURANCE</code> immediately.
272		Blank line.
273	<code>case RISK_TAKER:</code>	This case of the switch runs when its constant matches.
274	<code>/* only insures after already suffering a loss */</code>	Comment: <code>/* only insures after already suffering a loss */</code>
275	<code>if(!game[0].players[playerIndex].sufferedLoss)</code>	Condition: <code>!the current player.sufferedLoss</code> -> run the next block if true.
276	<code>return NO_INSURANCE;</code>	Returns <code>NO_INSURANCE</code> immediately.
277	<code>if(hasHotel)</code>	Condition: <code>hasHotel</code> -> run the next block if true.
278	<code>return BUSINESS_INTERRUPTION;</code>	Returns <code>BUSINESS_INTERRUPTION</code> immediately.
279	<code>return BASIC_INSURANCE;</code>	Returns <code>BASIC_INSURANCE</code> immediately.
280		Blank line.
281	<code>case OPPORTUNISTIC_TRADER:</code>	This case of the switch runs when its constant matches.
282	<code>if(purchasePrice >= EXPENSIVE_PROP_PRICE)</code>	Condition: <code>purchasePrice >= EXPENSIVE_PROP_PRICE</code> -> run the next block if true.
283	<code>return COMPREHENSIVE_INSURANCE;</code>	Returns <code>COMPREHENSIVE_INSURANCE</code> immediately.
284	<code>return NO_INSURANCE;</code>	Returns <code>NO_INSURANCE</code> immediately.
285		Blank line.
286	<code>default:</code>	The fallback switch branch when no case matched.
287	<code>return NO_INSURANCE;</code>	Returns <code>NO_INSURANCE</code> immediately.
288	<code>}</code>	Closes the current block.
289	<code>}</code>	Closes the current block.

Nested functions inside `desiredInsurance()` - study these

- `desiredInsurance()` (Returns the policy type the player wants for a given property.)

MAIN PURPOSE

Returns the policy type the player wants for a given property.

4.10 `shouldRenovateAgeDepreciation()`

```

int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciatio
nPercent)

```

Parameters: *playerIndex*, *depreciationPercent* | Returns: 1 = renovate, 0 = don't.

The function, verbatim from the source (starts at line 290)

```
291  /* Whether a player will pay to renovate away age depreciation. */
292  int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
293  {
294      switch(game[0].players[playerIndex].strategy)
295      {
296          case CONSERVATIVE_BANKER:
297              return (depreciationPercent >= 10);
298
299          case OPPORTUNISTIC_TRADER:
300              return (depreciationPercent >= 15);
301
302          default:
303              /* not specified for these two strategies - reasonable default */
304              return (depreciationPercent >= 20);
305      }
306  }
```

Line-by-line explanation

Line	Code	Explanation
290		Blank line.
291	/* Whether a player will pay to renovate away age depreciation. */	Comment: /* Whether a player will pay to renovate away age depreciation. */
292	int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)	int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
293	{	Opens a new block.
294	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
295	{	Opens a new block.
296	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.
297	return (depreciationPercent >= 10);	Returns (depreciationPercent >= 10) immediately.
298		Blank line.
299	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
300	return (depreciationPercent >= 15);	Returns (depreciationPercent >= 15) immediately.
301		Blank line.
302	default:	The fallback switch branch when no case matched.
303	/* not specified for these two strategies - reasonable default */	Comment: /* not specified for these two strategies - reasonable default */
304	return (depreciationPercent >= 20);	Returns (depreciationPercent >= 20) immediately.
305	}	Closes the current block.
306	}	Closes the current block.

Nested functions inside shouldRenovateAgeDepreciation() - study these

- shouldRenovateAgeDepreciation() (Whether the player pays to renovate away age depreciation.)

MAIN PURPOSE

Whether the player pays to renovate away age depreciation.

4.11 shouldMaintain()

```
int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
```

Parameters: *playerIndex*, *condition*, *cost* | Returns: 1 = maintain, 0 = don't.

The function, verbatim from the source (starts at line 308)

```
308  int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
309  {
310      int cash;
311
312      cash = game[0].players[playerIndex].cash;
```

```

314     if(game[0].players[playerIndex].strategy == RISK_TAKER)
315     {
316         /* ignores depreciation until repair becomes unavoidable */
317         return (condition < 25 && cash >= cost);
318     }

320     return (cash - cost >= cash * 20 / 100);
321 }

```

Line-by-line explanation

Line	Code	Explanation
308	int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)	int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
309	{	Opens a new block.
310	int cash;	Declares int cash.
311		Blank line.
312	cash = game[0].players[playerIndex].cash;	cash = game[0].players[playerIndex].cash;
313		Blank line.
314	if(game[0].players[playerIndex].strategy == RISK_TAKER)	Condition: the current player.strategy == RISK_TAKER -> run the next block if true.
315	{	Opens a new block.
316	/* ignores depreciation until repair becomes unavoidable */	Comment: /* ignores depreciation until repair becomes unavoidable */
317	return (condition < 25 && cash >= cost);	Returns (condition < 25 && cash >= cost) immediately.
318	}	Closes the current block.
319		Blank line.
320	return (cash - cost >= cash * 20 / 100);	Returns (cash - cost >= cash * 20 / 100) immediately.
321	}	Closes the current block.

Nested functions inside shouldMaintain() - study these

- shouldMaintain() (Whether the player performs maintenance on a building.)

MAIN PURPOSE

Whether the player performs maintenance on a building.

4.12 willingToBid()

```
int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
```

Parameters: playerIndex, propIndex, candidateBid | Returns: 1 = bid, 0 = withdraw from this auction for good.

The function, verbatim from the source (starts at line 323)

```

323 int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
324 {
325     int marketValue;

327     if(game[0].players[playerIndex].cash < candidateBid)
328         return 0;

330     marketValue = currentMarketValue(game, propIndex);

332     switch(game[0].players[playerIndex].strategy)
333     {
334         case AGGRESSIVE_INVESTOR:
335             return (candidateBid <= (marketValue * 120) / 100);

337         case CONSERVATIVE_BANKER:
338             return (candidateBid < marketValue);

340         case RISK_TAKER:
341             return 1;

343         case OPPORTUNISTIC_TRADER:
344             return (candidateBid <= (marketValue * 80) / 100);

```

```

346         default:
347             return 0;
348     }
349 }

```

Line-by-line explanation

Line	Code	Explanation
323	int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)	int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
324	{	Opens a new block.
325	int marketValue;	Declares int marketValue.
326		Blank line.
327	if(game[0].players[playerIndex].cash < candidateBid)	Condition: the current player.cash < candidateBid -> run the next block if true.
328	return 0;	Returns 0 immediately.
329		Blank line.
330	marketValue = currentMarketValue(game, propIndex);	marketValue = currentMarketValue(game, propIndex);
331		Blank line.
332	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
333	{	Opens a new block.
334	case AGGRESSIVE_INVESTOR:	This case of the switch runs when its constant matches.
335	return (candidateBid <= (marketValue * 120) / 100);	Returns (candidateBid <= (marketValue * 120) / 100) immediately.
336		Blank line.
337	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.
338	return (candidateBid < marketValue);	Returns (candidateBid < marketValue) immediately.
339		Blank line.
340	case RISK_TAKER:	This case of the switch runs when its constant matches.
341	return 1;	Returns 1 immediately.
342		Blank line.
343	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
344	return (candidateBid <= (marketValue * 80) / 100);	Returns (candidateBid <= (marketValue * 80) / 100) immediately.
345		Blank line.
346	default:	The fallback switch branch when no case matched.
347	return 0;	Returns 0 immediately.
348	}	Closes the current block.
349	}	Closes the current block.

Nested functions inside willingToBid() - study these

- willingToBid() (Whether the player raises the bid at an auction.)
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)

MAIN PURPOSE

Whether the player raises the bid at an auction.

4.13 shouldMortgage()

```
int shouldMortgage(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: 1 = mortgage, 0 = don't.

The function, verbatim from the source (starts at line 351)

```

351 int shouldMortgage(GameState game[], int playerIndex)
352 {
353     int cash;

355     cash = game[0].players[playerIndex].cash;

357     switch(game[0].players[playerIndex].strategy)
358     {
359         case CONSERVATIVE_BANKER:

```

```

360         return (cash < COMFORTABLE_CASH);

362     case RISK_TAKER:
363         return (cash < MIN_CASH_FLOOR);

365     case AGGRESSIVE_INVESTOR:
366         return (cash < MIN_CASH_SAFETY);

368     case OPPORTUNISTIC_TRADER:
369         return (cash < TRADER_CASH_FLOOR);

371     default:
372         return 0;
373 }
374 }

```

Line-by-line explanation

Line	Code	Explanation
351	int shouldMortgage(GameState game[], int playerIndex)	int shouldMortgage(GameState game[], int playerIndex)
352	{	Opens a new block.
353	int cash;	Declares int cash.
354		Blank line.
355	cash = game[0].players[playerIndex].cash;	cash = game[0].players[playerIndex].cash;
356		Blank line.
357	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
358	{	Opens a new block.
359	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.
360	return (cash < COMFORTABLE_CASH);	Returns (cash < COMFORTABLE_CASH) immediately.
361		Blank line.
362	case RISK_TAKER:	This case of the switch runs when its constant matches.
363	return (cash < MIN_CASH_FLOOR);	Returns (cash < MIN_CASH_FLOOR) immediately.
364		Blank line.
365	case AGGRESSIVE_INVESTOR:	This case of the switch runs when its constant matches.
366	return (cash < MIN_CASH_SAFETY);	Returns (cash < MIN_CASH_SAFETY) immediately.
367		Blank line.
368	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
369	return (cash < TRADER_CASH_FLOOR);	Returns (cash < TRADER_CASH_FLOOR) immediately.
370		Blank line.
371	default:	The fallback switch branch when no case matched.
372	return 0;	Returns 0 immediately.
373	}	Closes the current block.
374	}	Closes the current block.

Nested functions inside shouldMortgage() - study these

- shouldMortgage() (Whether the player mortgages a property.)

MAIN PURPOSE

Whether the player mortgages a property.

4.14 shouldRedeemMortgage()

```
int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
```

Parameters: playerIndex, redeemCost | Returns: 1 = redeem, 0 = don't.

The function, verbatim from the source (starts at line 376)

```

376 int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
377 {
378     return (game[0].players[playerIndex].cash - redeemCost >= redeemCost);
379 }

```

Line-by-line explanation

Line	Code	Explanation
376	<code>int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)</code>	<code>int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)</code>
377	<code>{</code>	Opens a new block.
378	<code>return (game[0].players[playerIndex].cash - redeemCost >= redeemCost);</code>	Returns <code>(game[0].players[playerIndex].cash - redeemCost >= redeemCost)</code> immediately.
379	<code>}</code>	Closes the current block.

Nested functions inside `shouldRedeemMortgage()` - study these

- `shouldRedeemMortgage()` (Whether the player redeems a mortgaged property.)

MAIN PURPOSE

Whether the player redeems a mortgaged property.

4.15 `shouldPayBail()`

```
int shouldPayBail(GameState game[], int playerIndex)
```

Parameters: `playerIndex` | Returns: 1 = pay bail, 0 = stay in Jail and keep rolling.

The function, verbatim from the source (starts at line 380)

```
381  /* Whether a player pays bail after failing to roll doubles.
382     The assignment doesn't specify which strategy prefers which
383     option, so this is a reasonable assumption per strategy. */
384  int shouldPayBail(GameState game[], int playerIndex)
385  {
386      int cash;

388      cash = game[0].players[playerIndex].cash;

390      switch(game[0].players[playerIndex].strategy)
391      {
392          case CONSERVATIVE_BANKER:
393              return (cash >= BAIL_CASH_SPARE);

395          case RISK_TAKER:
396              return 0;

398          case AGGRESSIVE_INVESTOR:
399          case OPPORTUNISTIC_TRADER:
400              return (cash >= RICH_CASH_LINE);

402          default:
403              return 0;
404      }
405  }
```

Line-by-line explanation

Line	Code	Explanation
380		Blank line.
381	<code>/* Whether a player pays bail after failing to roll doubles.</code>	Comment: <code>/* Whether a player pays bail after failing to roll doubles.</code>
382	<code>The assignment doesn't specify which strategy prefers which</code>	The assignment doesn't specify which strategy prefers which
383	<code>option, so this is a reasonable assumption per strategy. */</code>	option, so this is a reasonable assumption per strategy. <code>*/</code>
384	<code>int shouldPayBail(GameState game[], int playerIndex)</code>	<code>int shouldPayBail(GameState game[], int playerIndex)</code>
385	<code>{</code>	Opens a new block.
386	<code>int cash;</code>	Declares <code>int cash</code> .
387		Blank line.
388	<code>cash = game[0].players[playerIndex].cash;</code>	<code>cash = game[0].players[playerIndex].cash;</code>

389		Blank line.
390	switch(game[0].players[playerIndex].strategy)	Chooses a branch based on the value of the current player.strategy.
391	{	Opens a new block.
392	case CONSERVATIVE_BANKER:	This case of the switch runs when its constant matches.
393	return (cash >= BAIL_CASH_SPARE);	Returns (cash >= BAIL_CASH_SPARE) immediately.
394		Blank line.
395	case RISK_TAKER:	This case of the switch runs when its constant matches.
396	return 0;	Returns 0 immediately.
397		Blank line.
398	case AGGRESSIVE_INVESTOR:	This case of the switch runs when its constant matches.
399	case OPPORTUNISTIC_TRADER:	This case of the switch runs when its constant matches.
400	return (cash >= RICH_CASH_LINE);	Returns (cash >= RICH_CASH_LINE) immediately.
401		Blank line.
402	default:	The fallback switch branch when no case matched.
403	return 0;	Returns 0 immediately.
404	}	Closes the current block.
405	}	Closes the current block.

Nested functions inside shouldPayBail() - study these

- shouldPayBail() (Whether the player pays bail after failing to roll doubles.)

MAIN PURPOSE

Whether the player pays bail after failing to roll doubles.

Part 5 game.c

The game engine: turn loop, dice, movement, jail, round flow, buying, mortgage decisions, building, selling, anti-speculation, winner detection.

The sub-topics below are the 14 functions implemented in game.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in game.c

5.1	startGame()	(source line 615)	p. 49
5.2	playGame()	(source line 404)	p. 50
5.3	playTurn()	(source line 123)	p. 55
5.4	movePlayer()	(source line 15)	p. 59
5.5	rollDice()	(source line 8)	p. 61
5.6	handleJail()	(source line 51)	p. 61
5.7	resolveGroup()	(source line 209)	p. 64
5.8	determineTurnOrder()	(source line 287)	p. 67
5.9	countHouses()	(source line 306)	p. 68
5.10	countHotels()	(source line 326)	p. 69
5.11	displayRoundSummary()	(source line 346)	p. 70
5.12	countSolventPlayers()	(source line 389)	p. 72
5.13	determineWinner()	(source line 510)	p. 72
5.14	displayFinalResults()	(source line 555)	p. 74

5.1 startGame()

```
void startGame(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 615)

```
615 void startGame(GameState game[])
616 {
617     int turnOrder[MAX_PLAYERS];
618     char moneyBuffer[32];
619
620     srand((unsigned)time(NULL));
621
622     initializeBoard(game);
623     initializePlayers(game);
624     initEconomy(game);
625     initMarket(game);
626
627     printf("Player 1 : %s\n", game[0].players[0].name);
628     printf("Player 2 : %s\n", game[0].players[1].name);
629     printf("Player 3 : %s\n", game[0].players[2].name);
630     printf("Player 4 : %s\n", game[0].players[3].name);
631
632     formatLKR(START_MONEY, moneyBuffer);
633     printf("\nEach player begins with LKR %s.\n", moneyBuffer);
634
635     determineTurnOrder(game, turnOrder);
636
637     playGame(game, turnOrder);
638
639     displayFinalResults(game);
640 }
```

Line-by-line explanation

Line	Code	Explanation
615	void startGame(GameState game[])	void startGame(GameState game[])
616	{	Opens a new block.
617	int turnOrder[MAX_PLAYERS];	Declares int turnOrder[MAX_PLAYERS].

618	char moneyBuffer[32];	Declares char moneyBuffer[32].
619		Blank line.
620	srand((unsigned)time(NULL));	Calls srand((unsigned)time(NULL)).
621		Blank line.
622	initializeBoard(game);	Calls initializeBoard(game). (Builds the complete 40-square board: special squares, 4 railways, 2 utilities and every colour property, using the helper setters)
623	initializePlayers(game);	Calls initializePlayers(game). (Names the four players and gives their starting state)
624	initEconomy(game);	Calls initEconomy(game). (Resets all economy state: zero inflation, base rates, empty modifier list)
625	initMarket(game);	Calls initMarket(game). (Resets market state: all group cooldowns 0, no last boom/decline)
626		Blank line.
627	printf("Player 1 : %s\n", game[0].players[0].name);	Prints the message "Player 1 : %s\n".
628	printf("Player 2 : %s\n", game[0].players[1].name);	Prints the message "Player 2 : %s\n".
629	printf("Player 3 : %s\n", game[0].players[2].name);	Prints the message "Player 3 : %s\n".
630	printf("Player 4 : %s\n", game[0].players[3].name);	Prints the message "Player 4 : %s\n".
631		Blank line.
632	formatLKR(START_MONEY, moneyBuffer);	Calls formatLKR(START_MONEY, moneyBuffer). (Formats an amount with thousands separators, e)
633	printf("\nEach player begins with LKR %s.\n", moneyBuffer);	Prints the message "\nEach player begins with LKR %s.\n".
634		Blank line.
635	determineTurnOrder(game, turnOrder);	Calls determineTurnOrder(game, turnOrder). (Finds who plays first and the full order of play)
636		Blank line.
637	playGame(game, turnOrder);	Calls playGame(game, turnOrder). (Master loop)
638		Blank line.
639	displayFinalResults(game);	Calls displayFinalResults(game). (Prints the winner's cash/property/loan/net worth and every player's final standing)
640	}	Closes the current block.

Nested functions inside startGame() - study these

- startGame() (Entry point driven by main.c: seeds RNG, initialises everything, runs the game and prints)
- initializeBoard() (Builds the complete 40-square board: special squares, 4 railways, 2 utilities and every co)
- initializePlayers() (Names the four players and gives their starting state.)
- initEconomy() (Resets all economy state: zero inflation, base rates, empty modifier list.)
- initMarket() (Resets market state: all group cooldowns 0, no last boom/decline.)
- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)
- determineTurnOrder() (Finds who plays first and the full order of play.)
- playGame() (Master loop. A round completes only when EVERY solvent player has passed GO at least once)
- displayFinalResults() (Prints the winner's cash/property/loan/net worth and every player's final standing.)

MAIN PURPOSE

Entry point driven by main.c: seeds RNG, initialises everything, runs the game and prints results.

5.2 playGame()

```
void playGame(GameState game[], int turnOrder[])
```

Parameters: turnOrder | Returns: void.

The function, verbatim from the source (starts at line 404)

```
405  /* A round is not "each player took one turn" - it only finishes
406     once every player still in the game has passed GO at least once.
407     Since players move different distances each turn, this usually
408     takes several turns per player, not one - so this loop keeps
```

```

409     cycling through turnOrder until everyone has crossed GO. */
410 void playGame(GameState game[], int turnOrder[])
411 {
412     int round;
413     int turnPointer;
414     int playerIndex;
415     int passedGoThisRound[MAX_PLAYERS];
416     int i;
417     int allPassed;
418     int passedGo;

420     round = 1;
421     turnPointer = 0;

423     for(i = 0; i < MAX_PLAYERS; i++)
424         passedGoThisRound[i] = 0;

426     printf("\n=====\\n");
427     printf("ROUND %d\\n", round);
428     printf("=====\\n");

430     while(round <= MAX_ROUNDS)
431     {
432         playerIndex = turnOrder[turnPointer];

434         if(!game[0].players[playerIndex].bankrupt)
435         {
436             passedGo = playTurn(game, playerIndex);

438             if(passedGo)
439                 passedGoThisRound[playerIndex] = 1;
440         }

442         turnPointer = (turnPointer + 1) % MAX_PLAYERS;

444         /* has every player still playing passed GO this round? */
445         allPassed = 1;

447         for(i = 0; i < MAX_PLAYERS; i++)
448         {
449             if(game[0].players[i].bankrupt)
450                 continue;

452             if(!passedGoThisRound[i])
453             {
454                 allPassed = 0;
455                 break;
456             }
457         }

459         if(!allPassed)
460             continue;

462         processLoans(game);
463         processInsuranceExpiry(game);
464         ageProperties(game);
465         ageBuildings(game);
466         decrementEventTimers(game);
467         decrementModifiers(game);
468         enforceAntiSpeculation(game);

470         if(round % ECONOMY_CYCLE == 0)
471         {
472             triggerDisaster(game);
473             applyInflation(game);
474             reviewPropertyMarket(game, round);
475         }

477         if(round % EVENT_CYCLE == 0)

```

```

478         {
479             triggerEconomicEvent(game);
480             drawRegionalCard(game);
481         }

483         if(round % REGULATION_CYCLE == 0)
484         {
485             triggerGovernmentRegulation(game);
486         }

488         displayRoundSummary(game, round);
489         displayMarketConditions(game);

491         if(countSolventPlayers(game) <= 1)
492         {
493             printf("\nOnly one solvent player remains. Ending game.\n");
494             break;
495         }

497         round++;

499         if(round <= MAX_ROUNDS)
500         {
501             printf("\n=====\\n");
502             printf("ROUND %d\\n", round);
503             printf("=====\\n");
504         }

506         for(i = 0; i < MAX_PLAYERS; i++)
507             passedGoThisRound[i] = 0;
508     }
509 }

```

Line-by-line explanation

Line	Code	Explanation
404		Blank line.
405	/* A round is not "each player took one turn" - it only finishes	Comment: /* A round is not "each player took one turn" - it only finishes
406	once every player still in the game has passed GO at least once.	once every player still in the game has passed GO at least once.
407	Since players move different distances each turn, this usually	Since players move different distances each turn, this usually
408	takes several turns per player, not one - so this loop keeps	takes several turns per player, not one - so this loop keeps
409	cycling through turnOrder until everyone has crossed GO. */	cycling through turnOrder until everyone has crossed GO. */
410	void playGame(GameState game[], int turnOrder[])	void playGame(GameState game[], int turnOrder[])
411	{	Opens a new block.
412	int round;	Declares int round.
413	int turnPointer;	Declares int turnPointer.
414	int playerIndex;	Declares int playerIndex.
415	int passedGoThisRound[MAX_PLAYERS];	Declares int passedGoThisRound[MAX_PLAYERS].
416	int i;	Declares int i.
417	int allPassed;	Declares int allPassed.
418	int passedGo;	Declares int passedGo.
419		Blank line.
420	round = 1;	round = 1;
421	turnPointer = 0;	turnPointer = 0;
422		Blank line.
423	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
424	passedGoThisRound[i] = 0;	passedGoThisRound[i] = 0;
425		Blank line.

426	printf("\n=====\\n");	Prints the message "\\n=====\\n".
427	printf("ROUND %d\\n", round);	Prints the message "ROUND %d\\n".
428	printf("=====\\n");	Prints the message "=====\\n".
429		Blank line.
430	while(round <= MAX_ROUNDS)	Loop: repeat while round <= MAX_ROUNDS is true.
431	{	Opens a new block.
432	playerIndex = turnOrder[turnPointer];	playerIndex = turnOrder[turnPointer];
433		Blank line.
434	if(!game[0].players[playerIndex].bankrupt)	Condition: !the current player.bankrupt -> run the next block if true.
435	{	Opens a new block.
436	passedGo = playTurn(game, playerIndex);	passedGo = playTurn(game, playerIndex);
437		Blank line.
438	if(passedGo)	Condition: passedGo -> run the next block if true.
439	passedGoThisRound[playerIndex] = 1;	passedGoThisRound[playerIndex] = 1;
440	}	Closes the current block.
441		Blank line.
442	turnPointer = (turnPointer + 1) % MAX_PLAYERS;	turnPointer = (turnPointer + 1) % MAX_PLAYERS;
443		Blank line.
444	/* has every player still playing passed GO this round? */	Comment: /* has every player still playing passed GO this round? */
445	allPassed = 1;	allPassed = 1;
446		Blank line.
447	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
448	{	Opens a new block.
449	if(game[0].players[i].bankrupt)	Condition: player i.bankrupt -> run the next block if true.
450	continue;	Skips to the next loop iteration.
451		Blank line.
452	if(!passedGoThisRound[i])	Condition: !passedGoThisRound[i] -> run the next block if true.
453	{	Opens a new block.
454	allPassed = 0;	allPassed = 0;
455	break;	Breaks out of the current loop or switch.
456	}	Closes the current block.
457	}	Closes the current block.
458		Blank line.
459	if(!allPassed)	Condition: !allPassed -> run the next block if true.
460	continue;	Skips to the next loop iteration.
461		Blank line.
462	processLoans(game);	Calls processLoans(game). (Round-end loan heartbeat: compounds interest into the principal and counts down; at 0 rounds, forecloses)
463	processInsuranceExpiry(game);	Calls processInsuranceExpiry(game). (Counts every policy down each round; warns at 3 rounds and expires (removes) at 0)
464	ageProperties(game);	Calls ageProperties(game). (Rule-LK 15/16: ages every property; past age 50 depreciation ticks +5% every 5 rounds, capped at 30%, printing when it changes)
465	ageBuildings(game);	Calls ageBuildings(game). (Rule-LK 25/28: condition drops 2 per round; lost-income countdown runs; 20+ rounds without maintenance causes structural damage (-15% value, -25% rent, 150% repairs))
466	decrementEventTimers(game);	Calls decrementEventTimers(game). (Ticks the non-modifier event timers down: construction suspension and the closed property)
467	decrementModifiers(game);	Calls decrementModifiers(game). (Ticks every active modifier down and compacts out expired ones)
468	enforceAntiSpeculation(game);	Calls enforceAntiSpeculation(game). (Rule-LK 8: once active, force-auctions excess undeveloped property after 5 consecutive rounds over the limit)
469		Blank line.

470	if(round % ECONOMY_CYCLE == 0)	Condition: round % ECONOMY_CYCLE == 0 -> run the next block if true.
471	{	Opens a new block.
472	triggerDisaster(game);	Calls triggerDisaster(game). (Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owners get compensation, uninsured owners are marked sufferedLoss and own the full repair cost)
473	applyInflation(game);	Calls applyInflation(game). (Every 10 rounds picks a random rate {-3, 0, 2, 5, 8, 12} and applies it to purchase prices, mortgage values, rents, construction costs and the loan interest rate)
474	reviewPropertyMarket(game, round);	Calls reviewPropertyMarket(game, round). (Rule-LK 30-34: one group booms (+20% value, +25% rent, +15% mortgage/purchase, +10% construction) and a different one declines (-15% value, -20% rent, -10% mortgage, -25% auction), each for 10 rounds)
475	}	Closes the current block.
476		Blank line.
477	if(round % EVENT_CYCLE == 0)	Condition: round % EVENT_CYCLE == 0 -> run the next block if true.
478	{	Opens a new block.
479	triggerEconomicEvent(game);	Calls triggerEconomicEvent(game). (One of 8 random economy-wide events every 15 rounds (Tourism Boom, Fuel Crisis, Heavy Monsoon, Economic Recession, Stock Market Boom, Government Housing, Foreign Investment, Political Unrest))
480	drawRegionalCard(game);	Calls drawRegionalCard(game). (Table 4: one of 12 regional development cards every 15 rounds, active for 15, all as SRC_REGIONAL modifiers)
481	}	Closes the current block.
482		Blank line.
483	if(round % REGULATION_CYCLE == 0)	Condition: round % REGULATION_CYCLE == 0 -> run the next block if true.
484	{	Opens a new block.
485	triggerGovernmentRegulation(game);	Calls triggerGovernmentRegulation(game). (One of 8 random government regulations every 20 rounds (Property Tax raise, Loan Interest cut, Housing Subsidy, Luxury Property Tax on hotels, Railway Modernization, Electricity Tariff, Insurance Regulation, Anti-Speculation Act))
486	}	Closes the current block.
487		Blank line.
488	displayRoundSummary(game, round);	Calls displayRoundSummary(game, round). (Prints every player's cash, net worth, properties, buildings, loan and bankrupt status)
489	displayMarketConditions(game);	Calls displayMarketConditions(game). (Rule-LK 36: the end-of-round market report - boom, decline, regional development, other conditions, inflation, loan interest and tax rate)
490		Blank line.
491	if(countSolventPlayers(game) <= 1)	Condition: countSolventPlayers(game) <= 1 -> run the next block if true.
492	{	Opens a new block.
493	printf("\nOnly one solvent player remains. Ending game.\n");	Prints the message "\nOnly one solvent player remains. Ending game.\n".
494	break;	Breaks out of the current loop or switch.
495	}	Closes the current block.
496		Blank line.
497	round++;	round++;
498		Blank line.
499	if(round <= MAX_ROUNDS)	Condition: round <= MAX_ROUNDS -> run the next block if true.
500	{	Opens a new block.
501	printf("\n=====\\n");	Prints the message "\\n=====\\n".
502	printf("ROUND %d\\n", round);	Prints the message "ROUND %d\\n".
503	printf("=====\\n");	Prints the message "=====\\n".
504	}	Closes the current block.
505		Blank line.
506	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.

507 passedGoThisRound[i] = 0; 508 } 509 }	passedGoThisRound[i] = 0; Closes the current block. Closes the current block.
--	---

Nested functions inside playGame() - study these

- playGame() (Master loop. A round completes only when EVERY solvent player has passed GO at least once)
- playTurn() (One whole turn for one player (Rule 3): repair/maintain, handle jail, roll, move, resolve)
- processLoans() (Round-end loan heartbeat: compounds interest into the principal and counts down; at 0 round)
- processInsuranceExpiry() (Counts every policy down each round; warns at 3 rounds and expires (removes) at 0.)
- ageProperties() (Rule-LK 15/16: ages every property; past age 50 depreciation ticks +5% every 5 rounds, cap)
- ageBuildings() (Rule-LK 25/28: condition drops 2 per round; lost-income countdown runs; 20+ rounds without)
- decrementEventTimers() (Ticks the non-modifier event timers down: construction suspension and the closed property.)
- decrementModifiers() (Ticks every active modifier down and compacts out expired ones.)
- enforceAntiSpeculation() (Rule-LK 8: once active, force-auctions excess undeveloped property after 5 consecutive rounds)
- triggerDisaster() (Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owners)
- applyInflation() (Every 10 rounds picks a random rate {-3, 0, 2, 5, 8, 12} and applies it to purchase prices)
- reviewPropertyMarket() (Rule-LK 30-34: one group booms (+20% value, +25% rent, +15% mortgage/purchase, +10% constr)
- triggerEconomicEvent() (One of 8 random economy-wide events every 15 rounds (Tourism Boom, Fuel Crisis, Heavy Mons)
- drawRegionalCard() (Table 4: one of 12 regional development cards every 15 rounds, active for 15, all as SRC_R)
- triggerGovernmentRegulation() (One of 8 random government regulations every 20 rounds (Property Tax raise, Loan Interest)
- displayRoundSummary() (Prints every player's cash, net worth, properties, buildings, loan and bankrupt status.)
- displayMarketConditions() (Rule-LK 36: the end-of-round market report - boom, decline, regional development, other co)
- countSolventPlayers() (Counts players who are not bankrupt.)

MAIN PURPOSE

Master loop. A round completes only when EVERY solvent player has passed GO at least once (not after one turn each).

5.3 playTurn()

```
int playTurn(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: 1 if the player passed GO during the turn, else 0.

The function, verbatim from the source (starts at line 123)

```

124  /* One player's whole turn, matching Rule 3. Returns 1 if GO was
125     passed at any point during the turn. */
126  int playTurn(GameState game[], int playerIndex)
127  {
128      int dice;
129      int pos;
130      int passedGo;
131      int jailResult;
132      SquareType squareType;

134      if(game[0].players[playerIndex].bankrupt)
135          return 0;

137      printf("\n-----\n");
138      printf("%s's Turn\n", game[0].players[playerIndex].name);

140      /* Step 1: fix any damaged, worn, or neglected buildings first */
141      tryAutoRepair(game, playerIndex);
142      performMaintenance(game, playerIndex);
143      renovateStructuralDamage(game, playerIndex);

145      passedGo = 0;

147      jailResult = handleJail(game, playerIndex);

149      if(jailResult == 1)
150          return 0;  /* was in jail - released or not, no move this turn */

```

```

152     /* jailResult == 0: wasn't in jail - take a completely normal turn. */
153     dice = rollDice();
154     printf("%s rolled %d.\n", game[0].players[playerIndex].name, dice);

156     if(movePlayer(game, playerIndex, dice))
157         passedGo = 1;

159     /* Step 4: resolve landing action */
160     pos = game[0].players[playerIndex].position;
161     squareType = game[0].board[pos].type;

163     if(squareType == PROPERTY || squareType == RAILWAY || squareType == UTILITY)
164     {
165         payRent(game, playerIndex, dice);
166         buyProperty(game, playerIndex);
167     }
168     else if(squareType == EVENT)
169     {
170         executeEvent(game, playerIndex);
171     }
172     else if(squareType == TAX)
173     {
174         payTax(game, playerIndex);
175     }
176     else if(squareType == COMMUNITY_FUND)
177     {
178         payCommunityFundTax(game, playerIndex);
179     }
180     else if(squareType == GO_TO_JAIL)
181     {
182         printf("%s is sent to Jail!\n", game[0].players[playerIndex].name);
183         game[0].players[playerIndex].position = JAIL_SQUARE;
184         game[0].players[playerIndex].inJail = 1;
185         game[0].players[playerIndex].jailTurns = 0;
186     }
187     else if(squareType == BANK)
188     {
189         handleBankVisit(game, playerIndex);
190     }
191     else if(squareType == INSURANCE)
192     {
193         handleInsuranceVisit(game, playerIndex);
194     }
195     /* GO, JAIL (just visiting), FREE_PARKING need no action */

197     /* Step 6: build if eligible */
198     constructBuildings(game, playerIndex);

200     /* Section 3.4: the Opportunistic Trader first dumps any property
201        that economic events have marked for a decline. */
202     sellDecliningProperties(game, playerIndex);

204     /* Step 7: mortgage or redeem if the player's situation calls for it */
205     handleMortgageDecisions(game, playerIndex);

207     return passedGo;
208 }

```

Line-by-line explanation

Line	Code	Explanation
123		Blank line.
124	/* One player's whole turn, matching Rule 3. Returns 1 if GO was	Comment: /* One player's whole turn, matching Rule 3. Returns 1 if GO was
125	passed at any point during the turn. */	passed at any point during the turn. */
126	int playTurn(GameState game[], int playerIndex)	int playTurn(GameState game[], int playerIndex)
127	{	Opens a new block.

128	<code>int dice;</code>	Declares int dice.
129	<code>int pos;</code>	Declares int pos.
130	<code>int passedGo;</code>	Declares int passedGo.
131	<code>int jailResult;</code>	Declares int jailResult.
132	<code>SquareType squareType;</code>	SquareType squareType;
133		Blank line.
134	<code>if(game[0].players[playerIndex].bankrupt)</code>	Condition: the current player.bankrupt -> run the next block if true.
135	<code>return 0;</code>	Returns 0 immediately.
136		Blank line.
137	<code>printf("\n-----\n");</code>	Prints the message "\n-----\n".
138	<code>printf("%s's Turn\n", game[0].players[playerIndex].name);</code>	Prints the message "%s's Turn\n".
139		Blank line.
140	<code>/* Step 1: fix any damaged, worn, or neglected buildings first */</code>	Comment: /* Step 1: fix any damaged, worn, or neglected buildings first */
141	<code>tryAutoRepair(game, playerIndex);</code>	Calls tryAutoRepair(game, playerIndex). (Rule-LK 11: at the start of a turn, repairs any disaster-damaged building the owner can afford)
142	<code>performMaintenance(game, playerIndex);</code>	Calls performMaintenance(game, playerIndex). (Rule-LK 27: restores a building's condition to 100% for a fee (hotel 8%, house 5%), if the strategy agrees)
143	<code>renovateStructuralDamage(game, playerIndex);</code>	Calls renovateStructuralDamage(game, playerIndex). (Rule-LK 29: fully restores a structurally damaged building at 150% of replacement value, restoring pre-damage price/rent)
144		Blank line.
145	<code>passedGo = 0;</code>	passedGo = 0;
146		Blank line.
147	<code>jailResult = handleJail(game, playerIndex);</code>	jailResult = handleJail(game, playerIndex);
148		Blank line.
149	<code>if(jailResult == 1)</code>	Condition: jailResult == 1 -> run the next block if true.
150	<code>return 0; /* was in jail - released or not, no move this turn */</code>	Returns 0; /* was in jail - released or not, no move this turn */ immediately.
151		Blank line.
152	<code>/* jailResult == 0: wasn't in jail - take a completely normal turn. */</code>	Comment: /* jailResult == 0: wasn't in jail - take a completely normal turn. */
153	<code>dice = rollDice();</code>	dice = rollDice();
154	<code>printf("%s rolled %d.\n", game[0].players[playerIndex].name, dice);</code>	Prints the message "%s rolled %d.\n".
155		Blank line.
156	<code>if(movePlayer(game, playerIndex, dice))</code>	Condition: movePlayer(game, playerIndex, dice) -> run the next block if true.
157	<code>passedGo = 1;</code>	passedGo = 1;
158		Blank line.
159	<code>/* Step 4: resolve landing action */</code>	Comment: /* Step 4: resolve landing action */
160	<code>pos = game[0].players[playerIndex].position;</code>	pos = game[0].players[playerIndex].position;
161	<code>squareType = game[0].board[pos].type;</code>	squareType = game[0].board[pos].type;
162		Blank line.
163	<code>if(squareType == PROPERTY squareType == RAILWAY squareType == UTILITY)</code>	Condition: squareType == PROPERTY squareType == RAILWAY squareType == UTILITY -> run the next block if true.
164	<code>{</code>	Opens a new block.
165	<code>payRent(game, playerIndex, dice);</code>	Calls payRent(game, playerIndex, dice). (Resolves rent when landing on an owned, unmortgaged, working square)
166	<code>buyProperty(game, playerIndex);</code>	Calls buyProperty(game, playerIndex). (Handles landing on an unowned square: prices it, asks the strategy, buys or sends to auction)
167	<code>}</code>	Closes the current block.
168	<code>else if(squareType == EVENT)</code>	A further condition checked only if the previous one failed.
169	<code>{</code>	Opens a new block.

170	executeEvent(game, playerIndex);	Calls executeEvent(game, playerIndex). (Draws the current National Event Card (0)
171	}	Closes the current block.
172	else if(squareType == TAX)	A further condition checked only if the previous one failed.
173	{	Opens a new block.
174	payTax(game, playerIndex);	Calls payTax(game, playerIndex). (Income Tax: currentTaxRatePercent(INCOME_TAX_RATE) x net worth)
175	}	Closes the current block.
176	else if(squareType == COMMUNITY_FUND)	A further condition checked only if the previous one failed.
177	{	Opens a new block.
178	payCommunityFundTax(game, playerIndex);	Calls payCommunityFundTax(game, playerIndex). (Community Development Fund: currentTaxRatePercent(COMMUNITY_FUND_TAX_RATE) x property (land) value, not buildings)
179	}	Closes the current block.
180	else if(squareType == GO_TO_JAIL)	A further condition checked only if the previous one failed.
181	{	Opens a new block.
182	printf("%s is sent to Jail!\n", game[0].players[playerIndex].name);	Prints the message "%s is sent to Jail!\n".
183	game[0].players[playerIndex].position = JAIL_SQUARE;	Sets the field position of the player (i.e. Current board position 0.39.) = JAIL_SQUARE.
184	game[0].players[playerIndex].inJail = 1;	Sets the field inJail of the player (i.e. 1 while in Jail.) = 1.
185	game[0].players[playerIndex].jailTurns = 0;	Sets the field jailTurns of the player (i.e. Consecutive jail turns while failing to roll doubles.) = 0.
186	}	Closes the current block.
187	else if(squareType == BANK)	A further condition checked only if the previous one failed.
188	{	Opens a new block.
189	handleBankVisit(game, playerIndex);	Calls handleBankVisit(game, playerIndex). (The Bank square: repay full / increase / extend / refinance (existing loan), or obtain a new loan (Strategy 3)
190	}	Closes the current block.
191	else if(squareType == INSURANCE)	A further condition checked only if the previous one failed.
192	{	Opens a new block.
193	handleInsuranceVisit(game, playerIndex);	Calls handleInsuranceVisit(game, playerIndex). (The Insurance square: renew an expiring policy first, otherwise buy a new one per the strategy)
194	}	Closes the current block.
195	/* GO, JAIL (just visiting), FREE_PARKING need no action */	Comment: /* GO, JAIL (just visiting), FREE_PARKING need no action */
196		Blank line.
197	/* Step 6: build if eligible */	Comment: /* Step 6: build if eligible */
198	constructBuildings(game, playerIndex);	Calls constructBuildings(game, playerIndex). (Builds the maximum affordable buildings in one turn by repeatedly calling developGroup per group)
199		Blank line.
200	/* Section 3.4: the Opportunistic Trader first dumps any property	Comment: /* Section 3.4: the Opportunistic Trader first dumps any property
201	that economic events have marked for a decline. */	that economic events have marked for a decline. */
202	sellDecliningProperties(game, playerIndex);	Calls sellDecliningProperties(game, playerIndex). (Opportunistic Trader dumps undeveloped property in groups currently declining, under a global downturn, or with a negative index modifier)
203		Blank line.
204	/* Step 7: mortgage or redeem if the player's situation calls for it */	Comment: /* Step 7: mortgage or redeem if the player's situation calls for it */
205	handleMortgageDecisions(game, playerIndex);	Calls handleMortgageDecisions(game, playerIndex). (The turn-end mortgage step: try redeeming first, then mortgaging)
206		Blank line.
207	return passedGo;	Returns passedGo immediately.
208	}	Closes the current block.

Nested functions inside playTurn() - study these

- playTurn() (One whole turn for one player (Rule 3): repair/maintain, handle jail, roll, move, resolve)
- tryAutoRepair() (Rule-LK 11: at the start of a turn, repairs any disaster-damaged building the owner can af)
- performMaintenance() (Rule-LK 27: restores a building's condition to 100% for a fee (hotel 8%, house 5%), if the)
- renovateStructuralDamage() (Rule-LK 29: fully restores a structurally damaged building at 150% of replacement value, r)
- handleJail() (Resolves a Jail turn. Returns 1 when the player is in jail (their turn ends), 0 when they)
- rollDice() (Rolls two six-sided dice and returns the sum.)
- movePlayer() (Advances the player; collects GO money if they passed or landed on GO.)
- payRent() (Resolves rent when landing on an owned, unmortgaged, working square.)
- buyProperty() (Handles landing on an unowned square: prices it, asks the strategy, buys or sends to aucti)
- executeEvent() (Draws the current National Event Card (0..19, cycling like draw-top-put-bottom). Effects a)
- payTax() (Income Tax: currentTaxRatePercent(INCOME_TAX_RATE) x net worth.)
- payCommunityFundTax() (Community Development Fund: currentTaxRatePercent(COMMUNITY_FUND_TAX_RATE) x property (lan)
- handleBankVisit() (The Bank square: repay full / increase / extend / refinance (existing loan), or obtain a n)
- handleInsuranceVisit() (The Insurance square: renew an expiring policy first, otherwise buy a new one per the stra)
- constructBuildings() (Builds the maximum affordable buildings in one turn by repeatedly calling developGroup per)
- sellDecliningProperties() (Opportunistic Trader dumps undeveloped property in groups currently declining, under a glo)
- handleMortgageDecisions() (The turn-end mortgage step: try redeeming first, then mortgaging.)

MAIN PURPOSE

One whole turn for one player (Rule 3): repair/maintain, handle jail, roll, move, resolve landing, build, sell declining, mortgage.

5.4 movePlayer()

```
int movePlayer(GameState game[], int playerIndex, int dice)
```

Parameters: playerIndex, dice | Returns: 1 if the player passed GO, else 0.

The function, verbatim from the source (starts at line 15)

```
16  /* Moves a player, returns 1 if they passed or landed on GO. */
17  int movePlayer(GameState game[], int playerIndex, int dice)
18  {
19      int oldPosition;
20      int passedGo;
21      char moneyBuffer[32];

23      oldPosition = game[0].players[playerIndex].position;
24      game[0].players[playerIndex].position = (oldPosition + dice) % BOARD_SIZE;

26      printf("%s moves from Square %d to Square %d.\n",
27             game[0].players[playerIndex].name,
28             oldPosition,
29             game[0].players[playerIndex].position);

31      passedGo = 0;

33      if(oldPosition + dice >= BOARD_SIZE)
34      {
35          passedGo = 1;

37          receiveMoney(game, playerIndex, GO_MONEY);

39          printf("%s passed GO.\n", game[0].players[playerIndex].name);

41          formatLKR(GO_MONEY, moneyBuffer);

43          printf("Collected LKR %s\n", moneyBuffer);

45          printf("Current Balance : LKR %d\n",
46                 game[0].players[playerIndex].cash);
47      }
```

```

49         return passedGo;
50     }

```

Line-by-line explanation

Line	Code	Explanation
15		Blank line.
16	<code>/* Moves a player, returns 1 if they passed or landed on GO. */</code>	Comment: <code>/* Moves a player, returns 1 if they passed or landed on GO. */</code>
17	<code>int movePlayer(GameState game[], int playerIndex, int dice)</code>	<code>int movePlayer(GameState game[], int playerIndex, int dice)</code>
18	<code>{</code>	Opens a new block.
19	<code>int oldPosition;</code>	Declares <code>int oldPosition</code> .
20	<code>int passedGo;</code>	Declares <code>int passedGo</code> .
21	<code>char moneyBuffer[32];</code>	Declares <code>char moneyBuffer[32]</code> .
22		Blank line.
23	<code>oldPosition = game[0].players[playerIndex].position;</code>	<code>oldPosition = game[0].players[playerIndex].position;</code>
24	<code>game[0].players[playerIndex].position = (oldPosition + dice) % BOARD_SIZE;</code>	Sets the field position of the player (i.e. Current board position 0..39.) = <code>(oldPosition + dice) % BOARD_SIZE</code> .
25		Blank line.
26	<code>printf("%s moves from Square %d to Square %d.\n",</code>	Prints the message <code>"%s moves from Square %d to Square %d.\n"</code> .
27	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
28	<code>oldPosition,</code>	<code>oldPosition,</code>
29	<code>game[0].players[playerIndex].position);</code>	<code>game[0].players[playerIndex].position);</code>
30		Blank line.
31	<code>passedGo = 0;</code>	<code>passedGo = 0;</code>
32		Blank line.
33	<code>if(oldPosition + dice >= BOARD_SIZE)</code>	Condition: <code>oldPosition + dice >= BOARD_SIZE</code> -> run the next block if true.
34	<code>{</code>	Opens a new block.
35	<code>passedGo = 1;</code>	<code>passedGo = 1;</code>
36		Blank line.
37	<code>receiveMoney(game, playerIndex, GO_MONEY);</code>	Calls <code>receiveMoney(game, playerIndex, GO_MONEY)</code> . (Adds money to a player, but only if they are not bankrupt)
38		Blank line.
39	<code>printf("%s passed GO.\n",</code>	Prints the message <code>"%s passed GO.\n"</code> .
	<code>game[0].players[playerIndex].name);</code>	
40		Blank line.
41	<code>formatLKR(GO_MONEY, moneyBuffer);</code>	Calls <code>formatLKR(GO_MONEY, moneyBuffer)</code> . (Formats an amount with thousands separators, e)
42		Blank line.
43	<code>printf("Collected LKR %s\n", moneyBuffer);</code>	Prints the message <code>"Collected LKR %s\n"</code> .
44		Blank line.
45	<code>printf("Current Balance : LKR %d\n",</code>	Prints the message <code>"Current Balance : LKR %d\n"</code> .
46	<code>game[0].players[playerIndex].cash);</code>	<code>game[0].players[playerIndex].cash);</code>
47	<code>}</code>	Closes the current block.
48		Blank line.
49	<code>return passedGo;</code>	Returns <code>passedGo</code> immediately.
50	<code>}</code>	Closes the current block.

Nested functions inside movePlayer() - study these

- `movePlayer()` (Advances the player; collects GO money if they passed or landed on GO.)
- `receiveMoney()` (Adds money to a player, but only if they are not bankrupt.)
- `formatLKR()` (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Advances the player; collects GO money if they passed or landed on GO.

5.5 rollDice()

```
int rollDice(void)
```

Parameters: none | Returns: Sum 2..12.

The function, verbatim from the source (starts at line 8)

```
8    int rollDice(void)
9    {
10       int die1 = rand() % 6 + 1;
11       int die2 = rand() % 6 + 1;

13       return die1 + die2;
14    }
```

Line-by-line explanation

Line	Code	Explanation
8	int rollDice(void)	int rollDice(void)
9	{	Opens a new block.
10	int die1 = rand() % 6 + 1;	Declares int die1 = rand() % 6 + 1.
11	int die2 = rand() % 6 + 1;	Declares int die2 = rand() % 6 + 1.
12		Blank line.
13	return die1 + die2;	Returns die1 + die2 immediately.
14	}	Closes the current block.

Nested functions inside rollDice() - study these

- rollDice() (Rolls two six-sided dice and returns the sum.)

MAIN PURPOSE

Rolls two six-sided dice and returns the sum.

5.6 handleJail()

```
int handleJail(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: 0 = not in jail; 1 = was in jail (released or not).

The function, verbatim from the source (starts at line 51)

```
52    /* Handles a player's turn while they're in jail.
53       Return codes:
54       0 = wasn't in jail - take a normal turn
55       1 = was in jail (released or not) - turn ends, no move this turn */
56    int handleJail(GameState game[], int playerIndex)
57    {
58       int die1, die2;

60       if(!game[0].players[playerIndex].inJail)
61          return 0;

63       die1 = rand() % 6 + 1;
64       die2 = rand() % 6 + 1;

66       printf("%s is in Jail. Rolled %d and %d.\n",
67              game[0].players[playerIndex].name, die1, die2);

69       if(die1 == die2)
70       {
71          printf("Doubles! %s is released from Jail.\n",
72                 game[0].players[playerIndex].name);

74          /* Rule 13: rolling doubles releases the player, but they do
75             not move this turn - they remain on the Jail square and
76             move on their next turn. */
77          game[0].players[playerIndex].inJail = 0;
78          game[0].players[playerIndex].jailTurns = 0;
```

```

80         return 1;
81     }

82
83     /* Rule 13: paying bail is a strategy choice made after a failed
84     doubles roll. The player is released but does not move this
85     turn. */
86     if(shouldPayBail(game, playerIndex) &&
87        game[0].players[playerIndex].cash >= JAIL_BAIL)
88     {
89         printf("%s pays bail of LKR %d and is released from Jail.\n",
90             game[0].players[playerIndex].name, JAIL_BAIL);

91
92         payMoney(game, playerIndex, JAIL_BAIL);

93
94         game[0].players[playerIndex].inJail = 0;
95         game[0].players[playerIndex].jailTurns = 0;

96
97         return 1;
98     }

99
100    game[0].players[playerIndex].jailTurns++;

101
102    if(game[0].players[playerIndex].jailTurns >= JAIL_MAX_TURNS)
103    {
104        printf("%s has been in Jail for %d turns - must pay bail of "
105            "LKR %d and is released.\n",
106            game[0].players[playerIndex].name, JAIL_MAX_TURNS, JAIL_BAIL);

107
108        payMoney(game, playerIndex, JAIL_BAIL);

109
110        game[0].players[playerIndex].inJail = 0;
111        game[0].players[playerIndex].jailTurns = 0;

112
113        return 1;
114    }

115
116    printf("%s remains in Jail (%d/%d turns).\n",
117        game[0].players[playerIndex].name,
118        game[0].players[playerIndex].jailTurns,
119        JAIL_MAX_TURNS);

120
121    return 1;
122 }

```

Line-by-line explanation

Line	Code	Explanation
51		Blank line.
52	/* Handles a player's turn while they're in jail.	Comment: /* Handles a player's turn while they're in jail.
53	Return codes:	Return codes:
54	0 = wasn't in jail - take a normal turn	0 = wasn't in jail - take a normal turn
55	1 = was in jail (released or not) - turn ends, no move this turn */	1 = was in jail (released or not) - turn ends, no move this turn */
56	int handleJail(GameState game[], int playerIndex)	int handleJail(GameState game[], int playerIndex)
57	{	Opens a new block.
58	int die1, die2;	Declares int die1, die2.
59		Blank line.
60	if(!game[0].players[playerIndex].inJail)	Condition: !the current player.inJail -> run the next block if true.
61	return 0;	Returns 0 immediately.
62		Blank line.
63	die1 = rand() % 6 + 1;	die1 = rand() % 6 + 1;
64	die2 = rand() % 6 + 1;	die2 = rand() % 6 + 1;
65		Blank line.
66	printf("%s is in Jail. Rolled %d and %d.\n",	Prints the message "%s is in Jail. Rolled %d and %d.\n".
67	game[0].players[playerIndex].name, die1, die2);	game[0].players[playerIndex].name, die1, die2);

68		Blank line.
69	if(die1 == die2)	Condition: die1 == die2 -> run the next block if true.
70	{	Opens a new block.
71	printf("Doubles! %s is released from Jail.\n",	Prints the message "Doubles! %s is released from Jail.\n".
72	game[0].players[playerIndex].name);	game[0].players[playerIndex].name);
73		Blank line.
74	/* Rule 13: rolling doubles releases the player, but	Comment: /* Rule 13: rolling doubles releases the player, but they do
	they do	
75	not move this turn - they remain on the Jail square	not move this turn - they remain on the Jail square and
	and	
76	move on their next turn. */	move on their next turn. */
77	game[0].players[playerIndex].inJail = 0;	Sets the field inJail of the player (i.e. 1 while in Jail.) = 0.
78	game[0].players[playerIndex].jailTurns = 0;	Sets the field jailTurns of the player (i.e. Consecutive jail turns while failing to roll doubles.) = 0.
79		Blank line.
80	return 1;	Returns 1 immediately.
81	}	Closes the current block.
82		Blank line.
83	/* Rule 13: paying bail is a strategy choice made	Comment: /* Rule 13: paying bail is a strategy choice made after a failed
	after a failed	
84	doubles roll. The player is released but does not	doubles roll. The player is released but does not move this
	move this	
85	turn. */	turn. */
86	if(shouldPayBail(game, playerIndex) &&	Condition: shouldPayBail(game, playerIndex -> run the next block if true.
87	game[0].players[playerIndex].cash >= JAIL_BAIL)	game[0].players[playerIndex].cash >= JAIL_BAIL)
88	{	Opens a new block.
89	printf("%s pays bail of LKR %d and is released from	Prints the message "%s pays bail of LKR %d and is released from Jail.\n".
	Jail.\n",	
90	game[0].players[playerIndex].name, JAIL_BAIL);	game[0].players[playerIndex].name, JAIL_BAIL);
91		Blank line.
92	payMoney(game, playerIndex, JAIL_BAIL);	Calls payMoney(game, playerIndex, JAIL_BAIL). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
93		Blank line.
94	game[0].players[playerIndex].inJail = 0;	Sets the field inJail of the player (i.e. 1 while in Jail.) = 0.
95	game[0].players[playerIndex].jailTurns = 0;	Sets the field jailTurns of the player (i.e. Consecutive jail turns while failing to roll doubles.) = 0.
96		Blank line.
97	return 1;	Returns 1 immediately.
98	}	Closes the current block.
99		Blank line.
100	game[0].players[playerIndex].jailTurns++;	game[0].players[playerIndex].jailTurns++;
101		Blank line.
102	if(game[0].players[playerIndex].jailTurns >=	Condition: the current player.jailTurns >= JAIL_MAX_TURNS -> run the
	JAIL_MAX_TURNS)	next block if true.
103	{	Opens a new block.
104	printf("%s has been in Jail for %d turns - must pay	Prints the message "%s has been in Jail for %d turns - must pay bail of "
	bail of "	
105	"LKR %d and is released.\n",	"LKR %d and is released.\n",
106	game[0].players[playerIndex].name, JAIL_MAX_TURNS,	game[0].players[playerIndex].name, JAIL_MAX_TURNS, JAIL_BAIL);
	JAIL_BAIL);	
107		Blank line.
108	payMoney(game, playerIndex, JAIL_BAIL);	Calls payMoney(game, playerIndex, JAIL_BAIL). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
109		Blank line.
110	game[0].players[playerIndex].inJail = 0;	Sets the field inJail of the player (i.e. 1 while in Jail.) = 0.

111	game[0].players[playerIndex].jailTurns = 0;	Sets the field jailTurns of the player (i.e. Consecutive jail turns while failing to roll doubles.) = 0.
112		Blank line.
113	return 1;	Returns 1 immediately.
114	}	Closes the current block.
115		Blank line.
116	printf("%s remains in Jail (%d/%d turns).\n",	Prints the message "%s remains in Jail (%d/%d turns).\n".
117	game[0].players[playerIndex].name,	game[0].players[playerIndex].name,
118	game[0].players[playerIndex].jailTurns,	game[0].players[playerIndex].jailTurns,
119	JAIL_MAX_TURNS);	JAIL_MAX_TURNS);
120		Blank line.
121	return 1;	Returns 1 immediately.
122	}	Closes the current block.

Nested functions inside handleJail() - study these

- handleJail() (Resolves a Jail turn. Returns 1 when the player is in jail (their turn ends), 0 when they)
- shouldPayBail() (Whether the player pays bail after failing to roll doubles.)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Resolves a Jail turn. Returns 1 when the player is in jail (their turn ends), 0 when they take a normal turn.

5.7 resolveGroup()

```
int resolveGroup(GameState game[], int groupPlayers[], int groupSize, int output[], int startIndex)
```

Parameters: groupPlayers, groupSize, output, startIndex | Returns: How many players were written.

The function, verbatim from the source (starts at line 209)

```
210  /* Resolves one group of (possibly tied) players into a strict order.
211     Writes the result into output[] starting at startIndex, and
212     returns how many players it wrote. Any players still tied after
213     rolling call this function again on just themselves. */
214  int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
215                  int output[], int startIndex)
216  {
217      int rolls[MAX_PLAYERS];
218      int i, j, temp;
219      int runStart, runLength;
220      int tiedPlayers[MAX_PLAYERS];
221      int written;
222      int k;
223
224      written = 0;
225
226      for(i = 0; i < groupSize; i++)
227      {
228          rolls[i] = rollDice();
229          printf("%s rolls %d.\n", game[0].players[groupPlayers[i]].name, rolls[i]);
230      }
231
232      /* sort this group by roll, highest first */
233      for(i = 0; i < groupSize - 1; i++)
234      {
235          for(j = 0; j < groupSize - 1 - i; j++)
236          {
237              if(rolls[j] < rolls[j + 1])
238              {
239                  temp = rolls[j];
240                  rolls[j] = rolls[j + 1];
241                  rolls[j + 1] = temp;
242              }
243              temp = groupPlayers[j];
```

```

244         groupPlayers[j] = groupPlayers[j + 1];
245         groupPlayers[j + 1] = temp;
246     }
247 }
248 }

250 /* walk the sorted list - a run of equal rolls is a tie */
251 i = 0;

253 while(i < groupSize)
254 {
255     runStart = i;
256     runLength = 1;

258     while(runStart + runLength < groupSize &&
259           rolls[runStart + runLength] == rolls[runStart])
260     {
261         runLength++;
262     }

264     if(runLength == 1)
265     {
266         output[startIndex + written] = groupPlayers[runStart];
267         written++;
268     }
269     else
270     {
271         printf("Tie detected between %d players at %d. Rolling again.\n",
272               runLength, rolls[runStart]);

274         for(k = 0; k < runLength; k++)
275             tiedPlayers[k] = groupPlayers[runStart + k];

277         written += resolveGroup(game, tiedPlayers, runLength,
278                                output, startIndex + written);
279     }

281     i = runStart + runLength;
282 }

284 return written;
285 }

```

Line-by-line explanation

Line	Code	Explanation
209		Blank line.
210	/* Resolves one group of (possibly tied) players into a strict order.	Comment: /* Resolves one group of (possibly tied) players into a strict order.
211	Writes the result into output[] starting at startIndex, and	Writes the result into output[] starting at startIndex, and
212	returns how many players it wrote. Any players still tied after	returns how many players it wrote. Any players still tied after
213	rolling call this function again on just themselves. */	rolling call this function again on just themselves. */
214	int resolveGroup(GameState game[], int groupPlayers[], int groupSize,	int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
215	int output[], int startIndex)	int output[], int startIndex)
216	{	Opens a new block.
217	int rolls[MAX_PLAYERS];	Declares int rolls[MAX_PLAYERS].
218	int i, j, temp;	Declares int i, j, temp.
219	int runStart, runLength;	Declares int runStart, runLength.
220	int tiedPlayers[MAX_PLAYERS];	Declares int tiedPlayers[MAX_PLAYERS].
221	int written;	Declares int written.
222	int k;	Declares int k.

223		Blank line.
224	written = 0;	written = 0;
225		Blank line.
226	for(i = 0; i < groupSize; i++)	Loop over all entries: init i = 0; run while i < groupSize; every loop i++.
227	{	Opens a new block.
228	rolls[i] = rollDice();	rolls[i] = rollDice();
229	printf("%s rolls %d.\n", game[0].players[groupPlayers[i]].name, rolls[i]);	Prints the message "%s rolls %d.\n".
230	}	Closes the current block.
231		Blank line.
232	/* sort this group by roll, highest first */	Comment: /* sort this group by roll, highest first */
233	for(i = 0; i < groupSize - 1; i++)	Loop over all entries: init i = 0; run while i < groupSize - 1; every loop i++.
234	{	Opens a new block.
235	for(j = 0; j < groupSize - 1 - i; j++)	Loop over all entries: init j = 0; run while j < groupSize - 1 - i; every loop j++.
236	{	Opens a new block.
237	if(rolls[j] < rolls[j + 1])	Condition: rolls[j] < rolls[j + 1] -> run the next block if true.
238	{	Opens a new block.
239	temp = rolls[j];	temp = rolls[j];
240	rolls[j] = rolls[j + 1];	rolls[j] = rolls[j + 1];
241	rolls[j + 1] = temp;	rolls[j + 1] = temp;
242		Blank line.
243	temp = groupPlayers[j];	temp = groupPlayers[j];
244	groupPlayers[j] = groupPlayers[j + 1];	groupPlayers[j] = groupPlayers[j + 1];
245	groupPlayers[j + 1] = temp;	groupPlayers[j + 1] = temp;
246	}	Closes the current block.
247	}	Closes the current block.
248	}	Closes the current block.
249		Blank line.
250	/* walk the sorted list - a run of equal rolls is a tie */	Comment: /* walk the sorted list - a run of equal rolls is a tie */
251	i = 0;	i = 0;
252		Blank line.
253	while(i < groupSize)	Loop: repeat while i < groupSize is true.
254	{	Opens a new block.
255	runStart = i;	runStart = i;
256	runLength = 1;	runLength = 1;
257		Blank line.
258	while(runStart + runLength < groupSize && rolls[runStart + runLength] == rolls[runStart])	Loop: repeat while is true. rolls[runStart + runLength] == rolls[runStart])
260	{	Opens a new block.
261	runLength++;	runLength++;
262	}	Closes the current block.
263		Blank line.
264	if(runLength == 1)	Condition: runLength == 1 -> run the next block if true.
265	{	Opens a new block.
266	output[startIndex + written] = groupPlayers[runStart];	output[startIndex + written] = groupPlayers[runStart];
267	written++;	written++;
268	}	Closes the current block.
269	else	The alternative branch when the condition above is false.
270	{	Opens a new block.
271	printf("Tie detected between %d players at %d. Rolling again.\n", runLength, rolls[runStart]);	Prints the message "Tie detected between %d players at %d. Rolling again.\n". runLength, rolls[runStart]);
272		
273		Blank line.
274	for(k = 0; k < runLength; k++)	Loop over all entries: init k = 0; run while k < runLength; every loop k++.

275	<code>tiedPlayers[k] = groupPlayers[runStart + k];</code>	<code>tiedPlayers[k] = groupPlayers[runStart + k];</code>
276		Blank line.
277	<code>written += resolveGroup(game, tiedPlayers,</code> <code>runLength,</code>	<code>written += resolveGroup(game, tiedPlayers, runLength,</code>
278	<code>output, startIndex + written);</code>	<code>output, startIndex + written);</code>
279	<code>}</code>	Closes the current block.
280		Blank line.
281	<code>i = runStart + runLength;</code>	<code>i = runStart + runLength;</code>
282	<code>}</code>	Closes the current block.
283		Blank line.
284	<code>return written;</code>	Returns written immediately.
285	<code>}</code>	Closes the current block.

Nested functions inside resolveGroup() - study these

- resolveGroup() (Resolves a group of (possibly tied) players into a strict order by rolling dice, recursing)
- rollDice() (Rolls two six-sided dice and returns the sum.)

MAIN PURPOSE

Resolves a group of (possibly tied) players into a strict order by rolling dice, recursing on ties.

5.8 determineTurnOrder()

```
void determineTurnOrder(GameState game[], int turnOrder[])
```

Parameters: turnOrder | Returns: void.

The function, verbatim from the source (starts at line 287)

```
287 void determineTurnOrder(GameState game[], int turnOrder[])
288 {
289     int allPlayers[MAX_PLAYERS];
290     int i;

292     for(i = 0; i < MAX_PLAYERS; i++)
293         allPlayers[i] = i;

295     printf("\nDetermining the First Player\n");

297     resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0);

299     printf("\n%s will begin the game.\n", game[0].players[turnOrder[0]].name);

301     printf("\nTurn order:\n");
302     for(i = 0; i < MAX_PLAYERS; i++)
303         printf("%s\n", game[0].players[turnOrder[i]].name);
304 }
```

Line-by-line explanation

Line	Code	Explanation
287	<code>void determineTurnOrder(GameState game[], int turnOrder[])</code>	<code>void determineTurnOrder(GameState game[], int turnOrder[])</code>
288	<code>{</code>	Opens a new block.
289	<code>int allPlayers[MAX_PLAYERS];</code>	Declares int allPlayers[MAX_PLAYERS].
290	<code>int i;</code>	Declares int i.
291		Blank line.
292	<code>for(i = 0; i < MAX_PLAYERS; i++)</code>	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
293	<code>allPlayers[i] = i;</code>	<code>allPlayers[i] = i;</code>
294		Blank line.
295	<code>printf("\nDetermining the First Player\n");</code>	Prints the message "\nDetermining the First Player\n".
296		Blank line.

297	<code>resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0);</code>	Calls <code>resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0)</code> . (Resolves a group of (possibly tied) players into a strict order by rolling dice, recursing on ties)
298		Blank line.
299	<code>printf("\n%s will begin the game.\n", game[0].players[turnOrder[0]].name);</code>	Prints the message <code>"\n%s will begin the game.\n"</code> .
300		Blank line.
301	<code>printf("\nTurn order:\n");</code>	Prints the message <code>"\nTurn order:\n"</code> .
302	<code>for(i = 0; i < MAX_PLAYERS; i++)</code>	Loop over all entries: init <code>i = 0</code> ; run while <code>i < MAX_PLAYERS</code> ; every loop <code>i++</code> .
303	<code>printf("%s\n", game[0].players[turnOrder[i]].name);</code>	Prints the message <code>"%s\n"</code> .
304	<code>}</code>	Closes the current block.

Nested functions inside `determineTurnOrder()` - study these

- `determineTurnOrder()` (Finds who plays first and the full order of play.)
- `resolveGroup()` (Resolves a group of (possibly tied) players into a strict order by rolling dice, recursing)

MAIN PURPOSE

Finds who plays first and the full order of play.

5.9 countHouses()

```
int countHouses(GameState game[], int playerIndex)
```

Parameters: `playerIndex` | Returns: Number of houses 0..many.

The function, verbatim from the source (starts at line 306)

```

306  int countHouses(GameState game[], int playerIndex)
307  {
308      int i;
309      int count;

311      count = 0;

313      for(i = 0; i < BOARD_SIZE; i++)
314      {
315          if(game[0].board[i].type == PROPERTY &&
316             game[0].board[i].property.owner == playerIndex &&
317             !game[0].board[i].property.hotel)
318          {
319              count += game[0].board[i].property.houses;
320          }
321      }

323      return count;
324  }
```

Line-by-line explanation

Line	Code	Explanation
306	<code>int countHouses(GameState game[], int playerIndex)</code>	<code>int countHouses(GameState game[], int playerIndex)</code>
307	<code>{</code>	Opens a new block.
308	<code>int i;</code>	Declares <code>int i</code> .
309	<code>int count;</code>	Declares <code>int count</code> .
310		Blank line.
311	<code>count = 0;</code>	<code>count = 0;</code>
312		Blank line.
313	<code>for(i = 0; i < BOARD_SIZE; i++)</code>	Loop over all entries: init <code>i = 0</code> ; run while <code>i < BOARD_SIZE</code> ; every loop <code>i++</code> .
314	<code>{</code>	Opens a new block.
315	<code>if(game[0].board[i].type == PROPERTY &&</code>	Condition: -> run the next block if true.
316	<code>game[0].board[i].property.owner == playerIndex &&</code>	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) <code>= playerIndex &&</code> .
317	<code>!game[0].board[i].property.hotel)</code>	<code>!game[0].board[i].property.hotel)</code>

318	{	Opens a new block.
319	count += game[0].board[i].property.houses;	count += game[0].board[i].property.houses;
320	}	Closes the current block.
321	}	Closes the current block.
322		Blank line.
323	return count;	Returns count immediately.
324	}	Closes the current block.

Nested functions inside countHouses() - study these

- countHouses() (Counts houses (not hotels) owned by a player.)

MAIN PURPOSE

Counts houses (not hotels) owned by a player.

5.10 countHotels()

int countHotels(GameState game[], int playerIndex)

Parameters: playerIndex | Returns: Number of hotels.

The function, verbatim from the source (starts at line 326)

```
326 int countHotels(GameState game[], int playerIndex)
327 {
328     int i;
329     int count;

331     count = 0;

333     for(i = 0; i < BOARD_SIZE; i++)
334     {
335         if(game[0].board[i].type == PROPERTY &&
336            game[0].board[i].property.owner == playerIndex &&
337            game[0].board[i].property.hotel)
338         {
339             count++;
340         }
341     }

343     return count;
344 }
```

Line-by-line explanation

Line	Code	Explanation
326	int countHotels(GameState game[], int playerIndex)	int countHotels(GameState game[], int playerIndex)
327	{	Opens a new block.
328	int i;	Declares int i.
329	int count;	Declares int count.
330		Blank line.
331	count = 0;	count = 0;
332		Blank line.
333	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
334	{	Opens a new block.
335	if(game[0].board[i].type == PROPERTY &&	Condition: -> run the next block if true.
336	game[0].board[i].property.owner == playerIndex &&	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = = playerIndex &&.
337	game[0].board[i].property.hotel)	game[0].board[i].property.hotel)
338	{	Opens a new block.
339	count++;	count++;
340	}	Closes the current block.
341	}	Closes the current block.
342		Blank line.
343	return count;	Returns count immediately.

344 }

Closes the current block.

Nested functions inside countHotels() - study these

- countHotels() (Counts hotels owned by a player.)

MAIN PURPOSE

Counts hotels owned by a player.

5.11 displayRoundSummary()

```
void displayRoundSummary(GameState game[], int round)
```

Parameters: round | Returns: void.

The function, verbatim from the source (starts at line 346)

```
346 void displayRoundSummary(GameState game[], int round)
347 {
348     int i;

350     printf("\n=====\\n");
351     printf("Round %d Summary\\n", round);
352     printf("=====\\n");

354     for(i = 0; i < MAX_PLAYERS; i++)
355     {
356         char cashBuf[32];
357         char netBuf[32];
358         char loanBuf[32];

360         formatLKR(game[0].players[i].cash, cashBuf);
361         formatLKR(calculateNetWorth(game, i), netBuf);

363         printf("%s\\n", game[0].players[i].name);
364         printf("Cash : LKR %s\\n", cashBuf);
365         printf("Net Worth : LKR %s\\n", netBuf);
366         printf("Properties : %d\\n", game[0].players[i].propertiesOwned);
367         printf("Railways : %d\\n", game[0].players[i].railwaysOwned);
368         printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);
369         printf("Houses : %d\\n", countHouses(game, i));
370         printf("Hotels : %d\\n", countHotels(game, i));

372         if(game[0].players[i].loan.active)
373         {
374             formatLKR(game[0].players[i].loan.amount, loanBuf);
375             printf("Outstanding Loan : LKR %s\\n", loanBuf);
376         }
377         else
378         {
379             printf("Outstanding Loan : None\\n");
380         }

382         if(game[0].players[i].bankrupt)
383             printf("Status : BANKRUPT\\n");

385         printf("-----\\n");
386     }
387 }
```

Line-by-line explanation

Line	Code	Explanation
346	void displayRoundSummary(GameState game[], int round)	void displayRoundSummary(GameState game[], int round)
347	{	Opens a new block.
348	int i;	Declares int i.
349		Blank line.

350	printf("\n=====	Prints the message
	====\n");	"\n=====\\n".
351	printf("Round %d Summary\n", round);	Prints the message "Round %d Summary\n".
352	printf("=====	Prints the message
	=\n");	"=====\\n".
353		Blank line.
354	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
355	{	Opens a new block.
356	char cashBuf[32];	Declares char cashBuf[32].
357	char netBuf[32];	Declares char netBuf[32].
358	char loanBuf[32];	Declares char loanBuf[32].
359		Blank line.
360	formatLKR(game[0].players[i].cash, cashBuf);	Calls formatLKR(game[0].players[i].cash, cashBuf). (Formats an amount with thousands separators, e)
361	formatLKR(calculateNetWorth(game, i), netBuf);	Calls formatLKR(calculateNetWorth(game, i), netBuf). (Formats an amount with thousands separators, e)
362		Blank line.
363	printf("%s\n", game[0].players[i].name);	Prints the message "%s\n".
364	printf("Cash : LKR %s\n", cashBuf);	Prints the message "Cash : LKR %s\n".
365	printf("Net Worth : LKR %s\n", netBuf);	Prints the message "Net Worth : LKR %s\n".
366	printf("Properties : %d\n", game[0].players[i].propertiesOwned);	Prints the message "Properties : %d\n".
367	printf("Railways : %d\n", game[0].players[i].railwaysOwned);	Prints the message "Railways : %d\n".
368	printf("Utilities : %d\n", game[0].players[i].utilitiesOwned);	Prints the message "Utilities : %d\n".
369	printf("Houses : %d\n", countHouses(game, i));	Prints the message "Houses : %d\n".
370	printf("Hotels : %d\n", countHotels(game, i));	Prints the message "Hotels : %d\n".
371		Blank line.
372	if(game[0].players[i].loan.active)	Condition: player i.loan.active -> run the next block if true.
373	{	Opens a new block.
374	formatLKR(game[0].players[i].loan.amount, loanBuf);	Calls formatLKR(game[0].players[i].loan.amount, loanBuf). (Formats an amount with thousands separators, e)
375	printf("Outstanding Loan : LKR %s\n", loanBuf);	Prints the message "Outstanding Loan : LKR %s\n".
376	}	Closes the current block.
377	else	The alternative branch when the condition above is false.
378	{	Opens a new block.
379	printf("Outstanding Loan : None\n");	Prints the message "Outstanding Loan : None\n".
380	}	Closes the current block.
381		Blank line.
382	if(game[0].players[i].bankrupt)	Condition: player i.bankrupt -> run the next block if true.
383	printf("Status : BANKRUPT\n");	Prints the message "Status : BANKRUPT\n".
384		Blank line.
385	printf("-----	Prints the message "-----\\n".
	-\\n");	
386	}	Closes the current block.
387	}	Closes the current block.

Nested functions inside displayRoundSummary() - study these

- displayRoundSummary() (Prints every player's cash, net worth, properties, buildings, loan and bankrupt status.)
- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)
- calculateNetWorth() (Rule 15 net worth: cash + property + buildings - outstanding loan.)
- countHouses() (Counts houses (not hotels) owned by a player.)
- countHotels() (Counts hotels owned by a player.)

MAIN PURPOSE

Prints every player's cash, net worth, properties, buildings, loan and bankrupt status.

5.12 countSolventPlayers()

```
int countSolventPlayers(GameState game[])
```

Parameters: none | Returns: Number of solvent players.

The function, verbatim from the source (starts at line 389)

```
389 int countSolventPlayers(GameState game[])
390 {
391     int i;
392     int count;

394     count = 0;

396     for(i = 0; i < MAX_PLAYERS; i++)
397     {
398         if(!game[0].players[i].bankrupt)
399             count++;
400     }

402     return count;
403 }
```

Line-by-line explanation

Line	Code	Explanation
389	int countSolventPlayers(GameState game[])	int countSolventPlayers(GameState game[])
390	{	Opens a new block.
391	int i;	Declares int i.
392	int count;	Declares int count.
393		Blank line.
394	count = 0;	count = 0;
395		Blank line.
396	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
397	{	Opens a new block.
398	if(!game[0].players[i].bankrupt)	Condition: !player i.bankrupt -> run the next block if true.
399	count++;	count++;
400	}	Closes the current block.
401		Blank line.
402	return count;	Returns count immediately.
403	}	Closes the current block.

Nested functions inside countSolventPlayers() - study these

- countSolventPlayers() (Counts players who are not bankrupt.)

MAIN PURPOSE

Counts players who are not bankrupt.

5.13 determineWinner()

```
int determineWinner(GameState game[])
```

Parameters: none | Returns: Winner's player index.

The function, verbatim from the source (starts at line 510)

```
511 /* Whoever is still solvent wins. If several are still solvent at
512    round 500, the highest net worth wins (Rule 15). */
513 int determineWinner(GameState game[])
514 {
515     int i;
```

```

516     int winner;
517     int bestNetWorth;
518     int netWorth;

520     winner = -1;
521     bestNetWorth = 0;

523     for(i = 0; i < MAX_PLAYERS; i++)
524     {
525         if(game[0].players[i].bankrupt)
526             continue;

528         netWorth = calculateNetWorth(game, i);

530         if(winner == -1 || netWorth > bestNetWorth)
531         {
532             winner = i;
533             bestNetWorth = netWorth;
534         }
535     }

537     if(winner == -1)
538     {
539         /* edge case: everyone bankrupt at once - compare everyone anyway */
540         for(i = 0; i < MAX_PLAYERS; i++)
541         {
542             netWorth = calculateNetWorth(game, i);

544             if(winner == -1 || netWorth > bestNetWorth)
545             {
546                 winner = i;
547                 bestNetWorth = netWorth;
548             }
549         }
550     }

552     return winner;
553 }

```

Line-by-line explanation

Line	Code	Explanation
510		Blank line.
511	/* Whoever is still solvent wins. If several are still solvent at	Comment: /* Whoever is still solvent wins. If several are still solvent at
512	round 500, the highest net worth wins (Rule 15). */	round 500, the highest net worth wins (Rule 15). */
513	int determineWinner(GameState game[])	int determineWinner(GameState game[])
514	{	Opens a new block.
515	int i;	Declares int i.
516	int winner;	Declares int winner.
517	int bestNetWorth;	Declares int bestNetWorth.
518	int netWorth;	Declares int netWorth.
519		Blank line.
520	winner = -1;	winner = -1;
521	bestNetWorth = 0;	bestNetWorth = 0;
522		Blank line.
523	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
524	{	Opens a new block.
525	if(game[0].players[i].bankrupt)	Condition: player i.bankrupt -> run the next block if true.
526	continue;	Skips to the next loop iteration.
527		Blank line.
528	netWorth = calculateNetWorth(game, i);	netWorth = calculateNetWorth(game, i);
529		Blank line.

530	if(winner == -1 netWorth > bestNetWorth)	Condition: winner == -1 netWorth > bestNetWorth -> run the next block if true.
531	{	Opens a new block.
532	winner = i;	winner = i;
533	bestNetWorth = netWorth;	bestNetWorth = netWorth;
534	}	Closes the current block.
535	}	Closes the current block.
536		Blank line.
537	if(winner == -1)	Condition: winner == -1 -> run the next block if true.
538	{	Opens a new block.
539	/* edge case: everyone bankrupt at once - compare everyone anyway */	Comment: /* edge case: everyone bankrupt at once - compare everyone anyway */
540	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
541	{	Opens a new block.
542	netWorth = calculateNetWorth(game, i);	netWorth = calculateNetWorth(game, i);
543		Blank line.
544	if(winner == -1 netWorth > bestNetWorth)	Condition: winner == -1 netWorth > bestNetWorth -> run the next block if true.
545	{	Opens a new block.
546	winner = i;	winner = i;
547	bestNetWorth = netWorth;	bestNetWorth = netWorth;
548	}	Closes the current block.
549	}	Closes the current block.
550	}	Closes the current block.
551		Blank line.
552	return winner;	Returns winner immediately.
553	}	Closes the current block.

Nested functions inside determineWinner() - study these

- determineWinner() (Finds the winner: the last solvent player, or the highest net worth at round 500 (Rule 15))
- calculateNetWorth() (Rule 15 net worth: cash + property + buildings - outstanding loan.)

MAIN PURPOSE

Finds the winner: the last solvent player, or the highest net worth at round 500 (Rule 15).

5.14 displayFinalResults()

```
void displayFinalResults(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 555)

```
555 void displayFinalResults(GameState game[])
556 {
557     int winner;
558     int i;
559     char cashBuf[32];
560     char propBuf[32];
561     char loanBuf[32];
562     char netBuf[32];

564     winner = determineWinner(game);

566     printf("\n=====\\n");
567     printf("GAME OVER\\n");
568     printf("=====\\n");

570     printf("Winner\\n%s\\n", game[0].players[winner].name);

572     formatLKR(game[0].players[winner].cash, cashBuf);
573     printf("Total Cash\\nLKR %s\\n", cashBuf);
```

```

575     formatLKR(calculatePropertyValue(game, winner) +
576               calculateBuildingValue(game, winner), propBuf);
577     printf("Total Property Value\nLKR %s\n", propBuf);

579     if(game[0].players[winner].loan.active)
580     {
581         formatLKR(game[0].players[winner].loan.amount, loanBuf);
582         printf("Outstanding Loans\nLKR %s\n", loanBuf);
583     }
584     else
585     {
586         printf("Outstanding Loans\nNone\n");
587     }

589     formatLKR(calculateNetWorth(game, winner), netBuf);
590     printf("Net Worth\nLKR %s\n", netBuf);
591     printf("=====\n");

593     printf("\nFinal Standings (all players)\n");
594     printf("=====\n");

596     for(i = 0; i < MAX_PLAYERS; i++)
597     {
598         printf("\n%s\n", game[0].players[i].name);
599         printf("Cash : LKR %d\n", game[0].players[i].cash);
600         printf("Net Worth : LKR %d\n", calculateNetWorth(game, i));
601         printf("Properties : %d\n", game[0].players[i].propertiesOwned);
602         printf("Railways : %d\n", game[0].players[i].railwaysOwned);
603         printf("Utilities : %d\n", game[0].players[i].utilitiesOwned);
604         if(game[0].players[i].bankrupt)
605         {
606             printf("Status : Bankrupt\n");
607         }
608         else
609         {
610             printf("Status : Active\n");
611         }
612     }
613 }

```

Line-by-line explanation

Line	Code	Explanation
555	void displayFinalResults(GameState game[])	void displayFinalResults(GameState game[])
556	{	Opens a new block.
557	int winner;	Declares int winner.
558	int i;	Declares int i.
559	char cashBuf[32];	Declares char cashBuf[32].
560	char propBuf[32];	Declares char propBuf[32].
561	char loanBuf[32];	Declares char loanBuf[32].
562	char netBuf[32];	Declares char netBuf[32].
563		Blank line.
564	winner = determinewinner(game);	winner = determineWinner(game);
565		Blank line.
566	printf("\n===== ==\n");	Prints the message "\n===== ==\n".
567	printf("GAME OVER\n");	Prints the message "GAME OVER\n".
568	printf("===== =\n");	Prints the message "===== =\n".
569		Blank line.
570	printf("Winner\n%s\n", game[0].players[winner].name);	Prints the message "Winner\n%s\n".
571		Blank line.

572	formatLKR(game[0].players[winner].cash, cashBuf);	Calls formatLKR(game[0].players[winner].cash, cashBuf). (Formats an amount with thousands separators, e)
573	printf("Total Cash\nLKR %s\n", cashBuf);	Prints the message "Total Cash\nLKR %s\n".
574		Blank line.
575	formatLKR(calculatePropertyValue(game, winner) +	formatLKR(calculatePropertyValue(game, winner) +
576	calculateBuildingValue(game, winner), propBuf);	Calls calculateBuildingValue(game, winner), propBuf). (Cash value of all houses and hotels)
577	printf("Total Property Value\nLKR %s\n", propBuf);	Prints the message "Total Property Value\nLKR %s\n".
578		Blank line.
579	if(game[0].players[winner].loan.active)	Condition: the winner.loan.active -> run the next block if true.
580	{	Opens a new block.
581	formatLKR(game[0].players[winner].loan.amount,	Calls formatLKR(game[0].players[winner].loan.amount, loanBuf). (Formats an amount with thousands separators, e)
	loanBuf);	
582	printf("Outstanding Loans\nLKR %s\n", loanBuf);	Prints the message "Outstanding Loans\nLKR %s\n".
583	}	Closes the current block.
584	else	The alternative branch when the condition above is false.
585	{	Opens a new block.
586	printf("Outstanding Loans\nNone\n");	Prints the message "Outstanding Loans\nNone\n".
587	}	Closes the current block.
588		Blank line.
589	formatLKR(calculateNetWorth(game, winner), netBuf);	Calls formatLKR(calculateNetWorth(game, winner), netBuf). (Formats an amount with thousands separators, e)
590	printf("Net Worth\nLKR %s\n", netBuf);	Prints the message "Net Worth\nLKR %s\n".
591	printf("===== =\\n");	Prints the message "===== =\\n".
592		Blank line.
593	printf("\\nFinal Standings (all players)\\n");	Prints the message "\\nFinal Standings (all players)\\n".
594	printf("===== =\\n");	Prints the message "===== =\\n".
595		Blank line.
596	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
597	{	Opens a new block.
598	printf("\\n%s\\n", game[0].players[i].name);	Prints the message "\\n%s\\n".
599	printf("Cash : LKR %d\\n", game[0].players[i].cash);	Prints the message "Cash : LKR %d\\n".
600	printf("Net Worth : LKR %d\\n", calculateNetWorth(game, i));	Prints the message "Net Worth : LKR %d\\n".
601	printf("Properties : %d\\n", game[0].players[i].propertiesOwned);	Prints the message "Properties : %d\\n".
602	printf("Railways : %d\\n", game[0].players[i].railwaysOwned);	Prints the message "Railways : %d\\n".
603	printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);	Prints the message "Utilities : %d\\n".
604	if(game[0].players[i].bankrupt)	Condition: player i.bankrupt -> run the next block if true.
605	{	Opens a new block.
606	printf("Status : Bankrupt\\n");	Prints the message "Status : Bankrupt\\n".
607	}	Closes the current block.
608	else	The alternative branch when the condition above is false.
609	{	Opens a new block.
610	printf("Status : Active\\n");	Prints the message "Status : Active\\n".
611	}	Closes the current block.
612	}	Closes the current block.
613	}	Closes the current block.

Nested functions inside displayFinalResults() - study these

- displayFinalResults() (Prints the winner's cash/property/loan/net worth and every player's final standing.)
- determineWinner() (Finds the winner: the last solvent player, or the highest net worth at round 500 (Rule 15))

- `formatLKR()` (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)
- `calculatePropertyValue()` (Total current market value of owned, unmortgaged squares.)
- `calculateBuildingValue()` (Cash value of all houses and hotels.)
- `calculateNetWorth()` (Rule 15 net worth: cash + property + buildings - outstanding loan.)

MAIN PURPOSE

Prints the winner's cash/property/loan/net worth and every player's final standing.

All money movement: buying, rent, taxes, mortgages, buildings, net worth, selling, forced sales, the Anti-Speculation Act.

The sub-topics below are the 33 functions implemented in finance.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in finance.c

6.1	stripOwnership()	(source line 4)	p. 78
6.2	resetLoan()	(source line 15)	p. 79
6.3	adjustOwnedCount()	(source line 24)	p. 80
6.4	groupOf()	(source line 37)	p. 80
6.5	receiveMoney()	(source line 47)	p. 81
6.6	payMoney()	(source line 56)	p. 82
6.7	liquidateBankruptAssets()	(source line 92)	p. 83
6.8	payTax()	(source line 120)	p. 85
6.9	payCommunityFundTax()	(source line 149)	p. 86
6.10	findPropertyToMortgage()	(source line 176)	p. 88
6.11	findMortgagedProperty()	(source line 204)	p. 89
6.12	mortgageProperty()	(source line 220)	p. 90
6.13	redeemMortgage()	(source line 252)	p. 91
6.14	handleMortgageDecisions()	(source line 281)	p. 92
6.15	calculateRent()	(source line 286)	p. 93
6.16	calculateRailwayRent()	(source line 338)	p. 95
6.17	calculateUtilityRent()	(source line 373)	p. 97
6.18	groupSize()	(source line 405)	p. 98
6.19	ownsMonopoly()	(source line 421)	p. 99
6.20	developGroup()	(source line 439)	p. 100
6.21	constructBuildings()	(source line 562)	p. 105
6.22	calculatePropertyValue()	(source line 590)	p. 106
6.23	calculateBuildingValue()	(source line 614)	p. 107
6.24	calculateNetWorth()	(source line 637)	p. 108
6.25	countUndevelopedProperties()	(source line 659)	p. 109
6.26	wouldCompleteMonopoly()	(source line 679)	p. 110
6.27	sellLowValueProperty()	(source line 706)	p. 111
6.28	sellDecliningProperties()	(source line 757)	p. 113
6.29	sellUndevelopedPropertyToAuction()	(source line 816)	p. 116
6.30	forceSellExcess()	(source line 866)	p. 118
6.31	enforceAntiSpeculation()	(source line 889)	p. 119
6.32	buyProperty()	(source line 933)	p. 121
6.33	payRent()	(source line 1002)	p. 124

6.1 stripOwnership()

```
void stripOwnership(GameState game[], int index)
```

Parameters: *index* | Returns: *void*.

The function, verbatim from the source (starts at line 4)

```
5      /* Sets a square back to "unowned": no owner, no mortgage, no loan
6         pledge and no insurance. Used when selling, foreclosing or
7         liquidating a property. */
8      void stripOwnership(GameState game[], int index)
9      {
10         game[0].board[index].property.owner = -1;
11         game[0].board[index].property.mortgaged = 0;
12         game[0].board[index].property.loanLocked = 0;
```

```

13         game[0].board[index].property.insurance = NO_INSURANCE;
14     }

```

Line-by-line explanation

Line	Code	Explanation
4		Blank line.
5	<code>/* Sets a square back to "unowned": no owner, no mortgage, no loan</code>	Comment: <code>/* Sets a square back to "unowned": no owner, no mortgage, no loan</code>
6	<code>pledge and no insurance. Used when selling, foreclosing or</code>	pledge and no insurance. Used when selling, foreclosing or
7	<code>liquidating a property. */</code>	liquidating a property. <code>*/</code>
8	<code>void stripOwnership(GameState game[], int index)</code>	<code>void stripOwnership(GameState game[], int index)</code>
9	<code>{</code>	Opens a new block.
10	<code>game[0].board[index].property.owner = -1;</code>	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = -1.
11	<code>game[0].board[index].property.mortgaged = 0;</code>	Sets the field mortgaged of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed).) = 0.
12	<code>game[0].board[index].property.loanLocked = 0;</code>	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 0.
13	<code>game[0].board[index].property.insurance = NO_INSURANCE;</code>	Sets the field insurance of the property on the board (i.e. Currently held policy type.) = NO_INSURANCE.
14	<code>}</code>	Closes the current block.

Nested functions inside stripOwnership() - study these

- stripOwnership() (Resets a square to unowned: no owner, mortgage, loan pledge or insurance.)

MAIN PURPOSE

Resets a square to unowned: no owner, mortgage, loan pledge or insurance.

6.2 resetLoan()

```
void resetLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 15)

```

16     /* Clears a player's loan completely (amount, rate and rounds). */
17     void resetLoan(GameState game[], int playerIndex)
18     {
19         game[0].players[playerIndex].loan.active = 0;
20         game[0].players[playerIndex].loan.amount = 0;
21         game[0].players[playerIndex].loan.interestRate = 0;
22         game[0].players[playerIndex].loan.remainingRounds = 0;
23     }

```

Line-by-line explanation

Line	Code	Explanation
15		Blank line.
16	<code>/* Clears a player's loan completely (amount, rate and rounds). */</code>	Comment: <code>/* Clears a player's loan completely (amount, rate and rounds). */</code>
17	<code>void resetLoan(GameState game[], int playerIndex)</code>	<code>void resetLoan(GameState game[], int playerIndex)</code>
18	<code>{</code>	Opens a new block.
19	<code>game[0].players[playerIndex].loan.active = 0;</code>	Sets the field active of the player (i.e. 1 when the player has an outstanding loan (only one loan at a time).) = 0.
20	<code>game[0].players[playerIndex].loan.amount = 0;</code>	Sets the field amount of the player (i.e. Current outstanding balance in LKR (grows with compound interest).) = 0.
21	<code>game[0].players[playerIndex].loan.interestRate = 0;</code>	Sets the field interestRate of the player (i.e. Annual/round interest rate the loan is charged at.) = 0.
22	<code>game[0].players[playerIndex].loan.remainingRounds = 0;</code>	Sets the field remainingRounds of the player (i.e. Rounds left before foreclosure.) = 0.

Nested functions inside resetLoan() - study these

- resetLoan() (Wipes a player's loan (active, amount, rate, rounds).)

MAIN PURPOSE

Wipes a player's loan (active, amount, rate, rounds).

6.3 adjustOwnedCount()

```
void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
```

Parameters: *playerIndex, index, delta* | Returns: *void*.

The function, verbatim from the source (starts at line 24)

```
25  /* Adds delta to the owned-count that matches the square's type
26     (property, railway or utility). Pass 1 when buying, -1 when
27     selling or losing a square. */
28  void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
29  {
30      if(game[0].board[index].type == PROPERTY)
31          game[0].players[playerIndex].propertiesOwned += delta;
32      else if(game[0].board[index].type == RAILWAY)
33          game[0].players[playerIndex].railwaysOwned += delta;
34      else if(game[0].board[index].type == UTILITY)
35          game[0].players[playerIndex].utilitiesOwned += delta;
36  }
```

Line-by-line explanation

Line	Code	Explanation
24		Blank line.
25	/* Adds delta to the owned-count that matches the square's type	Comment: /* Adds delta to the owned-count that matches the square's type
26	(property, railway or utility). Pass 1 when buying, -1 when	(property, railway or utility). Pass 1 when buying, -1 when
27	selling or losing a square. */	selling or losing a square. */
28	void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)	void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
29	{	Opens a new block.
30	if(game[0].board[index].type == PROPERTY)	Condition: this square.type == PROPERTY -> run the next block if true.
31	game[0].players[playerIndex].propertiesOwned += delta;	Updates the field propertiesOwned of the player (i.e. Count of colour properties owned.): increases it by delta.
32	else if(game[0].board[index].type == RAILWAY)	A further condition checked only if the previous one failed.
33	game[0].players[playerIndex].railwaysOwned += delta;	Updates the field railwaysOwned of the player (i.e. Count of railways owned.): increases it by delta.
34	else if(game[0].board[index].type == UTILITY)	A further condition checked only if the previous one failed.
35	game[0].players[playerIndex].utilitiesOwned += delta;	Updates the field utilitiesOwned of the player (i.e. Count of utilities owned.): increases it by delta.
36	}	Closes the current block.

Nested functions inside adjustOwnedCount() - study these

- adjustOwnedCount() (Updates the player's owned counter to match the square's type.)

MAIN PURPOSE

Updates the player's owned counter to match the square's type.

6.4 groupOf()

```
PropertyGroup groupOf(GameState game[], int index)
```

Parameters: *index* | Returns: *PropertyGroup, or (PropertyGroup)-1*.

The function, verbatim from the source (starts at line 37)

```
38  /* The colour group of a square, or -1 if it isn't a property. */
39  PropertyGroup groupOf(GameState game[], int index)
40  {
41      if(game[0].board[index].type == PROPERTY)
42          return game[0].board[index].property.group;
43
44      return (PropertyGroup)-1;
45  }
```

Line-by-line explanation

Line	Code	Explanation
37		Blank line.
38	/* The colour group of a square, or -1 if it isn't a property. */	Comment: /* The colour group of a square, or -1 if it isn't a property. */
39	PropertyGroup groupOf(GameState game[], int index)	PropertyGroup groupOf(GameState game[], int index)
40	{	Opens a new block.
41	if(game[0].board[index].type == PROPERTY)	Condition: this square.type == PROPERTY -> run the next block if true.
42	return game[0].board[index].property.group;	Returns game[0].board[index].property.group immediately.
43		Blank line.
44	return (PropertyGroup)-1;	Returns (PropertyGroup)-1 immediately.
45	}	Closes the current block.

Nested functions inside groupOf() - study these

- groupOf() (Returns the colour group of a square, or -1 if it isn't a property.)

MAIN PURPOSE

Returns the colour group of a square, or -1 if it isn't a property.

6.5 receiveMoney()

```
void receiveMoney(GameState game[], int playerIndex, int amount)
```

Parameters: playerIndex, amount | Returns: void.

The function, verbatim from the source (starts at line 47)

```
47  void receiveMoney(GameState game[], int playerIndex, int amount)
48  {
49      /* bankrupt players take no further part in the game (Section 1.3) */
50      if(game[0].players[playerIndex].bankrupt)
51          return;
52
53      game[0].players[playerIndex].cash += amount;
54  }
```

Line-by-line explanation

Line	Code	Explanation
47	void receiveMoney(GameState game[], int playerIndex, int amount)	void receiveMoney(GameState game[], int playerIndex, int amount)
48	{	Opens a new block.
49	/* bankrupt players take no further part in the game (Section 1.3) */	Comment: /* bankrupt players take no further part in the game (Section 1.3) */
50	if(game[0].players[playerIndex].bankrupt)	Condition: the current player.bankrupt -> run the next block if true.
51	return;	Returns the computed value immediately.
52		Blank line.
53	game[0].players[playerIndex].cash += amount;	Updates the field cash of the player (i.e. Cash balance in LKR.): increases it by amount.
54	}	Closes the current block.

Nested functions inside receiveMoney() - study these

- receiveMoney() (Adds money to a player, but only if they are not bankrupt.)

MAIN PURPOSE

Adds money to a player, but only if they are not bankrupt.

6.6 payMoney()

```
void payMoney(GameState game[], int playerIndex, int amount)
```

Parameters: *playerIndex, amount* | Returns: *void*.

The function, verbatim from the source (starts at line 56)

```
56 void payMoney(GameState game[], int playerIndex, int amount)
57 {
58     game[0].players[playerIndex].cash -= amount;

60     /* Section 3.4: a Risk Taker facing bankruptcy first sells their
61        cheapest undeveloped property to the Bank to raise cash.
62        Loan-locked collateral (Rule-LK 3) can't be sold, so stop as
63        soon as a sale fails to reduce the stock of undeveloped
64        properties - otherwise this loop would never end. */
65     while(game[0].players[playerIndex].cash < 0 &&
66           !game[0].players[playerIndex].bankrupt &&
67           game[0].players[playerIndex].strategy == RISK_TAKER)
68     {
69         int before = countUndevelopedProperties(game, playerIndex);

71         sellLowValueProperty(game, playerIndex);

73         if(countUndevelopedProperties(game, playerIndex) >= before)
74             break;
75     }

77     if(game[0].players[playerIndex].cash < 0 && !game[0].players[playerIndex].bankrupt)
78     {
79         game[0].players[playerIndex].bankrupt = 1;

81         printf("\n*** BANKRUPTCY ***\n");
82         printf("%s has been declared bankrupt.\n",
83               game[0].players[playerIndex].name);

85         liquidateBankruptAssets(game, playerIndex);

87         /* cash never stays negative - assets were liquidated, so any
88            remaining debt is written off (Section 1.3) */
89         game[0].players[playerIndex].cash = 0;
90     }
91 }
```

Line-by-line explanation

Line	Code	Explanation
56	void payMoney(GameState game[], int playerIndex, int amount)	void payMoney(GameState game[], int playerIndex, int amount)
57	{	Opens a new block.
58	game[0].players[playerIndex].cash -= amount;	Updates the field cash of the player (i.e. Cash balance in LKR.): decreases it by amount.
59		Blank line.
60	/* Section 3.4: a Risk Taker facing bankruptcy first sells their	Comment: /* Section 3.4: a Risk Taker facing bankruptcy first sells their
61	cheapest undeveloped property to the Bank to raise cash.	cheapest undeveloped property to the Bank to raise cash.
62	Loan-locked collateral (Rule-LK 3) can't be sold, so stop as	Loan-locked collateral (Rule-LK 3) can't be sold, so stop as
	stop as	

63	soon as a sale fails to reduce the stock of undeveloped	soon as a sale fails to reduce the stock of undeveloped
64	properties - otherwise this loop would never end. */	properties - otherwise this loop would never end. */
65	while(game[0].players[playerIndex].cash < 0 &&	Loop: repeat while is true.
66	!game[0].players[playerIndex].bankrupt &&	!game[0].players[playerIndex].bankrupt &&
67	game[0].players[playerIndex].strategy == RISK_TAKER)	Sets the field strategy of the player (i.e. Which strategy drives every decision.) == RISK_TAKER).
68	{	Opens a new block.
69	int before = countUndevelopedProperties(game, playerIndex);	Declares int before = countUndevelopedProperties(game, playerIndex).
70		Blank line.
71	sellLowValueProperty(game, playerIndex);	Calls sellLowValueProperty(game, playerIndex). (Sells the player's cheapest undeveloped property to the Bank at market value (Risk Taker bankruptcy tactic))
72		Blank line.
73	if(countUndevelopedProperties(game, playerIndex) >= before)	Condition: countUndevelopedProperties(game, playerIndex) >= before -> run the next block if true.
74	break;	Breaks out of the current loop or switch.
75	}	Closes the current block.
76		Blank line.
77	if(game[0].players[playerIndex].cash < 0 && !game[0].players[playerIndex].bankrupt)	Condition: the current player.cash < 0 && !the current player.bankrupt -> run the next block if true.
78	{	Opens a new block.
79	game[0].players[playerIndex].bankrupt = 1;	Sets the field bankrupt of the player (i.e. 1 once the player is out of the game.) = 1.
80		Blank line.
81	printf("\n*** BANKRUPTCY ***\n");	Prints the message "\n*** BANKRUPTCY ***\n".
82	printf("%s has been declared bankrupt.\n", game[0].players[playerIndex].name);	Prints the message "%s has been declared bankrupt.\n".
83		game[0].players[playerIndex].name);
84		Blank line.
85	liquidateBankruptAssets(game, playerIndex);	Calls liquidateBankruptAssets(game, playerIndex). (Rule 14: on bankruptcy, demolishes buildings and auctions every owned square)
86		Blank line.
87	/* cash never stays negative - assets were liquidated, so any	Comment: /* cash never stays negative - assets were liquidated, so any
88	remaining debt is written off (Section 1.3) */	remaining debt is written off (Section 1.3) */
89	game[0].players[playerIndex].cash = 0;	Sets the field cash of the player (i.e. Cash balance in LKR.) = 0.
90	}	Closes the current block.
91	}	Closes the current block.

Nested functions inside payMoney() - study these

- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- countUndevelopedProperties() (Counts owned colour properties with no houses and no hotel.)
- sellLowValueProperty() (Sells the player's cheapest undeveloped property to the Bank at market value (Risk Taker b)
- liquidateBankruptAssets() (Rule 14: on bankruptcy, demolishes buildings and auctions every owned square.)

MAIN PURPOSE

Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated.

6.7 liquidateBankruptAssets()

```
void liquidateBankruptAssets(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 92)

```
93 /* Rule 14: on bankruptcy, buildings are demolished and every
```

```

94     property/railway/utility owned is auctioned off (Rule-LK 19). */
95     void liquidateBankruptAssets(GameState game[], int playerIndex)
96     {
97         int i;

99         printf("%s's remaining assets are being liquidated.\n",
100             game[0].players[playerIndex].name);

102         resetLoan(game, playerIndex);

104         for(i = 0; i < BOARD_SIZE; i++)
105         {
106             if(game[0].board[i].property.owner != playerIndex)
107                 continue;

109             demolishBuildingsOn(game, i);

111             stripOwnership(game, i);
112             game[0].board[i].property.damaged = 0;
113             game[0].board[i].property.repairCostOwed = 0;

115             adjustOwnedCount(game, playerIndex, i, -1);

117             runAuction(game, i, -1);
118         }
119     }

```

Line-by-line explanation

Line	Code	Explanation
92		Blank line.
93	/* Rule 14: on bankruptcy, buildings are demolished and every	Comment: /* Rule 14: on bankruptcy, buildings are demolished and every
94	property/railway/utility owned is auctioned off (Rule-LK 19). */	property/railway/utility owned is auctioned off (Rule-LK 19). */
95	void liquidateBankruptAssets(GameState game[], int playerIndex)	void liquidateBankruptAssets(GameState game[], int playerIndex)
96	{	Opens a new block.
97	int i;	Declares int i.
98		Blank line.
99	printf("%s's remaining assets are being liquidated.\n",	Prints the message "%s's remaining assets are being liquidated.\n".
100	game[0].players[playerIndex].name);	game[0].players[playerIndex].name);
101		Blank line.
102	resetLoan(game, playerIndex);	Calls resetLoan(game, playerIndex). (Wipes a player's loan (active, amount, rate, rounds))
103		Blank line.
104	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
105	{	Opens a new block.
106	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
107	continue;	Skips to the next loop iteration.
108		Blank line.
109	demolishBuildingsOn(game, i);	Calls demolishBuildingsOn(game, i). (Zeroes houses and hotel on a property (used on seizure and bankruptcy))
110		Blank line.
111	stripOwnership(game, i);	Calls stripOwnership(game, i). (Resets a square to unowned: no owner, mortgage, loan pledge or insurance)
112	game[0].board[i].property.damaged = 0;	Sets the field damaged of the property on the board (i.e. 1 after a disaster until repaired (no rent).) = 0.
113	game[0].board[i].property.repairCostOwed = 0;	Sets the field repairCostOwed of the property on the board (i.e. Remaining repair cost after insurance compensation.) = 0.
114		Blank line.

115	adjustOwnedCount(game, playerIndex, i, -1);	Calls adjustOwnedCount(game, playerIndex, i, -1). (Updates the player's owned counter to match the square's type)
116		Blank line.
117	runAuction(game, i, -1);	Calls runAuction(game, i, -1). (Sells a property to the highest bidder among solvent players)
118	}	Closes the current block.
119	}	Closes the current block.

Nested functions inside liquidateBankruptAssets() - study these

- liquidateBankruptAssets() (Rule 14: on bankruptcy, demolishes buildings and auctions every owned square.)
- resetLoan() (Wipes a player's loan (active, amount, rate, rounds).)
- demolishBuildingsOn() (Zeroes houses and hotel on a property (used on seizure and bankruptcy).)
- stripOwnership() (Resets a square to unowned: no owner, mortgage, loan pledge or insurance.)
- adjustOwnedCount() (Updates the player's owned counter to match the square's type.)
- runAuction() (Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise)

MAIN PURPOSE

Rule 14: on bankruptcy, demolishes buildings and auctions every owned square.

6.8 payTax()

```
void payTax(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 120)

```

121  /* Income Tax (Rule-LK): 15% of the player's net worth at the base
122     rate. Market conditions scale the rate (currentTaxRatePercent). */
123  void payTax(GameState game[], int playerIndex)
124  {
125      int rate;
126      int netWorth;
127      int amount;

129      rate = currentTaxRatePercent(game, INCOME_TAX_RATE);
130      netWorth = calculateNetWorth(game, playerIndex);
131      amount = (netWorth * rate) / 100;

133      /* a bankrupt player's net worth can be negative - a tax can never
134         pay the player money, so clamp it at zero */
135      if(amount < 0)
136          amount = 0;

138      printf("\n%s landed on Income Tax.\n",
139             game[0].players[playerIndex].name);

141      printf("Tax Rate : %d%%\n", rate);
142      printf("Net Worth : LKR %d\n", netWorth);
143      printf("%s paid tax : LKR %d\n",
144             game[0].players[playerIndex].name,
145             amount);

147      payMoney(game, playerIndex, amount);
148  }
```

Line-by-line explanation

Line	Code	Explanation
120		Blank line.
121	/* Income Tax (Rule-LK): 15% of the player's net worth at the base	Comment: /* Income Tax (Rule-LK): 15% of the player's net worth at the base
122	rate. Market conditions scale the rate (currentTaxRatePercent). */	rate. Market conditions scale the rate (currentTaxRatePercent). */
123	void payTax(GameState game[], int playerIndex)	void payTax(GameState game[], int playerIndex)

124	{	Opens a new block.
125	int rate;	Declares int rate.
126	int netWorth;	Declares int netWorth.
127	int amount;	Declares int amount.
128		Blank line.
129	rate = currentTaxRatePercent(game, INCOME_TAX_RATE);	rate = currentTaxRatePercent(game, INCOME_TAX_RATE);
130	netWorth = calculateNetWorth(game, playerIndex);	netWorth = calculateNetWorth(game, playerIndex);
131	amount = (netWorth * rate) / 100;	amount = (netWorth * rate) / 100;
132		Blank line.
133	/* a bankrupt player's net worth can be negative - a tax can never	Comment: /* a bankrupt player's net worth can be negative - a tax can never
134	pay the player money, so clamp it at zero */	pay the player money, so clamp it at zero */
135	if(amount < 0)	Condition: amount < 0 -> run the next block if true.
136	amount = 0;	amount = 0;
137		Blank line.
138	printf("\n%s landed on Income Tax.\n",	Prints the message "\n%s landed on Income Tax.\n".
139	game[0].players[playerIndex].name);	game[0].players[playerIndex].name);
140		Blank line.
141	printf("Tax Rate : %d%%\n", rate);	Prints the message "Tax Rate : %d%%\n".
142	printf("Net Worth : LKR %d\n", netWorth);	Prints the message "Net Worth : LKR %d\n".
143	printf("%s paid tax : LKR %d\n",	Prints the message "%s paid tax : LKR %d\n".
144	game[0].players[playerIndex].name,	game[0].players[playerIndex].name,
145	amount);	amount);
146		Blank line.
147	payMoney(game, playerIndex, amount);	Calls payMoney(game, playerIndex, amount). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
148	}	Closes the current block.

Nested functions inside payTax() - study these

- payTax() (Income Tax: currentTaxRatePercent(INCOME_TAX_RATE) x net worth.)
- currentTaxRatePercent() (The effective tax rate: base rate scaled by MOD_VALUE_GLOBAL.)
- calculateNetWorth() (Rule 15 net worth: cash + property + buildings - outstanding loan.)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Income Tax: currentTaxRatePercent(INCOME_TAX_RATE) x net worth.

6.9 payCommunityFundTax()

```
void payCommunityFundTax(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 149)

```
150  /* Community Development Fund (Rule-LK): 10% of the player's total
151     assets at current market rate. Only property (land) counts -
152     buildings are not included. The 10% rate is scaled by market
153     conditions just like the Income Tax rate. */
154  void payCommunityFundTax(GameState game[], int playerIndex)
155  {
156      int rate;
157      int assets;
158      int amount;

160      rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);
161      assets = calculatePropertyValue(game, playerIndex);
162      amount = (assets * rate) / 100;

164      printf("\n%s landed on Community Development Fund.\n",
165             game[0].players[playerIndex].name);
```

```

167     printf("Tax Rate : %d%%\n", rate);
168     printf("Property Assets (market rate) : LKR %d\n", assets);
169     printf("%s paid community development tax : LKR %d\n",
170           game[0].players[playerIndex].name,
171           amount);

173     payMoney(game, playerIndex, amount);
174 }

```

Line-by-line explanation

Line	Code	Explanation
149		Blank line.
150	<code>/* Community Development Fund (Rule-LK): 10% of the player's total</code>	Comment: <code>/* Community Development Fund (Rule-LK): 10% of the player's total</code>
151	<code>assets at current market rate. Only property (land) counts -</code>	<code>assets at current market rate. Only property (land) counts -</code>
152	<code>buildings are not included. The 10% rate is scaled by market</code>	<code>buildings are not included. The 10% rate is scaled by market</code>
153	<code>conditions just like the Income Tax rate. */</code>	<code>conditions just like the Income Tax rate. */</code>
154	<code>void payCommunityFundTax(GameState game[], int playerIndex)</code>	<code>void payCommunityFundTax(GameState game[], int playerIndex)</code>
155	<code>{</code>	Opens a new block.
156	<code>int rate;</code>	Declares int rate.
157	<code>int assets;</code>	Declares int assets.
158	<code>int amount;</code>	Declares int amount.
159		Blank line.
160	<code>rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);</code>	<code>rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);</code>
161	<code>assets = calculatePropertyValue(game, playerIndex);</code>	<code>assets = calculatePropertyValue(game, playerIndex);</code>
162	<code>amount = (assets * rate) / 100;</code>	<code>amount = (assets * rate) / 100;</code>
163		Blank line.
164	<code>printf("\n%s landed on Community Development Fund.\n",</code>	<code>Prints the message "\n%s landed on Community Development Fund.\n".</code>
165	<code>game[0].players[playerIndex].name);</code>	<code>game[0].players[playerIndex].name);</code>
166		Blank line.
167	<code>printf("Tax Rate : %d%%\n", rate);</code>	<code>Prints the message "Tax Rate : %d%%\n".</code>
168	<code>printf("Property Assets (market rate) : LKR %d\n", assets);</code>	<code>Prints the message "Property Assets (market rate) : LKR %d\n".</code>
169	<code>printf("%s paid community development tax : LKR %d\n",</code>	<code>Prints the message "%s paid community development tax : LKR %d\n".</code>
170	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
171	<code>amount);</code>	<code>amount);</code>
172		Blank line.
173	<code>payMoney(game, playerIndex, amount);</code>	<code>Calls payMoney(game, playerIndex, amount). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)</code>
174	<code>}</code>	<code>Closes the current block.</code>

Nested functions inside payCommunityFundTax() - study these

- `payCommunityFundTax()` (Community Development Fund: `currentTaxRatePercent(COMMUNITY_FUND_TAX_RATE)` x property (lan)
- `currentTaxRatePercent()` (The effective tax rate: base rate scaled by `MOD_VALUE_GLOBAL`.)
- `calculatePropertyValue()` (Total current market value of owned, unmortgaged squares.)
- `payMoney()` (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Community Development Fund: `currentTaxRatePercent(COMMUNITY_FUND_TAX_RATE)` x property (land) value, not buildings.

6.10 findPropertyToMortgage()

```
int findPropertyToMortgage(GameState game[], int playerIndex)
```

Parameters: none | Returns: Square index, or -1 if none.

The function, verbatim from the source (starts at line 176)

```
176 int findPropertyToMortgage(GameState game[], int playerIndex)
177 {
178     int i;

180     for(i = 0; i < BOARD_SIZE; i++)
181     {
182         if(game[0].board[i].property.owner != playerIndex)
183             continue;

185         if(game[0].board[i].property.mortgaged)
186             continue;

188         if(game[0].board[i].property.loanLocked)
189             continue;

191         /* must be undeveloped before it can be mortgaged */
192         if(game[0].board[i].type == PROPERTY &&
193            (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
194         {
195             continue;
196         }

198         return i;
199     }

201     return -1;
202 }
```

Line-by-line explanation

Line	Code	Explanation
176	int findPropertyToMortgage(GameState game[], int playerIndex)	int findPropertyToMortgage(GameState game[], int playerIndex)
177	{	Opens a new block.
178	int i;	Declares int i.
179		Blank line.
180	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
181	{	Opens a new block.
182	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
183	continue;	Skips to the next loop iteration.
184		Blank line.
185	if(game[0].board[i].property.mortgaged)	Condition: square i.property.mortgaged -> run the next block if true.
186	continue;	Skips to the next loop iteration.
187		Blank line.
188	if(game[0].board[i].property.loanLocked)	Condition: square i.property.loanLocked -> run the next block if true.
189	continue;	Skips to the next loop iteration.
190		Blank line.
191	/* must be undeveloped before it can be mortgaged */	Comment: /* must be undeveloped before it can be mortgaged */
192	if(game[0].board[i].type == PROPERTY &&	Condition: -> run the next block if true.
193	(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel))	(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel))
194	{	Opens a new block.
195	continue;	Skips to the next loop iteration.
196	}	Closes the current block.
197		Blank line.
198	return i;	Returns i immediately.

199	}	Closes the current block.
200		Blank line.
201	return -1;	Returns -1 immediately.
202	}	Closes the current block.

Nested functions inside findPropertyToMortgage() - study these

- findPropertyToMortgage() (Finds the first mortgageable owned property.)

MAIN PURPOSE
 Finds the first mortgageable owned property.

6.11 findMortgagedProperty()

```
int findMortgagedProperty(GameState game[], int playerIndex)
```

Parameters: none | Returns: Square index, or -1 if none.

The function, verbatim from the source (starts at line 204)

```

204  int findMortgagedProperty(GameState game[], int playerIndex)
205  {
206      int i;

208      for(i = 0; i < BOARD_SIZE; i++)
209      {
210          if(game[0].board[i].property.owner == playerIndex &&
211             game[0].board[i].property.mortgaged)
212          {
213              return i;
214          }
215      }

217      return -1;
218  }
```

Line-by-line explanation

Line	Code	Explanation
204	int findMortgagedProperty(GameState game[], int playerIndex)	int findMortgagedProperty(GameState game[], int playerIndex)
205	{	Opens a new block.
206	int i;	Declares int i.
207		Blank line.
208	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
209	{	Opens a new block.
210	if(game[0].board[i].property.owner == playerIndex &&	Condition: -> run the next block if true.
211	game[0].board[i].property.mortgaged)	game[0].board[i].property.mortgaged)
212	{	Opens a new block.
213	return i;	Returns i immediately.
214	}	Closes the current block.
215	}	Closes the current block.
216		Blank line.
217	return -1;	Returns -1 immediately.
218	}	Closes the current block.

Nested functions inside findMortgagedProperty() - study these

- findMortgagedProperty() (Finds the first mortgaged owned property.)

MAIN PURPOSE
 Finds the first mortgaged owned property.

6.12 mortgageProperty()

```
void mortgageProperty(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 220)

```
220 void mortgageProperty(GameState game[], int playerIndex)
221 {
222     int propIndex;
223     int payout;
224     PropertyGroup group;
225
226     if(!shouldMortgage(game, playerIndex))
227         return;
228
229     propIndex = findPropertyToMortgage(game, playerIndex);
230
231     if(propIndex == -1)
232         return;
233
234     /* Rule-LK 34: a booming group's mortgage values rise by 15%, a
235        declining group's fall by 10%. Railways and utilities are
236        unaffected. */
237     group = groupOf(game, propIndex);
238
239     payout = (game[0].board[propIndex].property.mortgageValue *
240              modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;
241
242     game[0].board[propIndex].property.mortgaged = 1;
243
244     receiveMoney(game, playerIndex, payout);
245
246     printf("\n%s mortgaged %s for LKR %d.\n",
247           game[0].players[playerIndex].name,
248           game[0].board[propIndex].name,
249           payout);
250 }
```

Line-by-line explanation

Line	Code	Explanation
220	<code>void mortgageProperty(GameState game[], int playerIndex)</code>	<code>void mortgageProperty(GameState game[], int playerIndex)</code>
221	<code>{</code>	Opens a new block.
222	<code>int propIndex;</code>	Declares <code>int propIndex</code> .
223	<code>int payout;</code>	Declares <code>int payout</code> .
224	<code>PropertyGroup group;</code>	<code>PropertyGroup group</code> ;
225		Blank line.
226	<code>if(!shouldMortgage(game, playerIndex))</code>	Condition: <code>!shouldMortgage(game, playerIndex)</code> -> run the next block if true.
227	<code>return;</code>	Returns the computed value immediately.
228		Blank line.
229	<code>propIndex = findPropertyToMortgage(game, playerIndex);</code>	<code>propIndex = findPropertyToMortgage(game, playerIndex);</code>
230		Blank line.
231	<code>if(propIndex == -1)</code>	Condition: <code>propIndex == -1</code> -> run the next block if true.
232	<code>return;</code>	Returns the computed value immediately.
233		Blank line.
234	<code>/* Rule-LK 34: a booming group's mortgage values rise by 15%, a</code>	Comment: <code>/* Rule-LK 34: a booming group's mortgage values rise by 15%, a</code>
235	<code>declining group's fall by 10%. Railways and utilities are</code>	<code>declining group's fall by 10%. Railways and utilities are</code>
236	<code>unaffected. */</code>	<code>unaffected. */</code>
237	<code>group = groupOf(game, propIndex);</code>	<code>group = groupOf(game, propIndex);</code>

238		Blank line.
239	payout = (game[0].board[propIndex].property.mortgageValue *	payout = (game[0].board[propIndex].property.mortgageValue *
240	modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;	modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;
241		Blank line.
242	game[0].board[propIndex].property.mortgaged = 1;	Sets the field mortgaged of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed).) = 1.
243		Blank line.
244	receiveMoney(game, playerIndex, payout);	Calls receiveMoney(game, playerIndex, payout). (Adds money to a player, but only if they are not bankrupt)
245		Blank line.
246	printf("\n%s mortgaged %s for LKR %d.\n",	Prints the message "\n%s mortgaged %s for LKR %d.\n".
247	game[0].players[playerIndex].name,	game[0].players[playerIndex].name,
248	game[0].board[propIndex].name,	game[0].board[propIndex].name,
249	payout);	payout);
250	}	Closes the current block.

Nested functions inside mortgageProperty() - study these

- mortgageProperty() (Mortgages the first eligible property if the strategy wants to.)
- shouldMortgage() (Whether the player mortgages a property.)
- findPropertyToMortgage() (Finds the first mortgageable owned property.)
- groupOf() (Returns the colour group of a square, or -1 if it isn't a property.)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- receiveMoney() (Adds money to a player, but only if they are not bankrupt.)

MAIN PURPOSE

Mortgages the first eligible property if the strategy wants to.

6.13 redeemMortgage()

```
void redeemMortgage(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 252)

```

252 void redeemMortgage(GameState game[], int playerIndex)
253 {
254     int propIndex;
255     int redeemCost;

257     propIndex = findMortgagedProperty(game, playerIndex);

259     if(propIndex == -1)
260         return;

262     /* pay back the mortgage value plus 10% interest */
263     redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;

265     if(!shouldRedeemMortgage(game, playerIndex, redeemCost))
266         return;

268     if(game[0].players[playerIndex].cash < redeemCost)
269         return;

271     payMoney(game, playerIndex, redeemCost);

273     game[0].board[propIndex].property.mortgaged = 0;

275     printf("\n%s redeemed the mortgage on %s for LKR %d.\n",
276           game[0].players[playerIndex].name,
277           game[0].board[propIndex].name,
278           redeemCost);

```

Line-by-line explanation

Line	Code	Explanation
252	void redeemMortgage(GameState game[], int playerIndex)	void redeemMortgage(GameState game[], int playerIndex)
253	{	Opens a new block.
254	int propIndex;	Declares int propIndex.
255	int redeemCost;	Declares int redeemCost.
256		Blank line.
257	propIndex = findMortgagedProperty(game, playerIndex);	propIndex = findMortgagedProperty(game, playerIndex);
258		Blank line.
259	if(propIndex == -1)	Condition: propIndex == -1 -> run the next block if true.
260	return;	Returns the computed value immediately.
261		Blank line.
262	/* pay back the mortgage value plus 10% interest */	Comment: /* pay back the mortgage value plus 10% interest */
263	redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;	redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;
264		Blank line.
265	if(!shouldRedeemMortgage(game, playerIndex, redeemCost))	Condition: !shouldRedeemMortgage(game, playerIndex, redeemCost) -> run the next block if true.
266	return;	Returns the computed value immediately.
267		Blank line.
268	if(game[0].players[playerIndex].cash < redeemCost)	Condition: the current player.cash < redeemCost -> run the next block if true.
269	return;	Returns the computed value immediately.
270		Blank line.
271	payMoney(game, playerIndex, redeemCost);	Calls payMoney(game, playerIndex, redeemCost). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
272		Blank line.
273	game[0].board[propIndex].property.mortgaged = 0;	Sets the field mortgaged of the property on the board (i.e. 1 if the property is mortgaged (no rent until redeemed).) = 0.
274		Blank line.
275	printf("\n%s redeemed the mortgage on %s for LKR %d.\n",	Prints the message "\n%s redeemed the mortgage on %s for LKR %d.\n".
276	game[0].players[playerIndex].name,	game[0].players[playerIndex].name,
277	game[0].board[propIndex].name,	game[0].board[propIndex].name,
278	redeemCost);	redeemCost);
279	}	Closes the current block.

Nested functions inside redeemMortgage() - study these

- redeemMortgage() (Redeems the first mortgaged property at 110% of value.)
- findMortgagedProperty() (Finds the first mortgaged owned property.)
- shouldRedeemMortgage() (Whether the player redeems a mortgaged property.)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Redeems the first mortgaged property at 110% of value.

6.14 handleMortgageDecisions()

```
void handleMortgageDecisions(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 281)

```
281 void handleMortgageDecisions(GameState game[], int playerIndex)
282 {
283     redeemMortgage(game, playerIndex);
284     mortgageProperty(game, playerIndex);
285 }
```

Line-by-line explanation

Line	Code	Explanation
281	void handleMortgageDecisions(GameState game[], int playerIndex)	void handleMortgageDecisions(GameState game[], int playerIndex)
282	{	Opens a new block.
283	redeemMortgage(game, playerIndex);	Calls redeemMortgage(game, playerIndex). (Redeems the first mortgaged property at 110% of value)
284	mortgageProperty(game, playerIndex);	Calls mortgageProperty(game, playerIndex). (Mortgages the first eligible property if the strategy wants to)
285	}	Closes the current block.

Nested functions inside handleMortgageDecisions() - study these

- handleMortgageDecisions() (The turn-end mortgage step: try redeeming first, then mortgaging.)
- redeemMortgage() (Redeems the first mortgaged property at 110% of value.)
- mortgageProperty() (Mortgages the first eligible property if the strategy wants to.)

MAIN PURPOSE

The turn-end mortgage step: try redeeming first, then mortgaging.

6.15 calculateRent()

```
int calculateRent(GameState game[], int position)
```

Parameters: position | Returns: Rent in LKR.

The function, verbatim from the source (starts at line 286)

```
287 /* Rent for a normal property, based on houses/hotel (Table 6),
288    scaled by any active bonuses and by building condition (Table 3). */
289 int calculateRent(GameState game[], int position)
290 {
291     int rent;
292     int conditionPercent;
293     int houses;
294     int hotel;
295     int baseRent;
296     PropertyGroup group;
297     int condition;
298
299     houses = game[0].board[position].property.houses;
300     hotel = game[0].board[position].property.hotel;
301     baseRent = game[0].board[position].property.baseRent;
302     group = game[0].board[position].property.group;
303     condition = game[0].board[position].property.condition;
304
305     if(hotel)
306     {
307         rent = baseRent * 10;
308     }
309     else
310     {
311         if(houses == 0)
312             rent = baseRent * 1;
313         else if(houses == 1)
314             rent = baseRent * 2;
315         else if(houses == 2)
316             rent = baseRent * 3;
317         else if(houses == 3)
```

```

318         rent = baseRent * 5;
319     else
320         rent = baseRent * 7;
321     }

323     /* Rule-LK 34: all matching modifiers multiply together. */
324     rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;
325     rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

327     if(hotel)
328         rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;

330     if(houses > 0 || hotel)
331     {
332         conditionPercent = rentConditionPercent(condition);
333         rent = (rent * conditionPercent) / 100;
334     }

336     return rent;
337 }

```

Line-by-line explanation

Line	Code	Explanation
286		Blank line.
287	/* Rent for a normal property, based on houses/hotel (Table 6),	Comment: /* Rent for a normal property, based on houses/hotel (Table 6),
288	scaled by any active bonuses and by building condition (Table 3). */	scaled by any active bonuses and by building condition (Table 3). */
289	int calculateRent(GameState game[], int position)	int calculateRent(GameState game[], int position)
290	{	Opens a new block.
291	int rent;	Declares int rent.
292	int conditionPercent;	Declares int conditionPercent.
293	int houses;	Declares int houses.
294	int hotel;	Declares int hotel.
295	int baseRent;	Declares int baseRent.
296	PropertyGroup group;	PropertyGroup group;
297	int condition;	Declares int condition.
298		Blank line.
299	houses = game[0].board[position].property.houses;	houses = game[0].board[position].property.houses;
300	hotel = game[0].board[position].property.hotel;	hotel = game[0].board[position].property.hotel;
301	baseRent = game[0].board[position].property.baseRent;	baseRent = game[0].board[position].property.baseRent;
302	group = game[0].board[position].property.group;	group = game[0].board[position].property.group;
303	condition = game[0].board[position].property.condition;	condition = game[0].board[position].property.condition;
304		Blank line.
305	if(hotel)	Condition: hotel -> run the next block if true.
306	{	Opens a new block.
307	rent = baseRent * 10;	rent = baseRent * 10;
308	}	Closes the current block.
309	else	The alternative branch when the condition above is false.
310	{	Opens a new block.
311	if(houses == 0)	Condition: houses == 0 -> run the next block if true.
312	rent = baseRent * 1;	rent = baseRent * 1;
313	else if(houses == 1)	A further condition checked only if the previous one failed.
314	rent = baseRent * 2;	rent = baseRent * 2;
315	else if(houses == 2)	A further condition checked only if the previous one failed.
316	rent = baseRent * 3;	rent = baseRent * 3;
317	else if(houses == 3)	A further condition checked only if the previous one failed.
318	rent = baseRent * 5;	rent = baseRent * 5;
319	else	The alternative branch when the condition above is false.

320	rent = baseRent * 7;	rent = baseRent * 7;
321	}	Closes the current block.
322		Blank line.
323	/* Rule-LK 34: all matching modifiers multiply together. */	Comment: /* Rule-LK 34: all matching modifiers multiply together. */
324	rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;	rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;
325	rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;	rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;
326		Blank line.
327	if(hotel)	Condition: hotel -> run the next block if true.
328	rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;	rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;
329		Blank line.
330	if(houses > 0 hotel)	Condition: houses > 0 hotel -> run the next block if true.
331	{	Opens a new block.
332	conditionPercent = rentConditionPercent(condition);	conditionPercent = rentConditionPercent(condition);
333	rent = (rent * conditionPercent) / 100;	rent = (rent * conditionPercent) / 100;
334	}	Closes the current block.
335		Blank line.
336	return rent;	Returns rent immediately.
337	}	Closes the current block.

Nested functions inside calculateRent() - study these

- calculateRent() (Rent for a normal property from Table 6 (houses/hotel), scaled by group/global/hotel modif)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- rentConditionPercent() (Table 3: maps building condition to the fraction of rent collected.)

MAIN PURPOSE

Rent for a normal property from Table 6 (houses/hotel), scaled by group/global/hotel modifiers and building condition (Table 3).

6.16 calculateRailwayRent()

```
int calculateRailwayRent(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *Rent in LKR.*

The function, verbatim from the source (starts at line 338)

```

339  /* Rent for a railway, based on how many the same owner has (Table 7). */
340  int calculateRailwayRent(GameState game[], int playerIndex)
341  {
342      int i;
343      int count;
344      int baseRent;

346      count = 0;

348      for(i = 0; i < BOARD_SIZE; i++)
349      {
350          if(game[0].board[i].type == RAILWAY &&
351             game[0].board[i].property.owner == playerIndex)
352          {
353              count++;
354          }
355      }

357      if(count == 1)
358          baseRent = RAILWAY_RENT_1;
359      else if(count == 2)
360          baseRent = RAILWAY_RENT_2;
361      else if(count == 3)
362          baseRent = RAILWAY_RENT_3;
```

```

363     else if(count == 4)
364         baseRent = RAILWAY_RENT_4;
365     else
366         baseRent = 0;

368     baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;
369     baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

371     return baseRent;
372 }

```

Line-by-line explanation

Line	Code	Explanation
338		Blank line.
339	<code>/* Rent for a railway, based on how many the same owner has (Table 7). */</code>	Comment: <code>/* Rent for a railway, based on how many the same owner has (Table 7). */</code>
340	<code>int calculateRailwayRent(GameState game[], int playerIndex)</code>	<code>int calculateRailwayRent(GameState game[], int playerIndex)</code>
341	<code>{</code>	Opens a new block.
342	<code>int i;</code>	Declares int i.
343	<code>int count;</code>	Declares int count.
344	<code>int baseRent;</code>	Declares int baseRent.
345		Blank line.
346	<code>count = 0;</code>	<code>count = 0;</code>
347		Blank line.
348	<code>for(i = 0; i < BOARD_SIZE; i++)</code>	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
349	<code>{</code>	Opens a new block.
350	<code>if(game[0].board[i].type == RAILWAY &&</code>	Condition: -> run the next block if true.
351	<code>game[0].board[i].property.owner == playerIndex)</code>	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) == playerIndex).
352	<code>{</code>	Opens a new block.
353	<code>count++;</code>	<code>count++;</code>
354	<code>}</code>	Closes the current block.
355	<code>}</code>	Closes the current block.
356		Blank line.
357	<code>if(count == 1)</code>	Condition: <code>count == 1</code> -> run the next block if true.
358	<code>baseRent = RAILWAY_RENT_1;</code>	<code>baseRent = RAILWAY_RENT_1;</code>
359	<code>else if(count == 2)</code>	A further condition checked only if the previous one failed.
360	<code>baseRent = RAILWAY_RENT_2;</code>	<code>baseRent = RAILWAY_RENT_2;</code>
361	<code>else if(count == 3)</code>	A further condition checked only if the previous one failed.
362	<code>baseRent = RAILWAY_RENT_3;</code>	<code>baseRent = RAILWAY_RENT_3;</code>
363	<code>else if(count == 4)</code>	A further condition checked only if the previous one failed.
364	<code>baseRent = RAILWAY_RENT_4;</code>	<code>baseRent = RAILWAY_RENT_4;</code>
365	<code>else</code>	The alternative branch when the condition above is false.
366	<code>baseRent = 0;</code>	<code>baseRent = 0;</code>
367		Blank line.
368	<code>baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;</code>	<code>baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;</code>
369	<code>baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;</code>	<code>baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;</code>
370		Blank line.
371	<code>return baseRent;</code>	Returns baseRent immediately.
372	<code>}</code>	Closes the current block.

Nested functions inside calculateRailwayRent() - study these

- `calculateRailwayRent()` (Railway rent from Table 7 based on how many stations the OWNER has, scaled by rail/global)
- `modifierMultiplier()` (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

Railway rent from Table 7 based on how many stations the OWNER has, scaled by rail/global rent modifiers.

6.17 calculateUtilityRent()

```
int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
```

Parameters: *playerIndex, diceValue* | Returns: *Rent in LKR.*

The function, verbatim from the source (starts at line 373)

```
374  /* Rent for a utility, based on the dice roll just made (Table 8). */
375  int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
376  {
377      int i;
378      int count;
379      int rent;

381      count = 0;

383      for(i = 0; i < BOARD_SIZE; i++)
384      {
385          if(game[0].board[i].type == UTILITY &&
386             game[0].board[i].property.owner == playerIndex)
387          {
388              count++;
389          }
390      }

392      if(count == 2)
393          rent = UTILITY_RENT_TWO * diceValue;
394      else if(count == 1)
395          rent = UTILITY_RENT_ONE * diceValue;
396      else
397          rent = 0;

399      rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;
400      rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

402      return rent;
403  }
```

Line-by-line explanation

Line	Code	Explanation
373		Blank line.
374	/* Rent for a utility, based on the dice roll just made (Table 8). */	Comment: /* Rent for a utility, based on the dice roll just made (Table 8). */
375	int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)	int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
376	{	Opens a new block.
377	int i;	Declares int i.
378	int count;	Declares int count.
379	int rent;	Declares int rent.
380		Blank line.
381	count = 0;	count = 0;
382		Blank line.
383	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
384	{	Opens a new block.
385	if(game[0].board[i].type == UTILITY &&	Condition: -> run the next block if true.
386	game[0].board[i].property.owner == playerIndex)	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = playerIndex).
387	{	Opens a new block.
388	count++;	count++;
389	}	Closes the current block.

390	}	Closes the current block.
391		Blank line.
392	if(count == 2)	Condition: count == 2 -> run the next block if true.
393	rent = UTILITY_RENT_TWO * diceValue;	rent = UTILITY_RENT_TWO * diceValue;
394	else if(count == 1)	A further condition checked only if the previous one failed.
395	rent = UTILITY_RENT_ONE * diceValue;	rent = UTILITY_RENT_ONE * diceValue;
396	else	The alternative branch when the condition above is false.
397	rent = 0;	rent = 0;
398		Blank line.
399	rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;	rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;
400	rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;	rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;
401		Blank line.
402	return rent;	Returns rent immediately.
403	}	Closes the current block.

Nested functions inside calculateUtilityRent() - study these

- calculateUtilityRent() (Utility rent from Table 8: dice x 4 (one utility) or dice x 10 (both), scaled by util/glob)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

Utility rent from Table 8: dice x 4 (one utility) or dice x 10 (both), scaled by util/global modifiers.

6.18 groupSize()

```
int groupSize(GameState game[], PropertyGroup group)
```

Parameters: group | Returns: Count (2 or 3 in this board).

The function, verbatim from the source (starts at line 405)

```

405  int groupSize(GameState game[], PropertyGroup group)
406  {
407      int i;
408      int count;

410      count = 0;

412      for(i = 0; i < BOARD_SIZE; i++)
413      {
414          if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
415              count++;
416      }

418      return count;
419  }
```

Line-by-line explanation

Line	Code	Explanation
405	int groupSize(GameState game[], PropertyGroup group)	int groupSize(GameState game[], PropertyGroup group)
406	{	Opens a new block.
407	int i;	Declares int i.
408	int count;	Declares int count.
409		Blank line.
410	count = 0;	count = 0;
411		Blank line.
412	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
413	{	Opens a new block.
414	if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)	Condition: square i.type == PROPERTY && square i.property.group == group -> run the next block if true.
415	count++;	count++;
416	}	Closes the current block.

417		Blank line.
418	return count;	Returns count immediately.
419	}	Closes the current block.

Nested functions inside groupSize() - study these

- groupSize() (How many squares belong to a colour group.)

MAIN PURPOSE

How many squares belong to a colour group.

6.19 ownsMonopoly()

```
int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
```

Parameters: playerIndex, group | Returns: 1 = full monopoly, 0 = not.

The function, verbatim from the source (starts at line 421)

```
421 int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
422 {
423     int i;
424     int owned;
425
426     owned = 0;
427
428     for(i = 0; i < BOARD_SIZE; i++)
429     {
430         if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
431         {
432             if(game[0].board[i].property.owner == playerIndex)
433                 owned++;
434         }
435     }
436
437     return (owned == groupSize(game, group));
438 }
```

Line-by-line explanation

Line	Code	Explanation
421	int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)	int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
422	{	Opens a new block.
423	int i;	Declares int i.
424	int owned;	Declares int owned.
425		Blank line.
426	owned = 0;	owned = 0;
427		Blank line.
428	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
429	{	Opens a new block.
430	if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)	Condition: square i.type == PROPERTY && square i.property.group == group -> run the next block if true.
431	{	Opens a new block.
432	if(game[0].board[i].property.owner == playerIndex)	Condition: square i.property.owner == playerIndex -> run the next block if true.
433	owned++;	owned++;
434	}	Closes the current block.
435	}	Closes the current block.
436		Blank line.
437	return (owned == groupSize(game, group));	Returns (owned == groupSize(game, group)) immediately.
438	}	Closes the current block.

Nested functions inside ownsMonopoly() - study these

- ownsMonopoly() (Whether the player owns every member of the group.)
- groupSize() (How many squares belong to a colour group.)

MAIN PURPOSE

Whether the player owns every member of the group.

6.20 developGroup()

```
int developGroup(GameState game[], int playerIndex, PropertyGroup group)
```

Parameters: *playerIndex, group* | Returns: *1 = built, 0 = nothing built.*

The function, verbatim from the source (starts at line 439)

```
440  /* Builds one house (or upgrades to a hotel) somewhere in this
441  colour group. Always adds to whichever property currently has the
442  fewest houses, so development stays even (Rule 9). Returns 1 if
443  something was actually built, 0 if not - constructBuildings()
444  uses this to keep building repeatedly in the same turn. */
445  int developGroup(GameState game[], int playerIndex, PropertyGroup group)
446  {
447      int i;
448      int minHouses;
449      int targetIndex;
450      int allFourHouses;

452      if(!ownsMonopoly(game, playerIndex, group))
453          return 0;

455      minHouses = MAX_HOUSES + 1;
456      targetIndex = -1;
457      allFourHouses = 1;

459      for(i = 0; i < BOARD_SIZE; i++)
460      {
461          if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
462          {
463              /* can't build on a mortgaged, damaged or loan-locked
464              property - those earn no rent and/or are pledged */
465              if(game[0].board[i].property.mortgaged ||
466                 game[0].board[i].property.damaged ||
467                 game[0].board[i].property.loanLocked)
468              {
469                  /* Rule 9: a blocked member that isn't fully developed
470                  means the group can't support a hotel yet */
471                  if(!game[0].board[i].property.hotel &&
472                     game[0].board[i].property.houses < MAX_HOUSES)
473                  {
474                      allFourHouses = 0;
475                  }

477                  continue;
478              }

480              if(game[0].board[i].property.hotel)
481                  continue;

483              if(game[0].board[i].property.houses < MAX_HOUSES)
484                  allFourHouses = 0;

486              if(game[0].board[i].property.houses < minHouses)
487              {
488                  minHouses = game[0].board[i].property.houses;
489                  targetIndex = i;
490              }
491          }
492      }
```

```

494     if(targetIndex == -1)
495         return 0; /* every property already has a hotel */

497     /* Rule 9: never pile a 5th house onto one property just because a
498        group member is blocked from building */
499     if(game[0].board[targetIndex].property.houses >= MAX_HOUSES)
500         return 0;

502     if(allFourHouses)
503     {
504         int hotelCost;
505         char costBuf[32];

507         /* Rule-LK 13/31/34: base hotel cost is permanent; temporary
508            events only add modifiers on top. A market boom's +10%
509            applies only to the booming group, while event/regulation
510            modifiers (group -1) are global. */
511         hotelCost = (game[0].board[targetIndex].property.hotelCost *
512                     modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

514         if(!shouldConstruct(game, playerIndex, hotelCost, 1))
515             return 0;

517         if(game[0].players[playerIndex].cash < hotelCost)
518             return 0;

520         payMoney(game, playerIndex, hotelCost);

522         game[0].board[targetIndex].property.houses = 0;
523         game[0].board[targetIndex].property.hotel = 1;

525         printf("\n%s upgraded %s to a Hotel.\n",
526                game[0].players[playerIndex].name,
527                game[0].board[targetIndex].name);

529         formatLKR(hotelCost, costBuf);
530         printf("Construction Cost : LKR %s.\n", costBuf);

532         return 1;
533     }

535     {
536         int houseCost;
537         char costBuf[32];

539         houseCost = (game[0].board[targetIndex].property.houseCost *
540                     modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

542         if(!shouldConstruct(game, playerIndex, houseCost, 0))
543             return 0;

545         if(game[0].players[playerIndex].cash < houseCost)
546             return 0;

548         payMoney(game, playerIndex, houseCost);

550         game[0].board[targetIndex].property.houses++;

552         printf("\n%s constructed one house on %s.\n",
553                game[0].players[playerIndex].name,
554                game[0].board[targetIndex].name);

556         formatLKR(houseCost, costBuf);
557         printf("Construction Cost : LKR %s.\n", costBuf);

559         return 1;
560     }
561 }

```

Line-by-line explanation

Line	Code	Explanation
439		Blank line.
440	<code>/* Builds one house (or upgrades to a hotel) somewhere in this</code>	Comment: <code>/* Builds one house (or upgrades to a hotel) somewhere in this</code>
441	<code>colour group. Always adds to whichever property currently has the</code>	colour group. Always adds to whichever property currently has the
442	<code>fewest houses, so development stays even (Rule 9). Returns 1 if</code>	fewest houses, so development stays even (Rule 9). Returns 1 if
443	<code>something was actually built, 0 if not - constructBuildings()</code>	something was actually built, 0 if not - <code>constructBuildings()</code>
444	<code>uses this to keep building repeatedly in the same turn. */</code>	uses this to keep building repeatedly in the same turn. <code>*/</code>
445	<code>int developGroup(GameState game[], int playerIndex, PropertyGroup group)</code>	<code>int developGroup(GameState game[], int playerIndex, PropertyGroup group)</code>
446	<code>{</code>	Opens a new block.
447	<code>int i;</code>	Declares <code>int i</code> .
448	<code>int minHouses;</code>	Declares <code>int minHouses</code> .
449	<code>int targetIndex;</code>	Declares <code>int targetIndex</code> .
450	<code>int allFourHouses;</code>	Declares <code>int allFourHouses</code> .
451		Blank line.
452	<code>if(!ownsMonopoly(game, playerIndex, group))</code>	Condition: <code>!ownsMonopoly(game, playerIndex, group)</code> -> run the next block if true.
453	<code>return 0;</code>	Returns 0 immediately.
454		Blank line.
455	<code>minHouses = MAX_HOUSES + 1;</code>	<code>minHouses = MAX_HOUSES + 1;</code>
456	<code>targetIndex = -1;</code>	<code>targetIndex = -1;</code>
457	<code>allFourHouses = 1;</code>	<code>allFourHouses = 1;</code>
458		Blank line.
459	<code>for(i = 0; i < BOARD_SIZE; i++)</code>	Loop over all entries: <code>init i = 0; run while i < BOARD_SIZE; every loop i++</code> .
460	<code>{</code>	Opens a new block.
461	<code>if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)</code>	Condition: <code>square i.type == PROPERTY && square i.property.group == group</code> -> run the next block if true.
462	<code>{</code>	Opens a new block.
463	<code>/* can't build on a mortgaged, damaged or loan-locked</code>	Comment: <code>/* can't build on a mortgaged, damaged or loan-locked</code>
464	<code>property - those earn no rent and/or are pledged */</code>	<code>property - those earn no rent and/or are pledged */</code>
465	<code>if(game[0].board[i].property.mortgaged </code>	Condition: -> run the next block if true.
466	<code>game[0].board[i].property.damaged </code>	<code>game[0].board[i].property.damaged </code>
467	<code>game[0].board[i].property.loanLocked)</code>	<code>game[0].board[i].property.loanLocked)</code>
468	<code>{</code>	Opens a new block.
469	<code>/* Rule 9: a blocked member that isn't fully developed</code>	Comment: <code>/* Rule 9: a blocked member that isn't fully developed</code>
470	<code>means the group can't support a hotel yet */</code>	<code>means the group can't support a hotel yet */</code>
471	<code>if(!game[0].board[i].property.hotel &&</code>	Condition: -> run the next block if true.
472	<code>game[0].board[i].property.houses < MAX_HOUSES)</code>	<code>game[0].board[i].property.houses < MAX_HOUSES)</code>
473	<code>{</code>	Opens a new block.
474	<code>allFourHouses = 0;</code>	<code>allFourHouses = 0;</code>
475	<code>}</code>	Closes the current block.
476		Blank line.
477	<code>continue;</code>	Skips to the next loop iteration.
478	<code>}</code>	Closes the current block.
479		Blank line.
480	<code>if(game[0].board[i].property.hotel)</code>	Condition: <code>square i.property.hotel</code> -> run the next block if true.
481	<code>continue;</code>	Skips to the next loop iteration.
482		Blank line.

483	if(game[0].board[i].property.houses < MAX_HOUSES)	Condition: square i.property.houses < MAX_HOUSES -> run the next block if true.
484	allFourHouses = 0;	allFourHouses = 0;
485		Blank line.
486	if(game[0].board[i].property.houses < minHouses)	Condition: square i.property.houses < minHouses -> run the next block if true.
487	{	Opens a new block.
488	minHouses = game[0].board[i].property.houses;	minHouses = game[0].board[i].property.houses;
489	targetIndex = i;	targetIndex = i;
490	}	Closes the current block.
491	}	Closes the current block.
492	}	Closes the current block.
493		Blank line.
494	if(targetIndex == -1)	Condition: targetIndex == -1 -> run the next block if true.
495	return 0; /* every property already has a hotel */	Returns 0; /* every property already has a hotel */ immediately.
496		Blank line.
497	/* Rule 9: never pile a 5th house onto one property just because a	Comment: /* Rule 9: never pile a 5th house onto one property just
498	group member is blocked from building */	because a
499	if(game[0].board[targetIndex].property.houses >= MAX_HOUSES)	Condition: game[0].board[targetIndex].property.houses >= MAX_HOUSES
500	return 0;	-> run the next block if true.
501		Returns 0 immediately.
502	if(allFourHouses)	Blank line.
503	{	Condition: allFourHouses -> run the next block if true.
504	int hotelCost;	Opens a new block.
505	char costBuf[32];	Declares int hotelCost.
506		Declares char costBuf[32].
507	/* Rule-LK 13/31/34: base hotel cost is permanent; temporary	Blank line.
508	events only add modifiers on top. A market boom's +10%	Comment: /* Rule-LK 13/31/34: base hotel cost is permanent; temporary
509	applies only to the booming group, while event/regulation	events only add modifiers on top. A market boom's +10%
510	modifiers (group -1) are global. */	applies only to the booming group, while event/regulation
511	hotelCost =	modifiers (group -1) are global. */
512	(game[0].board[targetIndex].property.hotelCost *	hotelCost = (game[0].board[targetIndex].property.hotelCost *
513	modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;	modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;
514	if(!shouldConstruct(game, playerIndex, hotelCost, 1))	Blank line.
515	return 0;	Condition: !shouldConstruct(game, playerIndex, hotelCost, 1) -> run the next block if true.
516		Returns 0 immediately.
517	if(game[0].players[playerIndex].cash < hotelCost)	Blank line.
518	return 0;	Condition: the current player.cash < hotelCost -> run the next block if true.
519		Returns 0 immediately.
520	payMoney(game, playerIndex, hotelCost);	Blank line.
521		Calls payMoney(game, playerIndex, hotelCost). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
522	game[0].board[targetIndex].property.houses = 0;	Blank line.
523	game[0].board[targetIndex].property.hotel = 1;	Sets the field houses of the property on the board (i.e. Number of houses (0..4).) = 0.
524		Sets the field hotel of the property on the board (i.e. 1 if a hotel is built (replaces the 4 houses).) = 1.
525	printf("\n%s upgraded %s to a Hotel.\n",	Blank line.
		Prints the message "\n%s upgraded %s to a Hotel.\n".

526	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
527	<code>game[0].board[targetIndex].name);</code>	<code>game[0].board[targetIndex].name);</code>
528		Blank line.
529	<code>formatLKR(hotelCost, costBuf);</code>	Calls <code>formatLKR(hotelCost, costBuf)</code> . (Formats an amount with thousands separators, e)
530	<code>printf("Construction Cost : LKR %s.\n", costBuf);</code>	Prints the message "Construction Cost : LKR %s.\n".
531		Blank line.
532	<code>return 1;</code>	Returns 1 immediately.
533	<code>}</code>	Closes the current block.
534		Blank line.
535	<code>{</code>	Opens a new block.
536	<code>int houseCost;</code>	Declares <code>int houseCost</code> .
537	<code>char costBuf[32];</code>	Declares <code>char costBuf[32]</code> .
538		Blank line.
539	<code>houseCost =</code> <code>(game[0].board[targetIndex].property.houseCost *</code>	<code>houseCost = (game[0].board[targetIndex].property.houseCost *</code>
540	<code>modifierMultiplier(game, MOD_CONSTRUCTION, group,</code> <code>-1)) / 100;</code>	<code>modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;</code>
541		Blank line.
542	<code>if(!shouldConstruct(game, playerIndex, houseCost,</code> <code>0))</code>	Condition: <code>!shouldConstruct(game, playerIndex, houseCost, 0)</code> -> run the next block if true.
543	<code>return 0;</code>	Returns 0 immediately.
544		Blank line.
545	<code>if(game[0].players[playerIndex].cash < houseCost)</code>	Condition: the current player.cash < houseCost -> run the next block if true.
546	<code>return 0;</code>	Returns 0 immediately.
547		Blank line.
548	<code>payMoney(game, playerIndex, houseCost);</code>	Calls <code>payMoney(game, playerIndex, houseCost)</code> . (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
549		Blank line.
550	<code>game[0].board[targetIndex].property.houses++;</code>	<code>game[0].board[targetIndex].property.houses++;</code>
551		Blank line.
552	<code>printf("\n%s constructed one house on %s.\n",</code>	Prints the message "\n%s constructed one house on %s.\n".
553	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
554	<code>game[0].board[targetIndex].name);</code>	<code>game[0].board[targetIndex].name);</code>
555		Blank line.
556	<code>formatLKR(houseCost, costBuf);</code>	Calls <code>formatLKR(houseCost, costBuf)</code> . (Formats an amount with thousands separators, e)
557	<code>printf("Construction Cost : LKR %s.\n", costBuf);</code>	Prints the message "Construction Cost : LKR %s.\n".
558		Blank line.
559	<code>return 1;</code>	Returns 1 immediately.
560	<code>}</code>	Closes the current block.
561	<code>}</code>	Closes the current block.

Nested functions inside `developGroup()` - study these

- `constructBuildings()` (Builds the maximum affordable buildings in one turn by repeatedly calling `developGroup` per)
- `developGroup()` (Builds one house (or the group's hotel) keeping development even (Rule 9). Returns 1 if an)
- `ownsMonopoly()` (Whether the player owns every member of the group.)
- `modifierMultiplier()` (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- `shouldConstruct()` (Decides whether the player spends on a house/hotel now.)
- `payMoney()` (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- `formatLKR()` (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Builds one house (or the group's hotel) keeping development even (Rule 9). Returns 1 if anything was built.

6.21 constructBuildings()

```
void constructBuildings(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 562)

```
563  /* Keeps building on each colour group, one step at a time, until
564     either nothing more can be built there this turn (unaffordable,
565     the strategy doesn't want to, or the group is fully built up) -
566     this is what lets a strategy build "the maximum possible number
567     of houses immediately" in a single turn instead of one per turn. */
568  void constructBuildings(GameState game[], int playerIndex)
569  {
570      PropertyGroup group;
571      int builtSomething;
572      int safetyLimit;

574      if(game[0].economy.constructionSuspendedRoundsLeft > 0)
575          return;

577      for(group = BROWN; group < NO_GROUP; group++)
578      {
579          safetyLimit = 0;

581          do
582          {
583              builtSomething = developGroup(game, playerIndex, group);
584              safetyLimit++;
585          }
586          while(builtSomething && safetyLimit < 20);
587      }
588  }
```

Line-by-line explanation

Line	Code	Explanation
562		Blank line.
563	/* Keeps building on each colour group, one step at a time, until	Comment: /* Keeps building on each colour group, one step at a time, until
564	either nothing more can be built there this turn (unaffordable,	either nothing more can be built there this turn (unaffordable,
565	the strategy doesn't want to, or the group is fully built up) -	the strategy doesn't want to, or the group is fully built up) -
566	this is what lets a strategy build "the maximum possible number	this is what lets a strategy build "the maximum possible number
567	of houses immediately" in a single turn instead of one per turn. */	of houses immediately" in a single turn instead of one per turn. */
568	void constructBuildings(GameState game[], int playerIndex)	void constructBuildings(GameState game[], int playerIndex)
569	{	Opens a new block.
570	PropertyGroup group;	PropertyGroup group;
571	int builtSomething;	Declares int builtSomething.
572	int safetyLimit;	Declares int safetyLimit.
573		Blank line.
574	if(game[0].economy.constructionSuspendedRoundsLeft > 0)	Condition: game[0].economy.constructionSuspendedRoundsLeft > 0 ->
575	return;	run the next block if true.
576		Returns the computed value immediately.
577	for(group = BROWN; group < NO_GROUP; group++)	Blank line.
578	{	Loop over all entries: init group = BROWN; run while group < NO_GROUP; every loop group++.
579	safetyLimit = 0;	Opens a new block.
		safetyLimit = 0;

580		Blank line.
581	do	do
582	{	Opens a new block.
583	builtSomething = developGroup(game, playerIndex, group);	builtSomething = developGroup(game, playerIndex, group);
584	safetyLimit++;	safetyLimit++;
585	}	Closes the current block.
586	while(builtSomething && safetyLimit < 20);	Loop: repeat while builtSomething && safetyLimit < 20 is true.
587	}	Closes the current block.
588	}	Closes the current block.

Nested functions inside constructBuildings() - study these

- constructBuildings() (Builds the maximum affordable buildings in one turn by repeatedly calling developGroup per)
- developGroup() (Builds one house (or the group's hotel) keeping development even (Rule 9). Returns 1 if an)

MAIN PURPOSE

Builds the maximum affordable buildings in one turn by repeatedly calling developGroup per group.

6.22 calculatePropertyValue()

```
int calculatePropertyValue(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: Value in LKR.

The function, verbatim from the source (starts at line 590)

```
590 int calculatePropertyValue(GameState game[], int playerIndex)
591 {
592     int i;
593     int total;

595     total = 0;

597     for(i = 0; i < BOARD_SIZE; i++)
598     {
599         if(game[0].board[i].property.owner != playerIndex)
600             continue;

602         /* mortgaged properties contribute zero to net worth */
603         if(game[0].board[i].property.mortgaged)
604             continue;

606         /* railways and utilities also use currentMarketValue so their
607            value modifiers (rail/port/water effects) apply too */
608         total += currentMarketValue(game, i);
609     }

611     return total;
612 }
```

Line-by-line explanation

Line	Code	Explanation
590	int calculatePropertyValue(GameState game[], int playerIndex)	int calculatePropertyValue(GameState game[], int playerIndex)
591	{	Opens a new block.
592	int i;	Declares int i.
593	int total;	Declares int total.
594		Blank line.
595	total = 0;	total = 0;
596		Blank line.
597	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
598	{	Opens a new block.
599	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.

600	continue;	Skips to the next loop iteration.
601		Blank line.
602	/* mortgaged properties contribute zero to net worth */	Comment: /* mortgaged properties contribute zero to net worth */
603	if(game[0].board[i].property.mortgaged)	Condition: square i.property.mortgaged -> run the next block if true.
604	continue;	Skips to the next loop iteration.
605		Blank line.
606	/* railways and utilities also use currentMarketValue so their	Comment: /* railways and utilities also use currentMarketValue so their
607	value modifiers (rail/port/water effects) apply too */	value modifiers (rail/port/water effects) apply too */
608	total += currentMarketValue(game, i);	total += currentMarketValue(game, i);
609	}	Closes the current block.
610		Blank line.
611	return total;	Returns total immediately.
612	}	Closes the current block.

Nested functions inside calculatePropertyValue() - study these

- calculatePropertyValue() (Total current market value of owned, unmortgaged squares.)
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)

MAIN PURPOSE

Total current market value of owned, unmortgaged squares.

6.23 calculateBuildingValue()

```
int calculateBuildingValue(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: Value in LKR.

The function, verbatim from the source (starts at line 614)

```

614 int calculateBuildingValue(GameState game[], int playerIndex)
615 {
616     int i;
617     int total;

619     total = 0;

621     for(i = 0; i < BOARD_SIZE; i++)
622     {
623         if(game[0].board[i].type != PROPERTY)
624             continue;

626         if(game[0].board[i].property.owner != playerIndex)
627             continue;

629         if(game[0].board[i].property.hotel)
630             total += game[0].board[i].property.hotelCost;
631         else
632             total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;
633     }

635     return total;
636 }
```

Line-by-line explanation

Line	Code	Explanation
614	int calculateBuildingValue(GameState game[], int playerIndex)	int calculateBuildingValue(GameState game[], int playerIndex)
615	{	Opens a new block.
616	int i;	Declares int i.
617	int total;	Declares int total.
618		Blank line.

619	total = 0;	total = 0;
620		Blank line.
621	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
622	{	Opens a new block.
623	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
624	continue;	Skips to the next loop iteration.
625		Blank line.
626	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
627	continue;	Skips to the next loop iteration.
628		Blank line.
629	if(game[0].board[i].property.hotel)	Condition: square i.property.hotel -> run the next block if true.
630	total += game[0].board[i].property.hotelCost;	total += game[0].board[i].property.hotelCost;
631	else	The alternative branch when the condition above is false.
632	total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;	total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;
633	}	Closes the current block.
634		Blank line.
635	return total;	Returns total immediately.
636	}	Closes the current block.

Nested functions inside calculateBuildingValue() - study these

- calculateBuildingValue() (Cash value of all houses and hotels.)

MAIN PURPOSE

Cash value of all houses and hotels.

6.24 calculateNetWorth()

```
int calculateNetWorth(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *Net worth in LKR.*

The function, verbatim from the source (starts at line 637)

```
638  /* Rule 15 net worth formula: cash + property value + building value,
639     minus any outstanding loan amount. */
640  int calculateNetWorth(GameState game[], int playerIndex)
641  {
642      int cash;
643      int propertyValue;
644      int buildingValue;
645      int outstandingLoans;

647      cash = game[0].players[playerIndex].cash;
648      propertyValue = calculatePropertyValue(game, playerIndex);
649      buildingValue = calculateBuildingValue(game, playerIndex);

651      outstandingLoans = 0;

653      if(game[0].players[playerIndex].loan.active)
654          outstandingLoans = game[0].players[playerIndex].loan.amount;

656      return cash + propertyValue + buildingValue - outstandingLoans;
657  }
```

Line-by-line explanation

Line	Code	Explanation
637		Blank line.
638	/* Rule 15 net worth formula: cash + property value + building value,	Comment: /* Rule 15 net worth formula: cash + property value + building value,
639	minus any outstanding loan amount. */	minus any outstanding loan amount. */

640	int calculateNetWorth(GameState game[], int playerIndex)	int calculateNetWorth(GameState game[], int playerIndex)
641	{	Opens a new block.
642	int cash;	Declares int cash.
643	int propertyValue;	Declares int propertyValue.
644	int buildingValue;	Declares int buildingValue.
645	int outstandingLoans;	Declares int outstandingLoans.
646		Blank line.
647	cash = game[0].players[playerIndex].cash;	cash = game[0].players[playerIndex].cash;
648	propertyValue = calculatePropertyValue(game, playerIndex);	propertyValue = calculatePropertyValue(game, playerIndex);
649	buildingValue = calculateBuildingValue(game, playerIndex);	buildingValue = calculateBuildingValue(game, playerIndex);
650		Blank line.
651	outstandingLoans = 0;	outstandingLoans = 0;
652		Blank line.
653	if(game[0].players[playerIndex].loan.active)	Condition: the current player.loan.active -> run the next block if true.
654	outstandingLoans = game[0].players[playerIndex].loan.amount;	outstandingLoans = game[0].players[playerIndex].loan.amount;
655		Blank line.
656	return cash + propertyValue + buildingValue - outstandingLoans;	Returns cash + propertyValue + buildingValue - outstandingLoans immediately.
657	}	Closes the current block.

Nested functions inside calculateNetWorth() - study these

- calculateNetWorth() (Rule 15 net worth: cash + property + buildings - outstanding loan.)
- calculatePropertyValue() (Total current market value of owned, unmortgaged squares.)
- calculateBuildingValue() (Cash value of all houses and hotels.)

MAIN PURPOSE

Rule 15 net worth: cash + property + buildings - outstanding loan.

6.25 countUndevelopedProperties()

```
int countUndevelopedProperties(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *Count*.

The function, verbatim from the source (starts at line 659)

```
659 int countUndevelopedProperties(GameState game[], int playerIndex)
660 {
661     int i;
662     int count;
663
664     count = 0;
665
666     for(i = 0; i < BOARD_SIZE; i++)
667     {
668         if(game[0].board[i].type == PROPERTY &&
669             game[0].board[i].property.owner == playerIndex &&
670             game[0].board[i].property.houses == 0 &&
671             !game[0].board[i].property.hotel)
672         {
673             count++;
674         }
675     }
676
677     return count;
678 }
```

Line-by-line explanation

Line	Code	Explanation
------	------	-------------

659	int countUndevelopedProperties(GameState game[], int playerIndex)	int countUndevelopedProperties(GameState game[], int playerIndex)
660	{	Opens a new block.
661	int i;	Declares int i.
662	int count;	Declares int count.
663		Blank line.
664	count = 0;	count = 0;
665		Blank line.
666	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
667	{	Opens a new block.
668	if(game[0].board[i].type == PROPERTY &&	Condition: -> run the next block if true.
669	game[0].board[i].property.owner == playerIndex &&	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = playerIndex &&.
670	game[0].board[i].property.houses == 0 &&	Sets the field houses of the property on the board (i.e. Number of houses (0..4).) = 0 &&.
671	!game[0].board[i].property.hotel)	!game[0].board[i].property.hotel)
672	{	Opens a new block.
673	count++;	count++;
674	}	Closes the current block.
675	}	Closes the current block.
676		Blank line.
677	return count;	Returns count immediately.
678	}	Closes the current block.

Nested functions inside countUndevelopedProperties() - study these

- countUndevelopedProperties() (Counts owned colour properties with no houses and no hotel.)

MAIN PURPOSE

Counts owned colour properties with no houses and no hotel.

6.26 wouldCompleteMonopoly()

```
int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
```

Parameters: playerIndex, pos | Returns: 1 = completes a monopoly, 0 = not.

The function, verbatim from the source (starts at line 679)

```
680  /* True if buying the property at pos would complete the whole colour
681     group for the player (Section 3.1). */
682  int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
683  {
684      int i;
685      PropertyGroup group;

687      if(game[0].board[pos].type != PROPERTY)
688          return 0;

690      if(game[0].board[pos].property.owner != -1)
691          return 0;

693      group = game[0].board[pos].property.group;

695      for(i = 0; i < BOARD_SIZE; i++)
696      {
697          if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
698          {
699              if(i != pos && game[0].board[i].property.owner != playerIndex)
700                  return 0;
701          }
702      }

704      return 1;
705  }
```

Line-by-line explanation

Line	Code	Explanation
679		Blank line.
680	/* True if buying the property at pos would complete the whole colour	Comment: /* True if buying the property at pos would complete the whole colour
681	group for the player (Section 3.1). */	group for the player (Section 3.1). */
682	int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)	int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
683	{	Opens a new block.
684	int i;	Declares int i.
685	PropertyGroup group;	PropertyGroup group;
686		Blank line.
687	if(game[0].board[pos].type != PROPERTY)	Condition: game[0].board[pos].type != PROPERTY -> run the next block if true.
688	return 0;	Returns 0 immediately.
689		Blank line.
690	if(game[0].board[pos].property.owner != -1)	Condition: game[0].board[pos].property.owner != -1 -> run the next block if true.
691	return 0;	Returns 0 immediately.
692		Blank line.
693	group = game[0].board[pos].property.group;	group = game[0].board[pos].property.group;
694		Blank line.
695	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
696	{	Opens a new block.
697	if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)	Condition: square i.type == PROPERTY && square i.property.group == group -> run the next block if true.
698	{	Opens a new block.
699	if(i != pos && game[0].board[i].property.owner != playerIndex)	Condition: i != pos && square i.property.owner != playerIndex -> run the next block if true.
700	return 0;	Returns 0 immediately.
701	}	Closes the current block.
702	}	Closes the current block.
703		Blank line.
704	return 1;	Returns 1 immediately.
705	}	Closes the current block.

Nested functions inside wouldCompleteMonopoly() - study these

- wouldCompleteMonopoly() (True if buying the square at pos would complete a whole colour group.)

MAIN PURPOSE

True if buying the square at pos would complete a whole colour group.

6.27 sellLowValueProperty()

```
void sellLowValueProperty(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 706)

```
707  /* Sells the player's lowest-value undeveloped property back to the
708     Bank at market value (Section 3.4 bankruptcy tactic). */
709  void sellLowValueProperty(GameState game[], int playerIndex)
710  {
711      int i;
712      int best;
713      int bestValue;
714      int value;
715
716      best = -1;
```

```

717     bestValue = 0;

719     for(i = 0; i < BOARD_SIZE; i++)
720     {
721         if(game[0].board[i].type != PROPERTY)
722             continue;

724         if(game[0].board[i].property.owner != playerIndex)
725             continue;

727         if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
728             continue;

730         /* Rule-LK 3: loan-locked collateral can never be sold */
731         if(game[0].board[i].property.loanLocked)
732             continue;

734         value = currentMarketValue(game, i);

736         if(best == -1 || value < bestValue)
737         {
738             best = i;
739             bestValue = value;
740         }
741     }

743     if(best == -1)
744         return;

746     receiveMoney(game, playerIndex, bestValue);

748     stripOwnership(game, best);

750     adjustOwnedCount(game, playerIndex, best, -1);

752     printf("\n%s sold %s to the Bank for LKR %d.\n",
753           game[0].players[playerIndex].name,
754           game[0].board[best].name,
755           bestValue);
756 }

```

Line-by-line explanation

Line	Code	Explanation
706		Blank line.
707	/* Sells the player's lowest-value undeveloped property back to the	Comment: /* Sells the player's lowest-value undeveloped property back to the
708	Bank at market value (Section 3.4 bankruptcy tactic). */	Bank at market value (Section 3.4 bankruptcy tactic). */
709	void sellLowValueProperty(GameState game[], int playerIndex)	void sellLowValueProperty(GameState game[], int playerIndex)
710	{	Opens a new block.
711	int i;	Declares int i.
712	int best;	Declares int best.
713	int bestValue;	Declares int bestValue.
714	int value;	Declares int value.
715		Blank line.
716	best = -1;	best = -1;
717	bestValue = 0;	bestValue = 0;
718		Blank line.
719	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
720	{	Opens a new block.
721	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
722	continue;	Skips to the next loop iteration.
723		Blank line.

724	<code>if(game[0].board[i].property.owner != playerIndex)</code>	Condition: square i.property.owner != playerIndex -> run the next block if true.
725	<code>continue;</code>	Skips to the next loop iteration.
726		Blank line.
727	<code>if(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel)</code>	Condition: square i.property.houses > 0 square i.property.hotel -> run the next block if true.
728	<code>continue;</code>	Skips to the next loop iteration.
729		Blank line.
730	<code>/* Rule-LK 3: loan-locked collateral can never be sold */</code>	Comment: /* Rule-LK 3: loan-locked collateral can never be sold */
731	<code>if(game[0].board[i].property.loanLocked)</code>	Condition: square i.property.loanLocked -> run the next block if true.
732	<code>continue;</code>	Skips to the next loop iteration.
733		Blank line.
734	<code>value = currentMarketValue(game, i);</code>	<code>value = currentMarketValue(game, i);</code>
735		Blank line.
736	<code>if(best == -1 value < bestValue)</code>	Condition: best == -1 value < bestValue -> run the next block if true.
737	<code>{</code>	Opens a new block.
738	<code>best = i;</code>	<code>best = i;</code>
739	<code>bestValue = value;</code>	<code>bestValue = value;</code>
740	<code>}</code>	Closes the current block.
741	<code>}</code>	Closes the current block.
742		Blank line.
743	<code>if(best == -1)</code>	Condition: best == -1 -> run the next block if true.
744	<code>return;</code>	Returns the computed value immediately.
745		Blank line.
746	<code>receiveMoney(game, playerIndex, bestValue);</code>	<code>receiveMoney(game, playerIndex, bestValue).</code> (Adds money to a player, but only if they are not bankrupt)
747		Blank line.
748	<code>stripOwnership(game, best);</code>	<code>stripOwnership(game, best).</code> (Resets a square to unowned: no owner, mortgage, loan pledge or insurance)
749		Blank line.
750	<code>adjustOwnedCount(game, playerIndex, best, -1);</code>	<code>adjustOwnedCount(game, playerIndex, best, -1).</code> (Updates the player's owned counter to match the square's type)
751		Blank line.
752	<code>printf("\n%s sold %s to the Bank for LKR %d.\n",</code>	Prints the message " <code>\n%s sold %s to the Bank for LKR %d.\n</code> ".
753	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
754	<code>game[0].board[best].name,</code>	<code>game[0].board[best].name,</code>
755	<code>bestValue);</code>	<code>bestValue);</code>
756	<code>}</code>	Closes the current block.

Nested functions inside `sellLowValueProperty()` - study these

- `sellLowValueProperty()` (Sells the player's cheapest undeveloped property to the Bank at market value (Risk Taker b))
- `currentMarketValue()` (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- `receiveMoney()` (Adds money to a player, but only if they are not bankrupt.)
- `stripOwnership()` (Resets a square to unowned: no owner, mortgage, loan pledge or insurance.)
- `adjustOwnedCount()` (Updates the player's owned counter to match the square's type.)

MAIN PURPOSE

Sells the player's cheapest undeveloped property to the Bank at market value (Risk Taker bankruptcy tactic).

6.28 `sellDecliningProperties()`

```
void sellDecliningProperties(GameState game[], int playerIndex)
```

Parameters: `playerIndex` | Returns: `void`.

The function, verbatim from the source (starts at line 757)

```
758 /* Section 3.4: the Opportunistic Trader sells properties expected to
```

```

759     lose value following economic events - any undeveloped property in
760     a group that is currently declining, under a global downturn, or
761     carrying its own negative value modifier. Loan-locked collateral
762     (Rule-LK 3) is never sold. */
763 void sellDecliningProperties(GameState game[], int playerIndex)
764 {
765     int i;
766     int globalMult;

768     if(game[0].players[playerIndex].strategy != OPPORTUNISTIC_TRADER)
769         return;

771     if(game[0].players[playerIndex].bankrupt)
772         return;

774     globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

776     for(i = 0; i < BOARD_SIZE; i++)
777     {
778         int groupMult;
779         int indexMult;
780         int value;

782         if(game[0].board[i].type != PROPERTY)
783             continue;

785         if(game[0].board[i].property.owner != playerIndex)
786             continue;

788         if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
789             continue;

791         if(game[0].board[i].property.loanLocked)
792             continue;

794         groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,
795                                         game[0].board[i].property.group, -1);
796         indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);

798         if(globalMult >= 100 && groupMult >= 100 && indexMult >= 100)
799             continue;

801         value = currentMarketValue(game, i);

803         receiveMoney(game, playerIndex, value);

805         stripOwnership(game, i);

807         adjustOwnedCount(game, playerIndex, i, -1);

809         printf("\n%s sold %s to the Bank for LKR %d "
810                "(expected to lose value).\n",
811                game[0].players[playerIndex].name,
812                game[0].board[i].name,
813                value);
814     }
815 }

```

Line-by-line explanation

Line	Code	Explanation
757		Blank line.
758	/* Section 3.4: the Opportunistic Trader sells properties expected to	Comment: /* Section 3.4: the Opportunistic Trader sells properties expected to
759	lose value following economic events - any undeveloped property in	lose value following economic events - any undeveloped property in
760	a group that is currently declining, under a global downturn, or	a group that is currently declining, under a global downturn, or

761	carrying its own negative value modifier. Loan-locked collateral	carrying its own negative value modifier. Loan-locked collateral
762	(Rule-LK 3) is never sold. */	(Rule-LK 3) is never sold. */
763	void sellDecliningProperties(GameState game[], int playerIndex)	void sellDecliningProperties(GameState game[], int playerIndex)
764	{	Opens a new block.
765	int i;	Declares int i.
766	int globalMult;	Declares int globalMult.
767		Blank line.
768	if(game[0].players[playerIndex].strategy != OPPORTUNISTIC_TRADER)	Condition: the current player.strategy != OPPORTUNISTIC_TRADER -> run the next block if true.
769	return;	Returns the computed value immediately.
770		Blank line.
771	if(game[0].players[playerIndex].bankrupt)	Condition: the current player.bankrupt -> run the next block if true.
772	return;	Returns the computed value immediately.
773		Blank line.
774	globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);	globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);
775		Blank line.
776	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
777	{	Opens a new block.
778	int groupMult;	Declares int groupMult.
779	int indexMult;	Declares int indexMult.
780	int value;	Declares int value.
781		Blank line.
782	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
783	continue;	Skips to the next loop iteration.
784		Blank line.
785	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
786	continue;	Skips to the next loop iteration.
787		Blank line.
788	if(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel)	Condition: square i.property.houses > 0 square i.property.hotel -> run the next block if true.
789	continue;	Skips to the next loop iteration.
790		Blank line.
791	if(game[0].board[i].property.loanLocked)	Condition: square i.property.loanLocked -> run the next block if true.
792	continue;	Skips to the next loop iteration.
793		Blank line.
794	groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,	groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,
795	game[0].board[i].property.group, -1);	game[0].board[i].property.group, -1);
796	indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);	indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);
797		Blank line.
798	if(globalMult >= 100 && groupMult >= 100 && indexMult >= 100)	Condition: globalMult >= 100 && groupMult >= 100 && indexMult >= 100 -> run the next block if true.
799	continue;	Skips to the next loop iteration.
800		Blank line.
801	value = currentMarketValue(game, i);	value = currentMarketValue(game, i);
802		Blank line.
803	receiveMoney(game, playerIndex, value);	Calls receiveMoney(game, playerIndex, value). (Adds money to a player, but only if they are not bankrupt)
804		Blank line.
805	stripOwnership(game, i);	Calls stripOwnership(game, i). (Resets a square to unowned: no owner, mortgage, loan pledge or insurance)
806		Blank line.

807	<code>adjustOwnedCount(game, playerIndex, i, -1);</code>	Calls <code>adjustOwnedCount(game, playerIndex, i, -1)</code> . (Updates the player's owned counter to match the square's type)
808		Blank line.
809	<code>printf("\n%s sold %s to the Bank for LKR %d "</code>	Prints the message " <code>\n%s sold %s to the Bank for LKR %d "</code> .
810	<code>"(expected to lose value).\n",</code>	<code>"(expected to lose value).\n",</code>
811	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
812	<code>game[0].board[i].name,</code>	<code>game[0].board[i].name,</code>
813	<code>value);</code>	<code>value);</code>
814	<code>}</code>	Closes the current block.
815	<code>}</code>	Closes the current block.

Nested functions inside `sellDecliningProperties()` - study these

- `sellDecliningProperties()` (Opportunistic Trader dumps undeveloped property in groups currently declining, under a glo)
- `modifierMultiplier()` (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- `currentMarketValue()` (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- `receiveMoney()` (Adds money to a player, but only if they are not bankrupt.)
- `stripOwnership()` (Resets a square to unowned: no owner, mortgage, loan pledge or insurance.)
- `adjustOwnedCount()` (Updates the player's owned counter to match the square's type.)

MAIN PURPOSE

Opportunistic Trader dumps undeveloped property in groups currently declining, under a global downturn, or with a negative index modifier.

6.29 `sellUndevelopedPropertyToAuction()`

```
void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 816)

```

817  /* Releases the player's lowest-value undeveloped property and puts
818     it up for auction (Anti-Speculation Act enforcement). */
819  void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
820  {
821      int i;
822      int best;
823      int bestValue;
824      int value;

826      best = -1;
827      bestValue = 0;

829      for(i = 0; i < BOARD_SIZE; i++)
830      {
831          if(game[0].board[i].type != PROPERTY)
832              continue;

834          if(game[0].board[i].property.owner != playerIndex)
835              continue;

837          if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
838              continue;

840          /* Rule-LK 3: loan-locked collateral can never be auctioned */
841          if(game[0].board[i].property.loanLocked)
842              continue;

844          value = currentMarketValue(game, i);

846          if(best == -1 || value < bestValue)
847          {
848              best = i;
849              bestValue = value;
850          }

```

```

851     }

853     if(best == -1)
854         return;

856     printf("\nAnti-Speculation Act : %s must sell %s.\n",
857           game[0].players[playerIndex].name,
858           game[0].board[best].name);

860     stripOwnership(game, best);

862     adjustOwnedCount(game, playerIndex, best, -1);

864     runAuction(game, best, playerIndex);
865 }

```

Line-by-line explanation

Line	Code	Explanation
816		Blank line.
817	/* Releases the player's lowest-value undeveloped property and puts	Comment: /* Releases the player's lowest-value undeveloped property and puts
818	it up for auction (Anti-Speculation Act enforcement). */	it up for auction (Anti-Speculation Act enforcement). */
819	void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)	void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
820	{	Opens a new block.
821	int i;	Declares int i.
822	int best;	Declares int best.
823	int bestValue;	Declares int bestValue.
824	int value;	Declares int value.
825		Blank line.
826	best = -1;	best = -1;
827	bestValue = 0;	bestValue = 0;
828		Blank line.
829	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
830	{	Opens a new block.
831	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
832	continue;	Skips to the next loop iteration.
833		Blank line.
834	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
835	continue;	Skips to the next loop iteration.
836		Blank line.
837	if(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel)	Condition: square i.property.houses > 0 square i.property.hotel -> run the next block if true.
838	continue;	Skips to the next loop iteration.
839		Blank line.
840	/* Rule-LK 3: loan-locked collateral can never be auctioned */	Comment: /* Rule-LK 3: loan-locked collateral can never be auctioned */
841	if(game[0].board[i].property.loanLocked)	Condition: square i.property.loanLocked -> run the next block if true.
842	continue;	Skips to the next loop iteration.
843		Blank line.
844	value = currentMarketValue(game, i);	value = currentMarketValue(game, i);
845		Blank line.
846	if(best == -1 value < bestValue)	Condition: best == -1 value < bestValue -> run the next block if true.
847	{	Opens a new block.
848	best = i;	best = i;
849	bestValue = value;	bestValue = value;
850	}	Closes the current block.
851	}	Closes the current block.

852		Blank line.
853	<code>if(best == -1)</code>	Condition: best == -1 -> run the next block if true.
854	<code>return;</code>	Returns the computed value immediately.
855		Blank line.
856	<code>printf("\nAnti-Speculation Act : %s must sell %s.\n",</code>	Prints the message "\nAnti-Speculation Act : %s must sell %s.\n".
857	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
858	<code>game[0].board[best].name);</code>	<code>game[0].board[best].name);</code>
859		Blank line.
860	<code>stripOwnership(game, best);</code>	Calls <code>stripOwnership(game, best)</code> . (Resets a square to unowned: no owner, mortgage, loan pledge or insurance)
861		Blank line.
862	<code>adjustOwnedCount(game, playerIndex, best, -1);</code>	Calls <code>adjustOwnedCount(game, playerIndex, best, -1)</code> . (Updates the player's owned counter to match the square's type)
863		Blank line.
864	<code>runAuction(game, best, playerIndex);</code>	Calls <code>runAuction(game, best, playerIndex)</code> . (Sells a property to the highest bidder among solvent players)
865	<code>}</code>	Closes the current block.

Nested functions inside `sellUndevelopedPropertyToAuction()` - study these

- `sellUndevelopedPropertyToAuction()` (Anti-Speculation Act: releases the player's lowest-value undeveloped property and auctions)
- `currentMarketValue()` (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- `stripOwnership()` (Resets a square to unowned: no owner, mortgage, loan pledge or insurance.)
- `adjustOwnedCount()` (Updates the player's owned counter to match the square's type.)
- `runAuction()` (Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise)

MAIN PURPOSE

Anti-Speculation Act: releases the player's lowest-value undeveloped property and auctions it.

6.30 forceSellExcess()

```
void forceSellExcess(GameState game[], int playerIndex)
```

Parameters: `playerIndex` | Returns: `void`.

The function, verbatim from the source (starts at line 866)

```
867  /* Sells excess undeveloped properties until the player is back at
868     the allowed limit (or nothing more can be sold). */
869  void forceSellExcess(GameState game[], int playerIndex)
870  {
871      int excess;

873      excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;

875      while(excess > 0)
876      {
877          int before = countUndevelopedProperties(game, playerIndex);

879          sellUndevelopedPropertyToAuction(game, playerIndex);

881          /* nothing sellable left (e.g. every excess property is
882             loan-locked) - stop to avoid a non-terminating loop */
883          if(countUndevelopedProperties(game, playerIndex) >= before)
884              break;

886          excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
887      }
888  }
```

Line-by-line explanation

Line	Code	Explanation
866		Blank line.

867	/* Sells excess undeveloped properties until the player is back at	Comment: /* Sells excess undeveloped properties until the player is back at
868	the allowed limit (or nothing more can be sold). */	the allowed limit (or nothing more can be sold). */
869	void forceSellExcess(GameState game[], int playerIndex)	void forceSellExcess(GameState game[], int playerIndex)
870	{	Opens a new block.
871	int excess;	Declares int excess.
872		Blank line.
873	excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;	excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
874		Blank line.
875	while(excess > 0)	Loop: repeat while excess > 0 is true.
876	{	Opens a new block.
877	int before = countUndevelopedProperties(game, playerIndex);	Declares int before = countUndevelopedProperties(game, playerIndex).
878		Blank line.
879	sellUndevelopedPropertyToAuction(game, playerIndex);	Calls sellUndevelopedPropertyToAuction(game, playerIndex). (Anti-Speculation Act: releases the player's lowest-value undeveloped property and auctions it)
880		Blank line.
881	/* nothing sellable left (e.g. every excess property is	Comment: /* nothing sellable left (e.g. every excess property is
882	loan-locked) - stop to avoid a non-terminating loop */	loan-locked) - stop to avoid a non-terminating loop */
883	if(countUndevelopedProperties(game, playerIndex) >= before)	Condition: countUndevelopedProperties(game, playerIndex) >= before -> run the next block if true.
884	break;	Breaks out of the current loop or switch.
885		Blank line.
886	excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;	excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
887	}	Closes the current block.
888	}	Closes the current block.

Nested functions inside forceSellExcess() - study these

- forceSellExcess() (Sells excess undeveloped property until the player is back at the 3-property limit.)
- countUndevelopedProperties() (Counts owned colour properties with no houses and no hotel.)
- sellUndevelopedPropertyToAuction() (Anti-Speculation Act: releases the player's lowest-value undeveloped property and auctions)

MAIN PURPOSE

Sells excess undeveloped property until the player is back at the 3-property limit.

6.31 enforceAntiSpeculation()

```
void enforceAntiSpeculation(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 889)

```
890  /* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep
891      at most three undeveloped properties. After 5 consecutive rounds
892      above the limit the excess is force-auctioned. */
893  void enforceAntiSpeculation(GameState game[])
894  {
895      int i;
896      int undeveloped;

898      if(!game[0].economy.antiSpeculationActive)
899      {
900          for(i = 0; i < MAX_PLAYERS; i++)
901              game[0].players[i].antiSpecRounds = 0;
```

```

903     return;
904 }

906 for(i = 0; i < MAX_PLAYERS; i++)
907 {
908     if(game[0].players[i].bankrupt)
909     {
910         game[0].players[i].antiSpecRounds = 0;
911         continue;
912     }

914     undeveloped = countUndevelopedProperties(game, i);

916     if(undeveloped > MAX_UNDEVELOPED_PROPS)
917     {
918         game[0].players[i].antiSpecRounds++;

920         if(game[0].players[i].antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS)
921         {
922             forceSellExcess(game, i);
923             game[0].players[i].antiSpecRounds = 0;
924         }
925     }
926     else
927     {
928         game[0].players[i].antiSpecRounds = 0;
929     }
930 }
931 }

```

Line-by-line explanation

Line	Code	Explanation
889		Blank line.
890	/* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep	Comment: /* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep
891	at most three undeveloped properties. After 5 consecutive rounds	at most three undeveloped properties. After 5 consecutive rounds
892	above the limit the excess is force-auctioned. */	above the limit the excess is force-auctioned. */
893	void enforceAntiSpeculation(GameState game[])	void enforceAntiSpeculation(GameState game[])
894	{	Opens a new block.
895	int i;	Declares int i.
896	int undeveloped;	Declares int undeveloped.
897		Blank line.
898	if(!game[0].economy.antiSpeculationActive)	Condition: !game[0].economy.antiSpeculationActive -> run the next block if true.
899	{	Opens a new block.
900	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
901	game[0].players[i].antiSpecRounds = 0;	Sets the field antiSpecRounds of the player (i.e. Consecutive rounds over the 3-undeveloped limit.) = 0.
902		Blank line.
903	return;	Returns the computed value immediately.
904	}	Closes the current block.
905		Blank line.
906	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
907	{	Opens a new block.
908	if(game[0].players[i].bankrupt)	Condition: player i.bankrupt -> run the next block if true.
909	{	Opens a new block.
910	game[0].players[i].antiSpecRounds = 0;	Sets the field antiSpecRounds of the player (i.e. Consecutive rounds over the 3-undeveloped limit.) = 0.

911	continue;	Skips to the next loop iteration.
912	}	Closes the current block.
913		Blank line.
914	undeveloped = countUndevelopedProperties(game, i);	undeveloped = countUndevelopedProperties(game, i);
915		Blank line.
916	if(undeveloped > MAX_UNDEVELOPED_PROPS)	Condition: undeveloped > MAX_UNDEVELOPED_PROPS -> run the next block if true.
917	{	Opens a new block.
918	game[0].players[i].antiSpecRounds++;	game[0].players[i].antiSpecRounds++;
919		Blank line.
920	if(game[0].players[i].antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS)	Condition: player i.antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS -> run the next block if true.
921	{	Opens a new block.
922	forceSellExcess(game, i);	Calls forceSellExcess(game, i). (Sells excess undeveloped property until the player is back at the 3-property limit)
923	game[0].players[i].antiSpecRounds = 0;	Sets the field antiSpecRounds of the player (i.e. Consecutive rounds over the 3-undeveloped limit.) = 0.
924	}	Closes the current block.
925	}	Closes the current block.
926	else	The alternative branch when the condition above is false.
927	{	Opens a new block.
928	game[0].players[i].antiSpecRounds = 0;	Sets the field antiSpecRounds of the player (i.e. Consecutive rounds over the 3-undeveloped limit.) = 0.
929	}	Closes the current block.
930	}	Closes the current block.
931	}	Closes the current block.

Nested functions inside enforceAntiSpeculation() - study these

- enforceAntiSpeculation() (Rule-LK 8: once active, force-auctions excess undeveloped property after 5 consecutive rou)
- countUndevelopedProperties() (Counts owned colour properties with no houses and no hotel.)
- forceSellExcess() (Sells excess undeveloped property until the player is back at the 3-property limit.)

MAIN PURPOSE

Rule-LK 8: once active, force-auctions excess undeveloped property after 5 consecutive rounds over the limit.

6.32 buyProperty()

```
void buyProperty(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 933)

```

933 void buyProperty(GameState game[], int playerIndex)
934 {
935     int pos;
936     int price;
937     int wantsToBuy;
938     char priceBuf[32];
939     char balanceBuf[32];

941     pos = game[0].players[playerIndex].position;

943     if(game[0].board[pos].type != PROPERTY &&
944        game[0].board[pos].type != RAILWAY &&
945        game[0].board[pos].type != UTILITY)
946     {
947         return;
948     }

950     if(game[0].board[pos].property.owner != -1)
951         return; /* already owned */

```

```

953     price = currentMarketValue(game, pos);

955     if(game[0].board[pos].type == PROPERTY)
956     {
957         /* Rule-LK 31/34: a booming group's direct purchase prices
958            rise by 15% (group-scoped, not global). Railways and
959            utilities are priced at their current market value, so
960            their value modifiers (MOD_RAIL_VALUE, MOD_INDEX_VALUE)
961            affect their purchase price too. */
962         price = (price *
963                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
964                                     game[0].board[pos].property.group, -1)) / 100;
965     }

967     wantsToBuy = shouldBuyProperty(game, playerIndex);

969     if(game[0].players[playerIndex].cash < price)
970         wantsToBuy = 0;

972     /* Rule 5: if they don't buy directly, it goes to auction */
973     if(!wantsToBuy)
974     {
975         runAuction(game, pos, -1);
976         return;
977     }

979     payMoney(game, playerIndex, price);

981     game[0].board[pos].property.owner = playerIndex;

983     if(game[0].board[pos].type == PROPERTY)
984         game[0].players[playerIndex].propertiesOwned++;
985     else if(game[0].board[pos].type == RAILWAY)
986         game[0].players[playerIndex].railwaysOwned++;
987     else if(game[0].board[pos].type == UTILITY)
988         game[0].players[playerIndex].utilitiesOwned++;

990     formatLKR(price, priceBuf);
991     formatLKR(game[0].players[playerIndex].cash, balanceBuf);

993     printf("\n%s purchased %s for LKR %s.\n",
994           game[0].players[playerIndex].name,
995           game[0].board[pos].name,
996           priceBuf);

998     printf("Remaining Balance : LKR %s.\n",
999           balanceBuf);
1000 }

```

Line-by-line explanation

Line	Code	Explanation
933	void buyProperty(GameState game[], int playerIndex)	void buyProperty(GameState game[], int playerIndex)
934	{	Opens a new block.
935	int pos;	Declares int pos.
936	int price;	Declares int price.
937	int wantsToBuy;	Declares int wantsToBuy.
938	char priceBuf[32];	Declares char priceBuf[32].
939	char balanceBuf[32];	Declares char balanceBuf[32].
940		Blank line.
941	pos = game[0].players[playerIndex].position;	pos = game[0].players[playerIndex].position;
942		Blank line.
943	if(game[0].board[pos].type != PROPERTY &&	Condition: -> run the next block if true.
944	game[0].board[pos].type != RAILWAY &&	game[0].board[pos].type != RAILWAY &&
945	game[0].board[pos].type != UTILITY)	game[0].board[pos].type != UTILITY)
946	{	Opens a new block.

947	return;	Returns the computed value immediately.
948	}	Closes the current block.
949		Blank line.
950	if(game[0].board[pos].property.owner != -1)	Condition: game[0].board[pos].property.owner != -1 -> run the next block if true.
951	return; /* already owned */	Returns ; /* already owned */ immediately.
952		Blank line.
953	price = currentMarketValue(game, pos);	price = currentMarketValue(game, pos);
954		Blank line.
955	if(game[0].board[pos].type == PROPERTY)	Condition: game[0].board[pos].type == PROPERTY -> run the next block if true.
956	{	Opens a new block.
957	/* Rule-LK 31/34: a booming group's direct purchase prices	Comment: /* Rule-LK 31/34: a booming group's direct purchase prices
958	rise by 15% (group-scoped, not global). Railways and	rise by 15% (group-scoped, not global). Railways and
959	utilities are priced at their current market value,	utilities are priced at their current market value, so
960	so	
960	their value modifiers (MOD_RAIL_VALUE,	their value modifiers (MOD_RAIL_VALUE, MOD_INDEX_VALUE)
961	MOD_INDEX_VALUE)	
961	affect their purchase price too. */	affect their purchase price too. */
962	price = (price *	price = (price *
963	modifierMultiplier(game, MOD_PURCHASE_PRICE,	modifierMultiplier(game, MOD_PURCHASE_PRICE,
964	game[0].board[pos].property.group, -1)) / 100;	game[0].board[pos].property.group, -1)) / 100;
965	}	Closes the current block.
966		Blank line.
967	wantsToBuy = shouldBuyProperty(game, playerIndex);	wantsToBuy = shouldBuyProperty(game, playerIndex);
968		Blank line.
969	if(game[0].players[playerIndex].cash < price)	Condition: the current player.cash < price -> run the next block if true.
970	wantsToBuy = 0;	wantsToBuy = 0;
971		Blank line.
972	/* Rule 5: if they don't buy directly, it goes to	Comment: /* Rule 5: if they don't buy directly, it goes to auction */
973	auction */	
973	if(!wantsToBuy)	Condition: !wantsToBuy -> run the next block if true.
974	{	Opens a new block.
975	runAuction(game, pos, -1);	Calls runAuction(game, pos, -1). (Sells a property to the highest bidder among solvent players)
976		Returns the computed value immediately.
977	return;	Closes the current block.
978	}	Blank line.
979	payMoney(game, playerIndex, price);	Calls payMoney(game, playerIndex, price). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
980		Blank line.
981	game[0].board[pos].property.owner = playerIndex;	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = playerIndex.
982		Blank line.
983	if(game[0].board[pos].type == PROPERTY)	Condition: game[0].board[pos].type == PROPERTY -> run the next block if true.
984	game[0].players[playerIndex].propertiesOwned++;	game[0].players[playerIndex].propertiesOwned++;
985	else if(game[0].board[pos].type == RAILWAY)	A further condition checked only if the previous one failed.
986	game[0].players[playerIndex].railwaysOwned++;	game[0].players[playerIndex].railwaysOwned++;
987	else if(game[0].board[pos].type == UTILITY)	A further condition checked only if the previous one failed.
988	game[0].players[playerIndex].utilitiesOwned++;	game[0].players[playerIndex].utilitiesOwned++;
989		Blank line.
990	formatLKR(price, priceBuf);	Calls formatLKR(price, priceBuf). (Formats an amount with thousands separators, e)

991	<code>formatLKR(game[0].players[playerIndex].cash, balanceBuf);</code>	Calls <code>formatLKR(game[0].players[playerIndex].cash, balanceBuf)</code> . (Formats an amount with thousands separators, e)
992		Blank line.
993	<code>printf("\n%s purchased %s for LKR %s.\n",</code>	Prints the message " <code>\n%s purchased %s for LKR %s.\n</code> ".
994	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
995	<code>game[0].board[pos].name,</code>	<code>game[0].board[pos].name,</code>
996	<code>priceBuf);</code>	<code>priceBuf);</code>
997		Blank line.
998	<code>printf("Remaining Balance : LKR %s.\n",</code>	Prints the message " <code>Remaining Balance : LKR %s.\n</code> ".
999	<code>balanceBuf);</code>	<code>balanceBuf);</code>
1000	<code>}</code>	Closes the current block.

Nested functions inside `buyProperty()` - study these

- `buyProperty()` (Handles landing on an unowned square: prices it, asks the strategy, buys or sends to aucti)
- `currentMarketValue()` (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- `modifierMultiplier()` (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- `shouldBuyProperty()` (Decides whether the player buys the square they just landed on.)
- `runAuction()` (Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise)
- `payMoney()` (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- `formatLKR()` (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Handles landing on an unowned square: prices it, asks the strategy, buys or sends to auction.

6.33 `payRent()`

```
void payRent(GameState game[], int playerIndex, int diceValue)
```

Parameters: `playerIndex`, `diceValue` | Returns: `void`.

The function, verbatim from the source (starts at line 1002)

```
1002 void payRent(GameState game[], int playerIndex, int diceValue)
1003 {
1004     int pos;
1005     int owner;
1006     int rent;
1007     char rentBuf[32];

1009     pos = game[0].players[playerIndex].position;

1011     if(game[0].board[pos].type != PROPERTY &&
1012        game[0].board[pos].type != RAILWAY &&
1013        game[0].board[pos].type != UTILITY)
1014     {
1015         return;
1016     }

1018     owner = game[0].board[pos].property.owner;

1020     if(owner == playerIndex)
1021     {
1022         /* landed on your own square - no rent, maybe renovate */
1023         if(game[0].board[pos].type == PROPERTY)
1024             tryRenovateAgeDepreciation(game, playerIndex, pos);
1025         return;
1026     }

1028     if(owner == -1)
1029         return;

1031     if(game[0].board[pos].property.mortgaged)
1032         return;

1034     if(pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0)
```

```

1035     {
1036         printf("\n%s landed on %s, but it is closed (Political Rally) - no rent.\n",
1037             game[0].players[playerIndex].name, game[0].board[pos].name);
1038         return;
1039     }

1041     if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged)
1042     {
1043         printf("\n%s landed on %s, but it is damaged and collects no rent.\n",
1044             game[0].players[playerIndex].name, game[0].board[pos].name);
1045         return;
1046     }

1048     if(game[0].board[pos].type == PROPERTY &&
1049         game[0].board[pos].property.lostIncomeRoundsLeft > 0)
1050     {
1051         printf("\n%s landed on %s, but it earns no rent right now "
1052             "(Business Interruption).\n",
1053             game[0].players[playerIndex].name, game[0].board[pos].name);
1054         return;
1055     }

1057     if(game[0].board[pos].type == PROPERTY)
1058         rent = calculateRent(game, pos);
1059     else if(game[0].board[pos].type == RAILWAY)
1060         rent = calculateRailwayRent(game, owner);
1061     else
1062         rent = calculateUtilityRent(game, owner, diceValue);

1064     payMoney(game, playerIndex, rent);
1065     receiveMoney(game, owner, rent);

1067     formatLKR(rent, rentBuf);

1069     printf("\n%s landed on %s.\n",
1070         game[0].players[playerIndex].name,
1071         game[0].board[pos].name);

1073     printf("Rent Paid : LKR %s.\n", rentBuf);

1075     printf("Owner : %s.\n", game[0].players[owner].name);
1076 }

```

Line-by-line explanation

Line	Code	Explanation
1002	void payRent(GameState game[], int playerIndex, int diceValue)	void payRent(GameState game[], int playerIndex, int diceValue)
1003	{	Opens a new block.
1004	int pos;	Declares int pos.
1005	int owner;	Declares int owner.
1006	int rent;	Declares int rent.
1007	char rentBuf[32];	Declares char rentBuf[32].
1008		Blank line.
1009	pos = game[0].players[playerIndex].position;	pos = game[0].players[playerIndex].position;
1010		Blank line.
1011	if(game[0].board[pos].type != PROPERTY &&	Condition: -> run the next block if true.
1012	game[0].board[pos].type != RAILWAY &&	game[0].board[pos].type != RAILWAY &&
1013	game[0].board[pos].type != UTILITY)	game[0].board[pos].type != UTILITY)
1014	{	Opens a new block.
1015	return;	Returns the computed value immediately.
1016	}	Closes the current block.
1017		Blank line.
1018	owner = game[0].board[pos].property.owner;	owner = game[0].board[pos].property.owner;
1019		Blank line.

1020	<code>if(owner == playerIndex)</code>	Condition: owner == playerIndex -> run the next block if true.
1021	<code>{</code>	Opens a new block.
1022	<code>/* landed on your own square - no rent, maybe renovate */</code>	Comment: /* landed on your own square - no rent, maybe renovate */
1023	<code>if(game[0].board[pos].type == PROPERTY)</code>	Condition: game[0].board[pos].type == PROPERTY -> run the next block if true.
1024	<code>tryRenovateAgeDepreciation(game, playerIndex, pos);</code>	Calls tryRenovateAgeDepreciation(game, playerIndex, pos). (Rule-LK 17: landing on your own aged property offers a renovation - 10% of value, resetting age/depreciation and raising base rent +5%)
1025	<code>return;</code>	Returns the computed value immediately.
1026	<code>}</code>	Closes the current block.
1027		Blank line.
1028	<code>if(owner == -1)</code>	Condition: owner == -1 -> run the next block if true.
1029	<code>return;</code>	Returns the computed value immediately.
1030		Blank line.
1031	<code>if(game[0].board[pos].property.mortgaged)</code>	Condition: game[0].board[pos].property.mortgaged -> run the next block if true.
1032	<code>return;</code>	Returns the computed value immediately.
1033		Blank line.
1034	<code>if(pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0)</code>	Condition: pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0 -> run the next block if true.
1035	<code>{</code>	Opens a new block.
1036	<code>printf("\n%s landed on %s, but it is closed (Political Rally) - no rent.\n",</code>	Prints the message "\n%s landed on %s, but it is closed (Political Rally) - no r".
1037	<code>game[0].players[playerIndex].name,</code>	game[0].players[playerIndex].name, game[0].board[pos].name);
1038	<code>game[0].board[pos].name);</code>	
1038	<code>return;</code>	Returns the computed value immediately.
1039	<code>}</code>	Closes the current block.
1040		Blank line.
1041	<code>if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged)</code>	Condition: game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged -> run the next block if true.
1042	<code>{</code>	Opens a new block.
1043	<code>printf("\n%s landed on %s, but it is damaged and collects no rent.\n",</code>	Prints the message "\n%s landed on %s, but it is damaged and collects no rent.\n".
1044	<code>game[0].players[playerIndex].name,</code>	game[0].players[playerIndex].name, game[0].board[pos].name);
1045	<code>game[0].board[pos].name);</code>	
1045	<code>return;</code>	Returns the computed value immediately.
1046	<code>}</code>	Closes the current block.
1047		Blank line.
1048	<code>if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.lostIncomeRoundsLeft > 0)</code>	Condition: -> run the next block if true.
1049	<code>game[0].board[pos].property.lostIncomeRoundsLeft > 0)</code>	game[0].board[pos].property.lostIncomeRoundsLeft > 0)
1050	<code>{</code>	Opens a new block.
1051	<code>printf("\n%s landed on %s, but it earns no rent right now "</code>	Prints the message "\n%s landed on %s, but it earns no rent right now ".
1052	<code>"(Business Interruption).\n",</code>	"(Business Interruption).\n",
1053	<code>game[0].players[playerIndex].name,</code>	game[0].players[playerIndex].name, game[0].board[pos].name);
1054	<code>game[0].board[pos].name);</code>	
1054	<code>return;</code>	Returns the computed value immediately.
1055	<code>}</code>	Closes the current block.
1056		Blank line.
1057	<code>if(game[0].board[pos].type == PROPERTY)</code>	Condition: game[0].board[pos].type == PROPERTY -> run the next block if true.
1058	<code>rent = calculateRent(game, pos);</code>	rent = calculateRent(game, pos);
1059	<code>else if(game[0].board[pos].type == RAILWAY)</code>	A further condition checked only if the previous one failed.
1060	<code>rent = calculateRailwayRent(game, owner);</code>	rent = calculateRailwayRent(game, owner);

1061	<code>else</code>	The alternative branch when the condition above is false.
1062	<code>rent = calculateUtilityRent(game, owner, diceValue);</code>	<code>rent = calculateUtilityRent(game, owner, diceValue);</code>
1063		Blank line.
1064	<code>payMoney(game, playerIndex, rent);</code>	Calls <code>payMoney(game, playerIndex, rent)</code> . (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
1065	<code>receiveMoney(game, owner, rent);</code>	Calls <code>receiveMoney(game, owner, rent)</code> . (Adds money to a player, but only if they are not bankrupt)
1066		Blank line.
1067	<code>formatLKR(rent, rentBuf);</code>	Calls <code>formatLKR(rent, rentBuf)</code> . (Formats an amount with thousands separators, e)
1068		Blank line.
1069	<code>printf("\n%s landed on %s.\n",</code>	Prints the message " <code>\n%s landed on %s.\n</code> ".
1070	<code>game[0].players[playerIndex].name,</code>	<code>game[0].players[playerIndex].name,</code>
1071	<code>game[0].board[pos].name);</code>	<code>game[0].board[pos].name);</code>
1072		Blank line.
1073	<code>printf("Rent Paid : LKR %s.\n", rentBuf);</code>	Prints the message " <code>Rent Paid : LKR %s.\n</code> ".
1074		Blank line.
1075	<code>printf("Owner : %s.\n",</code> <code>game[0].players[owner].name);</code>	Prints the message " <code>Owner : %s.\n</code> ".
1076	<code>}</code>	Closes the current block.

Nested functions inside `payRent()` - study these

- `payRent()` (Resolves rent when landing on an owned, unmortgaged, working square.)
- `tryRenovateAgeDepreciation()` (Rule-LK 17: landing on your own aged property offers a renovation - 10% of value, resetin)
- `calculateRent()` (Rent for a normal property from Table 6 (houses/hotel), scaled by group/global/hotel modif)
- `calculateRailwayRent()` (Railway rent from Table 7 based on how many stations the OWNER has, scaled by rail/global)
- `calculateUtilityRent()` (Utility rent from Table 8: dice x 4 (one utility) or dice x 10 (both), scaled by util/glob)
- `payMoney()` (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- `receiveMoney()` (Adds money to a player, but only if they are not bankrupt.)
- `formatLKR()` (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Resolves rent when landing on an owned, unmortgaged, working square.

Part 7 bank.c

Loans: borrowing, repayment, extension, refinancing, foreclosure, collateral, the Bank square visit.

The sub-topics below are the 12 functions implemented in bank.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in bank.c

7.1	groupBasePrice()	(source line 4)	p. 128
7.2	totalEligibleCollateral()	(source line 26)	p. 129
7.3	calculateMaxLoan()	(source line 69)	p. 131
7.4	obtainLoan()	(source line 76)	p. 131
7.5	repayLoan()	(source line 125)	p. 133
7.6	handleBankVisit()	(source line 161)	p. 135
7.7	increaseLoan()	(source line 211)	p. 137
7.8	extendLoan()	(source line 287)	p. 140
7.9	refinanceLoan()	(source line 301)	p. 141
7.10	demolishBuildingsOn()	(source line 315)	p. 142
7.11	foreclose()	(source line 320)	p. 142
7.12	processLoans()	(source line 368)	p. 144

7.1 groupBasePrice()

```
int groupBasePrice(GameState game[], PropertyGroup group)
```

Parameters: group | Returns: Base price in LKR, 0 for no match.

The function, verbatim from the source (starts at line 4)

```
5      /* Rule-LK: each colour group has a base price that is used ONLY for
6      loan calculations (individual purchase prices are used for
7      buying). It matches the first property in the group on the board,
8      so this reads it straight from the board instead of storing a
9      second copy. Railways and utilities use their own mortgage value. */
10     int groupBasePrice(GameState game[], PropertyGroup group)
11     {
12         int i;

14         for(i = 0; i < BOARD_SIZE; i++)
15         {
16             if(game[0].board[i].type == PROPERTY &&
17                game[0].board[i].property.group == group)
18             {
19                 return game[0].board[i].property.purchasePrice;
20             }
21         }

23         return 0;
24     }
```

Line-by-line explanation

Line	Code	Explanation
4		Blank line.
5	/* Rule-LK: each colour group has a base price that is used ONLY for	Comment: /* Rule-LK: each colour group has a base price that is used ONLY for
6	loan calculations (individual purchase prices are used for	loan calculations (individual purchase prices are used for
7	buying). It matches the first property in the group on the board,	buying). It matches the first property in the group on the board,
8	so this reads it straight from the board instead of storing a	so this reads it straight from the board instead of storing a

9	second copy. Railways and utilities use their own mortgage value. */	second copy. Railways and utilities use their own mortgage value. */
10	int groupBasePrice(GameState game[], PropertyGroup group)	int groupBasePrice(GameState game[], PropertyGroup group)
11	{	Opens a new block.
12	int i;	Declares int i.
13		Blank line.
14	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
15	{	Opens a new block.
16	if(game[0].board[i].type == PROPERTY &&	Condition: -> run the next block if true.
17	game[0].board[i].property.group == group)	Sets the field group of the property on the board (i.e. Colour group if GROUP-scoped, else -1.) = = group).
18	{	Opens a new block.
19	return game[0].board[i].property.purchasePrice;	Returns game[0].board[i].property.purchasePrice immediately.
20	}	Closes the current block.
21	}	Closes the current block.
22		Blank line.
23	return 0;	Returns 0 immediately.
24	}	Closes the current block.

Nested functions inside groupBasePrice() - study these

- groupBasePrice() (The base price of a colour group - the first property's purchase price - used ONLY for loa)

MAIN PURPOSE

The base price of a colour group - the first property's purchase price - used ONLY for loan collateral maths.

7.2 totalEligibleCollateral()

```
int totalEligibleCollateral(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *Collateral value in LKR.*

The function, verbatim from the source (starts at line 26)

```

26  int totalEligibleCollateral(GameState game[], int playerIndex)
27  {
28      int i;
29      int total;
30      int group;
31      int mult;

33      total = 0;

35      for(i = 0; i < BOARD_SIZE; i++)
36      {
37          if(game[0].board[i].type != PROPERTY &&
38             game[0].board[i].type != RAILWAY &&
39             game[0].board[i].type != UTILITY)
40          {
41              continue;
42          }

44          if(game[0].board[i].property.owner != playerIndex)
45              continue;

47          if(game[0].board[i].property.mortgaged)
48              continue;

50          if(game[0].board[i].property.loanLocked)
51              continue;

53          /* Rule-LK 34: a booming group's collateral is worth 15% more,
54             a declining group's 10% less. Colour property collateral
55             is based on the group base price (not the individual
56             purchase price - that is only used for buying). */
57          group = groupOf(game, i);

```

```

59         mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);

61         if(game[0].board[i].type == PROPERTY)
62             total += (groupBasePrice(game, group) * mult) / 100;
63         else
64             total += (game[0].board[i].property.mortgageValue * mult) / 100;
65     }

67     return total;
68 }

```

Line-by-line explanation

Line	Code	Explanation
26	int totalEligibleCollateral(GameState game[], int playerIndex)	int totalEligibleCollateral(GameState game[], int playerIndex)
27	{	Opens a new block.
28	int i;	Declares int i.
29	int total;	Declares int total.
30	int group;	Declares int group.
31	int mult;	Declares int mult.
32		Blank line.
33	total = 0;	total = 0;
34		Blank line.
35	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
36	{	Opens a new block.
37	if(game[0].board[i].type != PROPERTY &&	Condition: -> run the next block if true.
38	game[0].board[i].type != RAILWAY &&	game[0].board[i].type != RAILWAY &&
39	game[0].board[i].type != UTILITY)	game[0].board[i].type != UTILITY)
40	{	Opens a new block.
41	continue;	Skips to the next loop iteration.
42	}	Closes the current block.
43		Blank line.
44	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
45	continue;	Skips to the next loop iteration.
46		Blank line.
47	if(game[0].board[i].property.mortgaged)	Condition: square i.property.mortgaged -> run the next block if true.
48	continue;	Skips to the next loop iteration.
49		Blank line.
50	if(game[0].board[i].property.loanLocked)	Condition: square i.property.loanLocked -> run the next block if true.
51	continue;	Skips to the next loop iteration.
52		Blank line.
53	/* Rule-LK 34: a booming group's collateral is worth 15% more,	Comment: /* Rule-LK 34: a booming group's collateral is worth 15% more,
54	a declining group's 10% less. Colour property collateral	a declining group's 10% less. Colour property collateral
55	is based on the group base price (not the individual	is based on the group base price (not the individual
56	purchase price - that is only used for buying). */	purchase price - that is only used for buying). */
57	group = groupOf(game, i);	group = groupOf(game, i);
58		Blank line.
59	mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);	mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);
60		Blank line.
61	if(game[0].board[i].type == PROPERTY)	Condition: square i.type == PROPERTY -> run the next block if true.
62	total += (groupBasePrice(game, group) * mult) / 100;	total += (groupBasePrice(game, group) * mult) / 100;
63	else	The alternative branch when the condition above is false.
64	total += (game[0].board[i].property.mortgageValue * mult) / 100;	total += (game[0].board[i].property.mortgageValue * mult) / 100;

65	}	Closes the current block.
66		Blank line.
67	return total;	Returns total immediately.
68	}	Closes the current block.

Nested functions inside totalEligibleCollateral() - study these

- totalEligibleCollateral() (Sum of the player's unmortgaged, un-pledged property values (group base price for colour p)
- groupOf() (Returns the colour group of a square, or -1 if it isn't a property.)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- groupBasePrice() (The base price of a colour group - the first property's purchase price - used ONLY for loa)

MAIN PURPOSE

Sum of the player's unmortgaged, un-pledged property values (group base price for colour property, mortgage value for rail/utility), each x its market modifier.

7.3 calculateMaxLoan()

```
int calculateMaxLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *Max loan in LKR.*

The function, verbatim from the source (starts at line 69)

```
70  /* Rule-LK 2: max loan is 75% of eligible collateral. */
71  int calculateMaxLoan(GameState game[], int playerIndex)
72  {
73      return (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100;
74  }
```

Line-by-line explanation

Line	Code	Explanation
69		Blank line.
70	/* Rule-LK 2: max loan is 75% of eligible collateral. */	Comment: /* Rule-LK 2: max loan is 75% of eligible collateral. */
71	int calculateMaxLoan(GameState game[], int playerIndex)	int calculateMaxLoan(GameState game[], int playerIndex)
72	{	Opens a new block.
73	return (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100;	Returns (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100 immediately.
74	}	Closes the current block.

Nested functions inside calculateMaxLoan() - study these

- calculateMaxLoan() (Rule-LK 2: the maximum loan = 75% of eligible collateral.)
- totalEligibleCollateral() (Sum of the player's unmortgaged, un-pledged property values (group base price for colour p)

MAIN PURPOSE

Rule-LK 2: the maximum loan = 75% of eligible collateral.

7.4 obtainLoan()

```
void obtainLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void.*

The function, verbatim from the source (starts at line 76)

```
76  void obtainLoan(GameState game[], int playerIndex)
77  {
78      int i;
79      int maxLoan;
80      char loanBuf[32];
81
82      if(game[0].players[playerIndex].loan.active)
83          return; /* only one active loan allowed at a time */
```

```

85     maxLoan = calculateMaxLoan(game, playerIndex);

87     if(maxLoan <= 0)
88         return;

90     game[0].players[playerIndex].loan.active = 1;
91     game[0].players[playerIndex].loan.amount = maxLoan;
92     game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;
93     game[0].players[playerIndex].loan.remainingRounds = LOAN_DURATION_ROUNDS;

95     receiveMoney(game, playerIndex, maxLoan);

97     printf("\n%s obtained a secured loan.\n", game[0].players[playerIndex].name);

99     formatLKR(maxLoan, loanBuf);
100    printf("Loan Amount : LKR %s.\n", loanBuf);
101    printf("Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
102    printf("Duration : %d Rounds\n", LOAN_DURATION_ROUNDS);

104    printf("Collateral :\n");

106    for(i = 0; i < BOARD_SIZE; i++)
107    {
108        if(game[0].board[i].type != PROPERTY &&
109           game[0].board[i].type != RAILWAY &&
110           game[0].board[i].type != UTILITY)
111        {
112            continue;
113        }

115        if(game[0].board[i].property.owner == playerIndex &&
116           !game[0].board[i].property.mortgaged &&
117           !game[0].board[i].property.loanLocked)
118        {
119            game[0].board[i].property.loanLocked = 1;
120            printf("%s\n", game[0].board[i].name);
121        }
122    }
123 }

```

Line-by-line explanation

Line	Code	Explanation
76	void obtainLoan(GameState game[], int playerIndex)	void obtainLoan(GameState game[], int playerIndex)
77	{	Opens a new block.
78	int i;	Declares int i.
79	int maxLoan;	Declares int maxLoan.
80	char loanBuf[32];	Declares char loanBuf[32].
81		Blank line.
82	if(game[0].players[playerIndex].loan.active)	Condition: the current player.loan.active -> run the next block if true.
83	return; /* only one active loan allowed at a time */	Returns ;/* only one active loan allowed at a time *// immediately.
84		Blank line.
85	maxLoan = calculateMaxLoan(game, playerIndex);	maxLoan = calculateMaxLoan(game, playerIndex);
86		Blank line.
87	if(maxLoan <= 0)	Condition: maxLoan <= 0 -> run the next block if true.
88	return;	Returns the computed value immediately.
89		Blank line.
90	game[0].players[playerIndex].loan.active = 1;	Sets the field active of the player (i.e. 1 when the player has an outstanding loan (only one loan at a time).) = 1.
91	game[0].players[playerIndex].loan.amount = maxLoan;	Sets the field amount of the player (i.e. Current outstanding balance in LKR (grows with compound interest).) = maxLoan.
92	game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;	Sets the field interestRate of the player (i.e. Annual/round interest rate the loan is charged at.) = game[0].economy.loanInterestRate.

93	game[0].players[playerIndex].loan.remainingRounds = LOAN_DURATION_ROUNDS;	Sets the field remainingRounds of the player (i.e. Rounds left before foreclosure.) = LOAN_DURATION_ROUNDS.
94		Blank line.
95	receiveMoney(game, playerIndex, maxLoan);	Calls receiveMoney(game, playerIndex, maxLoan). (Adds money to a player, but only if they are not bankrupt)
96		Blank line.
97	printf("\n%s obtained a secured loan.\n", game[0].players[playerIndex].name);	Prints the message "\n%s obtained a secured loan.\n".
98		Blank line.
99	formatLKR(maxLoan, loanBuf);	Calls formatLKR(maxLoan, loanBuf). (Formats an amount with thousands separators, e)
100	printf("Loan Amount : LKR %s.\n", loanBuf);	Prints the message "Loan Amount : LKR %s.\n".
101	printf("Interest Rate : %d%%\n", game[0].economy.loanInterestRate);	Prints the message "Interest Rate : %d%%\n".
102	printf("Duration : %d Rounds\n", LOAN_DURATION_ROUNDS);	Prints the message "Duration : %d Rounds\n".
103		Blank line.
104	printf("Collateral : \n");	Prints the message "Collateral : \n".
105		Blank line.
106	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
107	{	Opens a new block.
108	if(game[0].board[i].type != PROPERTY &&	Condition: -> run the next block if true.
109	game[0].board[i].type != RAILWAY &&	game[0].board[i].type != RAILWAY &&
110	game[0].board[i].type != UTILITY)	game[0].board[i].type != UTILITY)
111	{	Opens a new block.
112	continue;	Skips to the next loop iteration.
113	}	Closes the current block.
114		Blank line.
115	if(game[0].board[i].property.owner == playerIndex &&	Condition: -> run the next block if true.
116	!game[0].board[i].property.mortgaged &&	!game[0].board[i].property.mortgaged &&
117	!game[0].board[i].property.loanLocked)	!game[0].board[i].property.loanLocked)
118	{	Opens a new block.
119	game[0].board[i].property.loanLocked = 1;	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 1.
120	printf("%s\n", game[0].board[i].name);	Prints the message "%s\n".
121	}	Closes the current block.
122	}	Closes the current block.
123	}	Closes the current block.

Nested functions inside obtainLoan() - study these

- obtainLoan() (Takes a new loan: locks every eligible property as collateral and pays out the max loan at)
- calculateMaxLoan() (Rule-LK 2: the maximum loan = 75% of eligible collateral.)
- receiveMoney() (Adds money to a player, but only if they are not bankrupt.)
- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Takes a new loan: locks every eligible property as collateral and pays out the max loan at the current rate.

7.5 repayLoan()

```
void repayLoan(GameState game[], int playerIndex, int amount)
```

Parameters: playerIndex, amount | Returns: void.

The function, verbatim from the source (starts at line 125)

```
125 void repayLoan(GameState game[], int playerIndex, int amount)
126 {
127     int i;
128
129     if(!game[0].players[playerIndex].loan.active)
```

```

130         return;

132         if(amount > game[0].players[playerIndex].loan.amount)
133             amount = game[0].players[playerIndex].loan.amount;

135         if(game[0].players[playerIndex].cash < amount)
136             amount = game[0].players[playerIndex].cash;

138         payMoney(game, playerIndex, amount);
139         game[0].players[playerIndex].loan.amount -= amount;

141         printf("\n%s repaid LKR %d towards their loan.\n",
142             game[0].players[playerIndex].name, amount);

144         printf("Outstanding Balance : LKR %d\n",
145             game[0].players[playerIndex].loan.amount);

147         if(game[0].players[playerIndex].loan.amount <= 0)
148         {
149             resetLoan(game, playerIndex);

151             for(i = 0; i < BOARD_SIZE; i++)
152             {
153                 if(game[0].board[i].property.owner == playerIndex)
154                     game[0].board[i].property.loanLocked = 0;
155             }

157             printf("%s has fully repaid the loan. Collateral released.\n",
158                 game[0].players[playerIndex].name);
159         }
160     }

```

Line-by-line explanation

Line	Code	Explanation
125	void repayLoan(GameState game[], int playerIndex, int amount)	void repayLoan(GameState game[], int playerIndex, int amount)
126	{	Opens a new block.
127	int i;	Declares int i.
128		Blank line.
129	if(!game[0].players[playerIndex].loan.active)	Condition: !the current player.loan.active -> run the next block if true.
130	return;	Returns the computed value immediately.
131		Blank line.
132	if(amount > game[0].players[playerIndex].loan.amount)	Condition: amount > the current player.loan.amount -> run the next block if true.
133	amount = game[0].players[playerIndex].loan.amount;	amount = game[0].players[playerIndex].loan.amount;
134		Blank line.
135	if(game[0].players[playerIndex].cash < amount)	Condition: the current player.cash < amount -> run the next block if true.
136	amount = game[0].players[playerIndex].cash;	amount = game[0].players[playerIndex].cash;
137		Blank line.
138	payMoney(game, playerIndex, amount);	Calls payMoney(game, playerIndex, amount). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
139	game[0].players[playerIndex].loan.amount -= amount;	Updates the field amount of the player (i.e. Current outstanding balance in LKR (grows with compound interest).): decreases it by amount.
140		Blank line.
141	printf("\n%s repaid LKR %d towards their loan.\n",	Prints the message "\n%s repaid LKR %d towards their loan.\n".
142	game[0].players[playerIndex].name, amount);	game[0].players[playerIndex].name, amount);
143		Blank line.
144	printf("Outstanding Balance : LKR %d\n",	Prints the message "Outstanding Balance : LKR %d\n".
145	game[0].players[playerIndex].loan.amount);	game[0].players[playerIndex].loan.amount);
146		Blank line.
147	if(game[0].players[playerIndex].loan.amount <= 0)	Condition: the current player.loan.amount <= 0 -> run the next block if true.
148	{	Opens a new block.

149	resetLoan(game, playerIndex);	Calls resetLoan(game, playerIndex). (Wipes a player's loan (active, amount, rate, rounds))
150		Blank line.
151	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
152	{	Opens a new block.
153	if(game[0].board[i].property.owner == playerIndex)	Condition: square i.property.owner == playerIndex -> run the next block if true.
154	game[0].board[i].property.loanLocked = 0;	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 0.
155	}	Closes the current block.
156		Blank line.
157	printf("%s has fully repaid the loan. Collateral released.\n",	Prints the message "%s has fully repaid the loan. Collateral released.\n".
158	game[0].players[playerIndex].name);	game[0].players[playerIndex].name);
159	}	Closes the current block.
160	}	Closes the current block.

Nested functions inside repayLoan() - study these

- repayLoan() (Repays part or all of the loan; on full repayment all collateral is released.)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- resetLoan() (Wipes a player's loan (active, amount, rate, rounds).)

MAIN PURPOSE

Repays part or all of the loan; on full repayment all collateral is released.

7.6 handleBankVisit()

```
void handleBankVisit(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 161)

```

162  /* Rule-LK 5: landing on the Bank offers repaying part, repaying in
163     full, or (if no loan yet) obtaining one. With an existing loan a
164     player can also extend the term, refinance at the current rate, or
165     borrow more against newly acquired collateral. Only one
166     transaction happens per visit. */
167  void handleBankVisit(GameState game[], int playerIndex)
168  {
169      printf("\n%s landed on Bank of Ceylon.\n", game[0].players[playerIndex].name);

171      if(game[0].players[playerIndex].loan.active)
172      {
173          if(wantsToRepayLoan(game, playerIndex))
174          {
175              int repayAmount;

177              /* every strategy's wantsToRepayLoan() only triggers once
178                 cash can clear the whole loan, so always repay in full */
179              repayAmount = game[0].players[playerIndex].loan.amount;

181              repayLoan(game, playerIndex, repayAmount);
182              return;
183          }

185          if(wantsIncreaseLoan(game, playerIndex))
186          {
187              increaseLoan(game, playerIndex);
188              return;
189          }

191          if(wantsExtendLoan(game, playerIndex))
192          {
193              extendLoan(game, playerIndex);

```

```

194         return;
195     }

197     if(wantsRefinance(game, playerIndex))
198     {
199         refinanceLoan(game, playerIndex);
200         return;
201     }

203     return;
204 }

206     if(wantsLoan(game, playerIndex))
207     {
208         obtainLoan(game, playerIndex);
209     }
210 }

```

Line-by-line explanation

Line	Code	Explanation
161		Blank line.
162	/* Rule-LK 5: landing on the Bank offers repaying part, repaying in	Comment: /* Rule-LK 5: landing on the Bank offers repaying part, repaying in
163	full, or (if no loan yet) obtaining one. With an existing loan a	full, or (if no loan yet) obtaining one. With an existing loan a
164	player can also extend the term, refinance at the current rate, or	player can also extend the term, refinance at the current rate, or
165	borrow more against newly acquired collateral. Only one	borrow more against newly acquired collateral. Only one
166	transaction happens per visit. */	transaction happens per visit. */
167	void handleBankVisit(GameState game[], int playerIndex)	void handleBankVisit(GameState game[], int playerIndex)
168	{	Opens a new block.
169	printf("\n%s landed on Bank of Ceylon.\n", game[0].players[playerIndex].name);	Prints the message "\n%s landed on Bank of Ceylon.\n".
170		Blank line.
171	if(game[0].players[playerIndex].loan.active)	Condition: the current player.loan.active -> run the next block if true.
172	{	Opens a new block.
173	if(wantsToRepayLoan(game, playerIndex))	Condition: wantsToRepayLoan(game, playerIndex) -> run the next block if true.
174	{	Opens a new block.
175	int repayAmount;	Declares int repayAmount.
176		Blank line.
177	/* every strategy's wantsToRepayLoan() only triggers once	Comment: /* every strategy's wantsToRepayLoan() only triggers once
178	cash can clear the whole loan, so always repay in full */	cash can clear the whole loan, so always repay in full */
179	repayAmount = game[0].players[playerIndex].loan.amount;	repayAmount = game[0].players[playerIndex].loan.amount;
180		Blank line.
181	repayLoan(game, playerIndex, repayAmount);	Calls repayLoan(game, playerIndex, repayAmount). (Repays part or all of the loan; on full repayment all collateral is released)
182	return;	Returns the computed value immediately.
183	}	Closes the current block.
184		Blank line.
185	if(wantsIncreaseLoan(game, playerIndex))	Condition: wantsIncreaseLoan(game, playerIndex) -> run the next block if true.
186	{	Opens a new block.
187	increaseLoan(game, playerIndex);	Calls increaseLoan(game, playerIndex). (Rule-LK 5: borrows 75% of collateral acquired since the loan was taken out)

188	return;	Returns the computed value immediately.
189	}	Closes the current block.
190		Blank line.
191	if(wantsExtendLoan(game, playerIndex))	Condition: wantsExtendLoan(game, playerIndex) -> run the next block if true.
192	{	Opens a new block.
193	extendLoan(game, playerIndex);	Calls extendLoan(game, playerIndex). (Rule-LK 5: adds 10 rounds to the loan deadline)
194	return;	Returns the computed value immediately.
195	}	Closes the current block.
196		Blank line.
197	if(wantsRefinance(game, playerIndex))	Condition: wantsRefinance(game, playerIndex) -> run the next block if true.
198	{	Opens a new block.
199	refinanceLoan(game, playerIndex);	Calls refinanceLoan(game, playerIndex). (Rule-LK 5: switches the loan to the economy's current rate)
200	return;	Returns the computed value immediately.
201	}	Closes the current block.
202		Blank line.
203	return;	Returns the computed value immediately.
204	}	Closes the current block.
205		Blank line.
206	if(wantsLoan(game, playerIndex))	Condition: wantsLoan(game, playerIndex) -> run the next block if true.
207	{	Opens a new block.
208	obtainLoan(game, playerIndex);	Calls obtainLoan(game, playerIndex). (Takes a new loan: locks every eligible property as collateral and pays out the max loan at the current rate)
209	}	Closes the current block.
210	}	Closes the current block.

Nested functions inside handleBankVisit() - study these

- handleBankVisit() (The Bank square: repay full / increase / extend / refinance (existing loan), or obtain a n)
- wantsToRepayLoan() (Whether the player repays the loan (fully) during a Bank visit.)
- repayLoan() (Repays part or all of the loan; on full repayment all collateral is released.)
- wantsIncreaseLoan() (Rule-LK 5: borrow more against newly acquired collateral during a Bank visit.)
- increaseLoan() (Rule-LK 5: borrows 75% of collateral acquired since the loan was taken out.)
- wantsExtendLoan() (Rule-LK 5: extend the loan term by 10 rounds.)
- extendLoan() (Rule-LK 5: adds 10 rounds to the loan deadline.)
- wantsRefinance() (Rule-LK 5: refinance at the economy's current (lower) rate.)
- refinanceLoan() (Rule-LK 5: switches the loan to the economy's current rate.)
- wantsLoan() (Whether the player wants to take out a new loan (Bank visit, no current loan).)
- obtainLoan() (Takes a new loan: locks every eligible property as collateral and pays out the max loan at)

MAIN PURPOSE

The Bank square: repay full / increase / extend / refinance (existing loan), or obtain a new loan (Strategy 3.5). One transaction per visit.

7.7 increaseLoan()

```
void increaseLoan(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 211)

```
212  /* Rule-LK 5: a player with an active loan may borrow more against
213     collateral they've acquired since the loan was taken out. */
214  void increaseLoan(GameState game[], int playerIndex)
215  {
```

```

216     int i;
217     int availableCollateral;
218     int additional;

220     if(!game[0].players[playerIndex].loan.active)
221         return;

223     availableCollateral = 0;

225     for(i = 0; i < BOARD_SIZE; i++)
226     {
227         if(game[0].board[i].type != PROPERTY &&
228            game[0].board[i].type != RAILWAY &&
229            game[0].board[i].type != UTILITY)
230         {
231             continue;
232         }

234         if(game[0].board[i].property.owner != playerIndex)
235             continue;

237         if(game[0].board[i].property.mortgaged)
238             continue;

240         if(game[0].board[i].property.loanLocked)
241             continue;

243         if(game[0].board[i].type == PROPERTY)
244         {
245             availableCollateral +=
246                 (groupBasePrice(game, game[0].board[i].property.group) *
247                  modifierMultiplier(game, MOD_MARKET_MORTGAGE,
248                                     game[0].board[i].property.group, -1)) / 100;
249         }
250         else
251         {
252             availableCollateral += game[0].board[i].property.mortgageValue;
253         }
254     }

256     additional = (availableCollateral * LOAN_COLLATERAL_PERCENT) / 100;

258     if(additional <= 0)
259         return;

261     game[0].players[playerIndex].loan.amount += additional;

263     for(i = 0; i < BOARD_SIZE; i++)
264     {
265         if(game[0].board[i].type != PROPERTY &&
266            game[0].board[i].type != RAILWAY &&
267            game[0].board[i].type != UTILITY)
268         {
269             continue;
270         }

272         if(game[0].board[i].property.owner == playerIndex &&
273            !game[0].board[i].property.mortgaged &&
274            !game[0].board[i].property.loanLocked)
275         {
276             game[0].board[i].property.loanLocked = 1;
277         }
278     }

280     receiveMoney(game, playerIndex, additional);

282     printf("\n%s increased their loan by LKR %d.\n",
283            game[0].players[playerIndex].name, additional);
284     printf("New Outstanding Balance : LKR %d\n",

```

```

285         game[0].players[playerIndex].loan.amount);
286     }

```

Line-by-line explanation

Line	Code	Explanation
211		Blank line.
212	<code>/* Rule-LK 5: a player with an active loan may borrow more against</code>	Comment: <code>/* Rule-LK 5: a player with an active loan may borrow more against</code>
213	<code>collateral they've acquired since the loan was taken out. */</code>	<code>collateral they've acquired since the loan was taken out. */</code>
214	<code>void increaseLoan(GameState game[], int playerIndex)</code>	<code>void increaseLoan(GameState game[], int playerIndex)</code>
215	<code>{</code>	Opens a new block.
216	<code>int i;</code>	Declares <code>int i</code> .
217	<code>int availableCollateral;</code>	Declares <code>int availableCollateral</code> .
218	<code>int additional;</code>	Declares <code>int additional</code> .
219		Blank line.
220	<code>if(!game[0].players[playerIndex].loan.active)</code>	Condition: <code>!the current player.loan.active -> run the next block if true.</code>
221	<code>return;</code>	Returns the computed value immediately.
222		Blank line.
223	<code>availableCollateral = 0;</code>	<code>availableCollateral = 0;</code>
224		Blank line.
225	<code>for(i = 0; i < BOARD_SIZE; i++)</code>	Loop over all entries: <code>init i = 0; run while i < BOARD_SIZE; every loop i++.</code>
226	<code>{</code>	Opens a new block.
227	<code>if(game[0].board[i].type != PROPERTY &&</code>	Condition: <code>-> run the next block if true.</code>
228	<code>game[0].board[i].type != RAILWAY &&</code>	<code>game[0].board[i].type != RAILWAY &&</code>
229	<code>game[0].board[i].type != UTILITY)</code>	<code>game[0].board[i].type != UTILITY)</code>
230	<code>{</code>	Opens a new block.
231	<code>continue;</code>	Skips to the next loop iteration.
232	<code>}</code>	Closes the current block.
233		Blank line.
234	<code>if(game[0].board[i].property.owner != playerIndex)</code>	Condition: <code>square i.property.owner != playerIndex -> run the next block if true.</code>
235	<code>continue;</code>	Skips to the next loop iteration.
236		Blank line.
237	<code>if(game[0].board[i].property.mortgaged)</code>	Condition: <code>square i.property.mortgaged -> run the next block if true.</code>
238	<code>continue;</code>	Skips to the next loop iteration.
239		Blank line.
240	<code>if(game[0].board[i].property.loanLocked)</code>	Condition: <code>square i.property.loanLocked -> run the next block if true.</code>
241	<code>continue;</code>	Skips to the next loop iteration.
242		Blank line.
243	<code>if(game[0].board[i].type == PROPERTY)</code>	Condition: <code>square i.type == PROPERTY -> run the next block if true.</code>
244	<code>{</code>	Opens a new block.
245	<code>availableCollateral +=</code>	<code>availableCollateral +=</code>
246	<code>(groupBasePrice(game,</code>	<code>(groupBasePrice(game, game[0].board[i].property.group) *</code>
247	<code>game[0].board[i].property.group) *</code>	<code>modifierMultiplier(game, MOD_MARKET_MORTGAGE,</code>
248	<code>modifierMultiplier(game, MOD_MARKET_MORTGAGE,</code>	<code>game[0].board[i].property.group, -1)) / 100;</code>
249	<code>game[0].board[i].property.group, -1)) / 100;</code>	Closes the current block.
250	<code>}</code>	The alternative branch when the condition above is false.
251	<code>{</code>	Opens a new block.
252	<code>availableCollateral +=</code>	<code>availableCollateral += game[0].board[i].property.mortgageValue;</code>
253	<code>game[0].board[i].property.mortgageValue;</code>	
254	<code>}</code>	Closes the current block.
255		Blank line.
256	<code>additional = (availableCollateral *</code>	<code>additional = (availableCollateral * LOAN_COLLATERAL_PERCENT) /</code>
257	<code>LOAN_COLLATERAL_PERCENT) / 100;</code>	<code>100;</code>
		Blank line.

258	if(additional <= 0)	Condition: additional <= 0 -> run the next block if true.
259	return;	Returns the computed value immediately.
260		Blank line.
261	game[0].players[playerIndex].loan.amount += additional;	Updates the field amount of the player (i.e. Current outstanding balance in LKR (grows with compound interest).): increases it by additional.
262		Blank line.
263	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
264	{	Opens a new block.
265	if(game[0].board[i].type != PROPERTY &&	Condition: -> run the next block if true.
266	game[0].board[i].type != RAILWAY &&	game[0].board[i].type != RAILWAY &&
267	game[0].board[i].type != UTILITY)	game[0].board[i].type != UTILITY)
268	{	Opens a new block.
269	continue;	Skips to the next loop iteration.
270	}	Closes the current block.
271		Blank line.
272	if(game[0].board[i].property.owner == playerIndex &&	Condition: -> run the next block if true.
273	!game[0].board[i].property.mortgaged &&	!game[0].board[i].property.mortgaged &&
274	!game[0].board[i].property.loanLocked)	!game[0].board[i].property.loanLocked)
275	{	Opens a new block.
276	game[0].board[i].property.loanLocked = 1;	Sets the field loanLocked of the property on the board (i.e. 1 if the property is pledged as loan collateral (cannot be sold).) = 1.
277	}	Closes the current block.
278	}	Closes the current block.
279		Blank line.
280	receiveMoney(game, playerIndex, additional);	Calls receiveMoney(game, playerIndex, additional). (Adds money to a player, but only if they are not bankrupt)
281		Blank line.
282	printf("\n%s increased their loan by LKR %d.\n",	Prints the message "\n%s increased their loan by LKR %d.\n".
283	game[0].players[playerIndex].name, additional);	game[0].players[playerIndex].name, additional);
284	printf("New Outstanding Balance : LKR %d\n",	Prints the message "New Outstanding Balance : LKR %d\n".
285	game[0].players[playerIndex].loan.amount);	game[0].players[playerIndex].loan.amount);
286	}	Closes the current block.

Nested functions inside increaseLoan() - study these

- increaseLoan() (Rule-LK 5: borrows 75% of collateral acquired since the loan was taken out.)
- groupBasePrice() (The base price of a colour group - the first property's purchase price - used ONLY for loa)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- receiveMoney() (Adds money to a player, but only if they are not bankrupt.)

MAIN PURPOSE

Rule-LK 5: borrows 75% of collateral acquired since the loan was taken out.

7.8 extendLoan()

```
void extendLoan(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 287)

```
288  /* Rule-LK 5: extend the loan term by 10 rounds. */
289  void extendLoan(GameState game[], int playerIndex)
290  {
291      if(!game[0].players[playerIndex].loan.active)
292          return;

294      game[0].players[playerIndex].loan.remainingRounds += LOAN_EXTEND_ROUNDS;

296      printf("\n%s extended their loan by %d rounds.\n",
297             game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);
298      printf("New Duration : %d Rounds remaining\n",
```

```

299         game[0].players[playerIndex].loan.remainingRounds);
300     }

```

Line-by-line explanation

Line	Code	Explanation
287		Blank line.
288	<code>/* Rule-LK 5: extend the loan term by 10 rounds. */</code>	Comment: <code>/* Rule-LK 5: extend the loan term by 10 rounds. */</code>
289	<code>void extendLoan(GameState game[], int playerIndex)</code>	<code>void extendLoan(GameState game[], int playerIndex)</code>
290	<code>{</code>	Opens a new block.
291	<code>if(!game[0].players[playerIndex].loan.active)</code>	Condition: <code>!the current player.loan.active -> run the next block if true.</code>
292	<code>return;</code>	Returns the computed value immediately.
293		Blank line.
294	<code>game[0].players[playerIndex].loan.remainingRounds += LOAN_EXTEND_ROUNDS;</code>	Updates the field <code>remainingRounds</code> of the player (i.e. Rounds left before foreclosure.): increases it by <code>LOAN_EXTEND_ROUNDS</code> .
295		Blank line.
296	<code>printf("\n%s extended their loan by %d rounds.\n",</code>	Prints the message <code>"\n%s extended their loan by %d rounds.\n"</code> .
297	<code>game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);</code>	<code>game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);</code>
298	<code>printf("New Duration : %d Rounds remaining\n",</code>	Prints the message <code>"New Duration : %d Rounds remaining\n"</code> .
299	<code>game[0].players[playerIndex].loan.remainingRounds);</code>	<code>game[0].players[playerIndex].loan.remainingRounds);</code>
300	<code>}</code>	Closes the current block.

Nested functions inside `extendLoan()` - study these

- `extendLoan()` (Rule-LK 5: adds 10 rounds to the loan deadline.)

MAIN PURPOSE

Rule-LK 5: adds 10 rounds to the loan deadline.

7.9 `refinanceLoan()`

```
void refinanceLoan(GameState game[], int playerIndex)
```

Parameters: `playerIndex` | Returns: `void`.

The function, verbatim from the source (starts at line 301)

```

302     /* Rule-LK 5: refinance the loan at the current economy interest rate. */
303     void refinanceLoan(GameState game[], int playerIndex)
304     {
305         if(!game[0].players[playerIndex].loan.active)
306             return;

308         game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;

310         printf("\n%s refinanced their loan at the current rate of %d%.\n",
311             game[0].players[playerIndex].name,
312             game[0].economy.loanInterestRate);
313     }

```

Line-by-line explanation

Line	Code	Explanation
301		Blank line.
302	<code>/* Rule-LK 5: refinance the loan at the current economy interest rate. */</code>	Comment: <code>/* Rule-LK 5: refinance the loan at the current economy interest rate. */</code>
303	<code>void refinanceLoan(GameState game[], int playerIndex)</code>	<code>void refinanceLoan(GameState game[], int playerIndex)</code>
304	<code>{</code>	Opens a new block.
305	<code>if(!game[0].players[playerIndex].loan.active)</code>	Condition: <code>!the current player.loan.active -> run the next block if true.</code>
306	<code>return;</code>	Returns the computed value immediately.
307		Blank line.
308	<code>game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;</code>	Sets the field <code>interestRate</code> of the player (i.e. Annual/round interest rate the loan is charged at.) = <code>game[0].economy.loanInterestRate</code> .

309		Blank line.
310	printf("\n%s refinanced their loan at the current rate of %d%%.\n",	Prints the message "\n%s refinanced their loan at the current rate of %d%%.\n".
311	game[0].players[playerIndex].name,	game[0].players[playerIndex].name,
312	game[0].economy.loanInterestRate);	game[0].economy.loanInterestRate);
313	}	Closes the current block.

Nested functions inside refinanceLoan() - study these

- refinanceLoan() (Rule-LK 5: switches the loan to the economy's current rate.)

MAIN PURPOSE

Rule-LK 5: switches the loan to the economy's current rate.

7.10 demolishBuildingsOn()

```
void demolishBuildingsOn(GameState game[], int index)
```

Parameters: *index* | Returns: *void*.

The function, verbatim from the source (starts at line 315)

```
315 void demolishBuildingsOn(GameState game[], int index)
316 {
317     game[0].board[index].property.houses = 0;
318     game[0].board[index].property.hotel = 0;
319 }
```

Line-by-line explanation

Line	Code	Explanation
315	void demolishBuildingsOn(GameState game[], int index)	void demolishBuildingsOn(GameState game[], int index)
316	{	Opens a new block.
317	game[0].board[index].property.houses = 0;	Sets the field houses of the property on the board (i.e. Number of houses (0..4).) = 0.
318	game[0].board[index].property.hotel = 0;	Sets the field hotel of the property on the board (i.e. 1 if a hotel is built (replaces the 4 houses).) = 0.
319	}	Closes the current block.

Nested functions inside demolishBuildingsOn() - study these

- demolishBuildingsOn() (Zeroes houses and hotel on a property (used on seizure and bankruptcy).)

MAIN PURPOSE

Zeroes houses and hotel on a property (used on seizure and bankruptcy).

7.11 foreclose()

```
void foreclose(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 320)

```
321 /* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties
322    are seized and auctioned. If nothing is left, the player is
323    declared bankrupt. */
324 void foreclose(GameState game[], int playerIndex)
325 {
326     int i;
327     int ownsAnythingLeft;
328
329     printf("\n%s has defaulted on their loan!\n", game[0].players[playerIndex].name);
330
331     ownsAnythingLeft = 0;
332
333     for(i = 0; i < BOARD_SIZE; i++)
```

```

334     {
335         if(game[0].board[i].property.owner != playerIndex)
336             continue;

338         if(!game[0].board[i].property.loanLocked)
339         {
340             ownsAnythingLeft = 1;
341             continue;
342         }

344         printf("%s (pledged collateral) transferred to the Bank.\n",
345             game[0].board[i].name);

347         demolishBuildingsOn(game, i);

349         stripOwnership(game, i);

351         adjustOwnedCount(game, playerIndex, i, -1);

353         runAuction(game, i, -1);
354     }

356     resetLoan(game, playerIndex);

358     printf("Outstanding debt cleared.\n");

360     if(!ownsAnythingLeft && game[0].players[playerIndex].cash <= 0)
361     {
362         game[0].players[playerIndex].bankrupt = 1;
363         printf("%s has no remaining assets and is declared BANKRUPT.\n",
364             game[0].players[playerIndex].name);
365     }
366 }

```

Line-by-line explanation

Line	Code	Explanation
320		Blank line.
321	/* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties	Comment: /* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties
322	are seized and auctioned. If nothing is left, the player is	are seized and auctioned. If nothing is left, the player is
323	declared bankrupt. */	declared bankrupt. */
324	void foreclose(GameState game[], int playerIndex)	void foreclose(GameState game[], int playerIndex)
325	{	Opens a new block.
326	int i;	Declares int i.
327	int ownsAnythingLeft;	Declares int ownsAnythingLeft.
328		Blank line.
329	printf("\n%s has defaulted on their loan!\n", game[0].players[playerIndex].name);	Prints the message "\n%s has defaulted on their loan!\n".
330		Blank line.
331	ownsAnythingLeft = 0;	ownsAnythingLeft = 0;
332		Blank line.
333	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
334	{	Opens a new block.
335	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
336	continue;	Skips to the next loop iteration.
337		Blank line.
338	if(!game[0].board[i].property.loanLocked)	Condition: !square i.property.loanLocked -> run the next block if true.
339	{	Opens a new block.
340	ownsAnythingLeft = 1;	ownsAnythingLeft = 1;
341	continue;	Skips to the next loop iteration.
342	}	Closes the current block.

343		Blank line.
344	<code>printf("%s (pledged collateral) transferred to the Bank.\n",</code>	Prints the message "%s (pledged collateral) transferred to the Bank.\n".
345	<code>game[0].board[i].name);</code>	<code>game[0].board[i].name);</code>
346		Blank line.
347	<code>demolishBuildingsOn(game, i);</code>	Calls <code>demolishBuildingsOn(game, i)</code> . (Zeroes houses and hotel on a property (used on seizure and bankruptcy))
348		Blank line.
349	<code>stripOwnership(game, i);</code>	Calls <code>stripOwnership(game, i)</code> . (Resets a square to unowned: no owner, mortgage, loan pledge or insurance)
350		Blank line.
351	<code>adjustOwnedCount(game, playerIndex, i, -1);</code>	Calls <code>adjustOwnedCount(game, playerIndex, i, -1)</code> . (Updates the player's owned counter to match the square's type)
352		Blank line.
353	<code>runAuction(game, i, -1);</code>	Calls <code>runAuction(game, i, -1)</code> . (Sells a property to the highest bidder among solvent players)
354	<code>}</code>	Closes the current block.
355		Blank line.
356	<code>resetLoan(game, playerIndex);</code>	Calls <code>resetLoan(game, playerIndex)</code> . (Wipes a player's loan (active, amount, rate, rounds))
357		Blank line.
358	<code>printf("Outstanding debt cleared.\n");</code>	Prints the message "Outstanding debt cleared.\n".
359		Blank line.
360	<code>if(!ownsAnythingLeft && game[0].players[playerIndex].cash <= 0)</code>	Condition: <code>ownsAnythingLeft &&</code> the current player.cash <= 0 -> run the next block if true.
361	<code>{</code>	Opens a new block.
362	<code>game[0].players[playerIndex].bankrupt = 1;</code>	Sets the field <code>bankrupt</code> of the player (i.e. 1 once the player is out of the game.) = 1.
363	<code>printf("%s has no remaining assets and is declared BANKRUPT.\n",</code>	Prints the message "%s has no remaining assets and is declared BANKRUPT.\n".
364	<code>game[0].players[playerIndex].name);</code>	<code>game[0].players[playerIndex].name);</code>
365	<code>}</code>	Closes the current block.
366	<code>}</code>	Closes the current block.

Nested functions inside `foreclose()` - study these

- `foreclose()` (Rule-LK 6/7: a defaulted loan seizes and auctions every pledged property; the player is ba)
- `demolishBuildingsOn()` (Zeroes houses and hotel on a property (used on seizure and bankruptcy).)
- `stripOwnership()` (Resets a square to unowned: no owner, mortgage, loan pledge or insurance.)
- `adjustOwnedCount()` (Updates the player's owned counter to match the square's type.)
- `runAuction()` (Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise)
- `resetLoan()` (Wipes a player's loan (active, amount, rate, rounds).)

MAIN PURPOSE

Rule-LK 6/7: a defaulted loan seizes and auctions every pledged property; the player is bankrupt if nothing is left.

7.12 `processLoans()`

```
void processLoans(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 368)

```
368 void processLoans(GameState game[])
369 {
370     int i;
371     int interest;

373     for(i = 0; i < MAX_PLAYERS; i++)
374     {
375         if(game[0].players[i].bankrupt)
```

```

376         continue;

378         if(!game[0].players[i].loan.active)
379             continue;

381         interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100;
382         game[0].players[i].loan.amount += interest;

384         game[0].players[i].loan.remainingRounds--;

386         printf("\n%s's loan accrued LKR %d interest. New balance : LKR %d\n",
387             game[0].players[i].name, interest, game[0].players[i].loan.amount);

389         if(game[0].players[i].loan.remainingRounds <= 0)
390         {
391             foreclose(game, i);
392         }
393     }
394 }

```

Line-by-line explanation

Line	Code	Explanation
368	void processLoans(GameState game[])	void processLoans(GameState game[])
369	{	Opens a new block.
370	int i;	Declares int i.
371	int interest;	Declares int interest.
372		Blank line.
373	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
374	{	Opens a new block.
375	if(game[0].players[i].bankrupt)	Condition: player i.bankrupt -> run the next block if true.
376	continue;	Skips to the next loop iteration.
377		Blank line.
378	if(!game[0].players[i].loan.active)	Condition: !player i.loan.active -> run the next block if true.
379	continue;	Skips to the next loop iteration.
380		Blank line.
381	interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100;	Declares interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100.
382	game[0].players[i].loan.amount += interest;	Updates the field amount of the player (i.e. Current outstanding balance in LKR (grows with compound interest).); increases it by interest.
383		Blank line.
384	game[0].players[i].loan.remainingRounds--;	game[0].players[i].loan.remainingRounds--;
385		Blank line.
386	printf("\n%s's loan accrued LKR %d interest. New balance : LKR %d\n",	Prints the message "\n%s's loan accrued LKR %d interest. New balance : LKR %d\n".
387	game[0].players[i].name, interest, game[0].players[i].loan.amount);	game[0].players[i].name, interest, game[0].players[i].loan.amount);
388		Blank line.
389	if(game[0].players[i].loan.remainingRounds <= 0)	Condition: player i.loan.remainingRounds <= 0 -> run the next block if true.
390	{	Opens a new block.
391	foreclose(game, i);	Calls foreclose(game, i). (Rule-LK 6/7: a defaulted loan seizes and auctions every pledged property; the player is bankrupt if nothing is left)
392	}	Closes the current block.
393	}	Closes the current block.
394	}	Closes the current block.

Nested functions inside processLoans() - study these

- processLoans() (Round-end loan heartbeat: compounds interest into the principal and counts down; at 0 round)
- foreclose() (Rule-LK 6/7: a defaulted loan seizes and auctions every pledged property; the player is bankrupt)

MAIN PURPOSE

Round-end loan heartbeat: compounds interest into the principal and counts down; at 0 rounds, forecloses.

The auction engine (Rule-LK 19-23): starting price, bidding rounds, withdrawal, forced-sale handling.

The sub-topics below are the 2 functions implemented in auction.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in auction.c

- | | | |
|-----|----------------------------------|--------|
| 8.1 | getAskingValue() (source line 4) | p. 147 |
| 8.2 | runAuction() (source line 23) | p. 148 |

8.1 getAskingValue()

```
int getAskingValue(GameState game[], int propIndex)
```

Parameters: propIndex | Returns: Asking value in LKR.

The function, verbatim from the source (starts at line 4)

```
5      /* Rule-LK 19: opening bid is 50% of market value. A market decline
6         lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.
7         Railways and utilities use currentMarketValue too, so their value
8         modifiers (Port Expansion, Water Shortage, ...) apply the same way
9         they do for colour-group properties. */
10     int getAskingValue(GameState game[], int propIndex)
11     {
12         int value;
13         int group;

15         value = currentMarketValue(game, propIndex);

17         group = groupOf(game, propIndex);

19         value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;

21         return value;
22     }
```

Line-by-line explanation

Line	Code	Explanation
4		Blank line.
5	/* Rule-LK 19: opening bid is 50% of market value. A market decline	Comment: /* Rule-LK 19: opening bid is 50% of market value. A market decline
6	lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.	lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.
7	Railways and utilities use currentMarketValue too, so their value	Railways and utilities use currentMarketValue too, so their value
8	modifiers (Port Expansion, Water Shortage, ...) apply the same way	modifiers (Port Expansion, Water Shortage, ...) apply the same way
9	they do for colour-group properties. */	they do for colour-group properties. */
10	int getAskingValue(GameState game[], int propIndex)	int getAskingValue(GameState game[], int propIndex)
11	{	Opens a new block.
12	int value;	Declares int value.
13	int group;	Declares int group.
14		Blank line.
15	value = currentMarketValue(game, propIndex);	value = currentMarketValue(game, propIndex);
16		Blank line.
17	group = groupOf(game, propIndex);	group = groupOf(game, propIndex);
18		Blank line.
19	value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;	value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;
20		Blank line.
21	return value;	Returns value immediately.

Nested functions inside getAskingValue() - study these

- getAskingValue() (Rule-LK 19/32/34: asking value = current market value x auction modifier (declining groups))
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- groupOf() (Returns the colour group of a square, or -1 if it isn't a property.)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

Rule-LK 19/32/34: asking value = current market value x auction modifier (declining groups start -25%).

8.2 runAuction()

```
void runAuction(GameState game[], int propIndex, int sellerIndex)
```

Parameters: propIndex, sellerIndex | Returns: void.

The function, verbatim from the source (starts at line 23)

```

24  /* Rule-LK 19-23: runs an auction among all solvent players. A
25     player who declines to bid is out permanently, so the pool of
26     bidders can only shrink - guaranteeing the loop finishes.
27     sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the
28     proceeds to that player; sellerIndex == -1 pays the Bank. */
29  void runAuction(GameState game[], int propIndex, int sellerIndex)
30  {
31      int active[MAX_PLAYERS];
32      int activeCount;
33      int i;
34      int currentBid;
35      int highBidder;
36      int candidateBid;
37      int safetyRounds;
38      char moneyBuf[32];

40      printf("\n*** AUCTION ***\n");
41      printf("Property : %s\n", game[0].board[propIndex].name);

43      currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;

45      formatLKR(currentBid, moneyBuf);
46      printf("Opening Bid : LKR %s\n", moneyBuf);

48      highBidder = -1;
49      activeCount = 0;

51      for(i = 0; i < MAX_PLAYERS; i++)
52      {
53          active[i] = !game[0].players[i].bankrupt;

55          /* Anti-Speculation Act (Rule-LK 8): the forced seller must
56             actually part with the property, so they never bid. If the
57             seller were allowed to bid, they could win the auction back
58             and merely pay the bid to themselves (net-zero cost), which
59             defeats the forced sale. */
60          if(i == sellerIndex)
61              active[i] = 0;

63          if(active[i])
64              activeCount++;
65      }

67      safetyRounds = 0;

69      while(activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS)
70      {
71          for(i = 0; i < MAX_PLAYERS; i++)

```

```

72     {
73         if(!active[i])
74             continue;

76         if(activeCount <= 1)
77             break;

79         candidateBid = currentBid + AUCTION_BID_INCREMENT;

81         if(willingToBid(game, i, propIndex, candidateBid))
82         {
83             currentBid = candidateBid;
84             highBidder = i;

86             formatLKR(currentBid, moneyBuf);
87             printf("%s bids LKR %s.\n", game[0].players[i].name, moneyBuf);
88         }
89         else
90         {
91             active[i] = 0;
92             activeCount--;

94             printf("%s withdraws.\n", game[0].players[i].name);
95         }
96     }

98     safetyRounds++;
99 }

101 /* If exactly one bidder is still active, give them one final
102    chance to bid at the next increment before awarding. */
103 if(activeCount == 1 && highBidder == -1)
104 {
105     int remaining;

107     remaining = -1;

109     for(i = 0; i < MAX_PLAYERS; i++)
110     {
111         if(active[i])
112         {
113             remaining = i;
114             break;
115         }
116     }

118     if(remaining != -1)
119     {
120         candidateBid = currentBid + AUCTION_BID_INCREMENT;

122         if(willingToBid(game, remaining, propIndex, candidateBid))
123         {
124             currentBid = candidateBid;
125             highBidder = remaining;

127             formatLKR(currentBid, moneyBuf);
128             printf("%s bids LKR %s.\n",
129                 game[0].players[remaining].name, moneyBuf);
130         }
131     }
132 }

134 if(highBidder == -1)
135 {
136     int lastActive;

138     /* One player never got a chance to bid (e.g. the first active
139        player declined while only two were in) - they may take the
140        property at the opening bid. */

```

```

141         lastActive = -1;

143         for(i = 0; i < MAX_PLAYERS; i++)
144         {
145             if(active[i])
146             {
147                 lastActive = i;
148                 break;
149             }
150         }

152         /* No one left to bid, or they decline the opening price */
153         if(lastActive == -1 || !willingToBid(game, lastActive, propIndex, currentBid))
154         {
155             printf("No bids received. %s remains with the Bank.\n",
156                 game[0].board[propIndex].name);
157             return;
158         }

160         highBidder = lastActive;
161     }

163     /* Always format the final winning price so the display is correct
164        regardless of which code path set highBidder. */
165     formatLKR(currentBid, moneyBuf);

167     payMoney(game, highBidder, currentBid);

169     if(sellerIndex >= 0)
170         receiveMoney(game, sellerIndex, currentBid);

172     game[0].board[propIndex].property.owner = highBidder;

174     adjustOwnedCount(game, highBidder, propIndex, 1);

176     printf("%s wins the auction for LKR %s.\n",
177         game[0].players[highBidder].name, moneyBuf);
178 }

```

Line-by-line explanation

Line	Code	Explanation
23		Blank line.
24	/* Rule-LK 19-23: runs an auction among all solvent players. A	Comment: /* Rule-LK 19-23: runs an auction among all solvent players. A
25	player who declines to bid is out permanently, so the pool of	player who declines to bid is out permanently, so the pool of
26	bidders can only shrink - guaranteeing the loop finishes.	bidders can only shrink - guaranteeing the loop finishes.
27	sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the	sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the
28	proceeds to that player; sellerIndex == -1 pays the Bank. */	proceeds to that player; sellerIndex == -1 pays the Bank. */
29	void runAuction(GameState game[], int propIndex, int sellerIndex)	void runAuction(GameState game[], int propIndex, int sellerIndex)
30	{	Opens a new block.
31	int active[MAX_PLAYERS];	Declares int active[MAX_PLAYERS].
32	int activeCount;	Declares int activeCount.
33	int i;	Declares int i.
34	int currentBid;	Declares int currentBid.
35	int highBidder;	Declares int highBidder.
36	int candidateBid;	Declares int candidateBid.
37	int safetyRounds;	Declares int safetyRounds.
38	char moneyBuf[32];	Declares char moneyBuf[32].
39		Blank line.

40	printf("\n*** AUCTION ***\n");	Prints the message "\n*** AUCTION ***\n".
41	printf("Property : %s\n", game[0].board[propIndex].name);	Prints the message "Property : %s\n".
42		Blank line.
43	currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;	currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;
44		Blank line.
45	formatLKR(currentBid, moneyBuf);	Calls formatLKR(currentBid, moneyBuf). (Formats an amount with thousands separators, e)
46	printf("Opening Bid : LKR %s\n", moneyBuf);	Prints the message "Opening Bid : LKR %s\n".
47		Blank line.
48	highBidder = -1;	highBidder = -1;
49	activeCount = 0;	activeCount = 0;
50		Blank line.
51	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
52	{	Opens a new block.
53	active[i] = !game[0].players[i].bankrupt;	active[i] = !game[0].players[i].bankrupt;
54		Blank line.
55	/* Anti-Speculation Act (Rule-LK 8): the forced seller must	Comment: /* Anti-Speculation Act (Rule-LK 8): the forced seller must
56	actually part with the property, so they never bid. If the	actually part with the property, so they never bid. If the
57	seller were allowed to bid, they could win the auction back	seller were allowed to bid, they could win the auction back
58	and merely pay the bid to themselves (net-zero cost), which	and merely pay the bid to themselves (net-zero cost), which
59	defeats the forced sale. */	defeats the forced sale. */
60	if(i == sellerIndex)	Condition: i == sellerIndex -> run the next block if true.
61	active[i] = 0;	active[i] = 0;
62		Blank line.
63	if(active[i])	Condition: active[i] -> run the next block if true.
64	activeCount++;	activeCount++;
65	}	Closes the current block.
66		Blank line.
67	safetyRounds = 0;	safetyRounds = 0;
68		Blank line.
69	while(activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS)	Loop: repeat while activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS is true.
70	{	Opens a new block.
71	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
72	{	Opens a new block.
73	if(!active[i])	Condition: !active[i] -> run the next block if true.
74	continue;	Skips to the next loop iteration.
75		Blank line.
76	if(activeCount <= 1)	Condition: activeCount <= 1 -> run the next block if true.
77	break;	Breaks out of the current loop or switch.
78		Blank line.
79	candidateBid = currentBid + AUCTION_BID_INCREMENT;	candidateBid = currentBid + AUCTION_BID_INCREMENT;
80		Blank line.
81	if(willingToBid(game, i, propIndex, candidateBid))	Condition: willingToBid(game, i, propIndex, candidateBid) -> run the next block if true.
82	{	Opens a new block.
83	currentBid = candidateBid;	currentBid = candidateBid;
84	highBidder = i;	highBidder = i;
85		Blank line.

86	formatLKR(currentBid, moneyBuf);	Calls formatLKR(currentBid, moneyBuf). (Formats an amount with thousands separators, e)
87	printf("%s bids LKR %s.\n", game[0].players[i].name, moneyBuf);	Prints the message "%s bids LKR %s.\n".
88	}	Closes the current block.
89	else	The alternative branch when the condition above is false.
90	{	Opens a new block.
91	active[i] = 0;	active[i] = 0;
92	activeCount--;	activeCount--;
93		Blank line.
94	printf("%s withdraws.\n", game[0].players[i].name);	Prints the message "%s withdraws.\n".
95	}	Closes the current block.
96	}	Closes the current block.
97		Blank line.
98	safetyRounds++;	safetyRounds++;
99	}	Closes the current block.
100		Blank line.
101	/* If exactly one bidder is still active, give them one final	Comment: /* If exactly one bidder is still active, give them one final
102	chance to bid at the next increment before awarding. */	chance to bid at the next increment before awarding. */
103	if(activeCount == 1 && highBidder == -1)	Condition: activeCount == 1 && highBidder == -1 -> run the next block if true.
104	{	Opens a new block.
105	int remaining;	Declares int remaining.
106		Blank line.
107	remaining = -1;	remaining = -1;
108		Blank line.
109	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
110	{	Opens a new block.
111	if(active[i])	Condition: active[i] -> run the next block if true.
112	{	Opens a new block.
113	remaining = i;	remaining = i;
114	break;	Breaks out of the current loop or switch.
115	}	Closes the current block.
116	}	Closes the current block.
117		Blank line.
118	if(remaining != -1)	Condition: remaining != -1 -> run the next block if true.
119	{	Opens a new block.
120	candidateBid = currentBid + AUCTION_BID_INCREMENT;	candidateBid = currentBid + AUCTION_BID_INCREMENT;
121		Blank line.
122	if(willingToBid(game, remaining, propIndex, candidateBid))	Condition: willingToBid(game, remaining, propIndex, candidateBid) -> run the next block if true.
123	{	Opens a new block.
124	currentBid = candidateBid;	currentBid = candidateBid;
125	highBidder = remaining;	highBidder = remaining;
126		Blank line.
127	formatLKR(currentBid, moneyBuf);	Calls formatLKR(currentBid, moneyBuf). (Formats an amount with thousands separators, e)
128	printf("%s bids LKR %s.\n",	Prints the message "%s bids LKR %s.\n".
129	game[0].players[remaining].name, moneyBuf);	game[0].players[remaining].name, moneyBuf);
130	}	Closes the current block.
131	}	Closes the current block.
132	}	Closes the current block.
133		Blank line.
134	if(highBidder == -1)	Condition: highBidder == -1 -> run the next block if true.

135	{	Opens a new block.
136	int lastActive;	Declares int lastActive.
137		Blank line.
138	/* One player never got a chance to bid (e.g. the first active	Comment: /* One player never got a chance to bid (e.g. the first active
139	player declined while only two were in) - they may take the	player declined while only two were in) - they may take the
140	property at the opening bid. */	property at the opening bid. */
141	lastActive = -1;	lastActive = -1;
142		Blank line.
143	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
144	{	Opens a new block.
145	if(active[i])	Condition: active[i] -> run the next block if true.
146	{	Opens a new block.
147	lastActive = i;	lastActive = i;
148	break;	Breaks out of the current loop or switch.
149	}	Closes the current block.
150	}	Closes the current block.
151		Blank line.
152	/* No one left to bid, or they decline the opening price */	Comment: /* No one left to bid, or they decline the opening price */
153	if(lastActive == -1 !willingToBid(game, lastActive, propIndex, currentBid))	Condition: lastActive == -1 !willingToBid(game, lastActive, propIndex, currentBid) -> run the next block if true.
154	{	Opens a new block.
155	printf("No bids received. %s remains with the Bank.\n",	Prints the message "No bids received. %s remains with the Bank.\n".
156	game[0].board[propIndex].name);	game[0].board[propIndex].name);
157	return;	Returns the computed value immediately.
158	}	Closes the current block.
159		Blank line.
160	highBidder = lastActive;	highBidder = lastActive;
161	}	Closes the current block.
162		Blank line.
163	/* Always format the final winning price so the display is correct	Comment: /* Always format the final winning price so the display is correct
164	regardless of which code path set highBidder. */	regardless of which code path set highBidder. */
165	formatLKR(currentBid, moneyBuf);	Calls formatLKR(currentBid, moneyBuf). (Formats an amount with thousands separators, e)
166		Blank line.
167	payMoney(game, highBidder, currentBid);	Calls payMoney(game, highBidder, currentBid). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
168		Blank line.
169	if(sellerIndex >= 0)	Condition: sellerIndex >= 0 -> run the next block if true.
170	receiveMoney(game, sellerIndex, currentBid);	Calls receiveMoney(game, sellerIndex, currentBid). (Adds money to a player, but only if they are not bankrupt)
171		Blank line.
172	game[0].board[propIndex].property.owner = highBidder;	Sets the field owner of the property on the board (i.e. Player index who owns it; -1 means unowned.) = highBidder.
173		Blank line.
174	adjustOwnedCount(game, highBidder, propIndex, 1);	Calls adjustOwnedCount(game, highBidder, propIndex, 1). (Updates the player's owned counter to match the square's type)
175		Blank line.
176	printf("%s wins the auction for LKR %s.\n",	Prints the message "%s wins the auction for LKR %s.\n".
177	game[0].players[highBidder].name, moneyBuf);	game[0].players[highBidder].name, moneyBuf);
178	}	Closes the current block.

Nested functions inside runAuction() - study these

- runAuction() (Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise)
- getAskingValue() (Rule-LK 19/32/34: asking value = current market value x auction modifier (declining groups))
- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)
- willingToBid() (Whether the player raises the bid at an auction.)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- receiveMoney() (Adds money to a player, but only if they are not bankrupt.)
- adjustOwnedCount() (Updates the player's owned counter to match the square's type.)

MAIN PURPOSE

Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise by 250, passers are out forever. The forced seller never bids.

National Event Cards, economic events (every 15 rounds) and government regulations (every 20 rounds).

The sub-topics below are the 6 functions implemented in events.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in events.c

9.1	clampLoanInterest()	(source line 5)	p. 155
9.2	adjustLoanInterest()	(source line 15)	p. 155
9.3	executeEvent()	(source line 22)	p. 156
9.4	triggerEconomicEvent()	(source line 204)	p. 164
9.5	triggerGovernmentRegulation()	(source line 294)	p. 168
9.6	decrementEventTimers()	(source line 388)	p. 172

9.1 clampLoanInterest()

```
void clampLoanInterest(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 5)

```
6      /* Keeps the loan interest rate inside its allowed range. */
7      void clampLoanInterest(GameState game[])
8      {
9          if(game[0].economy.loanInterestRate < LOAN_INTEREST_MIN)
10             game[0].economy.loanInterestRate = LOAN_INTEREST_MIN;

12         if(game[0].economy.loanInterestRate > LOAN_INTEREST_MAX)
13             game[0].economy.loanInterestRate = LOAN_INTEREST_MAX;
14     }
```

Line-by-line explanation

Line	Code	Explanation
5		Blank line.
6	/* Keeps the loan interest rate inside its allowed range. */	Comment: /* Keeps the loan interest rate inside its allowed range. */
7	void clampLoanInterest(GameState game[])	void clampLoanInterest(GameState game[])
8	{	Opens a new block.
9	if(game[0].economy.loanInterestRate < LOAN_INTEREST_MIN)	Condition: game[0].economy.loanInterestRate < LOAN_INTEREST_MIN -> run the next block if true.
10	game[0].economy.loanInterestRate = LOAN_INTEREST_MIN;	Sets the field loanInterestRate of the shared economy (i.e. Current economy loan interest rate (1..25).) = LOAN_INTEREST_MIN.
11		Blank line.
12	if(game[0].economy.loanInterestRate > LOAN_INTEREST_MAX)	Condition: game[0].economy.loanInterestRate > LOAN_INTEREST_MAX -> run the next block if true.
13	game[0].economy.loanInterestRate = LOAN_INTEREST_MAX;	Sets the field loanInterestRate of the shared economy (i.e. Current economy loan interest rate (1..25).) = LOAN_INTEREST_MAX.
14	}	Closes the current block.

Nested functions inside clampLoanInterest() - study these

- clampLoanInterest() (Keeps the loan interest rate inside [1, 25].)

MAIN PURPOSE

Keeps the loan interest rate inside [1, 25].

9.2 adjustLoanInterest()

```
void adjustLoanInterest(GameState game[], int deltaPercent)
```

Parameters: deltaPercent | Returns: void.

The function, verbatim from the source (starts at line 15)

```
16  /* Moves the loan interest rate up or down by deltaPercent. */
17  void adjustLoanInterest(GameState game[], int deltaPercent)
18  {
19      game[0].economy.loanInterestRate += deltaPercent;
20      clampLoanInterest(game);
21  }
```

Line-by-line explanation

Line	Code	Explanation
15		Blank line.
16	/* Moves the loan interest rate up or down by deltaPercent. */	Comment: /* Moves the loan interest rate up or down by deltaPercent. */
17	void adjustLoanInterest(GameState game[], int deltaPercent)	void adjustLoanInterest(GameState game[], int deltaPercent)
18	{	Opens a new block.
19	game[0].economy.loanInterestRate += deltaPercent;	Updates the field loanInterestRate of the shared economy (i.e. Current economy loan interest rate (1..25).): increases it by deltaPercent.
20	clampLoanInterest(game);	Calls clampLoanInterest(game). (Keeps the loan interest rate inside [1, 25])
21	}	Closes the current block.

Nested functions inside adjustLoanInterest() - study these

- adjustLoanInterest() (Moves the loan interest rate by deltaPercent and clamps it.)
- clampLoanInterest() (Keeps the loan interest rate inside [1, 25].)

MAIN PURPOSE

Moves the loan interest rate by deltaPercent and clamps it.

9.3 executeEvent()

```
void executeEvent(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 22)

```
23  /* Appendix A: 20 National Event Cards, drawn when a player lands on
24  an EVENT square. currentCardIndex cycles 0-19 and wraps around,
25  which behaves the same as "draw the top card, put it at the
26  bottom" without needing a real deck data structure. */
27  void executeEvent(GameState game[], int playerIndex)
28  {
29      (void)playerIndex; /* not needed - these effects are global, not per-player */
30
31      printf("\n*** NATIONAL EVENT CARD ***\n");
32
33      switch(game[0].economy.currentCardIndex)
34      {
35          case 0:
36              printf("Tourism Hype : Hotels earn double rent for 5 rounds.\n");
37              addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5);
38              break;
39
40          case 1:
41              printf("Fuel Shortage : Railway rent doubles for 5 rounds.\n");
42              addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5);
43              break;
44
45          case 2:
46              printf("Heavy Floods : a random coastal property is damaged.\n");
47              triggerDisaster(game);
48              break;
```

```

50     case 3:
51     {
52         int candidates[BOARD_SIZE];
53         int count;
54         int i;

56         count = 0;

58         for(i = 0; i < BOARD_SIZE; i++)
59         {
60             if(game[0].board[i].type == PROPERTY && game[0].board[i].property.owner != -1)
61             {
62                 candidates[count] = i;
63                 count++;
64             }
65         }

67         if(count > 0)
68         {
69             game[0].economy.closedPropertyIndex = candidates[rand() % count];
70             game[0].economy.closedPropertyRoundsLeft = 2;

72             printf("Political Rally : %s is closed for 2 rounds.\n",
73                    game[0].board[game[0].economy.closedPropertyIndex].name);
74         }
75         else
76         {
77             printf("Political Rally : no owned property to close.\n");
78         }
79         break;
80     }

82     case 4:
83         printf("Stock Market Rise : All property values increase by 10%% for 15 rounds.\n");
84         addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
85         break;

87     case 5:
88         printf("Economic Downturn : Property values decrease by 15%% for 15 rounds.\n");
89         addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
90         break;

92     case 6:
93         printf("Housing Subsidy : House construction cost reduced by 30%% for 15 rounds.\n");
94         addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15);
95         break;

97     case 7:
98         adjustLoanInterest(game, -LOAN_INTEREST_STEP);

100         printf("Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.\n",
101                LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
102         break;

104     case 8:
105         adjustLoanInterest(game, LOAN_INTEREST_STEP);

107         printf("Interest Rate Increase : Loan interest increased by %d%%. Now %d%%.\n",
108                LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
109         break;

111     case 9:
112     {
113         int i;

115         printf("Tax Amnesty : Each player receives LKR 2,000.\n");

117         for(i = 0; i < MAX_PLAYERS; i++)

```

```

118         receiveMoney(game, i, 2000);

120         break;
121     }

123     case 10:
124         printf("Power Failure : Utility income halved for 3 rounds.\n");
125         addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3);
126         break;

128     case 11:
129         printf("Foreign Funding : Commercial property values increase by 15%% for 15 rounds.\n");
130         addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15);
131         break;

133     case 12:
134         printf("Port Expansion : Railway station values increase by 20%% for 15 rounds.\n");
135         addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15);
136         break;

138     case 13:
139         printf("Festival Season : Hotels receive 50%% additional rent for 5 rounds.\n");
140         addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5);
141         break;

143     case 14:
144         printf("Labour Strike : Construction suspended for 2 rounds.\n");
145         game[0].economy.constructionSuspendedRoundsLeft = 2;
146         break;

148     case 15:
149         printf("Insurance Discount : Premiums reduced by 20%% for 10 rounds.\n");
150         addModifier(game, MOD_INSURANCE, -1, -1, 80, 10);
151         break;

153     case 16:
154     {
155         PropertyGroup group;
156         char groupBuffer[32];

158         group = (PropertyGroup)(rand() % NO_GROUP);
159         groupName(group, groupBuffer);

161         printf("Property Revaluation : %s properties appreciate by 15%% for 15 rounds.\n",
162             groupBuffer);
163         addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15);
164         break;
165     }

167     case 17:
168         printf("Currency Depreciation : Construction costs increase by 10%% for 15 rounds.\n");
169         addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15);
170         break;

172     case 18:
173     {
174         int lucky;

176         /* Section 1.3: bankrupt players take no further part in the
177            game, so a grant must go to a solvent player. Pick a
178            random starting point and scan forward to the first
179            solvent player - at least one must exist, because a live
180            player drew this card. */
181         lucky = rand() % MAX_PLAYERS;

183         while(game[0].players[lucky].bankrupt)
184             lucky = (lucky + 1) % MAX_PLAYERS;

186         printf("Government Grant : %s receives LKR 5,000.\n",

```

```

187             game[0].players[lucky].name);

189             receiveMoney(game, lucky, 5000);
190             break;
191         }

193         case 19:
194             printf("National Disaster : a random developed property is damaged.\n");
195             triggerDisaster(game);
196             break;

198         default:
199             break;
200     }

202     game[0].economy.currentCardIndex = (game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT;
203 }

```

Line-by-line explanation

Line	Code	Explanation
22		Blank line.
23	/* Appendix A: 20 National Event Cards, drawn when a player lands on	Comment: /* Appendix A: 20 National Event Cards, drawn when a player lands on
24	an EVENT square. currentCardIndex cycles 0-19 and wraps around,	an EVENT square. currentCardIndex cycles 0-19 and wraps around,
25	which behaves the same as "draw the top card, put it at the	which behaves the same as "draw the top card, put it at the
26	bottom" without needing a real deck data structure.	bottom" without needing a real deck data structure. */
27	void executeEvent(GameState game[], int playerIndex)	void executeEvent(GameState game[], int playerIndex)
28	{	Opens a new block.
29	(void)playerIndex; /* not needed - these effects are global, not per-player */	(void)playerIndex; /* not needed - these effects are global, not per-player */
30		Blank line.
31	printf("\n*** NATIONAL EVENT CARD ***\n");	Prints the message "\n*** NATIONAL EVENT CARD ***\n".
32		Blank line.
33	switch(game[0].economy.currentCardIndex)	Chooses a branch based on the value of game[0].economy.currentCardIndex.
34	{	Opens a new block.
35	case 0:	case 0:
36	printf("Tourism Hype : Hotels earn double rent for 5 rounds.\n");	Prints the message "Tourism Hype : Hotels earn double rent for 5 rounds.\n".
37	addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5);	Calls addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
38	break;	Breaks out of the current loop or switch.
39		Blank line.
40	case 1:	case 1:
41	printf("Fuel Shortage : Railway rent doubles for 5 rounds.\n");	Prints the message "Fuel Shortage : Railway rent doubles for 5 rounds.\n".
42	addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5);	Calls addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
43	break;	Breaks out of the current loop or switch.
44		Blank line.
45	case 2:	case 2:
46	printf("Heavy Floods : a random coastal property is damaged.\n");	Prints the message "Heavy Floods : a random coastal property is damaged.\n".

47	triggerDisaster(game);	Calls triggerDisaster(game). (Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owners get compensation, uninsured owners are marked sufferedLoss and own the full repair cost)
48	break;	Breaks out of the current loop or switch.
49		Blank line.
50	case 3:	case 3:
51	{	Opens a new block.
52	int candidates[BOARD_SIZE];	Declares int candidates[BOARD_SIZE].
53	int count;	Declares int count.
54	int i;	Declares int i.
55		Blank line.
56	count = 0;	count = 0;
57		Blank line.
58	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
59	{	Opens a new block.
60	if(game[0].board[i].type == PROPERTY && game[0].board[i].property.owner != -1)	Condition: square i.type == PROPERTY && square i.property.owner != -1 -> run the next block if true.
61	{	Opens a new block.
62	candidates[count] = i;	candidates[count] = i;
63	count++;	count++;
64	}	Closes the current block.
65	}	Closes the current block.
66		Blank line.
67	if(count > 0)	Condition: count > 0 -> run the next block if true.
68	{	Opens a new block.
69	game[0].economy.closedPropertyIndex = candidates[rand() % count];	Sets the field closedPropertyIndex of the shared economy (i.e. Square closed by Political Rally (-1 none).) = candidates[rand() % count].
70	game[0].economy.closedPropertyRoundsLeft = 2;	Sets the field closedPropertyRoundsLeft of the shared economy (i.e. Rounds the closed property stays closed.) = 2.
71		Blank line.
72	printf("Political Rally : %s is closed for 2 rounds.\n",	Prints the message "Political Rally : %s is closed for 2 rounds.\n".
73	game[0].board[game[0].economy.closedPropertyIndex].name);	game[0].board[game[0].economy.closedPropertyIndex].name);
74	}	Closes the current block.
75	else	The alternative branch when the condition above is false.
76	{	Opens a new block.
77	printf("Political Rally : no owned property to close.\n");	Prints the message "Political Rally : no owned property to close.\n".
78	}	Closes the current block.
79	break;	Breaks out of the current loop or switch.
80	}	Closes the current block.
81		Blank line.
82	case 4:	case 4:
83	printf("Stock Market Rise : All property values increase by 10%% for 15 rounds.\n");	Prints the message "Stock Market Rise : All property values increase by 10%% for".
84	addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);	Calls addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
85	break;	Breaks out of the current loop or switch.
86		Blank line.
87	case 5:	case 5:
88	printf("Economic Downturn : Property values decrease by 15%% for 15 rounds.\n");	Prints the message "Economic Downturn : Property values decrease by 15%% for 15".
89	addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);	Calls addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))

90	break;	Breaks out of the current loop or switch.
91		Blank line.
92	case 6:	case 6:
93	printf("Housing Subsidy : House construction cost reduced by 30%% for 15 rounds.\n");	Prints the message "Housing Subsidy : House construction cost reduced by 30%% fo".
94	addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15);	Calls addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
95	break;	Breaks out of the current loop or switch.
96		Blank line.
97	case 7:	case 7:
98	adjustLoanInterest(game, -LOAN_INTEREST_STEP);	Calls adjustLoanInterest(game, -LOAN_INTEREST_STEP). (Moves the loan interest rate by deltaPercent and clamps it)
99		Blank line.
100	printf("Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.\n",	Prints the message "Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.".
101	LOAN_INTEREST_STEP,	LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
102	game[0].economy.loanInterestRate);	
103	break;	Breaks out of the current loop or switch.
104		Blank line.
105	case 8:	case 8:
106	adjustLoanInterest(game, LOAN_INTEREST_STEP);	Calls adjustLoanInterest(game, LOAN_INTEREST_STEP). (Moves the loan interest rate by deltaPercent and clamps it)
107		Blank line.
108	printf("Interest Rate Increase : Loan interest increased by %d%%. Now %d%%.\n",	Prints the message "Interest Rate Increase : Loan interest increased by %d%%. No".
109	LOAN_INTEREST_STEP,	LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
110	game[0].economy.loanInterestRate);	
111	break;	Breaks out of the current loop or switch.
112		Blank line.
113	case 9:	case 9:
114	{	Opens a new block.
115	int i;	Declares int i.
116		Blank line.
117	printf("Tax Amnesty : Each player receives LKR 2,000.\n");	Prints the message "Tax Amnesty : Each player receives LKR 2,000.\n".
118		Blank line.
119	for(i = 0; i < MAX_PLAYERS; i++)	Loop over all entries: init i = 0; run while i < MAX_PLAYERS; every loop i++.
120	receiveMoney(game, i, 2000);	Calls receiveMoney(game, i, 2000). (Adds money to a player, but only if they are not bankrupt)
121		Blank line.
122	break;	Breaks out of the current loop or switch.
123	}	Closes the current block.
124		Blank line.
125	case 10:	case 10:
126	printf("Power Failure : Utility income halved for 3 rounds.\n");	Prints the message "Power Failure : Utility income halved for 3 rounds.\n".
127	addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3);	Calls addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
128		Blank line.
129	break;	Breaks out of the current loop or switch.
130		Blank line.
131	case 11:	case 11:
132	printf("Foreign Funding : Commercial property values increase by 15%% for 15 rounds.\n");	Prints the message "Foreign Funding : Commercial property values increase by 15%".

130	<code>addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15);</code>	Calls <code>addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
131	<code>break;</code>	Breaks out of the current loop or switch.
132		Blank line.
133	<code>case 12:</code>	<code>case 12:</code>
134	<code>printf("Port Expansion : Railway station values increase by 20%% for 15 rounds.\n");</code>	Prints the message "Port Expansion : Railway station values increase by 20%% for".
135	<code>addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15);</code>	Calls <code>addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
136	<code>break;</code>	Breaks out of the current loop or switch.
137		Blank line.
138	<code>case 13:</code>	<code>case 13:</code>
139	<code>printf("Festival Season : Hotels receive 50%% additional rent for 5 rounds.\n");</code>	Prints the message "Festival Season : Hotels receive 50%% additional rent for 5".
140	<code>addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5);</code>	Calls <code>addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
141	<code>break;</code>	Breaks out of the current loop or switch.
142		Blank line.
143	<code>case 14:</code>	<code>case 14:</code>
144	<code>printf("Labour Strike : Construction suspended for 2 rounds.\n");</code>	Prints the message "Labour Strike : Construction suspended for 2 rounds.\n".
145	<code>game[0].economy.constructionSuspendedRoundsLeft = 2;</code>	Sets the field <code>constructionSuspendedRoundsLeft</code> of the shared economy (i.e. Labour Strike blocks building while <code>> 0</code> .) = 2.
146	<code>break;</code>	Breaks out of the current loop or switch.
147		Blank line.
148	<code>case 15:</code>	<code>case 15:</code>
149	<code>printf("Insurance Discount : Premiums reduced by 20%% for 10 rounds.\n");</code>	Prints the message "Insurance Discount : Premiums reduced by 20%% for 10 rounds.".
150	<code>addModifier(game, MOD_INSURANCE, -1, -1, 80, 10);</code>	Calls <code>addModifier(game, MOD_INSURANCE, -1, -1, 80, 10)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
151	<code>break;</code>	Breaks out of the current loop or switch.
152		Blank line.
153	<code>case 16:</code>	<code>case 16:</code>
154	<code>{</code>	Opens a new block.
155	<code>PropertyGroup group;</code>	<code>PropertyGroup group;</code>
156	<code>char groupBuffer[32];</code>	Declares <code>char groupBuffer[32]</code> .
157		Blank line.
158	<code>group = (PropertyGroup)(rand() % NO_GROUP);</code>	<code>group = (PropertyGroup)(rand() % NO_GROUP);</code>
159	<code>groupName(group, groupBuffer);</code>	Calls <code>groupName(group, groupBuffer)</code> . (Writes the display name of a colour group)
160		Blank line.
161	<code>printf("Property Revaluation : %s properties appreciate by 15%% for 15 rounds.\n",</code> <code>groupBuffer);</code>	Prints the message "Property Revaluation : %s properties appreciate by 15%% for". <code>groupBuffer);</code>
162		
163	<code>addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15);</code>	Calls <code>addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
164	<code>break;</code>	Breaks out of the current loop or switch.
165	<code>}</code>	Closes the current block.
166		Blank line.
167	<code>case 17:</code>	<code>case 17:</code>
168	<code>printf("Currency Depreciation : Construction costs increase by 10%% for 15 rounds.\n");</code>	Prints the message "Currency Depreciation : Construction costs increase by 10%%".

169	<code>addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15);</code>	Calls <code>addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
170	<code>break;</code>	Breaks out of the current loop or switch.
171		Blank line.
172	<code>case 18:</code>	case 18:
173	<code>{</code>	Opens a new block.
174	<code>int lucky;</code>	Declares <code>int lucky</code> .
175		Blank line.
176	<code>/* Section 1.3: bankrupt players take no further part in the</code>	Comment: <code>/* Section 1.3: bankrupt players take no further part in the</code>
177	<code>game, so a grant must go to a solvent player. Pick a</code>	<code>game, so a grant must go to a solvent player. Pick a</code>
178	<code>random starting point and scan forward to the first</code>	<code>random starting point and scan forward to the first</code>
179	<code>solvent player - at least one must exist, because a live</code>	<code>solvent player - at least one must exist, because a live</code>
180	<code>player drew this card. */</code>	<code>player drew this card. */</code>
181	<code>lucky = rand() % MAX_PLAYERS;</code>	<code>lucky = rand() % MAX_PLAYERS;</code>
182		Blank line.
183	<code>while(game[0].players[lucky].bankrupt)</code>	Loop: repeat while <code>game[0].players[lucky].bankrupt</code> is true.
184	<code>lucky = (lucky + 1) % MAX_PLAYERS;</code>	<code>lucky = (lucky + 1) % MAX_PLAYERS;</code>
185		Blank line.
186	<code>printf("Government Grant : %s receives LKR 5,000.\n",</code>	Prints the message "Government Grant : %s receives LKR 5,000.\n".
187	<code>game[0].players[lucky].name);</code>	<code>game[0].players[lucky].name);</code>
188		Blank line.
189	<code>receiveMoney(game, lucky, 5000);</code>	Calls <code>receiveMoney(game, lucky, 5000)</code> . (Adds money to a player, but only if they are not bankrupt)
190	<code>break;</code>	Breaks out of the current loop or switch.
191	<code>}</code>	Closes the current block.
192		Blank line.
193	<code>case 19:</code>	case 19:
194	<code>printf("National Disaster : a random developed property is damaged.\n");</code>	Prints the message "National Disaster : a random developed property is damaged.\n".
195	<code>triggerDisaster(game);</code>	Calls <code>triggerDisaster(game)</code> . (Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owners get compensation, uninsured owners are marked sufferedLoss and own the full repair cost)
196	<code>break;</code>	Breaks out of the current loop or switch.
197		Blank line.
198	<code>default:</code>	The fallback switch branch when no case matched.
199	<code>break;</code>	Breaks out of the current loop or switch.
200	<code>}</code>	Closes the current block.
201		Blank line.
202	<code>game[0].economy.currentCardIndex = (game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT;</code>	Sets the field <code>currentCardIndex</code> of the shared economy (i.e. Position in the National Event Card deck (0..19).) = <code>(game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT</code> .
203	<code>}</code>	Closes the current block.

Nested functions inside `executeEvent()` - study these

- `executeEvent()` (Draws the current National Event Card (0..19, cycling like draw-top-put-bottom). Effects a)
- `addModifier()` (`addSourcedModifier` with `SRC_GENERAL` (default for events/cards/regulations).)
- `triggerDisaster()` (Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owne)
- `adjustLoanInterest()` (Moves the loan interest rate by `deltaPercent` and clamps it.)
- `receiveMoney()` (Adds money to a player, but only if they are not bankrupt.)
- `groupName()` (Writes the display name of a colour group.)

MAIN PURPOSE

Draws the current National Event Card (0..19, cycling like draw-top-put-bottom). Effects are global, so playerIndex is unused.

9.4 triggerEconomicEvent()

```
void triggerEconomicEvent(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 204)

```
205  /* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */
206  void triggerEconomicEvent(GameState game[])
207  {
208      int choice;

210      choice = rand() % ECONOMIC_EVENT_COUNT;

212      printf("\n=== Economic Event ===\n");

214      switch(choice)
215      {
216          case 0:
217              printf("Tourism Boom\n");
218              printf("Hotels receive double rent for 15 rounds.\n");
219              printf("Southern Province properties increase in value by 15%.\n");

221              addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15);
222              addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15);
223              break;

225          case 1:
226              printf("Fuel Crisis\n");
227              printf("Railway rent doubles for 15 rounds.\n");
228              printf("Property development costs increase 20% for 15 rounds.\n");

230              addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15);
231              addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15);
232              break;

234          case 2:
235              printf("Heavy Monsoon\n");
236              printf("Insurance premiums increase for 15 rounds.\n");
237              printf("Coastal properties lose 10% value.\n");

239              addModifier(game, MOD_INSURANCE, -1, -1, 115, 15);
240              addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15);
241              addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15);
242              break;

244          case 3:
245              printf("Economic Recession\n");
246              printf("Property values decrease 15%. Rent decreases 10%.\n");
247              printf("Loan interest increases by 15%.\n");

249              addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
250              addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15);
251              addModifier(game, MOD_RECESSION, -1, -1, 100, 15);

253              game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, 15);
254              clampLoanInterest(game);
255              break;

257          case 4:
258              printf("Stock Market Boom\n");
259              printf("Property values increase 10%. Loan interest decreases 10%.\n");

261              addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
```

```

263         game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, -10);
264         clampLoanInterest(game);
265         break;

267     case 5:
268         printf("Government Housing Programme\n");
269         printf("House construction costs reduce 25%%.\n");

271         addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15);
272         break;

274     case 6:
275         printf("Foreign Investment\n");
276         printf("Commercial properties increase 20%%.\n");

278         addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15);
279         break;

281     case 7:
282         printf("Political Unrest\n");
283         printf("Hotel rent drops by 50%% for 15 rounds.\n");

285         addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15);
286         addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15);
287         addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15);
288         break;

290     default:
291         break;
292 }
293 }

```

Line-by-line explanation

Line	Code	Explanation
204		Blank line.
205	/* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */	Comment: /* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */
206	void triggerEconomicEvent(GameState game[])	void triggerEconomicEvent(GameState game[])
207	{	Opens a new block.
208	int choice;	Declares int choice.
209		Blank line.
210	choice = rand() % ECONOMIC_EVENT_COUNT;	choice = rand() % ECONOMIC_EVENT_COUNT;
211		Blank line.
212	printf("\n=== Economic Event ===\n");	Prints the message "\n=== Economic Event ===\n".
213		Blank line.
214	switch(choice)	Chooses a branch based on the value of choice.
215	{	Opens a new block.
216	case 0:	case 0:
217	printf("Tourism Boom\n");	Prints the message "Tourism Boom\n".
218	printf("Hotels receive double rent for 15 rounds.\n");	Prints the message "Hotels receive double rent for 15 rounds.\n".
219	printf("Southern Province properties increase in value by 15%%.\n");	Prints the message "Southern Province properties increase in value by 15%%.\n".
220		Blank line.
221	addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15);	Calls addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
222	addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15);	Calls addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
223	break;	Breaks out of the current loop or switch.
224		Blank line.

225	case 1:	case 1:
226	printf("Fuel Crisis\n");	Prints the message "Fuel Crisis\n".
227	printf("Railway rent doubles for 15 rounds.\n");	Prints the message "Railway rent doubles for 15 rounds.\n".
228	printf("Property development costs increase 20%% for 15 rounds.\n");	Prints the message "Property development costs increase 20%% for 15 rounds.\n".
229		Blank line.
230	addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15);	Calls addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
231	addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15);	Calls addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
232	break;	Breaks out of the current loop or switch.
233		Blank line.
234	case 2:	case 2:
235	printf("Heavy Monsoon\n");	Prints the message "Heavy Monsoon\n".
236	printf("Insurance premiums increase for 15 rounds.\n");	Prints the message "Insurance premiums increase for 15 rounds.\n".
237	printf("Coastal properties lose 10%% value.\n");	Prints the message "Coastal properties lose 10%% value.\n".
238		Blank line.
239	addModifier(game, MOD_INSURANCE, -1, -1, 115, 15);	Calls addModifier(game, MOD_INSURANCE, -1, -1, 115, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
240	addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15);	Calls addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
241	addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15);	Calls addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
242	break;	Breaks out of the current loop or switch.
243		Blank line.
244	case 3:	case 3:
245	printf("Economic Recession\n");	Prints the message "Economic Recession\n".
246	printf("Property values decrease 15%%. Rent decreases 10%%.\n");	Prints the message "Property values decrease 15%%. Rent decreases 10%%.\n".
247	printf("Loan interest increases by 15%%.\n");	Prints the message "Loan interest increases by 15%%.\n".
248		Blank line.
249	addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);	Calls addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
250	addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15);	Calls addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
251	addModifier(game, MOD RECESSION, -1, -1, 100, 15);	Calls addModifier(game, MOD RECESSION, -1, -1, 100, 15). (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations))
252		Blank line.
253	game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, 15);	Sets the field loanInterestRate of the shared economy (i.e. Current economy loan interest rate (1..25).) = applyRate(game[0].economy.loanInterestRate, 15).
254	clampLoanInterest(game);	Calls clampLoanInterest(game). (Keeps the loan interest rate inside [1, 25])
255	break;	Breaks out of the current loop or switch.
256		Blank line.
257	case 4:	case 4:
258	printf("Stock Market Boom\n");	Prints the message "Stock Market Boom\n".
259	printf("Property values increase 10%%. Loan interest decreases 10%%.\n");	Prints the message "Property values increase 10%%. Loan interest decreases 10%%."

260		Blank line.
261	<code>addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);</code>	Calls <code>addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
262		Blank line.
263	<code>game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, -10);</code>	Sets the field <code>loanInterestRate</code> of the shared economy (i.e. Current economy loan interest rate (1..25).) = <code>applyRate(game[0].economy.loanInterestRate, -10)</code> .
264	<code>clampLoanInterest(game);</code>	Calls <code>clampLoanInterest(game)</code> . (Keeps the loan interest rate inside [1, 25])
265	<code>break;</code>	Breaks out of the current loop or switch.
266		Blank line.
267	<code>case 5:</code>	<code>case 5:</code>
268	<code>printf("Government Housing Programme\n");</code>	Prints the message "Government Housing Programme\n".
269	<code>printf("House construction costs reduce 25%%.\n");</code>	Prints the message "House construction costs reduce 25%%.\n".
270		Blank line.
271	<code>addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15);</code>	Calls <code>addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
272	<code>break;</code>	Breaks out of the current loop or switch.
273		Blank line.
274	<code>case 6:</code>	<code>case 6:</code>
275	<code>printf("Foreign Investment\n");</code>	Prints the message "Foreign Investment\n".
276	<code>printf("Commercial properties increase 20%%.\n");</code>	Prints the message "Commercial properties increase 20%%.\n".
277		Blank line.
278	<code>addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15);</code>	Calls <code>addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
279	<code>break;</code>	Breaks out of the current loop or switch.
280		Blank line.
281	<code>case 7:</code>	<code>case 7:</code>
282	<code>printf("Political Unrest\n");</code>	Prints the message "Political Unrest\n".
283	<code>printf("Hotel rent drops by 50%% for 15 rounds.\n");</code>	Prints the message "Hotel rent drops by 50%% for 15 rounds.\n".
284		Blank line.
285	<code>addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15);</code>	Calls <code>addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
286	<code>addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15);</code>	Calls <code>addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
287	<code>addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15);</code>	Calls <code>addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
288	<code>break;</code>	Breaks out of the current loop or switch.
289		Blank line.
290	<code>default:</code>	The fallback switch branch when no case matched.
291	<code>break;</code>	Breaks out of the current loop or switch.
292	<code>}</code>	Closes the current block.
293	<code>}</code>	Closes the current block.

Nested functions inside `triggerEconomicEvent()` - study these

- `triggerEconomicEvent()` (One of 8 random economy-wide events every 15 rounds (Tourism Boom, Fuel Crisis, Heavy Mons))
- `addModifier()` (`addSourcedModifier` with `SRC_GENERAL` (default for events/cards/regulations).)
- `applyRate()` (Rule-LK 14: applies a percentage change, never returning below 1.)
- `clampLoanInterest()` (Keeps the loan interest rate inside [1, 25].)

MAIN PURPOSE

One of 8 random economy-wide events every 15 rounds (Tourism Boom, Fuel Crisis, Heavy Monsoon, Economic Recession, Stock Market Boom, Government Housing, Foreign Investment, Political Unrest).

9.5 triggerGovernmentRegulation()

```
void triggerGovernmentRegulation(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 294)

```
295  /* Section 2.7: one of 8 regulations, every 20 rounds. */
296  void triggerGovernmentRegulation(GameState game[])
297  {
298      int choice;

300      choice = rand() % REGULATION_COUNT;

302      printf("\n=== Government Regulation ===\n");

304      switch(choice)
305      {
306          case 0:
307              game[0].economy.incomeTaxRate = applyRate(game[0].economy.incomeTaxRate, 50);

309              if(game[0].economy.incomeTaxRate > TAX_RATE_MAX)
310                  game[0].economy.incomeTaxRate = TAX_RATE_MAX;

312              printf("Increase Property Tax\n");
313              printf("Income Tax rate increased by 50%%. Now %d%%.\n", game[0].economy.incomeTaxRate);
314              break;

316          case 1:
317              adjustLoanInterest(game, -LOAN_INTEREST_STEP);

319              printf("Reduce Loan Interest\n");
320              printf("Interest decreased by %d%%. Now %d%%.\n",
321                  LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
322              break;

324          case 2:
325              printf("Housing Subsidy\n");
326              printf("House construction costs reduced by 30%% for 20 rounds.\n");

328              addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20);
329              break;

331          case 3:
332          {
333              int i;
334              int tax;

336              printf("Luxury Property Tax\n");
337              printf("Hotels pay a maintenance tax of 25%% of their value.\n");

339              for(i = 0; i < BOARD_SIZE; i++)
340              {
341                  if(game[0].board[i].type == PROPERTY && game[0].board[i].property.hotel &&
342                     game[0].board[i].property.owner != -1)
343                  {
344                      tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;

346                      printf("%s pays LKR %d luxury tax on %s.\n",
347                          game[0].players[game[0].board[i].property.owner].name,
348                          tax, game[0].board[i].name);

350                      payMoney(game, game[0].board[i].property.owner, tax);
```

```

351         }
352     }
353     break;
354 }

356     case 4:
357         printf("Railway Modernization\n");
358         printf("Railway rents increase 25%% for 20 rounds.\n");

360         addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20);
361         break;

363     case 5:
364         printf("Electricity Tariff Revision\n");
365         printf("Utility rents increase 20%% for 20 rounds.\n");

367         addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20);
368         break;

370     case 6:
371         printf("Insurance Regulation\n");
372         printf("Insurance premiums decrease 15%% for 20 rounds. Coverage unchanged.\n");

374         addModifier(game, MOD_INSURANCE, -1, -1, 85, 20);
375         break;

377     case 7:
378         printf("Anti-Speculation Act\n");
379         printf("Players may now own at most three undeveloped properties.\n");

381         game[0].economy.antiSpeculationActive = 1;
382         break;

384     default:
385         break;
386 }
387 }

```

Line-by-line explanation

Line	Code	Explanation
294		Blank line.
295	<code>/* Section 2.7: one of 8 regulations, every 20 rounds. */</code>	Comment: <code>/* Section 2.7: one of 8 regulations, every 20 rounds. */</code>
296	<code>void triggerGovernmentRegulation(GameState game[])</code>	<code>void triggerGovernmentRegulation(GameState game[])</code>
297	<code>{</code>	Opens a new block.
298	<code>int choice;</code>	Declares <code>int choice</code> .
299		Blank line.
300	<code>choice = rand() % REGULATION_COUNT;</code>	<code>choice = rand() % REGULATION_COUNT;</code>
301		Blank line.
302	<code>printf("\n=== Government Regulation ===\n");</code>	Prints the message <code>"\n=== Government Regulation ===\n"</code> .
303		Blank line.
304	<code>switch(choice)</code>	Chooses a branch based on the value of <code>choice</code> .
305	<code>{</code>	Opens a new block.
306	<code>case 0:</code>	<code>case 0:</code>
307	<code>game[0].economy.incomeTaxRate = applyRate(game[0].economy.incomeTaxRate, 50);</code>	Sets the field <code>incomeTaxRate</code> of the shared economy (i.e. Current income tax rate (base 15, can rise via regulation).) = <code>applyRate(game[0].economy.incomeTaxRate, 50)</code> .
308		Blank line.
309	<code>if(game[0].economy.incomeTaxRate > TAX_RATE_MAX)</code>	Condition: <code>game[0].economy.incomeTaxRate > TAX_RATE_MAX</code> -> run the next block if true.
310	<code>game[0].economy.incomeTaxRate = TAX_RATE_MAX;</code>	Sets the field <code>incomeTaxRate</code> of the shared economy (i.e. Current income tax rate (base 15, can rise via regulation).) = <code>TAX_RATE_MAX</code> .
311		Blank line.
312	<code>printf("Increase Property Tax\n");</code>	Prints the message <code>"Increase Property Tax\n"</code> .

313	<code>printf("Income Tax rate increased by 50%%. Now %d%%.\n", game[0].economy.incomeTaxRate);</code>	Prints the message "Income Tax rate increased by 50%%. Now %d%%.\n".
314	<code>break;</code>	Breaks out of the current loop or switch.
315		Blank line.
316	<code>case 1:</code>	case 1:
317	<code>adjustLoanInterest(game, -LOAN_INTEREST_STEP);</code>	Calls <code>adjustLoanInterest(game, -LOAN_INTEREST_STEP)</code> . (Moves the loan interest rate by <code>deltaPercent</code> and clamps it)
318		Blank line.
319	<code>printf("Reduce Loan Interest\n");</code>	Prints the message "Reduce Loan Interest\n".
320	<code>printf("Interest decreased by %d%%. Now %d%%.\n",</code>	Prints the message "Interest decreased by %d%%. Now %d%%.\n".
321	<code>LOAN_INTEREST_STEP,</code>	LOAN_INTEREST_STEP, <code>game[0].economy.loanInterestRate</code>);
	<code>game[0].economy.loanInterestRate);</code>	
322	<code>break;</code>	Breaks out of the current loop or switch.
323		Blank line.
324	<code>case 2:</code>	case 2:
325	<code>printf("Housing Subsidy\n");</code>	Prints the message "Housing Subsidy\n".
326	<code>printf("House construction costs reduced by 30%% for 20 rounds.\n");</code>	Prints the message "House construction costs reduced by 30%% for 20 rounds.\n".
327		Blank line.
328	<code>addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20);</code>	Calls <code>addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
329	<code>break;</code>	Breaks out of the current loop or switch.
330		Blank line.
331	<code>case 3:</code>	case 3:
332	<code>{</code>	Opens a new block.
333	<code>int i;</code>	Declares <code>int i</code> .
334	<code>int tax;</code>	Declares <code>int tax</code> .
335		Blank line.
336	<code>printf("Luxury Property Tax\n");</code>	Prints the message "Luxury Property Tax\n".
337	<code>printf("Hotels pay a maintenance tax of 25%% of their value.\n");</code>	Prints the message "Hotels pay a maintenance tax of 25%% of their value.\n".
338		Blank line.
339	<code>for(i = 0; i < BOARD_SIZE; i++)</code>	Loop over all entries: <code>init i = 0</code> ; run while <code>i < BOARD_SIZE</code> ; every loop <code>i++</code> .
340	<code>{</code>	Opens a new block.
341	<code>if(game[0].board[i].type == PROPERTY &&</code>	Condition: -> run the next block if true.
	<code>game[0].board[i].property.hotel &&</code>	
342	<code>game[0].board[i].property.owner != -1)</code>	<code>game[0].board[i].property.owner != -1)</code>
343	<code>{</code>	Opens a new block.
344	<code>tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;</code>	<code>tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;</code>
345		Blank line.
346	<code>printf("%s pays LKR %d luxury tax on %s.\n",</code>	Prints the message "%s pays LKR %d luxury tax on %s.\n".
347	<code>game[0].players[game[0].board[i].property.owner].name,</code>	<code>game[0].players[game[0].board[i].property.owner].name,</code>
	<code>e,</code>	
348	<code>tax, game[0].board[i].name);</code>	<code>tax, game[0].board[i].name);</code>
349		Blank line.
350	<code>payMoney(game, game[0].board[i].property.owner,</code>	Calls <code>payMoney(game, game[0].board[i].property.owner, tax)</code> . (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
	<code>tax);</code>	
351	<code>}</code>	Closes the current block.
352	<code>}</code>	Closes the current block.
353	<code>break;</code>	Breaks out of the current loop or switch.
354	<code>}</code>	Closes the current block.
355		Blank line.
356	<code>case 4:</code>	case 4:

357	<code>printf("Railway Modernization\n");</code>	Prints the message "Railway Modernization\n".
358	<code>printf("Railway rents increase 25%% for 20 rounds.\n");</code>	Prints the message "Railway rents increase 25%% for 20 rounds.\n".
359		Blank line.
360	<code>addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20);</code>	Calls <code>addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
361	<code>break;</code>	Breaks out of the current loop or switch.
362		Blank line.
363	<code>case 5:</code>	case 5:
364	<code>printf("Electricity Tariff Revision\n");</code>	Prints the message "Electricity Tariff Revision\n".
365	<code>printf("Utility rents increase 20%% for 20 rounds.\n");</code>	Prints the message "Utility rents increase 20%% for 20 rounds.\n".
366		Blank line.
367	<code>addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20);</code>	Calls <code>addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
368	<code>break;</code>	Breaks out of the current loop or switch.
369		Blank line.
370	<code>case 6:</code>	case 6:
371	<code>printf("Insurance Regulation\n");</code>	Prints the message "Insurance Regulation\n".
372	<code>printf("Insurance premiums decrease 15%% for 20 rounds. Coverage unchanged.\n");</code>	Prints the message "Insurance premiums decrease 15%% for 20 rounds. Coverage unc".
373		Blank line.
374	<code>addModifier(game, MOD_INSURANCE, -1, -1, 85, 20);</code>	Calls <code>addModifier(game, MOD_INSURANCE, -1, -1, 85, 20)</code> . (<code>addSourcedModifier</code> with <code>SRC_GENERAL</code> (default for events/cards/regulations))
375	<code>break;</code>	Breaks out of the current loop or switch.
376		Blank line.
377	<code>case 7:</code>	case 7:
378	<code>printf("Anti-Speculation Act\n");</code>	Prints the message "Anti-Speculation Act\n".
379	<code>printf("Players may now own at most three undeveloped properties.\n");</code>	Prints the message "Players may now own at most three undeveloped properties.\n".
380		Blank line.
381	<code>game[0].economy.antiSpeculationActive = 1;</code>	Sets the field <code>antiSpeculationActive</code> of the shared economy (i.e. 1 once the Anti-Speculation regulation fires.) = 1.
382	<code>break;</code>	Breaks out of the current loop or switch.
383		Blank line.
384	<code>default:</code>	The fallback switch branch when no case matched.
385	<code>break;</code>	Breaks out of the current loop or switch.
386	<code>}</code>	Closes the current block.
387	<code>}</code>	Closes the current block.

Nested functions inside `triggerGovernmentRegulation()` - study these

- `triggerGovernmentRegulation()` (One of 8 random government regulations every 20 rounds (Property Tax raise, Loan Interest)
- `applyRate()` (Rule-LK 14: applies a percentage change, never returning below 1.)
- `adjustLoanInterest()` (Moves the loan interest rate by `deltaPercent` and clamps it.)
- `addModifier()` (`addSourcedModifier` with `SRC_GENERAL` (default for events/cards/regulations).)
- `currentMarketValue()` (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- `payMoney()` (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

One of 8 random government regulations every 20 rounds (Property Tax raise, Loan Interest cut, Housing Subsidy, Luxury Property Tax on hotels, Railway Modernization, Electricity Tariff, Insurance Regulation, Anti-Speculation Act).

9.6 decrementEventTimers()

```
void decrementEventTimers(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 388)

```
389  /* Decrements the non-modifier event timers - the modifier list itself
390     is ticked down separately by decrementModifiers(). */
391  void decrementEventTimers(GameState game[])
392  {
393      if(game[0].economy.constructionSuspendedRoundsLeft > 0)
394          game[0].economy.constructionSuspendedRoundsLeft--;
395
396      if(game[0].economy.closedPropertyRoundsLeft > 0)
397      {
398          game[0].economy.closedPropertyRoundsLeft--;
399          if(game[0].economy.closedPropertyRoundsLeft == 0)
400              game[0].economy.closedPropertyIndex = -1;
401      }
402  }
```

Line-by-line explanation

Line	Code	Explanation
388		Blank line.
389	/* Decrements the non-modifier event timers - the modifier list itself	Comment: /* Decrements the non-modifier event timers - the modifier list itself
390	is ticked down separately by decrementModifiers(). */	is ticked down separately by decrementModifiers(). */
391	void decrementEventTimers(GameState game[])	void decrementEventTimers(GameState game[])
392	{	Opens a new block.
393	if(game[0].economy.constructionSuspendedRoundsLeft > 0)	Condition: game[0].economy.constructionSuspendedRoundsLeft > 0 -> run the next block if true.
394	game[0].economy.constructionSuspendedRoundsLeft--;	game[0].economy.constructionSuspendedRoundsLeft--;
395		Blank line.
396	if(game[0].economy.closedPropertyRoundsLeft > 0)	Condition: game[0].economy.closedPropertyRoundsLeft > 0 -> run the next block if true.
397	{	Opens a new block.
398	game[0].economy.closedPropertyRoundsLeft--;	game[0].economy.closedPropertyRoundsLeft--;
399	if(game[0].economy.closedPropertyRoundsLeft == 0)	Condition: game[0].economy.closedPropertyRoundsLeft == 0 -> run the next block if true.
400	game[0].economy.closedPropertyIndex = -1;	Sets the field closedPropertyIndex of the shared economy (i.e. Square closed by Political Rally (-1 none).) = -1.
401	}	Closes the current block.
402	}	Closes the current block.

Nested functions inside decrementEventTimers() - study these

- decrementModifiers() (Ticks every active modifier down and compacts out expired ones.)
- decrementEventTimers() (Ticks the non-modifier event timers down: construction suspension and the closed property.)

MAIN PURPOSE

Ticks the non-modifier event timers down: construction suspension and the closed property.

Inflation, the timed-modifier economy, ageing/depreciation, maintenance, repairs and the shared value math.

The sub-topics below are the 17 functions implemented in economy.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in economy.c

10.1	initEconomy()	(source line 7)	p. 173
10.2	applyRate()	(source line 25)	p. 174
10.3	addSourcedModifier()	(source line 38)	p. 175
10.4	addModifier()	(source line 58)	p. 176
10.5	modifierMultiplier()	(source line 66)	p. 176
10.6	isModifierActive()	(source line 95)	p. 178
10.7	currentTaxRatePercent()	(source line 116)	p. 179
10.8	decrementModifiers()	(source line 125)	p. 180
10.9	formatLKR()	(source line 152)	p. 181
10.10	applyInflation()	(source line 188)	p. 182
10.11	currentMarketValue()	(source line 235)	p. 184
10.12	ageProperties()	(source line 269)	p. 186
10.13	tryRenovateAgeDepreciation()	(source line 309)	p. 188
10.14	rentConditionPercent()	(source line 342)	p. 189
10.15	ageBuildings()	(source line 357)	p. 190
10.16	performMaintenance()	(source line 411)	p. 192
10.17	renovateStructuralDamage()	(source line 460)	p. 195

10.1 initEconomy()

```
void initEconomy(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 7)

```

7   void initEconomy(GameState game[])
8   {
9       game[0].economy.inflationRate = 0;
10      game[0].economy.loanInterestRate = LOAN_INTEREST_RATE;

12      game[0].economy.constructionSuspendedRoundsLeft = 0;

14      game[0].economy.closedPropertyIndex = -1;
15      game[0].economy.closedPropertyRoundsLeft = 0;

17      game[0].economy.incomeTaxRate = INCOME_TAX_RATE;

19      game[0].economy.antiSpeculationActive = 0;

21      game[0].economy.currentCardIndex = 0;

23      game[0].economy.modifierCount = 0;
24  }
```

Line-by-line explanation

Line	Code	Explanation
7	void initEconomy(GameState game[])	void initEconomy(GameState game[])
8	{	Opens a new block.
9	game[0].economy.inflationRate = 0;	Sets the field inflationRate of the shared economy (i.e. Current inflation percent (-3..12) applied every 10 rounds.) = 0.
10	game[0].economy.loanInterestRate = LOAN_INTEREST_RATE;	Sets the field loanInterestRate of the shared economy (i.e. Current economy loan interest rate (1..25).) = LOAN_INTEREST_RATE.
11		Blank line.

12	<code>game[0].economy.constructionSuspendedRoundsLeft = 0;</code>	Sets the field <code>constructionSuspendedRoundsLeft</code> of the shared economy (i.e. Labour Strike blocks building while > 0 .) = 0.
13		Blank line.
14	<code>game[0].economy.closedPropertyIndex = -1;</code>	Sets the field <code>closedPropertyIndex</code> of the shared economy (i.e. Square closed by Political Rally (-1 none).) = -1.
15	<code>game[0].economy.closedPropertyRoundsLeft = 0;</code>	Sets the field <code>closedPropertyRoundsLeft</code> of the shared economy (i.e. Rounds the closed property stays closed.) = 0.
16		Blank line.
17	<code>game[0].economy.incomeTaxRate = INCOME_TAX_RATE;</code>	Sets the field <code>incomeTaxRate</code> of the shared economy (i.e. Current income tax rate (base 15, can rise via regulation).) = <code>INCOME_TAX_RATE</code> .
18		Blank line.
19	<code>game[0].economy.antiSpeculationActive = 0;</code>	Sets the field <code>antiSpeculationActive</code> of the shared economy (i.e. 1 once the Anti-Speculation regulation fires.) = 0.
20		Blank line.
21	<code>game[0].economy.currentCardIndex = 0;</code>	Sets the field <code>currentCardIndex</code> of the shared economy (i.e. Position in the National Event Card deck (0..19).) = 0.
22		Blank line.
23	<code>game[0].economy.modifierCount = 0;</code>	Sets the field <code>modifierCount</code> of the shared economy (i.e. How many modifiers are currently active.) = 0.
24	<code>}</code>	Closes the current block.

Nested functions inside `initEconomy()` - study these

- `initEconomy()` (Resets all economy state: zero inflation, base rates, empty modifier list.)

MAIN PURPOSE

Resets all economy state: zero inflation, base rates, empty modifier list.

10.2 `applyRate()`

```
int applyRate(int oldValue, int ratePercent)
```

Parameters: *oldValue, ratePercent* | Returns: *New value (≥ 1).*

The function, verbatim from the source (starts at line 25)

```

26  /* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */
27  int applyRate(int oldValue, int ratePercent)
28  {
29      int newValue;

31      newValue = oldValue + (oldValue * ratePercent) / 100;

33      if(newValue < 1)
34          newValue = 1;

36      return newValue;
37  }
```

Line-by-line explanation

Line	Code	Explanation
25		Blank line.
26	<code>/* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */</code>	Comment: <code>/* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */</code>
27	<code>int applyRate(int oldValue, int ratePercent)</code>	<code>int applyRate(int oldValue, int ratePercent)</code>
28	<code>{</code>	Opens a new block.
29	<code>int newValue;</code>	Declares <code>int newValue</code> .
30		Blank line.
31	<code>newValue = oldValue + (oldValue * ratePercent) / 100;</code>	<code>newValue = oldValue + (oldValue * ratePercent) / 100;</code>
32		Blank line.
33	<code>if(newValue < 1)</code>	Condition: <code>newValue < 1</code> -> run the next block if true.
34	<code>newValue = 1;</code>	<code>newValue = 1;</code>

35		Blank line.
36	return newValue;	Returns newValue immediately.
37	}	Closes the current block.

Nested functions inside applyRate() - study these

- applyRate() (Rule-LK 14: applies a percentage change, never returning below 1.)

MAIN PURPOSE

Rule-LK 14: applies a percentage change, never returning below 1.

10.3 addSourcedModifier()

```
void addSourcedModifier(GameState game[], ModifierType type, int group, int index, int
percent, int roundsLeft, ModifierSource source)
```

Parameters: type, group, index, percent, roundsLeft, source | Returns: void.

The function, verbatim from the source (starts at line 38)

```
39  /* Adds a timed modifier (Rule-LK 30-35).  group/index are -1 when not
40     relevant.  Effects accumulate: two active +25% rents give
41     roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */
42  void addSourcedModifier(GameState game[], ModifierType type, int group,
43                          int index, int percent, int roundsLeft,
44                          ModifierSource source)
45  {
46      if(game[0].economy.modifierCount >= MAX_MODIFIERS)
47          return;

49      game[0].economy.modifiers[game[0].economy.modifierCount].type = type;
50      game[0].economy.modifiers[game[0].economy.modifierCount].group = group;
51      game[0].economy.modifiers[game[0].economy.modifierCount].index = index;
52      game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent;
53      game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft;
54      game[0].economy.modifiers[game[0].economy.modifierCount].source = source;

56      game[0].economy.modifierCount++;
57  }
```

Line-by-line explanation

Line	Code	Explanation
38		Blank line.
39	/* Adds a timed modifier (Rule-LK 30-35). group/index are -1 when not	Comment: /* Adds a timed modifier (Rule-LK 30-35). group/index are -1 when not
40	relevant. Effects accumulate: two active +25% rents give	relevant. Effects accumulate: two active +25% rents give
41	roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */	roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */
42	void addSourcedModifier(GameState game[], ModifierType type, int group,	void addSourcedModifier(GameState game[], ModifierType type, int group,
43	int index, int percent, int roundsLeft,	int index, int percent, int roundsLeft,
44	ModifierSource source)	ModifierSource source)
45	{	Opens a new block.
46	if(game[0].economy.modifierCount >= MAX_MODIFIERS)	Condition: game[0].economy.modifierCount >= MAX_MODIFIERS -> run the next block if true.
47	return;	Returns the computed value immediately.
48		Blank line.
49	game[0].economy.modifiers[game[0].economy.modifierCo unt].type = type;	game[0].economy.modifiers[game[0].economy.modifierCount].type = type;
50	game[0].economy.modifiers[game[0].economy.modifierCo unt].group = group;	game[0].economy.modifiers[game[0].economy.modifierCount].group = group;
51	game[0].economy.modifiers[game[0].economy.modifierCo unt].index = index;	game[0].economy.modifiers[game[0].economy.modifierCount].index = index;

52 game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent; 53 game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft; 54 game[0].economy.modifiers[game[0].economy.modifierCount].source = source; 55 56 game[0].economy.modifierCount++; 57 }	game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent; game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft; game[0].economy.modifiers[game[0].economy.modifierCount].source = source; Blank line. game[0].economy.modifierCount++; Closes the current block.
--	---

Nested functions inside addSourcedModifier() - study these

- addSourcedModifier() (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting.)

MAIN PURPOSE

Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting.

10.4 addModifier()

```
void addModifier(GameState game[], ModifierType type, int group, int index, int percent, int roundsLeft)
```

Parameters: type, group, index, percent, roundsLeft | Returns: void.

The function, verbatim from the source (starts at line 58)

```
59    /* Events, national cards and regulations use this - they are all
60    generic effects, not Market Boom/Decline or Regional cards. */
61    void addModifier(GameState game[], ModifierType type, int group, int index,
62                    int percent, int roundsLeft)
63    {
64       addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL);
65    }
```

Line-by-line explanation

Line	Code	Explanation
58		Blank line.
59	/* Events, national cards and regulations use this - they are all	Comment: /* Events, national cards and regulations use this - they are all
60	generic effects, not Market Boom/Decline or Regional cards. */	generic effects, not Market Boom/Decline or Regional cards. */
61	void addModifier(GameState game[], ModifierType type, int group, int index,	void addModifier(GameState game[], ModifierType type, int group, int index,
62	int percent, int roundsLeft)	int percent, int roundsLeft)
63	{	Opens a new block.
64	addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL);	Calls addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL). (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
65	}	Closes the current block.

Nested functions inside addModifier() - study these

- addModifier() (addSourcedModifier with SRC_GENERAL (default for events/cards/regulations).)
- addSourcedModifier() (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting.)

MAIN PURPOSE

addSourcedModifier with SRC_GENERAL (default for events/cards/regulations).

10.5 modifierMultiplier()

```
int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
```

Parameters: type, group, index | Returns: Combined multiplier, 100 = no change.

The function, verbatim from the source (starts at line 66)

```
67  /* Effective percentage multiplier from every active matching modifier,  
68     multiplied together (Rule-LK 34). Returns 100 when none match.  
69     A modifier with group == -1 is global and applies to every group,  
70     so it matches regardless of the requested group (and likewise for  
71     index == -1). */  
72  int modifierMultiplier(GameState game[], ModifierType type, int group, int index)  
73  {  
74      int mult;  
75      int i;  
  
77      mult = 100;  
  
79      for(i = 0; i < game[0].economy.modifierCount; i++)  
80      {  
81          if(game[0].economy.modifiers[i].type != type)  
82              continue;  
83          if(group != -1 && game[0].economy.modifiers[i].group != -1 &&  
84             game[0].economy.modifiers[i].group != group)  
85              continue;  
86          if(index != -1 && game[0].economy.modifiers[i].index != -1 &&  
87             game[0].economy.modifiers[i].index != index)  
88              continue;  
  
90          mult = (mult * game[0].economy.modifiers[i].percent) / 100;  
91      }  
  
93      return mult;  
94  }
```

Line-by-line explanation

Line	Code	Explanation
66		Blank line.
67	/* Effective percentage multiplier from every active matching modifier,	Comment: /* Effective percentage multiplier from every active matching modifier,
68	multiplied together (Rule-LK 34). Returns 100 when none match.	multiplied together (Rule-LK 34). Returns 100 when none match.
69	A modifier with group == -1 is global and applies to every group,	A modifier with group == -1 is global and applies to every group,
70	so it matches regardless of the requested group (and likewise for	so it matches regardless of the requested group (and likewise for
71	index == -1). */	index == -1). */
72	int modifierMultiplier(GameState game[], ModifierType type, int group, int index)	int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
73	{	Opens a new block.
74	int mult;	Declares int mult.
75	int i;	Declares int i.
76		Blank line.
77	mult = 100;	mult = 100;
78		Blank line.
79	for(i = 0; i < game[0].economy.modifierCount; i++)	Loop over all entries: init i = 0; run while i < game[0].economy.modifierCount; every loop i++.
80	{	Opens a new block.
81	if(game[0].economy.modifiers[i].type != type)	Condition: game[0].economy.modifiers[i].type != type -> run the next block if true.
82	continue;	Skips to the next loop iteration.
83	if(group != -1 && game[0].economy.modifiers[i].group != -1 &&	Condition: -> run the next block if true.
84	game[0].economy.modifiers[i].group != group)	game[0].economy.modifiers[i].group != group)
85	continue;	Skips to the next loop iteration.

86	if(index != -1 && game[0].economy.modifiers[i].index	Condition: -> run the next block if true.
	!= -1 &&	
87	game[0].economy.modifiers[i].index != index)	game[0].economy.modifiers[i].index != index)
88	continue;	Skips to the next loop iteration.
89		Blank line.
90	mult = (mult * game[0].economy.modifiers[i].percent)	mult = (mult * game[0].economy.modifiers[i].percent) / 100;
	/ 100;	
91	}	Closes the current block.
92		Blank line.
93	return mult;	Returns mult immediately.
94	}	Closes the current block.

Nested functions inside modifierMultiplier() - study these

- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly +56%.

10.6 isModifierActive()

```
int isModifierActive(GameState game[], ModifierType type, int group, int index)
```

Parameters: type, group, index | Returns: 1 = active, 0 = not.

The function, verbatim from the source (starts at line 95)

```

96  /* Whether at least one matching modifier is currently active - used
97  for flag-style modifiers such as flood risk or recession. */
98  int isModifierActive(GameState game[], ModifierType type, int group, int index)
99  {
100     int i;

102     for(i = 0; i < game[0].economy.modifierCount; i++)
103     {
104         if(game[0].economy.modifiers[i].type != type)
105             continue;
106         if(group != -1 && game[0].economy.modifiers[i].group != group)
107             continue;
108         if(index != -1 && game[0].economy.modifiers[i].index != index)
109             continue;

111         return 1;
112     }

114     return 0;
115 }
```

Line-by-line explanation

Line	Code	Explanation
95		Blank line.
96	/* Whether at least one matching modifier is currently active - used	Comment: /* Whether at least one matching modifier is currently active - used
97	for flag-style modifiers such as flood risk or recession. */	for flag-style modifiers such as flood risk or recession. */
98	int isModifierActive(GameState game[], ModifierType type, int group, int index)	int isModifierActive(GameState game[], ModifierType type, int group, int index)
99	{	Opens a new block.
100	int i;	Declares int i.
101		Blank line.
102	for(i = 0; i < game[0].economy.modifierCount; i++)	Loop over all entries: init i = 0; run while i < game[0].economy.modifierCount; every loop i++.
103	{	Opens a new block.

104	<code>if(game[0].economy.modifiers[i].type != type)</code>	Condition: <code>game[0].economy.modifiers[i].type != type</code> -> run the next block if true.
105	<code>continue;</code>	Skips to the next loop iteration.
106	<code>if(group != -1 && game[0].economy.modifiers[i].group != group)</code>	Condition: <code>group != -1 && game[0].economy.modifiers[i].group != group</code> -> run the next block if true.
107	<code>continue;</code>	Skips to the next loop iteration.
108	<code>if(index != -1 && game[0].economy.modifiers[i].index != index)</code>	Condition: <code>index != -1 && game[0].economy.modifiers[i].index != index</code> -> run the next block if true.
109	<code>continue;</code>	Skips to the next loop iteration.
110		Blank line.
111	<code>return 1;</code>	Returns 1 immediately.
112	<code>}</code>	Closes the current block.
113		Blank line.
114	<code>return 0;</code>	Returns 0 immediately.
115	<code>}</code>	Closes the current block.

Nested functions inside `isModifierActive()` - study these

- `isModifierActive()` (Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active)

MAIN PURPOSE

Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active.

10.7 `currentTaxRatePercent()`

```
int currentTaxRatePercent(GameState game[], int baseRate)
```

Parameters: `baseRate` | Returns: Effective percent.

The function, verbatim from the source (starts at line 116)

```
117  /* The effective tax rate used by the Income Tax and Community
118     Development Fund squares. Starts at the given base rate (15% /
119     10%) and is scaled by the currently active global value
120     modifiers, so market fluctuations move the rate up or down. */
121  int currentTaxRatePercent(GameState game[], int baseRate)
122  {
123      return (baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100;
124  }
```

Line-by-line explanation

Line	Code	Explanation
116		Blank line.
117	<code>/* The effective tax rate used by the Income Tax and Community</code>	Comment: <code>/* The effective tax rate used by the Income Tax and Community</code>
118	<code>Development Fund squares. Starts at the given base rate (15% /</code>	Development Fund squares. Starts at the given base rate (15% /
119	<code>10%) and is scaled by the currently active global value</code>	10%) and is scaled by the currently active global value
120	<code>modifiers, so market fluctuations move the rate up or down. */</code>	modifiers, so market fluctuations move the rate up or down. */
121	<code>int currentTaxRatePercent(GameState game[], int baseRate)</code>	<code>int currentTaxRatePercent(GameState game[], int baseRate)</code>
122	<code>{</code>	Opens a new block.
123	<code>return (baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100;</code>	Returns <code>(baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100</code> immediately.
124	<code>}</code>	Closes the current block.

Nested functions inside `currentTaxRatePercent()` - study these

- `currentTaxRatePercent()` (The effective tax rate: base rate scaled by `MOD_VALUE_GLOBAL`.)

- `modifierMultiplier()` (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

The effective tax rate: base rate scaled by MOD_VALUE_GLOBAL.

10.8 decrementModifiers()

```
void decrementModifiers(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 125)

```
126  /* Ticks every active modifier down and removes the expired ones. */
127  void decrementModifiers(GameState game[])
128  {
129      int i;

131      i = 0;

133      while(i < game[0].economy.modifierCount)
134      {
135          game[0].economy.modifiers[i].roundsLeft--;

137          if(game[0].economy.modifiers[i].roundsLeft <= 0)
138          {
139              int j;

141              for(j = i; j < game[0].economy.modifierCount - 1; j++)
142                  game[0].economy.modifiers[j] = game[0].economy.modifiers[j + 1];

144              game[0].economy.modifierCount--;
145          }
146          else
147          {
148              i++;
149          }
150      }
151  }
```

Line-by-line explanation

Line	Code	Explanation
125		Blank line.
126	/* Ticks every active modifier down and removes the expired ones. */	Comment: /* Ticks every active modifier down and removes the expired ones. */
127	void decrementModifiers(GameState game[])	void decrementModifiers(GameState game[])
128	{	Opens a new block.
129	int i;	Declares int i.
130		Blank line.
131	i = 0;	i = 0;
132		Blank line.
133	while(i < game[0].economy.modifierCount)	Loop: repeat while i < game[0].economy.modifierCount is true.
134	{	Opens a new block.
135	game[0].economy.modifiers[i].roundsLeft--;	game[0].economy.modifiers[i].roundsLeft--;
136		Blank line.
137	if(game[0].economy.modifiers[i].roundsLeft <= 0)	Condition: game[0].economy.modifiers[i].roundsLeft <= 0 -> run the next block if true.
138	{	Opens a new block.
139	int j;	Declares int j.
140		Blank line.
141	for(j = i; j < game[0].economy.modifierCount - 1; j++)	Loop over all entries: init j = i; run while j < game[0].economy.modifierCount - 1; every loop j++.
142	game[0].economy.modifiers[j] = game[0].economy.modifiers[j + 1];	Sets the field modifiers of the shared economy (i.e. Array of up to 48 active timed modifiers.) = game[0].economy.modifiers[j + 1].
143		Blank line.

144	<code>game[0].economy.modifierCount--;</code>	<code>game[0].economy.modifierCount--;</code>
145	<code>}</code>	Closes the current block.
146	<code>else</code>	The alternative branch when the condition above is false.
147	<code>{</code>	Opens a new block.
148	<code>i++;</code>	<code>i++;</code>
149	<code>}</code>	Closes the current block.
150	<code>}</code>	Closes the current block.
151	<code>}</code>	Closes the current block.

Nested functions inside `decrementModifiers()` - study these

- `decrementModifiers()` (Ticks every active modifier down and compacts out expired ones.)

MAIN PURPOSE

Ticks every active modifier down and compacts out expired ones.

10.9 `formatLKR()`

```
void formatLKR(int amount, char out[])
```

Parameters: `amount`, `out` | Returns: `void` (writes into `out`).

The function, verbatim from the source (starts at line 152)

```

153  /* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */
154  void formatLKR(int amount, char out[])
155  {
156      char digits[32];
157      int len;
158      int i;
159      int w;
160      int neg;

162      neg = 0;

164      if(amount < 0)
165      {
166          neg = 1;
167          amount = -amount;
168      }

170      sprintf(digits, "%d", amount);
171      len = strlen(digits);

173      w = 0;

175      if(neg)
176          out[w++] = '-';

178      for(i = 0; i < len; i++)
179      {
180          if(i > 0 && (len - i) % 3 == 0)
181              out[w++] = ',';

183          out[w++] = digits[i];
184      }

186      out[w] = '\0';
187  }
```

Line-by-line explanation

Line	Code	Explanation
152		Blank line.
153	<code>/* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */</code>	Comment: <code>/* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */</code>
154	<code>void formatLKR(int amount, char out[])</code>	<code>void formatLKR(int amount, char out[])</code>

155	{	Opens a new block.
156	char digits[32];	Declares char digits[32].
157	int len;	Declares int len.
158	int i;	Declares int i.
159	int w;	Declares int w.
160	int neg;	Declares int neg.
161		Blank line.
162	neg = 0;	neg = 0;
163		Blank line.
164	if(amount < 0)	Condition: amount < 0 -> run the next block if true.
165	{	Opens a new block.
166	neg = 1;	neg = 1;
167	amount = -amount;	amount = -amount;
168	}	Closes the current block.
169		Blank line.
170	sprintf(digits, "%d", amount);	Calls sprintf(digits, "%d", amount).
171	len = strlen(digits);	len = strlen(digits);
172		Blank line.
173	w = 0;	w = 0;
174		Blank line.
175	if(neg)	Condition: neg -> run the next block if true.
176	out[w++] = '-';	out[w++] = '-';
177		Blank line.
178	for(i = 0; i < len; i++)	Loop over all entries: init i = 0; run while i < len; every loop i++.
179	{	Opens a new block.
180	if(i > 0 && (len - i) % 3 == 0)	Condition: i > 0 && (len - i) % 3 == 0 -> run the next block if true.
181	out[w++] = ',';	out[w++] = ',';
182		Blank line.
183	out[w++] = digits[i];	out[w++] = digits[i];
184	}	Closes the current block.
185		Blank line.
186	out[w] = '\0';	out[w] = '\0';
187	}	Closes the current block.

Nested functions inside formatLKR() - study these

- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Formats an amount with thousands separators, e.g. 30000 -> "30,000".

10.10 applyInflation()

```
void applyInflation(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 188)

```
189  /* Every 10 rounds, a random inflation rate is applied to every
190     property's price, rent, and construction costs. */
191  void applyInflation(GameState game[])
192  {
193      int possibleRates[6] = {-3, 0, 2, 5, 8, 12};
194      int rate;
195      int i;

197      rate = possibleRates[rand() % 6];
198      game[0].economy.inflationRate = rate;

200      printf("\n=== Inflation Update ===\n");
201      printf("New Inflation Rate : %+d%%\n", rate);
```

```

203     for(i = 0; i < BOARD_SIZE; i++)
204     {
205         if(game[0].board[i].type != PROPERTY &&
206            game[0].board[i].type != RAILWAY &&
207            game[0].board[i].type != UTILITY)
208         {
209             continue;
210         }

212         game[0].board[i].property.purchasePrice =
213             applyRate(game[0].board[i].property.purchasePrice, rate);

215         game[0].board[i].property.mortgageValue =
216             applyRate(game[0].board[i].property.mortgageValue, rate);

218         if(game[0].board[i].type == PROPERTY)
219         {
220             game[0].board[i].property.baseRent =
221                 applyRate(game[0].board[i].property.baseRent, rate);

223             game[0].board[i].property.houseCost =
224                 applyRate(game[0].board[i].property.houseCost, rate);

226             game[0].board[i].property.hotelCost =
227                 applyRate(game[0].board[i].property.hotelCost, rate);
228         }
229     }

231     game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, rate);

233     printf("New Loan Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
234 }

```

Line-by-line explanation

Line	Code	Explanation
188		Blank line.
189	/* Every 10 rounds, a random inflation rate is applied to every	Comment: /* Every 10 rounds, a random inflation rate is applied to every
190	property's price, rent, and construction costs. */	property's price, rent, and construction costs. */
191	void applyInflation(GameState game[])	void applyInflation(GameState game[])
192	{	Opens a new block.
193	int possibleRates[6] = {-3, 0, 2, 5, 8, 12};	Declares int possibleRates[6] = {-3, 0, 2, 5, 8, 12}.
194	int rate;	Declares int rate.
195	int i;	Declares int i.
196		Blank line.
197	rate = possibleRates[rand() % 6];	rate = possibleRates[rand() % 6];
198	game[0].economy.inflationRate = rate;	Sets the field inflationRate of the shared economy (i.e. Current inflation percent (-3..12) applied every 10 rounds.) = rate.
199		Blank line.
200	printf("\n=== Inflation Update ===\n");	Prints the message "\n=== Inflation Update ===\n".
201	printf("New Inflation Rate : %d%%\n", rate);	Prints the message "New Inflation Rate : %d%%\n".
202		Blank line.
203	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
204	{	Opens a new block.
205	if(game[0].board[i].type != PROPERTY &&	Condition: -> run the next block if true.
206	game[0].board[i].type != RAILWAY &&	game[0].board[i].type != RAILWAY &&
207	game[0].board[i].type != UTILITY)	game[0].board[i].type != UTILITY)
208	{	Opens a new block.
209	continue;	Skips to the next loop iteration.
210	}	Closes the current block.
211		Blank line.
212	game[0].board[i].property.purchasePrice =	Sets the field purchasePrice of the property on the board (i.e. Base price to buy (inflation and modifiers can move this).) = the computed value.

213	<code>applyRate(game[0].board[i].property.purchasePrice, rate);</code>	Calls <code>applyRate(game[0].board[i].property.purchasePrice, rate)</code> . (Rule-LK 14: applies a percentage change, never returning below 1)
214		Blank line.
215	<code>game[0].board[i].property.mortgageValue =</code>	Sets the field <code>mortgageValue</code> of the property on the board (i.e. Value received when mortgaging (redeem costs +10%.) = the computed value.
216	<code>applyRate(game[0].board[i].property.mortgageValue, rate);</code>	Calls <code>applyRate(game[0].board[i].property.mortgageValue, rate)</code> . (Rule-LK 14: applies a percentage change, never returning below 1)
217		Blank line.
218	<code>if(game[0].board[i].type == PROPERTY)</code>	Condition: <code>square i.type == PROPERTY</code> -> run the next block if true.
219	<code>{</code>	Opens a new block.
220	<code>game[0].board[i].property.baseRent =</code>	Sets the field <code>baseRent</code> of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = the computed value.
221	<code>applyRate(game[0].board[i].property.baseRent, rate);</code>	Calls <code>applyRate(game[0].board[i].property.baseRent, rate)</code> . (Rule-LK 14: applies a percentage change, never returning below 1)
222		Blank line.
223	<code>game[0].board[i].property.houseCost =</code>	Sets the field <code>houseCost</code> of the property on the board (i.e. Cost of building one house.) = the computed value.
224	<code>applyRate(game[0].board[i].property.houseCost, rate);</code>	Calls <code>applyRate(game[0].board[i].property.houseCost, rate)</code> . (Rule-LK 14: applies a percentage change, never returning below 1)
225		Blank line.
226	<code>game[0].board[i].property.hotelCost =</code>	Sets the field <code>hotelCost</code> of the property on the board (i.e. Cost of building a hotel.) = the computed value.
227	<code>applyRate(game[0].board[i].property.hotelCost, rate);</code>	Calls <code>applyRate(game[0].board[i].property.hotelCost, rate)</code> . (Rule-LK 14: applies a percentage change, never returning below 1)
228	<code>}</code>	Closes the current block.
229	<code>}</code>	Closes the current block.
230		Blank line.
231	<code>game[0].economy.loanInterestRate =</code> <code>applyRate(game[0].economy.loanInterestRate, rate);</code>	Sets the field <code>loanInterestRate</code> of the shared economy (i.e. Current economy loan interest rate (1..25).) = <code>applyRate(game[0].economy.loanInterestRate, rate)</code> .
232		Blank line.
233	<code>printf("New Loan Interest Rate : %d%%\n",</code> <code>game[0].economy.loanInterestRate);</code>	Prints the message "New Loan Interest Rate : %d%%\n".
234	<code>}</code>	Closes the current block.

Nested functions inside `applyInflation()` - study these

- `applyInflation()` (Every 10 rounds picks a random rate {-3, 0, 2, 5, 8, 12} and applies it to purchase prices)
- `applyRate()` (Rule-LK 14: applies a percentage change, never returning below 1.)

MAIN PURPOSE

Every 10 rounds picks a random rate {-3, 0, 2, 5, 8, 12} and applies it to purchase prices, mortgage values, rents, construction costs and the loan interest rate.

10.11 `currentMarketValue()`

```
int currentMarketValue(GameState game[], int propIndex)
```

Parameters: `propIndex` | Returns: Value in LKR.

The function, verbatim from the source (starts at line 235)

```
236  /* A property's real current worth: purchase price, minus age
237     depreciation, adjusted by any active market conditions. Reused
238     everywhere a "real price" is needed - buying, insurance, repairs,
239     renovation, net worth - so all of them stay consistent. */
240  /* Rule-LK 11-13, 24: the current market value of a property, after
241     age depreciation and every value modifier currently active
242     (Rule-LK 34 - all matching modifiers multiply together). */
243  int currentMarketValue(GameState game[], int propIndex)
244  {
245      int price;
```

```

246     int depreciation;
247     int mult;

249     price = game[0].board[propIndex].property.purchasePrice;
250     depreciation = game[0].board[propIndex].property.depreciation;

252     price = price - (price * depreciation) / 100;

254     mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

256     if(game[0].board[propIndex].type == PROPERTY)
257         mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,
258                                           game[0].board[propIndex].property.group, -1)) / 100;

260     if(game[0].board[propIndex].type == RAILWAY)
261         mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100;

263     mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100;

265     price = (price * mult) / 100;

267     return price;
268 }

```

Line-by-line explanation

Line	Code	Explanation
235		Blank line.
236	/* A property's real current worth: purchase price, minus age	Comment: /* A property's real current worth: purchase price, minus age
237	depreciation, adjusted by any active market conditions. Reused	depreciation, adjusted by any active market conditions. Reused
238	everywhere a "real price" is needed - buying, insurance, repairs,	everywhere a "real price" is needed - buying, insurance, repairs,
239	renovation, net worth - so all of them stay consistent. */	renovation, net worth - so all of them stay consistent. */
240	/* Rule-LK 11-13, 24: the current market value of a property, after	Comment: /* Rule-LK 11-13, 24: the current market value of a property, after
241	age depreciation and every value modifier currently active	age depreciation and every value modifier currently active
242	(Rule-LK 34 - all matching modifiers multiply together). */	(Rule-LK 34 - all matching modifiers multiply together). */
243	int currentMarketValue(GameState game[], int propIndex)	int currentMarketValue(GameState game[], int propIndex)
244	{	Opens a new block.
245	int price;	Declares int price.
246	int depreciation;	Declares int depreciation.
247	int mult;	Declares int mult.
248		Blank line.
249	price = game[0].board[propIndex].property.purchasePrice;	price = game[0].board[propIndex].property.purchasePrice;
250	depreciation = game[0].board[propIndex].property.depreciation;	depreciation = game[0].board[propIndex].property.depreciation;
251		Blank line.
252	price = price - (price * depreciation) / 100;	price = price - (price * depreciation) / 100;
253		Blank line.
254	mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);	mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);
255		Blank line.
256	if(game[0].board[propIndex].type == PROPERTY)	Condition: this property.type == PROPERTY -> run the next block if true.
257	mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,	mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,

258 game[0].board[propIndex].property.group, -1)) / 100; 259 260 if(game[0].board[propIndex].type == RAILWAY) 261 mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100; 262 263 mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100; 264 265 price = (price * mult) / 100; 266 267 return price; 268 }	game[0].board[propIndex].property.group, -1)) / 100; Blank line. Condition: this property.type == RAILWAY -> run the next block if true. mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100; Blank line. mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100; Blank line. price = (price * mult) / 100; Blank line. Returns price immediately. Closes the current block.
--	---

Nested functions inside currentMarketValue() - study these

- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)

MAIN PURPOSE

The real worth of a property: purchase price minus age depreciation, times every applicable value modifier (global, group, rail, ind ex).

10.12 ageProperties()

```
void ageProperties(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 269)

```

270  /* Rule-LK 15, 16: properties get older every round, and lose value
271     past age 50. Prints the required "Property Depreciation" message
272     (Section 5) whenever a property's depreciation actually changes. */
273  void ageProperties(GameState game[])
274  {
275      int i;
276      int roundsOver50;
277      int newDepreciation;
278      int oldDepreciation;

280      for(i = 0; i < BOARD_SIZE; i++)
281      {
282          if(game[0].board[i].type != PROPERTY)
283              continue;

285          game[0].board[i].property.age++;

287          if(game[0].board[i].property.age > PROPERTY_AGE_LIMIT)
288          {
289              oldDepreciation = game[0].board[i].property.depreciation;

291              roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;
292              newDepreciation = roundsOver50 / DEPRECIATION_STEP;

294              if(newDepreciation > MAX_DEPRECIATION)
295                  newDepreciation = MAX_DEPRECIATION;

297              game[0].board[i].property.depreciation = newDepreciation;

299              if(newDepreciation != oldDepreciation)
300              {
301                  printf("\nProperty Depreciation\n");
302                  printf("Property\n%s\n", game[0].board[i].name);
303                  printf("has depreciated by %d%%.\n", newDepreciation);
304                  printf("Current Value\nLKR %d\n", currentMarketValue(game, i));

```

```

305         }
306     }
307 }
308 }

```

Line-by-line explanation

Line	Code	Explanation
269		Blank line.
270	/* Rule-LK 15, 16: properties get older every round, and lose value	Comment: /* Rule-LK 15, 16: properties get older every round, and lose value
271	past age 50. Prints the required "Property Depreciation" message	past age 50. Prints the required "Property Depreciation" message
272	(Section 5) whenever a property's depreciation actually changes. */	(Section 5) whenever a property's depreciation actually changes. */
273	void ageProperties(GameState game[])	void ageProperties(GameState game[])
274	{	Opens a new block.
275	int i;	Declares int i.
276	int roundsOver50;	Declares int roundsOver50.
277	int newDepreciation;	Declares int newDepreciation.
278	int oldDepreciation;	Declares int oldDepreciation.
279		Blank line.
280	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
281	{	Opens a new block.
282	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
283	continue;	Skips to the next loop iteration.
284		Blank line.
285	game[0].board[i].property.age++;	game[0].board[i].property.age++;
286		Blank line.
287	if(game[0].board[i].property.age > PROPERTY_AGE_LIMIT)	Condition: square i.property.age > PROPERTY_AGE_LIMIT -> run the next block if true.
288	{	Opens a new block.
289	oldDepreciation = game[0].board[i].property.depreciation;	oldDepreciation = game[0].board[i].property.depreciation;
290		Blank line.
291	roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;	roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;
292	newDepreciation = roundsOver50 / DEPRECIATION_STEP;	newDepreciation = roundsOver50 / DEPRECIATION_STEP;
293		Blank line.
294	if(newDepreciation > MAX_DEPRECIATION)	Condition: newDepreciation > MAX_DEPRECIATION -> run the next block if true.
295	newDepreciation = MAX_DEPRECIATION;	newDepreciation = MAX_DEPRECIATION;
296		Blank line.
297	game[0].board[i].property.depreciation = newDepreciation;	Sets the field depreciation of the property on the board (i.e. Percent value loss from age (0..30%).) = newDepreciation.
298		Blank line.
299	if(newDepreciation != oldDepreciation)	Condition: newDepreciation != oldDepreciation -> run the next block if true.
300	{	Opens a new block.
301	printf("\nProperty Depreciation\n");	Prints the message "\nProperty Depreciation\n".
302	printf("Property\n%s\n", game[0].board[i].name);	Prints the message "Property\n%s\n".
303	printf("has depreciated by %d%%.\n", newDepreciation);	Prints the message "has depreciated by %d%%.\n".
304	printf("Current Value\nLKR %d\n", currentMarketValue(game, i));	Prints the message "Current Value\nLKR %d\n".
305	}	Closes the current block.
306	}	Closes the current block.
307	}	Closes the current block.
308	}	Closes the current block.

Nested functions inside ageProperties() - study these

- ageProperties() (Rule-LK 15/16: ages every property; past age 50 depreciation ticks +5% every 5 rounds, cap)
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)

MAIN PURPOSE

Rule-LK 15/16: ages every property; past age 50 depreciation ticks +5% every 5 rounds, capped at 30%, printing when it changes

10.13 tryRenovateAgeDepreciation()

```
void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
```

Parameters: *playerIndex, propIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 309)

```
310  /* Rule-LK 17: landing on your own aged property offers a renovation. */
311  void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
312  {
313      int cost;

315      if(game[0].board[propIndex].property.depreciation <= 0)
316          return;

318      if(!shouldRenovateAgeDepreciation(game, playerIndex,
319                                         game[0].board[propIndex].property.depreciation))
320      {
321          return;
322      }

324      cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;

326      if(game[0].players[playerIndex].cash < cost)
327          return;

329      payMoney(game, playerIndex, cost);

331      game[0].board[propIndex].property.age = 0;
332      game[0].board[propIndex].property.depreciation = 0;

334      game[0].board[propIndex].property.baseRent =
335          applyRate(game[0].board[propIndex].property.baseRent, RENOVATION_RENT_BOOST);

337      printf("\n%s renovated %s.\n",
338             game[0].players[playerIndex].name, game[0].board[propIndex].name);

340      printf("Renovation Cost : LKR %d\n", cost);
341  }
```

Line-by-line explanation

Line	Code	Explanation
309		Blank line.
310	/* Rule-LK 17: landing on your own aged property offers a renovation. */	Comment: /* Rule-LK 17: landing on your own aged property offers a renovation. */
311	void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)	void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
312	{	Opens a new block.
313	int cost;	Declares int cost.
314		Blank line.
315	if(game[0].board[propIndex].property.depreciation <= 0)	Condition: this property.property.depreciation <= 0 -> run the next block if true.
316	return;	Returns the computed value immediately.
317		Blank line.
318	if(!shouldRenovateAgeDepreciation(game, playerIndex,	Condition: -> run the next block if true.

319	game[0].board[propIndex].property.depreciation))	game[0].board[propIndex].property.depreciation))
320	{	Opens a new block.
321	return;	Returns the computed value immediately.
322	}	Closes the current block.
323		Blank line.
324	cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;	cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;
325		Blank line.
326	if(game[0].players[playerIndex].cash < cost)	Condition: the current player.cash < cost -> run the next block if true.
327	return;	Returns the computed value immediately.
328		Blank line.
329	payMoney(game, playerIndex, cost);	Calls payMoney(game, playerIndex, cost). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
330		Blank line.
331	game[0].board[propIndex].property.age = 0;	Sets the field age of the property on the board (i.e. Rounds since purchase/last renovation.) = 0.
332	game[0].board[propIndex].property.depreciation = 0;	Sets the field depreciation of the property on the board (i.e. Percent value loss from age (0..30%).) = 0.
333		Blank line.
334	game[0].board[propIndex].property.baseRent =	Sets the field baseRent of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = the computed value.
335	applyRate(game[0].board[propIndex].property.baseRent , RENOVATION_RENT_BOOST);	Calls applyRate(game[0].board[propIndex].property.baseRent, RENOVATION_RENT_BOOST). (Rule-LK 14: applies a percentage change, never returning below 1)
336		Blank line.
337	printf("\n%s renovated %s.\n",	Prints the message "\n%s renovated %s.\n".
338	game[0].players[playerIndex].name, game[0].board[propIndex].name);	game[0].players[playerIndex].name, game[0].board[propIndex].name);
339		Blank line.
340	printf("Renovation Cost : LKR %d\n", cost);	Prints the message "Renovation Cost : LKR %d\n".
341	}	Closes the current block.

Nested functions inside tryRenovateAgeDepreciation() - study these

- tryRenovateAgeDepreciation() (Rule-LK 17: landing on your own aged property offers a renovation - 10% of value, resetting)
- shouldRenovateAgeDepreciation() (Whether the player pays to renovate away age depreciation.)
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- applyRate() (Rule-LK 14: applies a percentage change, never returning below 1.)

MAIN PURPOSE

Rule-LK 17: landing on your own aged property offers a renovation - 10% of value, resetting age/depreciation and raising base rent +5%.

10.14 rentConditionPercent()

```
int rentConditionPercent(int condition)
```

Parameters: condition | Returns: Percent (0, 50, 75, 90 or 100).

The function, verbatim from the source (starts at line 342)

```
343  /* Table 3: rent collected as a percentage of building condition. */
344  int rentConditionPercent(int condition)
345  {
346      if(condition >= 90)
347          return 100;
348      if(condition >= 75)
349          return 90;
350      if(condition >= 50)
351          return 75;
```

```

352         if(condition >= 25)
353             return 50;

355     return 0;    /* below 25% - building closed */
356 }

```

Line-by-line explanation

Line	Code	Explanation
342		Blank line.
343	/* Table 3: rent collected as a percentage of building condition. */	Comment: /* Table 3: rent collected as a percentage of building condition. */
344	int rentConditionPercent(int condition)	int rentConditionPercent(int condition)
345	{	Opens a new block.
346	if(condition >= 90)	Condition: condition >= 90 -> run the next block if true.
347	return 100;	Returns 100 immediately.
348	if(condition >= 75)	Condition: condition >= 75 -> run the next block if true.
349	return 90;	Returns 90 immediately.
350	if(condition >= 50)	Condition: condition >= 50 -> run the next block if true.
351	return 75;	Returns 75 immediately.
352	if(condition >= 25)	Condition: condition >= 25 -> run the next block if true.
353	return 50;	Returns 50 immediately.
354		Blank line.
355	return 0; /* below 25% - building closed */	Returns 0; /* below 25% - building closed */ immediately.
356	}	Closes the current block.

Nested functions inside rentConditionPercent() - study these

- rentConditionPercent() (Table 3: maps building condition to the fraction of rent collected.)

MAIN PURPOSE

Table 3: maps building condition to the fraction of rent collected.

10.15 ageBuildings()

```
void ageBuildings(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 357)

```

358     /* Rule-LK 25, 28: building condition wears down every round, and
359        20+ rounds of neglect causes lasting structural damage. */
360     void ageBuildings(GameState game[])
361     {
362         int i;

364         for(i = 0; i < BOARD_SIZE; i++)
365         {
366             if(game[0].board[i].type != PROPERTY)
367                 continue;

369             /* Business Interruption's 5-round lost-income countdown runs
370                even if the property isn't currently developed with houses
371                (it still has a hotel that just isn't earning anything). */
372             if(game[0].board[i].property.lostIncomeRoundsLeft > 0)
373                 game[0].board[i].property.lostIncomeRoundsLeft--;

375             if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
376                 continue;

378             game[0].board[i].property.condition -= 2;

380             if(game[0].board[i].property.condition < 0)
381                 game[0].board[i].property.condition = 0;

```

```

383         game[0].board[i].property.roundsSinceMaintenance++;

385         if(game[0].board[i].property.roundsSinceMaintenance > NEGLECT_ROUNDS &&
386            !game[0].board[i].property.structurallyDamaged)
387         {
388             game[0].board[i].property.structurallyDamaged = 1;

390             game[0].board[i].property.preDamagePurchasePrice =
391                 game[0].board[i].property.purchasePrice;
392             game[0].board[i].property.preDamageBaseRent =
393                 game[0].board[i].property.baseRent;

395             game[0].board[i].property.purchasePrice =
396                 game[0].board[i].property.purchasePrice -
397                 (game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;

399             game[0].board[i].property.baseRent =
400                 game[0].board[i].property.baseRent -
401                 (game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;

403             game[0].board[i].property.maintenanceCostMultiplierPercent = DAMAGE_REPAIR_MULTIPLIER;
404             game[0].board[i].property.condition = 0;

406             printf("\n%s has suffered structural damage from neglect!\n",
407                 game[0].board[i].name);
408         }
409     }
410 }

```

Line-by-line explanation

Line	Code	Explanation
357		Blank line.
358	/* Rule-LK 25, 28: building condition wears down every round, and	Comment: /* Rule-LK 25, 28: building condition wears down every round, and
359	20+ rounds of neglect causes lasting structural damage. */	20+ rounds of neglect causes lasting structural damage. */
360	void ageBuildings(GameState game[])	void ageBuildings(GameState game[])
361	{	Opens a new block.
362	int i;	Declares int i.
363		Blank line.
364	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
365	{	Opens a new block.
366	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
367	continue;	Skips to the next loop iteration.
368		Blank line.
369	/* Business Interruption's 5-round lost-income countdown runs	Comment: /* Business Interruption's 5-round lost-income countdown runs
370	even if the property isn't currently developed with houses	even if the property isn't currently developed with houses
371	(it still has a hotel that just isn't earning anything). */	(it still has a hotel that just isn't earning anything). */
372	if(game[0].board[i].property.lostIncomeRoundsLeft > 0)	Condition: square i.property.lostIncomeRoundsLeft > 0 -> run the next block if true.
373	game[0].board[i].property.lostIncomeRoundsLeft--;	game[0].board[i].property.lostIncomeRoundsLeft--;
374		Blank line.
375	if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)	Condition: square i.property.houses == 0 && !square i.property.hotel -> run the next block if true.
376	continue;	Skips to the next loop iteration.
377		Blank line.
378	game[0].board[i].property.condition -= 2;	Updates the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).): decreases it by 2.
379		Blank line.

380	if(game[0].board[i].property.condition < 0)	Condition: square i.property.condition < 0 -> run the next block if true.
381	game[0].board[i].property.condition = 0;	Sets the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 0.
382		Blank line.
383	game[0].board[i].property.roundsSinceMaintenance++;	game[0].board[i].property.roundsSinceMaintenance++;
384		Blank line.
385	if(game[0].board[i].property.roundsSinceMaintenance > NEGLECT_ROUNDS &&	Condition: -> run the next block if true.
386	!game[0].board[i].property.structurallyDamaged)	!game[0].board[i].property.structurallyDamaged)
387	{	Opens a new block.
388	game[0].board[i].property.structurallyDamaged = 1;	Sets the field structurallyDamaged of the property on the board (i.e. 1 when neglect (20+ rounds) caused structural damage.) = 1.
389		Blank line.
390	game[0].board[i].property.preDamagePurchasePrice =	Sets the field preDamagePurchasePrice of the property on the board (i.e. Purchase price before structural damage (for restore).) = the computed value.
391	game[0].board[i].property.purchasePrice;	game[0].board[i].property.purchasePrice;
392	game[0].board[i].property.preDamageBaseRent =	Sets the field preDamageBaseRent of the property on the board (i.e. Base rent before structural damage (for restore).) = the computed value.
393	game[0].board[i].property.baseRent;	game[0].board[i].property.baseRent;
394		Blank line.
395	game[0].board[i].property.purchasePrice =	Sets the field purchasePrice of the property on the board (i.e. Base price to buy (inflation and modifiers can move this).) = the computed value.
396	game[0].board[i].property.purchasePrice -	game[0].board[i].property.purchasePrice -
397	(game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;	(game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;
398		Blank line.
399	game[0].board[i].property.baseRent =	Sets the field baseRent of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = the computed value.
400	game[0].board[i].property.baseRent -	game[0].board[i].property.baseRent -
401	(game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;	(game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;
402		Blank line.
403	game[0].board[i].property.maintenanceCostMultiplierPercent = DAMAGE_REPAIR_MULTIPLIER;	Sets the field maintenanceCostMultiplierPercent of the property on the board (i.e. Cost multiplier (150% after damage, else 100%).) = DAMAGE_REPAIR_MULTIPLIER.
404	game[0].board[i].property.condition = 0;	Sets the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 0.
405		Blank line.
406	printf("\n%s has suffered structural damage from neglect!\n",	Prints the message "\n%s has suffered structural damage from neglect!\n".
407	game[0].board[i].name);	game[0].board[i].name);
408	}	Closes the current block.
409	}	Closes the current block.
410	}	Closes the current block.

Nested functions inside ageBuildings() - study these

- ageBuildings() (Rule-LK 25/28: condition drops 2 per round; lost-income countdown runs; 20+ rounds without)

MAIN PURPOSE

Rule-LK 25/28: condition drops 2 per round; lost-income countdown runs; 20+ rounds without maintenance causes structural damage (-15% value, -25% rent, 150% repairs).

10.16 performMaintenance()

```
void performMaintenance(GameState game[], int playerIndex)
```

Parameters: playerIndex | Returns: void.

The function, verbatim from the source (starts at line 411)

```
412  /* Rule-LK 27: restores a building's condition to 100% for a fee. */
413  void performMaintenance(GameState game[], int playerIndex)
414  {
415      int i;
416      int baseCost;
417      int cost;

419      for(i = 0; i < BOARD_SIZE; i++)
420      {
421          if(game[0].board[i].type != PROPERTY)
422              continue;

424          if(game[0].board[i].property.owner != playerIndex)
425              continue;

427          if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
428              continue;

430          if(game[0].board[i].property.structurallyDamaged)
431              continue; /* needs a full renovation instead */

433          if(game[0].board[i].property.condition >= 100)
434              continue;

436          if(game[0].board[i].property.hotel)
437              baseCost = (game[0].board[i].property.hotelCost * 8) / 100;
438          else
439              baseCost = (game[0].board[i].property.houseCost * 5) / 100;

441          cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierPercent) / 100;

443          if(!shouldMaintain(game, playerIndex, game[0].board[i].property.condition, cost))
444              continue;

446          if(game[0].players[playerIndex].cash < cost)
447              continue;

449          payMoney(game, playerIndex, cost);

451          game[0].board[i].property.condition = 100;
452          game[0].board[i].property.roundsSinceMaintenance = 0;

454          printf("\n%s performed maintenance on %s.\n",
455                 game[0].players[playerIndex].name, game[0].board[i].name);

457          printf("Maintenance Cost : LKR %d\n", cost);
458      }
459  }
```

Line-by-line explanation

Line	Code	Explanation
411		Blank line.
412	/* Rule-LK 27: restores a building's condition to 100% for a fee. */	Comment: /* Rule-LK 27: restores a building's condition to 100% for a fee. */
413	void performMaintenance(GameState game[], int playerIndex)	void performMaintenance(GameState game[], int playerIndex)
414	{	Opens a new block.
415	int i;	Declares int i.
416	int baseCost;	Declares int baseCost.
417	int cost;	Declares int cost.
418		Blank line.
419	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
420	{	Opens a new block.

421	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
422	continue;	Skips to the next loop iteration.
423		Blank line.
424	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
425	continue;	Skips to the next loop iteration.
426		Blank line.
427	if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)	Condition: square i.property.houses == 0 && !square i.property.hotel -> run the next block if true.
428	continue;	Skips to the next loop iteration.
429		Blank line.
430	if(game[0].board[i].property.structurallyDamaged)	Condition: square i.property.structurallyDamaged -> run the next block if true.
431	continue; /* needs a full renovation instead */	continue; /* needs a full renovation instead */
432		Blank line.
433	if(game[0].board[i].property.condition >= 100)	Condition: square i.property.condition >= 100 -> run the next block if true.
434	continue;	Skips to the next loop iteration.
435		Blank line.
436	if(game[0].board[i].property.hotel)	Condition: square i.property.hotel -> run the next block if true.
437	baseCost = (game[0].board[i].property.hotelCost * 8) / 100;	baseCost = (game[0].board[i].property.hotelCost * 8) / 100;
438	else	The alternative branch when the condition above is false.
439	baseCost = (game[0].board[i].property.houseCost * 5) / 100;	baseCost = (game[0].board[i].property.houseCost * 5) / 100;
440		Blank line.
441	cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierP ercent) / 100;	cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierPercent) / 100;
442		Blank line.
443	if(!shouldMaintain(game, playerIndex, game[0].board[i].property.condition, cost))	Condition: !shouldMaintain(game, playerIndex, square i.property.condition, cost) -> run the next block if true.
444	continue;	Skips to the next loop iteration.
445		Blank line.
446	if(game[0].players[playerIndex].cash < cost)	Condition: the current player.cash < cost -> run the next block if true.
447	continue;	Skips to the next loop iteration.
448		Blank line.
449	payMoney(game, playerIndex, cost);	Calls payMoney(game, playerIndex, cost). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
450		Blank line.
451	game[0].board[i].property.condition = 100;	Sets the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 100.
452	game[0].board[i].property.roundsSinceMaintenance = 0;	Sets the field roundsSinceMaintenance of the property on the board (i.e. Rounds since last maintenance.) = 0.
453		Blank line.
454	printf("\n%s performed maintenance on %s.\n",	Prints the message "\n%s performed maintenance on %s.\n".
455	game[0].players[playerIndex].name, game[0].board[i].name);	game[0].players[playerIndex].name, game[0].board[i].name);
456		Blank line.
457	printf("Maintenance Cost : LKR %d\n", cost);	Prints the message "Maintenance Cost : LKR %d\n".
458	}	Closes the current block.
459	}	Closes the current block.

Nested functions inside performMaintenance() - study these

- performMaintenance() (Rule-LK 27: restores a building's condition to 100% for a fee (hotel 8%, house 5%), if the)
- shouldMaintain() (Whether the player performs maintenance on a building.)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Rule-LK 27: restores a building's condition to 100% for a fee (hotel 8%, house 5%), if the strategy agrees.

10.17 renovateStructuralDamage()

```
void renovateStructuralDamage(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 460)

```
461  /* Rule-LK 29: fully restores a structurally damaged building. */
462  void renovateStructuralDamage(GameState game[], int playerIndex)
463  {
464      int i;
465      int replacementValue;
466      int cost;

468      for(i = 0; i < BOARD_SIZE; i++)
469      {
470          if(game[0].board[i].type != PROPERTY)
471              continue;

473          if(game[0].board[i].property.owner != playerIndex)
474              continue;

476          if(!game[0].board[i].property.structurallyDamaged)
477              continue;

479          if(game[0].board[i].property.hotel)
480              replacementValue = game[0].board[i].property.hotelCost;
481          else
482              replacementValue = game[0].board[i].property.houseCost;

484          cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;

486          if(game[0].players[playerIndex].cash < cost)
487              continue;

489          payMoney(game, playerIndex, cost);

491          game[0].board[i].property.purchasePrice = game[0].board[i].property.preDamagePurchasePrice;
492          game[0].board[i].property.baseRent = game[0].board[i].property.preDamageBaseRent;
493          game[0].board[i].property.condition = 100;
494          game[0].board[i].property.structurallyDamaged = 0;
495          game[0].board[i].property.maintenanceCostMultiplierPercent = 100;
496          game[0].board[i].property.roundsSinceMaintenance = 0;

498          printf("\n%s fully renovated %s after structural damage.\n",
499                game[0].players[playerIndex].name, game[0].board[i].name);

501          printf("Renovation Cost : LKR %d\n", cost);
502      }
503 }
```

Line-by-line explanation

Line	Code	Explanation
460		Blank line.
461	/* Rule-LK 29: fully restores a structurally damaged building. */	Comment: /* Rule-LK 29: fully restores a structurally damaged building. */
462	void renovateStructuralDamage(GameState game[], int playerIndex)	void renovateStructuralDamage(GameState game[], int playerIndex)
463	{	Opens a new block.
464	int i;	Declares int i.
465	int replacementValue;	Declares int replacementValue.
466	int cost;	Declares int cost.

467		Blank line.
468	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
469	{	Opens a new block.
470	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
471	continue;	Skips to the next loop iteration.
472		Blank line.
473	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
474	continue;	Skips to the next loop iteration.
475		Blank line.
476	if(!game[0].board[i].property.structurallyDamaged)	Condition: !square i.property.structurallyDamaged -> run the next block if true.
477	continue;	Skips to the next loop iteration.
478		Blank line.
479	if(game[0].board[i].property.hotel)	Condition: square i.property.hotel -> run the next block if true.
480	replacementValue = game[0].board[i].property.hotelCost;	replacementValue = game[0].board[i].property.hotelCost;
481	else	The alternative branch when the condition above is false.
482	replacementValue = game[0].board[i].property.houseCost;	replacementValue = game[0].board[i].property.houseCost;
483		Blank line.
484	cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;	cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;
485		Blank line.
486	if(game[0].players[playerIndex].cash < cost)	Condition: the current player.cash < cost -> run the next block if true.
487	continue;	Skips to the next loop iteration.
488		Blank line.
489	payMoney(game, playerIndex, cost);	Calls payMoney(game, playerIndex, cost). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
490		Blank line.
491	game[0].board[i].property.purchasePrice = game[0].board[i].property.preDamagePurchasePrice;	Sets the field purchasePrice of the property on the board (i.e. Base price to buy (inflation and modifiers can move this).) = game[0].board[i].property.preDamagePurchasePrice.
492	game[0].board[i].property.baseRent = game[0].board[i].property.preDamageBaseRent;	Sets the field baseRent of the property on the board (i.e. Rent with zero houses (Table multipliers on top).) = game[0].board[i].property.preDamageBaseRent.
493	game[0].board[i].property.condition = 100;	Sets the field condition of the property on the board (i.e. Building condition 0..100 (Table 3 rent mapping).) = 100.
494	game[0].board[i].property.structurallyDamaged = 0;	Sets the field structurallyDamaged of the property on the board (i.e. 1 when neglect (20+ rounds) caused structural damage.) = 0.
495	game[0].board[i].property.maintenanceCostMultiplierPercent = 100;	Sets the field maintenanceCostMultiplierPercent of the property on the board (i.e. Cost multiplier (150% after damage, else 100%).) = 100.
496	game[0].board[i].property.roundsSinceMaintenance = 0;	Sets the field roundsSinceMaintenance of the property on the board (i.e. Rounds since last maintenance.) = 0.
497		Blank line.
498	printf("\n%s fully renovated %s after structural damage.\n",	Prints the message "\n%s fully renovated %s after structural damage.\n".
499	game[0].players[playerIndex].name, game[0].board[i].name);	game[0].players[playerIndex].name, game[0].board[i].name);
500		Blank line.
501	printf("Renovation Cost : LKR %d\n", cost);	Prints the message "Renovation Cost : LKR %d\n".
502	}	Closes the current block.
503	}	Closes the current block.

Nested functions inside renovateStructuralDamage() - study these

- renovateStructuralDamage() (Rule-LK 29: fully restores a structurally damaged building at 150% of replacement value, r)

- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Rule-LK 29: fully restores a structurally damaged building at 150% of replacement value, restoring pre-damage price/rent.

Insurance policies, disasters, compensation, repairs, policy renewal and expiry.

The sub-topics below are the 14 functions implemented in insurance.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in insurance.c

11.1	propertyValue()	(source line 7)	p. 198
11.2	repairCost()	(source line 11)	p. 198
11.3	isCovered()	(source line 20)	p. 199
11.4	compensationPercent()	(source line 31)	p. 200
11.5	pickDisaster()	(source line 41)	p. 200
11.6	purchaseInsurance()	(source line 87)	p. 202
11.7	findPropertyToInsure()	(source line 124)	p. 204
11.8	findPropertyToRenew()	(source line 148)	p. 205
11.9	handleInsuranceVisit()	(source line 172)	p. 206
11.10	tryAutoRepair()	(source line 205)	p. 208
11.11	disasterName()	(source line 244)	p. 209
11.12	payCompensation()	(source line 259)	p. 210
11.13	triggerDisaster()	(source line 293)	p. 212
11.14	processInsuranceExpiry()	(source line 364)	p. 215

11.1 propertyValue()

```
int propertyValue(GameState game[], int propIndex)
```

Parameters: propIndex | Returns: Value in LKR.

The function, verbatim from the source (starts at line 7)

```
7 int propertyValue(GameState game[], int propIndex)
8 {
9     return currentMarketValue(game, propIndex);
10 }
```

Line-by-line explanation

Line	Code	Explanation
7	int propertyValue(GameState game[], int propIndex)	int propertyValue(GameState game[], int propIndex)
8	{	Opens a new block.
9	return currentMarketValue(game, propIndex);	Returns currentMarketValue(game, propIndex) immediately.
10	}	Closes the current block.

Nested functions inside propertyValue() - study these

- propertyValue() (A property's value = currentMarketValue.)
- currentMarketValue() (The real worth of a property: purchase price minus age depreciation, times every applicabl)

MAIN PURPOSE

A property's value = currentMarketValue.

11.2 repairCost()

```
int repairCost(GameState game[], int propIndex)
```

Parameters: propIndex | Returns: Cost in LKR.

The function, verbatim from the source (starts at line 11)

```
12 /* Assumption: repair costs 30% of the property's value. The
13    assignment doesn't give an exact figure, so this is used
14    consistently everywhere repair cost is needed. */
15 int repairCost(GameState game[], int propIndex)
16 {
```

```

17         return (propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100;
18     }

```

Line-by-line explanation

Line	Code	Explanation
11		Blank line.
12	<code>/* Assumption: repair costs 30% of the property's value. The</code>	Comment: <code>/* Assumption: repair costs 30% of the property's value. The</code>
13	<code>assignment doesn't give an exact figure, so this is used</code>	<code>assignment doesn't give an exact figure, so this is used</code>
14	<code>consistently everywhere repair cost is needed. */</code>	<code>consistently everywhere repair cost is needed. */</code>
15	<code>int repairCost(GameState game[], int propIndex)</code>	<code>int repairCost(GameState game[], int propIndex)</code>
16	<code>{</code>	Opens a new block.
17	<code>return (propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100;</code>	Returns <code>(propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100</code> immediately.
18	<code>}</code>	Closes the current block.

Nested functions inside repairCost() - study these

- `repairCost()` (Assumption: repairs cost 30% of a property's value.)
- `propertyValue()` (A property's value = `currentMarketValue`.)

MAIN PURPOSE

Assumption: repairs cost 30% of a property's value.

11.3 isCovered()

```
int isCovered(InsuranceType policy, DisasterType disaster)
```

Parameters: `policy`, `disaster` | Returns: 1 = covered, 0 = not.

The function, verbatim from the source (starts at line 20)

```

20     int isCovered(InsuranceType policy, DisasterType disaster)
21     {
22         if(policy == NO_INSURANCE)
23             return 0;
24
25         if(policy == BASIC_INSURANCE)
26             return (disaster == FIRE || disaster == FLOOD);
27
28         return 1; /* Comprehensive and Business Interruption cover everything */
29     }

```

Line-by-line explanation

Line	Code	Explanation
20	<code>int isCovered(InsuranceType policy, DisasterType disaster)</code>	<code>int isCovered(InsuranceType policy, DisasterType disaster)</code>
21	<code>{</code>	Opens a new block.
22	<code>if(policy == NO_INSURANCE)</code>	Condition: <code>policy == NO_INSURANCE</code> -> run the next block if true.
23	<code>return 0;</code>	Returns 0 immediately.
24		Blank line.
25	<code>if(policy == BASIC_INSURANCE)</code>	Condition: <code>policy == BASIC_INSURANCE</code> -> run the next block if true.
26	<code>return (disaster == FIRE disaster == FLOOD);</code>	Returns <code>(disaster == FIRE disaster == FLOOD)</code> immediately.
27		Blank line.
28	<code>return 1; /* Comprehensive and Business Interruption cover everything */</code>	Returns 1; <code>/* Comprehensive and Business Interruption cover everything */</code> immediately.
29	<code>}</code>	Closes the current block.

Nested functions inside isCovered() - study these

- `isCovered()` (Whether a policy covers a disaster type.)

MAIN PURPOSE

Whether a policy covers a disaster type.

11.4 compensationPercent()

```
int compensationPercent(InsuranceType policy)
```

Parameters: *policy* | Returns: *Percent (0, 80 or 100).*

The function, verbatim from the source (starts at line 31)

```
31  int compensationPercent(InsuranceType policy)
32  {
33      if(policy == BASIC_INSURANCE)
34          return 80;

36      if(policy == COMPREHENSIVE_INSURANCE || policy == BUSINESS_INTERRUPTION)
37          return 100;

39      return 0;
40  }
```

Line-by-line explanation

Line	Code	Explanation
31	int compensationPercent(InsuranceType policy)	int compensationPercent(InsuranceType policy)
32	{	Opens a new block.
33	if(policy == BASIC_INSURANCE)	Condition: policy == BASIC_INSURANCE -> run the next block if true.
34	return 80;	Returns 80 immediately.
35		Blank line.
36	if(policy == COMPREHENSIVE_INSURANCE policy == BUSINESS_INTERRUPTION)	Condition: policy == COMPREHENSIVE_INSURANCE policy == BUSINESS_INTERRUPTION -> run the next block if true.
37	return 100;	Returns 100 immediately.
38		Blank line.
39	return 0;	Returns 0 immediately.
40	}	Closes the current block.

Nested functions inside compensationPercent() - study these

- compensationPercent() (What fraction of the repair cost a policy pays.)

MAIN PURPOSE

What fraction of the repair cost a policy pays.

11.5 pickDisaster()

```
DisasterType pickDisaster(GameState game[])
```

Parameters: *none* | Returns: *DisasterType.*

The function, verbatim from the source (starts at line 41)

```
42  /* Picks a disaster. Every type is equally likely at base, but Heavy
43     Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */
44  DisasterType pickDisaster(GameState game[])
45  {
46      int weights[5];
47      int total;
48      int i;
49      int roll;
50      DisasterType result;

52      weights[FIRE] = DISASTER_WEIGHT;
53      weights[FLOOD] = DISASTER_WEIGHT;
54      weights[RIOT] = DISASTER_WEIGHT;
55      weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;
56      weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;
```

```

58     if(isModifierActive(game, MOD_FLOOD_RISK, -1, -1))
59         weights[FLOOD] += DISASTER_BOOST;

61     if(isModifierActive(game, MOD_RIOT_RISK, -1, -1))
62         weights[RIOT] += DISASTER_BOOST;

64     total = 0;

66     for(i = 0; i < 5; i++)
67         total += weights[i];

69     roll = rand() % total;

71     result = FIRE;

73     for(i = 0; i < 5; i++)
74     {
75         if(roll < weights[i])
76         {
77             result = (DisasterType)i;
78             break;
79         }

81         roll -= weights[i];
82     }

84     return result;
85 }

```

Line-by-line explanation

Line	Code	Explanation
41		Blank line.
42	/* Picks a disaster. Every type is equally likely at base, but Heavy	Comment: /* Picks a disaster. Every type is equally likely at base, but Heavy
43	Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */	Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */
44	DisasterType pickDisaster(GameState game[])	DisasterType pickDisaster(GameState game[])
45	{	Opens a new block.
46	int weights[5];	Declares int weights[5].
47	int total;	Declares int total.
48	int i;	Declares int i.
49	int roll;	Declares int roll.
50	DisasterType result;	DisasterType result;
51		Blank line.
52	weights[FIRE] = DISASTER_WEIGHT;	weights[FIRE] = DISASTER_WEIGHT;
53	weights[FLOOD] = DISASTER_WEIGHT;	weights[FLOOD] = DISASTER_WEIGHT;
54	weights[RIOT] = DISASTER_WEIGHT;	weights[RIOT] = DISASTER_WEIGHT;
55	weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;	weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;
56	weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;	weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;
57		Blank line.
58	if(isModifierActive(game, MOD_FLOOD_RISK, -1, -1))	Condition: isModifierActive(game, MOD_FLOOD_RISK, -1, -1) -> run the next block if true.
59	weights[FLOOD] += DISASTER_BOOST;	weights[FLOOD] += DISASTER_BOOST;
60		Blank line.
61	if(isModifierActive(game, MOD_RIOT_RISK, -1, -1))	Condition: isModifierActive(game, MOD_RIOT_RISK, -1, -1) -> run the next block if true.
62	weights[RIOT] += DISASTER_BOOST;	weights[RIOT] += DISASTER_BOOST;
63		Blank line.
64	total = 0;	total = 0;
65		Blank line.
66	for(i = 0; i < 5; i++)	Loop over all entries: init i = 0; run while i < 5; every loop i++.
67	total += weights[i];	total += weights[i];

68		Blank line.
69	roll = rand() % total;	roll = rand() % total;
70		Blank line.
71	result = FIRE;	result = FIRE;
72		Blank line.
73	for(i = 0; i < 5; i++)	Loop over all entries: init i = 0; run while i < 5; every loop i++.
74	{	Opens a new block.
75	if(roll < weights[i])	Condition: roll < weights[i] -> run the next block if true.
76	{	Opens a new block.
77	result = (DisasterType)i;	result = (DisasterType)i;
78	break;	Breaks out of the current loop or switch.
79	}	Closes the current block.
80		Blank line.
81	roll -= weights[i];	roll -= weights[i];
82	}	Closes the current block.
83		Blank line.
84	return result;	Returns result immediately.
85	}	Closes the current block.

Nested functions inside pickDisaster() - study these

- pickDisaster() (Weighted random disaster draw; all five start at weight 20, and active flood/riot risks add)
- isModifierActive() (Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active)

MAIN PURPOSE

Weighted random disaster draw; all five start at weight 20, and active flood/riot risks add +30.

11.6 purchaseInsurance()

```
void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
```

Parameters: playerIndex, propIndex, type | Returns: void.

The function, verbatim from the source (starts at line 87)

```
87 void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
88 {
89     int value;
90     int premium;
91     int premiumPercent;
92     char premiumBuf[32];

94     if(type == BASIC_INSURANCE)
95         premiumPercent = PREMIUM_BASIC_PERCENT;
96     else if(type == COMPREHENSIVE_INSURANCE)
97         premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;
98     else if(type == BUSINESS_INTERRUPTION)
99         premiumPercent = PREMIUM_INTERRUPTION_PERCENT;
100     else
101         return;

103     value = propertyValue(game, propIndex);
104     premium = (value * premiumPercent) / 100;

106     /* Rule-LK 34: premiums respond to insurance modifiers. */
107     premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;

109     if(game[0].players[playerIndex].cash < premium)
110         return;

112     payMoney(game, playerIndex, premium);

114     game[0].board[propIndex].property.insurance = type;
115     game[0].board[propIndex].property.insuranceRoundsLeft = INSURANCE_DURATION_ROUNDS;
```

```

117     printf("\n%s purchased insurance for %s.\n",
118           game[0].players[playerIndex].name,
119           game[0].board[propIndex].name);

121     formatLKR(premium, premiumBuf);
122     printf("Premium : LKR %s.\n", premiumBuf);
123 }

```

Line-by-line explanation

Line	Code	Explanation
87	void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)	void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
88	{	Opens a new block.
89	int value;	Declares int value.
90	int premium;	Declares int premium.
91	int premiumPercent;	Declares int premiumPercent.
92	char premiumBuf[32];	Declares char premiumBuf[32].
93		Blank line.
94	if(type == BASIC_INSURANCE)	Condition: type == BASIC_INSURANCE -> run the next block if true.
95	premiumPercent = PREMIUM_BASIC_PERCENT;	premiumPercent = PREMIUM_BASIC_PERCENT;
96	else if(type == COMPREHENSIVE_INSURANCE)	A further condition checked only if the previous one failed.
97	premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;	premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;
98	else if(type == BUSINESS_INTERRUPTION)	A further condition checked only if the previous one failed.
99	premiumPercent = PREMIUM_INTERRUPTION_PERCENT;	premiumPercent = PREMIUM_INTERRUPTION_PERCENT;
100	else	The alternative branch when the condition above is false.
101	return;	Returns the computed value immediately.
102		Blank line.
103	value = propertyValue(game, propIndex);	value = propertyValue(game, propIndex);
104	premium = (value * premiumPercent) / 100;	premium = (value * premiumPercent) / 100;
105		Blank line.
106	/* Rule-LK 34: premiums respond to insurance modifiers. */	Comment: /* Rule-LK 34: premiums respond to insurance modifiers. */
107	premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;	premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;
108		Blank line.
109	if(game[0].players[playerIndex].cash < premium)	Condition: the current player.cash < premium -> run the next block if true.
110	return;	Returns the computed value immediately.
111		Blank line.
112	payMoney(game, playerIndex, premium);	Calls payMoney(game, playerIndex, premium). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
113		Blank line.
114	game[0].board[propIndex].property.insurance = type;	Sets the field insurance of the property on the board (i.e. Currently held policy type.) = type.
115	game[0].board[propIndex].property.insuranceRoundsLeft = INSURANCE_DURATION_ROUNDS;	Sets the field insuranceRoundsLeft of the property on the board (i.e. Rounds remaining on the current policy.) = INSURANCE_DURATION_ROUNDS.
116		Blank line.
117	printf("\n%s purchased insurance for %s.\n",	Prints the message "\n%s purchased insurance for %s.\n".
118	game[0].players[playerIndex].name,	game[0].players[playerIndex].name,
119	game[0].board[propIndex].name);	game[0].board[propIndex].name);
120		Blank line.
121	formatLKR(premium, premiumBuf);	Calls formatLKR(premium, premiumBuf). (Formats an amount with thousands separators, e)
122	printf("Premium : LKR %s.\n", premiumBuf);	Prints the message "Premium : LKR %s.\n".
123	}	Closes the current block.

Nested functions inside purchaseInsurance() - study these

- purchaseInsurance() (Buys a policy: premium = value x 5/10/15% x insurance modifier; policy lasts 20 rounds.)
- propertyValue() (A property's value = currentMarketValue.)
- modifierMultiplier() (Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)
- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)

MAIN PURPOSE

Buys a policy: premium = value x 5/10/15% x insurance modifier; policy lasts 20 rounds.

11.7 findPropertyToInsure()

```
int findPropertyToInsure(GameState game[], int playerIndex)
```

Parameters: none | Returns: Square index, or -1.

The function, verbatim from the source (starts at line 124)

```
125  /* Any owned property can be insured (Section 2.8) - no longer just
126     developed ones. */
127  int findPropertyToInsure(GameState game[], int playerIndex)
128  {
129      int i;

131      for(i = 0; i < BOARD_SIZE; i++)
132      {
133          if(game[0].board[i].type != PROPERTY)
134              continue;

136          if(game[0].board[i].property.owner != playerIndex)
137              continue;

139          if(game[0].board[i].property.insurance != NO_INSURANCE)
140              continue;

142          if(desiredInsurance(game, playerIndex, i) != NO_INSURANCE)
143              return i;
144      }

146      return -1;
147  }
```

Line-by-line explanation

Line	Code	Explanation
124		Blank line.
125	/* Any owned property can be insured (Section 2.8) - no longer just	Comment: /* Any owned property can be insured (Section 2.8) - no longer just
126	developed ones. */	developed ones. */
127	int findPropertyToInsure(GameState game[], int playerIndex)	int findPropertyToInsure(GameState game[], int playerIndex)
128	{	Opens a new block.
129	int i;	Declares int i.
130		Blank line.
131	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
132	{	Opens a new block.
133	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
134	continue;	Skips to the next loop iteration.
135		Blank line.
136	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
137	continue;	Skips to the next loop iteration.
138		Blank line.

139	<code>if(game[0].board[i].property.insurance != NO_INSURANCE)</code>	Condition: square i.property.insurance != NO_INSURANCE -> run the next block if true.
140	<code>continue;</code>	Skips to the next loop iteration.
141		Blank line.
142	<code>if(desiredInsurance(game, playerIndex, i) != NO_INSURANCE)</code>	Condition: desiredInsurance(game, playerIndex, i) != NO_INSURANCE -> run the next block if true.
143	<code>return i;</code>	Returns i immediately.
144	<code>}</code>	Closes the current block.
145		Blank line.
146	<code>return -1;</code>	Returns -1 immediately.
147	<code>}</code>	Closes the current block.

Nested functions inside findPropertyToInsure() - study these

- findPropertyToInsure() (Finds the first owned, uninsured property the strategy actually wants to insure.)
- desiredInsurance() (Returns the policy type the player wants for a given property.)

MAIN PURPOSE

Finds the first owned, uninsured property the strategy actually wants to insure.

11.8 findPropertyToRenew()

```
int findPropertyToRenew(GameState game[], int playerIndex)
```

Parameters: none | Returns: Square index, or -1.

The function, verbatim from the source (starts at line 148)

```

149  /* Finds a policy close to expiring so it can be renewed. */
150  int findPropertyToRenew(GameState game[], int playerIndex)
151  {
152      int i;

154      for(i = 0; i < BOARD_SIZE; i++)
155      {
156          if(game[0].board[i].type != PROPERTY)
157              continue;

159          if(game[0].board[i].property.owner != playerIndex)
160              continue;

162          if(game[0].board[i].property.insurance == NO_INSURANCE)
163              continue;

165          if(game[0].board[i].property.insuranceRoundsLeft <= INSURANCE_RENEW_AT)
166              return i;
167      }

169      return -1;
170  }
```

Line-by-line explanation

Line	Code	Explanation
148		Blank line.
149	<code>/* Finds a policy close to expiring so it can be renewed. */</code>	Comment: /* Finds a policy close to expiring so it can be renewed. */
150	<code>int findPropertyToRenew(GameState game[], int playerIndex)</code>	int findPropertyToRenew(GameState game[], int playerIndex)
151	<code>{</code>	Opens a new block.
152	<code>int i;</code>	Declares int i.
153		Blank line.
154	<code>for(i = 0; i < BOARD_SIZE; i++)</code>	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
155	<code>{</code>	Opens a new block.
156	<code>if(game[0].board[i].type != PROPERTY)</code>	Condition: square i.type != PROPERTY -> run the next block if true.

157	<code>continue;</code>	Skips to the next loop iteration.
158		Blank line.
159	<code>if(game[0].board[i].property.owner != playerIndex)</code>	Condition: square i.property.owner != playerIndex -> run the next block if true.
160	<code>continue;</code>	Skips to the next loop iteration.
161		Blank line.
162	<code>if(game[0].board[i].property.insurance == NO_INSURANCE)</code>	Condition: square i.property.insurance == NO_INSURANCE -> run the next block if true.
163	<code>continue;</code>	Skips to the next loop iteration.
164		Blank line.
165	<code>if(game[0].board[i].property.insuranceRoundsLeft <= INSURANCE_RENEW_AT)</code>	Condition: square i.property.insuranceRoundsLeft <= INSURANCE_RENEW_AT -> run the next block if true.
166	<code>return i;</code>	Returns i immediately.
167	<code>}</code>	Closes the current block.
168		Blank line.
169	<code>return -1;</code>	Returns -1 immediately.
170	<code>}</code>	Closes the current block.

Nested functions inside findPropertyToRenew() - study these

- findPropertyToRenew() (Finds a policy close to expiring (<= 10 rounds left) for renewal.)

MAIN PURPOSE

Finds a policy close to expiring (<= 10 rounds left) for renewal.

11.9 handleInsuranceVisit()

```
void handleInsuranceVisit(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 172)

```

172 void handleInsuranceVisit(GameState game[], int playerIndex)
173 {
174     int propIndex;
175     InsuranceType type;

177     printf("\n%s landed on an Insurance square.\n",
178           game[0].players[playerIndex].name);

180     propIndex = findPropertyToRenew(game, playerIndex);

182     if(propIndex != -1)
183     {
184         type = game[0].board[propIndex].property.insurance;

186         purchaseInsurance(game, playerIndex, propIndex, type);

188         printf("The policy on %s was renewed for another %d rounds.\n",
189               game[0].board[propIndex].name, INSURANCE_DURATION_ROUNDS);
190         return;
191     }

193     propIndex = findPropertyToInsure(game, playerIndex);

195     if(propIndex == -1)
196     {
197         printf("No suitable property to insure right now.\n");
198         return;
199     }

201     type = desiredInsurance(game, playerIndex, propIndex);

203     purchaseInsurance(game, playerIndex, propIndex, type);
204 }
```

Line-by-line explanation

Line	Code	Explanation
172	void handleInsuranceVisit(GameState game[], int playerIndex)	void handleInsuranceVisit(GameState game[], int playerIndex)
173	{	Opens a new block.
174	int propIndex;	Declares int propIndex.
175	InsuranceType type;	InsuranceType type;
176		Blank line.
177	printf("\n%s landed on an Insurance square.\n",	Prints the message "\n%s landed on an Insurance square.\n".
178	game[0].players[playerIndex].name);	game[0].players[playerIndex].name);
179		Blank line.
180	propIndex = findPropertyToRenew(game, playerIndex);	propIndex = findPropertyToRenew(game, playerIndex);
181		Blank line.
182	if(propIndex != -1)	Condition: propIndex != -1 -> run the next block if true.
183	{	Opens a new block.
184	type = game[0].board[propIndex].property.insurance;	type = game[0].board[propIndex].property.insurance;
185		Blank line.
186	purchaseInsurance(game, playerIndex, propIndex,	Calls purchaseInsurance(game, playerIndex, propIndex, type). (Buys a
	type);	policy: premium = value x 5/10/15% x insurance modifier; policy lasts 20
		rounds)
187		Blank line.
188	printf("The policy on %s was renewed for another %d	Prints the message "The policy on %s was renewed for another %d
	rounds.\n",	rounds.\n".
189	game[0].board[propIndex].name,	game[0].board[propIndex].name, INSURANCE_DURATION_ROUNDS);
	INSURANCE_DURATION_ROUNDS);	
190	return;	Returns the computed value immediately.
191	}	Closes the current block.
192		Blank line.
193	propIndex = findPropertyToInsure(game, playerIndex);	propIndex = findPropertyToInsure(game, playerIndex);
194		Blank line.
195	if(propIndex == -1)	Condition: propIndex == -1 -> run the next block if true.
196	{	Opens a new block.
197	printf("No suitable property to insure right	Prints the message "No suitable property to insure right now.\n".
	now.\n");	
198	return;	Returns the computed value immediately.
199	}	Closes the current block.
200		Blank line.
201	type = desiredInsurance(game, playerIndex,	type = desiredInsurance(game, playerIndex, propIndex);
	propIndex);	
202		Blank line.
203	purchaseInsurance(game, playerIndex, propIndex,	Calls purchaseInsurance(game, playerIndex, propIndex, type). (Buys a
	type);	policy: premium = value x 5/10/15% x insurance modifier; policy lasts 20
		rounds)
204	}	Closes the current block.

Nested functions inside handleInsuranceVisit() - study these

- handleInsuranceVisit() (The Insurance square: renew an expiring policy first, otherwise buy a new one per the strategy)
- findPropertyToRenew() (Finds a policy close to expiring (<= 10 rounds left) for renewal.)
- purchaseInsurance() (Buys a policy: premium = value x 5/10/15% x insurance modifier; policy lasts 20 rounds.)
- findPropertyToInsure() (Finds the first owned, uninsured property the strategy actually wants to insure.)
- desiredInsurance() (Returns the policy type the player wants for a given property.)

MAIN PURPOSE

The Insurance square: renew an expiring policy first, otherwise buy a new one per the strategy.

11.10 tryAutoRepair()

```
void tryAutoRepair(GameState game[], int playerIndex)
```

Parameters: *playerIndex* | Returns: *void*.

The function, verbatim from the source (starts at line 205)

```
206  /* Rule-LK 11: fixes a disaster-damaged building automatically once
207     the owner can afford it. Called at the start of every turn. */
208  void tryAutoRepair(GameState game[], int playerIndex)
209  {
210      int i;

212      for(i = 0; i < BOARD_SIZE; i++)
213      {
214          if(game[0].board[i].type != PROPERTY)
215              continue;

217          if(game[0].board[i].property.owner != playerIndex)
218              continue;

220          if(!game[0].board[i].property.damaged)
221              continue;

223          if(game[0].players[playerIndex].cash >= game[0].board[i].property.repairCostOwed)
224          {
225              int cost;
226              char costBuf[32];

228              cost = game[0].board[i].property.repairCostOwed;

230              formatLKR(cost, costBuf);

232              payMoney(game, playerIndex, cost);

234              printf("\n%s repaired %s. It can collect rent again.\n",
235                  game[0].players[playerIndex].name, game[0].board[i].name);

237              printf("Repair Cost : LKR %s.\n", costBuf);

239              game[0].board[i].property.damaged = 0;
240              game[0].board[i].property.repairCostOwed = 0;
241          }
242      }
243  }
```

Line-by-line explanation

Line	Code	Explanation
205		Blank line.
206	/* Rule-LK 11: fixes a disaster-damaged building automatically once	Comment: /* Rule-LK 11: fixes a disaster-damaged building automatically once
207	the owner can afford it. Called at the start of every turn. */	the owner can afford it. Called at the start of every turn. */
208	void tryAutoRepair(GameState game[], int playerIndex)	void tryAutoRepair(GameState game[], int playerIndex)
209	{	Opens a new block.
210	int i;	Declares int i.
211		Blank line.
212	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
213	{	Opens a new block.
214	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
215	continue;	Skips to the next loop iteration.
216		Blank line.

217	if(game[0].board[i].property.owner != playerIndex)	Condition: square i.property.owner != playerIndex -> run the next block if true.
218	continue;	Skips to the next loop iteration.
219		Blank line.
220	if(!game[0].board[i].property.damaged)	Condition: !square i.property.damaged -> run the next block if true.
221	continue;	Skips to the next loop iteration.
222		Blank line.
223	if(game[0].players[playerIndex].cash >= game[0].board[i].property.repairCostOwed)	Condition: the current player.cash >= square i.property.repairCostOwed -> run the next block if true.
224	{	Opens a new block.
225	int cost;	Declares int cost.
226	char costBuf[32];	Declares char costBuf[32].
227		Blank line.
228	cost = game[0].board[i].property.repairCostOwed;	cost = game[0].board[i].property.repairCostOwed;
229		Blank line.
230	formatLKR(cost, costBuf);	Calls formatLKR(cost, costBuf). (Formats an amount with thousands separators, e)
231		Blank line.
232	payMoney(game, playerIndex, cost);	Calls payMoney(game, playerIndex, cost). (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated)
233		Blank line.
234	printf("\n%s repaired %. It can collect rent again.\n",	Prints the message "\n%s repaired %. It can collect rent again.\n".
235	game[0].players[playerIndex].name, game[0].board[i].name);	game[0].players[playerIndex].name, game[0].board[i].name);
236		Blank line.
237	printf("Repair Cost : LKR %s.\n", costBuf);	Prints the message "Repair Cost : LKR %s.\n".
238		Blank line.
239	game[0].board[i].property.damaged = 0;	Sets the field damaged of the property on the board (i.e. 1 after a disaster until repaired (no rent).) = 0.
240	game[0].board[i].property.repairCostOwed = 0;	Sets the field repairCostOwed of the property on the board (i.e. Remaining repair cost after insurance compensation.) = 0.
241	}	Closes the current block.
242	}	Closes the current block.
243	}	Closes the current block.

Nested functions inside tryAutoRepair() - study these

- tryAutoRepair() (Rule-LK 11: at the start of a turn, repairs any disaster-damaged building the owner can afford)
- formatLKR() (Formats an amount with thousands separators, e.g. 30000 -> "30,000".)
- payMoney() (Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped pr)

MAIN PURPOSE

Rule-LK 11: at the start of a turn, repairs any disaster-damaged building the owner can afford.

11.11 disasterName()

```
void disasterName(DisasterType disaster, char out[])
```

Parameters: disaster, out | Returns: void.

The function, verbatim from the source (starts at line 244)

```
245  /* Writes the name of a disaster into out. */
246  void disasterName(DisasterType disaster, char out[])
247  {
248      if(disaster == FIRE)
249          strcpy(out, "Fire");
250      else if(disaster == FLOOD)
251          strcpy(out, "Flood");
```

```

252     else if(disaster == RIOT)
253         strcpy(out, "Riot");
254     else if(disaster == BUILDING_COLLAPSE)
255         strcpy(out, "Building Collapse");
256     else
257         strcpy(out, "Electrical Failure");
258 }

```

Line-by-line explanation

Line	Code	Explanation
244		Blank line.
245	<code>/* Writes the name of a disaster into out. */</code>	Comment: <code>/* Writes the name of a disaster into out. */</code>
246	<code>void disasterName(DisasterType disaster, char out[])</code>	<code>void disasterName(DisasterType disaster, char out[])</code>
247	<code>{</code>	Opens a new block.
248	<code>if(disaster == FIRE)</code>	Condition: <code>disaster == FIRE</code> -> run the next block if true.
249	<code>strcpy(out, "Fire");</code>	Calls <code>strcpy(out, "Fire")</code> .
250	<code>else if(disaster == FLOOD)</code>	A further condition checked only if the previous one failed.
251	<code>strcpy(out, "Flood");</code>	Calls <code>strcpy(out, "Flood")</code> .
252	<code>else if(disaster == RIOT)</code>	A further condition checked only if the previous one failed.
253	<code>strcpy(out, "Riot");</code>	Calls <code>strcpy(out, "Riot")</code> .
254	<code>else if(disaster == BUILDING_COLLAPSE)</code>	A further condition checked only if the previous one failed.
255	<code>strcpy(out, "Building Collapse");</code>	Calls <code>strcpy(out, "Building Collapse")</code> .
256	<code>else</code>	The alternative branch when the condition above is false.
257	<code>strcpy(out, "Electrical Failure");</code>	Calls <code>strcpy(out, "Electrical Failure")</code> .
258	<code>}</code>	Closes the current block.

Nested functions inside `disasterName()` - study these

- `disasterName()` (Writes the human name of a disaster into out.)

MAIN PURPOSE

Writes the human name of a disaster into out.

11.12 `payCompensation()`

```
int payCompensation(GameState game[], int chosen, int owner, int cost)
```

Parameters: *chosen, owner, cost* | Returns: *Compensation actually paid.*

The function, verbatim from the source (starts at line 259)

```

260     /* Pays an insured player's compensation for a disaster and returns
261        how much they received. */
262     int payCompensation(GameState game[], int chosen, int owner, int cost)
263     {
264         int compensation;

266         compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;

268         /* Political Unrest inflates Business Interruption claims (Rule-LK 34). */
269         if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION &&
270            isModifierActive(game, MOD_BI_CLAIMS, -1, -1))
271         {
272             compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;
273         }

275         receiveMoney(game, owner, compensation);

277         printf("Insurance Claim Approved.\n");
278         printf("Compensation Paid : LKR %d\n", compensation);

280         /* Business Interruption also covers 5 rounds of lost rental
281            income - the repair cost is fully paid above, but the
282            property still earns nothing for 5 rounds. */
283         if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION)
284         {

```

```

285         game[0].board[chosen].property.lostIncomeRoundsLeft = BI_LOST_INCOME_ROUNDS;
286         printf("%s will earn no rent for the next %d rounds "
287             "(Business Interruption).\n", game[0].board[chosen].name,
288             BI_LOST_INCOME_ROUNDS);
289     }

291     return compensation;
292 }

```

Line-by-line explanation

Line	Code	Explanation
259		Blank line.
260	<code>/* Pays an insured player's compensation for a disaster and returns</code>	Comment: <code>/* Pays an insured player's compensation for a disaster and returns</code>
261	<code>how much they received. */</code>	<code>how much they received. */</code>
262	<code>int payCompensation(GameState game[], int chosen, int owner, int cost)</code>	<code>int payCompensation(GameState game[], int chosen, int owner, int cost)</code>
263	<code>{</code>	Opens a new block.
264	<code>int compensation;</code>	Declares <code>int compensation</code> .
265		Blank line.
266	<code>compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;</code>	<code>compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;</code>
267		Blank line.
268	<code>/* Political Unrest inflates Business Interruption claims (Rule-LK 34). */</code>	Comment: <code>/* Political Unrest inflates Business Interruption claims (Rule-LK 34). */</code>
269	<code>if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION &&</code>	Condition: <code>-></code> run the next block if true.
270	<code>isModifierActive(game, MOD_BI_CLAIMS, -1, -1))</code>	<code>Calls isModifierActive(game, MOD_BI_CLAIMS, -1, -1)). (Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active)</code>
271	<code>{</code>	Opens a new block.
272	<code>compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;</code>	<code>compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;</code>
273	<code>}</code>	Closes the current block.
274		Blank line.
275	<code>receiveMoney(game, owner, compensation);</code>	<code>Calls receiveMoney(game, owner, compensation). (Adds money to a player, but only if they are not bankrupt)</code>
276		Blank line.
277	<code>printf("Insurance Claim Approved.\n");</code>	<code>Prints the message "Insurance Claim Approved.\n".</code>
278	<code>printf("Compensation Paid : LKR %d\n", compensation);</code>	<code>Prints the message "Compensation Paid : LKR %d\n".</code>
279		Blank line.
280	<code>/* Business Interruption also covers 5 rounds of lost rental</code>	Comment: <code>/* Business Interruption also covers 5 rounds of lost rental</code>
281	<code>income - the repair cost is fully paid above, but the</code>	<code>income - the repair cost is fully paid above, but the</code>
282	<code>property still earns nothing for 5 rounds. */</code>	<code>property still earns nothing for 5 rounds. */</code>
283	<code>if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION)</code>	Condition: <code>game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION -></code> run the next block if true.
284	<code>{</code>	Opens a new block.
285	<code>game[0].board[chosen].property.lostIncomeRoundsLeft = BI_LOST_INCOME_ROUNDS;</code>	<code>Sets the field lostIncomeRoundsLeft of the property on the board (i.e. Business Interruption: rounds with no rent.) = BI_LOST_INCOME_ROUNDS.</code>
286	<code>printf("%s will earn no rent for the next %d rounds "</code>	<code>Prints the message "%s will earn no rent for the next %d rounds ".</code>
287	<code>"(Business Interruption).\n", game[0].board[chosen].name,</code>	<code>"(Business Interruption).\n", game[0].board[chosen].name,</code>
288	<code>BI_LOST_INCOME_ROUNDS);</code>	<code>BI_LOST_INCOME_ROUNDS);</code>
289	<code>}</code>	Closes the current block.

290

Blank line.

291 return compensation;

Returns compensation immediately.

292 }

Closes the current block.

Nested functions inside payCompensation() - study these

- payCompensation() (Pays the insured owner compensation (80%/100%), inflating BI claims x150% during Political)
- compensationPercent() (What fraction of the repair cost a policy pays.)
- isModifierActive() (Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active)
- receiveMoney() (Adds money to a player, but only if they are not bankrupt.)

MAIN PURPOSE

Pays the insured owner compensation (80%/100%), inflating BI claims x150% during Political Unrest, and sets 5 rounds of lost income for BI policies.

11.13 triggerDisaster()

```
void triggerDisaster(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 293)

```
294   /* Rule-LK 10: every 10 rounds, one random developed property may
295    be hit by a disaster. */
296   void triggerDisaster(GameState game[])
297   {
298       int developed[BOARD_SIZE];
299       int developedCount;
300       int i;
301       int chosen;
302       DisasterType disaster;
303       int owner;
304       int cost;
305       int compensation;
306       char name[32];

308       developedCount = 0;

310       for(i = 0; i < BOARD_SIZE; i++)
311       {
312           if(game[0].board[i].type == PROPERTY &&
313            (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
314           {
315               developed[developedCount] = i;
316               developedCount++;
317           }
318       }

320       if(developedCount == 0)
321           return;

323       chosen = developed[rand() % developedCount];

325       disaster = pickDisaster(game);

327       owner = game[0].board[chosen].property.owner;

329       printf("\n*** DISASTER ***\n");

331       disasterName(disaster, name);
332       printf("%s occurred.\n", name);

334       printf("Affected Property : %s (owner : %s)\n",
335           game[0].board[chosen].name, game[0].players[owner].name);

337       cost = repairCost(game, chosen);
```

```

339     if(isCovered(game[0].board[chosen].property.insurance, disaster))
340     {
341         compensation = payCompensation(game, chosen, owner, cost);
342     }
343     else
344     {
345         compensation = 0;
346         game[0].players[owner].sufferedLoss = 1;

348         printf("Property was NOT insured against this disaster.\n");
349     }

351     game[0].board[chosen].property.repairCostOwed = cost - compensation;

353     if(game[0].board[chosen].property.repairCostOwed < 0)
354         game[0].board[chosen].property.repairCostOwed = 0;

356     if(game[0].board[chosen].property.repairCostOwed > 0)
357     {
358         game[0].board[chosen].property.damaged = 1;
359         printf("%s is damaged and stops earning rent until repaired.\n",
360             game[0].board[chosen].name);
361     }
362 }

```

Line-by-line explanation

Line	Code	Explanation
293		Blank line.
294	/* Rule-LK 10: every 10 rounds, one random developed property may	Comment: /* Rule-LK 10: every 10 rounds, one random developed property may
295	be hit by a disaster. */	be hit by a disaster. */
296	void triggerDisaster(GameState game[])	void triggerDisaster(GameState game[])
297	{	Opens a new block.
298	int developed[BOARD_SIZE];	Declares int developed[BOARD_SIZE].
299	int developedCount;	Declares int developedCount.
300	int i;	Declares int i.
301	int chosen;	Declares int chosen.
302	DisasterType disaster;	DisasterType disaster;
303	int owner;	Declares int owner.
304	int cost;	Declares int cost.
305	int compensation;	Declares int compensation.
306	char name[32];	Declares char name[32].
307		Blank line.
308	developedCount = 0;	developedCount = 0;
309		Blank line.
310	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
311	{	Opens a new block.
312	if(game[0].board[i].type == PROPERTY &&	Condition: -> run the next block if true.
313	(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel))	(game[0].board[i].property.houses > 0 game[0].board[i].property.hotel))
314	{	Opens a new block.
315	developed[developedCount] = i;	developed[developedCount] = i;
316	developedCount++;	developedCount++;
317	}	Closes the current block.
318	}	Closes the current block.
319		Blank line.
320	if(developedCount == 0)	Condition: developedCount == 0 -> run the next block if true.
321	return;	Returns the computed value immediately.
322		Blank line.
323	chosen = developed[rand() % developedCount];	chosen = developed[rand() % developedCount];
324		Blank line.

325	<code>disaster = pickDisaster(game);</code>	<code>disaster = pickDisaster(game);</code>
326		Blank line.
327	<code>owner = game[0].board[chosen].property.owner;</code>	<code>owner = game[0].board[chosen].property.owner;</code>
328		Blank line.
329	<code>printf("\n*** DISASTER ***\n");</code>	Prints the message <code>"\n*** DISASTER ***\n"</code> .
330		Blank line.
331	<code>disasterName(disaster, name);</code>	Calls <code>disasterName(disaster, name)</code> . (Writes the human name of a disaster into out)
332	<code>printf("%s occurred.\n", name);</code>	Prints the message <code>"%s occurred.\n"</code> .
333		Blank line.
334	<code>printf("Affected Property : %s (owner : %s)\n",</code>	Prints the message <code>"Affected Property : %s (owner : %s)\n"</code> .
335	<code>game[0].board[chosen].name,</code>	<code>game[0].board[chosen].name, game[0].players[owner].name);</code>
336	<code>game[0].players[owner].name);</code>	
337		Blank line.
338	<code>cost = repairCost(game, chosen);</code>	<code>cost = repairCost(game, chosen);</code>
339		Blank line.
340	<code>if(isCovered(game[0].board[chosen].property.insurance,</code>	Condition: <code>isCovered(game[0].board[chosen].property.insurance, disaster)</code>
341	<code>e, disaster))</code>	-> run the next block if true.
342	<code>{</code>	Opens a new block.
343	<code>compensation = payCompensation(game, chosen, owner,</code>	<code>compensation = payCompensation(game, chosen, owner, cost);</code>
344	<code>cost);</code>	
345	<code>}</code>	Closes the current block.
346	<code>else</code>	The alternative branch when the condition above is false.
347	<code>{</code>	Opens a new block.
348	<code>compensation = 0;</code>	<code>compensation = 0;</code>
349	<code>game[0].players[owner].sufferedLoss = 1;</code>	Sets the field <code>sufferedLoss</code> of the player (i.e. 1 after an uninsured disaster (Risk Taker now insures).) = 1.
350		Blank line.
351	<code>printf("Property was NOT insured against this</code>	Prints the message <code>"Property was NOT insured against this disaster.\n"</code> .
352	<code>disaster.\n");</code>	
353	<code>}</code>	Closes the current block.
354		Blank line.
355	<code>game[0].board[chosen].property.repairCostOwed = cost</code>	Sets the field <code>repairCostOwed</code> of the property on the board (i.e. Remaining repair cost after insurance compensation.) = <code>cost - compensation</code> .
356	<code>- compensation;</code>	
357		Blank line.
358	<code>if(game[0].board[chosen].property.repairCostOwed <</code>	Condition: <code>game[0].board[chosen].property.repairCostOwed < 0</code> -> run the next block if true.
359	<code>0)</code>	
360	<code>game[0].board[chosen].property.repairCostOwed = 0;</code>	Sets the field <code>repairCostOwed</code> of the property on the board (i.e. Remaining repair cost after insurance compensation.) = 0.
361		Blank line.
362	<code>if(game[0].board[chosen].property.repairCostOwed ></code>	Condition: <code>game[0].board[chosen].property.repairCostOwed > 0</code> -> run the next block if true.
363	<code>0)</code>	
364	<code>{</code>	Opens a new block.
365	<code>game[0].board[chosen].property.damaged = 1;</code>	Sets the field <code>damaged</code> of the property on the board (i.e. 1 after a disaster until repaired (no rent).) = 1.
366		
367	<code>printf("%s is damaged and stops earning rent until</code>	Prints the message <code>"%s is damaged and stops earning rent until repaired.\n"</code> .
368	<code>repaired.\n",</code>	
369	<code>game[0].board[chosen].name);</code>	<code>game[0].board[chosen].name);</code>
370	<code>}</code>	Closes the current block.
371	<code>}</code>	Closes the current block.

Nested functions inside triggerDisaster() - study these

- `triggerDisaster()` (Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owne)
- `pickDisaster()` (Weighted random disaster draw; all five start at weight 20, and active flood/riot risks ad)
- `disasterName()` (Writes the human name of a disaster into out.)
- `repairCost()` (Assumption: repairs cost 30% of a property's value.)
- `isCovered()` (Whether a policy covers a disaster type.)

- payCompensation() (Pays the insured owner compensation (80%/100%), inflating BI claims x150% during Political)

MAIN PURPOSE

Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owners get compensation, uninsured owners are marked sufferedLoss and own the full repair cost.

11.14 processInsuranceExpiry()

```
void processInsuranceExpiry(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 364)

```
364 void processInsuranceExpiry(GameState game[])
365 {
366     int i;
367
368     for(i = 0; i < BOARD_SIZE; i++)
369     {
370         if(game[0].board[i].type != PROPERTY)
371             continue;
372
373         if(game[0].board[i].property.insurance == NO_INSURANCE)
374             continue;
375
376         game[0].board[i].property.insuranceRoundsLeft--;
377
378         if(game[0].board[i].property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING)
379         {
380             printf("\nInsurance policy on %s expires in %d rounds.\n",
381                 game[0].board[i].name, INSURANCE_EXPIRY_WARNING);
382         }
383
384         if(game[0].board[i].property.insuranceRoundsLeft <= 0)
385         {
386             printf("\nInsurance policy on %s has expired.\n",
387                 game[0].board[i].name);
388
389             game[0].board[i].property.insurance = NO_INSURANCE;
390         }
391     }
392 }
```

Line-by-line explanation

Line	Code	Explanation
364	void processInsuranceExpiry(GameState game[])	void processInsuranceExpiry(GameState game[])
365	{	Opens a new block.
366	int i;	Declares int i.
367		Blank line.
368	for(i = 0; i < BOARD_SIZE; i++)	Loop over all entries: init i = 0; run while i < BOARD_SIZE; every loop i++.
369	{	Opens a new block.
370	if(game[0].board[i].type != PROPERTY)	Condition: square i.type != PROPERTY -> run the next block if true.
371	continue;	Skips to the next loop iteration.
372		Blank line.
373	if(game[0].board[i].property.insurance == NO_INSURANCE)	Condition: square i.property.insurance == NO_INSURANCE -> run the next block if true.
374	continue;	Skips to the next loop iteration.
375		Blank line.
376	game[0].board[i].property.insuranceRoundsLeft--;	game[0].board[i].property.insuranceRoundsLeft--;
377		Blank line.
378	if(game[0].board[i].property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING)	Condition: square i.property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING -> run the next block if true.
379	{	Opens a new block.

<pre> 380 printf("\nInsurance policy on %s expires in %d rounds.\n", 381 game[0].board[i].name, INSURANCE_EXPIRY_WARNING); 382 } 383 384 if(game[0].board[i].property.insuranceRoundsLeft <= 0) 385 { 386 printf("\nInsurance policy on %s has expired.\n", 387 game[0].board[i].name); 388 389 game[0].board[i].property.insurance = NO_INSURANCE; 390 } 391 } 392 }</pre>	<pre> Prints the message "\nInsurance policy on %s expires in %d rounds.\n". game[0].board[i].name, INSURANCE_EXPIRY_WARNING); Closes the current block. Blank line. Condition: square i.property.insuranceRoundsLeft <= 0 -> run the next block if true. Opens a new block. Prints the message "\nInsurance policy on %s has expired.\n". game[0].board[i].name); Blank line. Sets the field insurance of the property on the board (i.e. Currently held policy type.) = NO_INSURANCE. Closes the current block. Closes the current block. Closes the current block.</pre>
--	--

Nested functions inside processInsuranceExpiry() - study these

- processInsuranceExpiry() (Counts every policy down each round; warns at 3 rounds and expires (removes) at 0.)

MAIN PURPOSE

Counts every policy down each round; warns at 3 rounds and expires (removes) at 0.

Property market reviews (boom/decline every 10 rounds), cooldowns, Regional Development cards and the market display.

The sub-topics below are the 12 functions implemented in market.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in market.c

12.1	initMarket() (source line 7)	p. 217
12.2	groupName() (source line 18)	p. 218
12.3	pickEligibleGroup() (source line 39)	p. 219
12.4	reviewPropertyMarket() (source line 67)	p. 220
12.5	drawRegionalCard() (source line 112)	p. 222
12.6	printSectionHeader() (source line 196)	p. 226
12.7	printBoomDecline() (source line 203)	p. 226
12.8	describeRegionalCondition() (source line 211)	p. 227
12.9	printRegionalSection() (source line 250)	p. 229
12.10	describeOtherCondition() (source line 285)	p. 230
12.11	printOtherConditions() (source line 330)	p. 232
12.12	displayMarketConditions() (source line 362)	p. 234

12.1 initMarket()

```
void initMarket(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 7)

```
7 void initMarket(GameState game[])
8 {
9     int i;

11    for(i = 0; i < NO_GROUP; i++)
12        game[0].economy.groupCooldownUntilRound[i] = 0;

14    game[0].economy.lastBoomGroup = -1;
15    game[0].economy.lastDeclineGroup = -1;
16 }
```

Line-by-line explanation

Line	Code	Explanation
7	void initMarket(GameState game[])	void initMarket(GameState game[])
8	{	Opens a new block.
9	int i;	Declares int i.
10		Blank line.
11	for(i = 0; i < NO_GROUP; i++)	Loop over all entries: init i = 0; run while i < NO_GROUP; every loop i++.
12	game[0].economy.groupCooldownUntilRound[i] = 0;	Sets the field groupCooldownUntilRound of the shared economy (i.e. Per group: earliest round it may boom/decline again.) = 0.
13		Blank line.
14	game[0].economy.lastBoomGroup = -1;	Sets the field lastBoomGroup of the shared economy (i.e. Group boosted by the most recent market review.) = -1.
15	game[0].economy.lastDeclineGroup = -1;	Sets the field lastDeclineGroup of the shared economy (i.e. Group declined by the most recent market review.) = -1.
16	}	Closes the current block.

Nested functions inside initMarket() - study these

- initMarket() (Resets market state: all group cooldowns 0, no last boom/decline.)

MAIN PURPOSE

Resets market state: all group cooldowns 0, no last boom/decline.

12.2 groupName()

```
void groupName(PropertyGroup group, char buffer[])
```

Parameters: group, buffer | Returns: void.

The function, verbatim from the source (starts at line 18)

```
18 void groupName(PropertyGroup group, char buffer[])
19 {
20     if(group == BROWN)
21         strcpy(buffer, "Brown");
22     else if(group == LIGHT_BLUE)
23         strcpy(buffer, "Light Blue");
24     else if(group == PINK)
25         strcpy(buffer, "Pink");
26     else if(group == ORANGE)
27         strcpy(buffer, "Orange");
28     else if(group == RED)
29         strcpy(buffer, "Red");
30     else if(group == YELLOW)
31         strcpy(buffer, "Yellow");
32     else if(group == GREEN)
33         strcpy(buffer, "Green");
34     else if(group == DARK_BLUE)
35         strcpy(buffer, "Dark Blue");
36     else
37         strcpy(buffer, "Unknown");
38 }
```

Line-by-line explanation

Line	Code	Explanation
18	void groupName(PropertyGroup group, char buffer[])	void groupName(PropertyGroup group, char buffer[])
19	{	Opens a new block.
20	if(group == BROWN)	Condition: group == BROWN -> run the next block if true.
21	strcpy(buffer, "Brown");	Calls strcpy(buffer, "Brown").
22	else if(group == LIGHT_BLUE)	A further condition checked only if the previous one failed.
23	strcpy(buffer, "Light Blue");	Calls strcpy(buffer, "Light Blue").
24	else if(group == PINK)	A further condition checked only if the previous one failed.
25	strcpy(buffer, "Pink");	Calls strcpy(buffer, "Pink").
26	else if(group == ORANGE)	A further condition checked only if the previous one failed.
27	strcpy(buffer, "Orange");	Calls strcpy(buffer, "Orange").
28	else if(group == RED)	A further condition checked only if the previous one failed.
29	strcpy(buffer, "Red");	Calls strcpy(buffer, "Red").
30	else if(group == YELLOW)	A further condition checked only if the previous one failed.
31	strcpy(buffer, "Yellow");	Calls strcpy(buffer, "Yellow").
32	else if(group == GREEN)	A further condition checked only if the previous one failed.
33	strcpy(buffer, "Green");	Calls strcpy(buffer, "Green").
34	else if(group == DARK_BLUE)	A further condition checked only if the previous one failed.
35	strcpy(buffer, "Dark Blue");	Calls strcpy(buffer, "Dark Blue").
36	else	The alternative branch when the condition above is false.
37	strcpy(buffer, "Unknown");	Calls strcpy(buffer, "Unknown").
38	}	Closes the current block.

Nested functions inside groupName() - study these

- groupName() (Writes the display name of a colour group.)

MAIN PURPOSE

Writes the display name of a colour group.

12.3 pickEligibleGroup()

```
PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
```

Parameters: *currentRound*, *avoid1*, *avoid2* | Returns: *PropertyGroup*.

The function, verbatim from the source (starts at line 39)

```
40  /* Picks a random group that isn't on cooldown and isn't one of the
41     two groups to avoid (Rule-LK 30, 33). */
42  PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
43  {
44      int candidates[NO_GROUP];
45      int count;
46      int g;
47
48      count = 0;
49
50      for(g = 0; g < NO_GROUP; g++)
51      {
52          if(g == avoid1 || g == avoid2)
53              continue;
54
55          if(game[0].economy.groupCooldownUntilRound[g] > currentRound)
56              continue;
57
58          candidates[count] = g;
59          count++;
60      }
61
62      if(count == 0)
63          return (PropertyGroup)((avoid1 + 1) % NO_GROUP); /* rare fallback */
64
65      return (PropertyGroup)candidates[rand() % count];
66  }
```

Line-by-line explanation

Line	Code	Explanation
39		Blank line.
40	/* Picks a random group that isn't on cooldown and isn't one of the	Comment: /* Picks a random group that isn't on cooldown and isn't one of the
41	two groups to avoid (Rule-LK 30, 33). */	two groups to avoid (Rule-LK 30, 33). */
42	PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)	PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
43	{	Opens a new block.
44	int candidates[NO_GROUP];	Declares int candidates[NO_GROUP].
45	int count;	Declares int count.
46	int g;	Declares int g.
47		Blank line.
48	count = 0;	count = 0;
49		Blank line.
50	for(g = 0; g < NO_GROUP; g++)	Loop over all entries: init g = 0; run while g < NO_GROUP; every loop g++.
51	{	Opens a new block.
52	if(g == avoid1 g == avoid2)	Condition: g == avoid1 g == avoid2 -> run the next block if true.
53	continue;	Skips to the next loop iteration.
54		Blank line.
55	if(game[0].economy.groupCooldownUntilRound[g] > currentRound)	Condition: game[0].economy.groupCooldownUntilRound[g] > currentRound -> run the next block if true.
56	continue;	Skips to the next loop iteration.
57		Blank line.
58	candidates[count] = g;	candidates[count] = g;
59	count++;	count++;

60	}	Closes the current block.
61		Blank line.
62	if(count == 0)	Condition: count == 0 -> run the next block if true.
63	return (PropertyGroup)((avoid1 + 1) % NO_GROUP); /* rare fallback */	Returns (PropertyGroup)((avoid1 + 1) % NO_GROUP); /* rare fallback */ immediately.
64		Blank line.
65	return (PropertyGroup)candidates[rand() % count];	Returns (PropertyGroup)candidates[rand() % count] immediately.
66	}	Closes the current block.

Nested functions inside pickEligibleGroup() - study these

- pickEligibleGroup() (Picks a random group not on cooldown and not the groups to avoid.)

MAIN PURPOSE

Picks a random group not on cooldown and not the groups to avoid.

12.4 reviewPropertyMarket()

```
void reviewPropertyMarket(GameState game[], int currentRound)
```

Parameters: currentRound | Returns: void.

The function, verbatim from the source (starts at line 67)

```
68  /* Rule-LK 30-34: every 10 rounds, one group booms and a different
69     one declines, each for 10 rounds. All effects are temporary
70     modifiers (Rule-LK 34) so no base value or cost is ever changed. */
71  void reviewPropertyMarket(GameState game[], int currentRound)
72  {
73      PropertyGroup boomGroup;
74      PropertyGroup declineGroup;
75      char boomName[12];
76      char declineName[12];

78      boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);
79      declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup, boomGroup);

81      groupName(boomGroup, boomName);
82      groupName(declineGroup, declineName);

84      printf("\n=== Property Market Review ===\n");
85      printf("Market Boom : %s (values +20%%, rent +25%%) for 10 rounds.\n", boomName);
86      printf("Market Decline : %s (values -15%%, rent -20%%) for 10 rounds.\n", declineName);

88      addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM);
89      addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM);

91      addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE);
92      addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE);

94      /* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,
95         direct purchase prices rise by 15% and construction costs rise
96         by 10%; the decline group's mortgage values fall by 10% and
97         auction starting prices fall by 25%. All are scoped to the
98         affected group. */
99      addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM);
100     addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM);
101     addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM);

103     addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE);
104     addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE);

106     game[0].economy.groupCooldownUntilRound[boomGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;
107     game[0].economy.groupCooldownUntilRound[declineGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;

109     game[0].economy.lastBoomGroup = boomGroup;
110     game[0].economy.lastDeclineGroup = declineGroup;
111 }
```

Line-by-line explanation

Line	Code	Explanation
67		Blank line.
68	<code>/* Rule-LK 30-34: every 10 rounds, one group booms and a different</code>	Comment: <code>/* Rule-LK 30-34: every 10 rounds, one group booms and a different</code>
69	<code>one declines, each for 10 rounds. All effects are temporary</code>	<code>one declines, each for 10 rounds. All effects are temporary</code>
70	<code>modifiers (Rule-LK 34) so no base value or cost is ever changed. */</code>	<code>modifiers (Rule-LK 34) so no base value or cost is ever changed. */</code>
71	<code>void reviewPropertyMarket(GameState game[], int currentRound)</code>	<code>void reviewPropertyMarket(GameState game[], int currentRound)</code>
72	<code>{</code>	Opens a new block.
73	<code>PropertyGroup boomGroup;</code>	<code>PropertyGroup boomGroup;</code>
74	<code>PropertyGroup declineGroup;</code>	<code>PropertyGroup declineGroup;</code>
75	<code>char boomName[12];</code>	Declares <code>char boomName[12]</code> .
76	<code>char declineName[12];</code>	Declares <code>char declineName[12]</code> .
77		Blank line.
78	<code>boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);</code>	<code>boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);</code>
79	<code>declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup, boomGroup);</code>	<code>declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup, boomGroup);</code>
80		Blank line.
81	<code>groupName(boomGroup, boomName);</code>	<code>groupName(boomGroup, boomName)</code> . (Writes the display name of a colour group)
82	<code>groupName(declineGroup, declineName);</code>	<code>groupName(declineGroup, declineName)</code> . (Writes the display name of a colour group)
83		Blank line.
84	<code>printf("\n=== Property Market Review ===\n");</code>	Prints the message <code>"\n=== Property Market Review ===\n"</code> .
85	<code>printf("Market Boom : %s (values +20%%, rent +25%%) for 10 rounds.\n", boomName);</code>	Prints the message <code>"Market Boom : %s (values +20%%, rent +25%%) for 10 rounds.\n"</code> .
86	<code>printf("Market Decline : %s (values -15%%, rent -20%%) for 10 rounds.\n", declineName);</code>	Prints the message <code>"Market Decline : %s (values -15%%, rent -20%%) for 10 rounds."</code> .
87		Blank line.
88	<code>addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM);</code>	<code>addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
89	<code>addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM);</code>	<code>addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
90		Blank line.
91	<code>addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE);</code>	<code>addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
92	<code>addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE);</code>	<code>addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
93		Blank line.
94	<code>/* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,</code>	Comment: <code>/* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,</code>
95	<code>direct purchase prices rise by 15% and construction costs rise</code>	<code>direct purchase prices rise by 15% and construction costs rise</code>
96	<code>by 10%; the decline group's mortgage values fall by 10% and</code>	<code>by 10%; the decline group's mortgage values fall by 10% and</code>
97	<code>auction starting prices fall by 25%. All are scoped to the</code>	<code>auction starting prices fall by 25%. All are scoped to the</code>
98	<code>affected group. */</code>	<code>affected group. */</code>

99	<code>addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM);</code>	Calls <code>addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
100	<code>addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM);</code>	Calls <code>addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
101	<code>addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM);</code>	Calls <code>addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
102		Blank line.
103	<code>addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE);</code>	Calls <code>addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
104	<code>addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE);</code>	Calls <code>addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
105		Blank line.
106	<code>game[0].economy.groupCooldownUntilRound[boomGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;</code>	Sets the field <code>groupCooldownUntilRound</code> of the shared economy (i.e. Per group: earliest round it may boom/decline again.) = <code>currentRound + MARKET_COOLDOWN_ROUNDS</code> .
107	<code>game[0].economy.groupCooldownUntilRound[declineGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;</code>	Sets the field <code>groupCooldownUntilRound</code> of the shared economy (i.e. Per group: earliest round it may boom/decline again.) = <code>currentRound + MARKET_COOLDOWN_ROUNDS</code> .
108		Blank line.
109	<code>game[0].economy.lastBoomGroup = boomGroup;</code>	Sets the field <code>lastBoomGroup</code> of the shared economy (i.e. Group boosted by the most recent market review.) = <code>boomGroup</code> .
110	<code>game[0].economy.lastDeclineGroup = declineGroup;</code>	Sets the field <code>lastDeclineGroup</code> of the shared economy (i.e. Group declined by the most recent market review.) = <code>declineGroup</code> .
111	<code>}</code>	Closes the current block.

Nested functions inside `reviewPropertyMarket()` - study these

- `reviewPropertyMarket()` (Rule-LK 30-34: one group booms (+20% value, +25% rent, +15% mortgage/purchase, +10% constr)
- `pickEligibleGroup()` (Picks a random group not on cooldown and not the groups to avoid.)
- `groupName()` (Writes the display name of a colour group.)
- `addSourcedModifier()` (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting.)

MAIN PURPOSE

Rule-LK 30-34: one group booms (+20% value, +25% rent, +15% mortgage/purchase, +10% construction) and a different one declines (-15% value, -20% rent, -10% mortgage, -25% auction), each for 10 rounds.

12.5 `drawRegionalCard()`

```
void drawRegionalCard(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 112)

```
113  /* Table 4: one card every 15 rounds, active for 15 rounds. */
114  void drawRegionalCard(GameState game[])
115  {
116      int choice;

118      choice = rand() % REGIONAL_CARD_COUNT;

120      printf("\n=== Regional Development Card ===\n");

122      switch(choice)
123      {
124          case 0:
125              printf("Southern Tourism Boom : Galle Fort, Unawatuna and "
126                  "Hikkaduwa rental income +40%\n");
```

```

127         addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL);
128         break;

130     case 1:
131         printf("Port City Expansion : Pettah and Maradana values +25%%.\n");
132         addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL);
133         addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL); /* Colombo Fort Station
134     */
135     break;

136     case 2:
137         printf("IT Industry Growth : Maharagama, Nugegoda and "
138             "Kottawa values +20%%.\n");
139         addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL);
140         break;

142     case 3:
143         printf("Northern Development Programme : Jaffna Town, Nallur "
144             "and Trincomalee values +30%%.\n");
145         addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL);
146         break;

148     case 4:
149         printf("Tea Export Boom : Nuwara Eliya value +35%%.\n");
150         addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL);
151         break;

153     case 5:
154         printf("Airport Expansion : Negombo, Katunayake and "
155             "Ja-Ela rents +30%%.\n");
156         addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL);
157         break;

159     case 6:
160         printf("University City Growth : Peradeniya and "
161             "Kandy City values +20%%.\n");
162         addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL);
163         break;

165     case 7:
166         printf("Beach Pollution : Southern coastal rents -30%%.\n");
167         addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL);
168         break;

170     case 8:
171         printf("Flood Damage : Low-lying coastal properties lose 20%% value.\n");
172         addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL);
173         break;

175     case 9:
176         printf("Transport Strike : Railway revenue reduced by 40%%.\n");
177         addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL);
178         break;

180     case 10:
181         printf("Electricity Tariff Increase : Utility rent +25%%.\n");
182         addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL);
183         break;

185     case 11:
186         printf("Water Shortage : Utility revenue +20%%.\n");
187         addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL);
188         addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL);
189         addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL);
190         break;

192     default:
193         break;
194 }

```

Line-by-line explanation

Line	Code	Explanation
112		Blank line.
113	<code>/* Table 4: one card every 15 rounds, active for 15 rounds. */</code>	Comment: <code>/* Table 4: one card every 15 rounds, active for 15 rounds. */</code>
114	<code>void drawRegionalCard(GameState game[])</code>	<code>void drawRegionalCard(GameState game[])</code>
115	<code>{</code>	Opens a new block.
116	<code>int choice;</code>	Declares int choice.
117		Blank line.
118	<code>choice = rand() % REGIONAL_CARD_COUNT;</code>	<code>choice = rand() % REGIONAL_CARD_COUNT;</code>
119		Blank line.
120	<code>printf("\n=== Regional Development Card ===\n");</code>	Prints the message <code>"\n=== Regional Development Card ===\n"</code> .
121		Blank line.
122	<code>switch(choice)</code>	Chooses a branch based on the value of choice.
123	<code>{</code>	Opens a new block.
124	<code>case 0:</code>	case 0:
125	<code>printf("Southern Tourism Boom : Galle Fort, Unawatuna and "</code>	Prints the message <code>"Southern Tourism Boom : Galle Fort, Unawatuna and "</code> .
126	<code>"Hikkaduwa rental income +40%%.\n");</code>	<code>"Hikkaduwa rental income +40%%.\n");</code>
127	<code>addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL);</code>	<code>addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
128	<code>break;</code>	Breaks out of the current loop or switch.
129		Blank line.
130	<code>case 1:</code>	case 1:
131	<code>printf("Port City Expansion : Pettah and Maradana values +25%%.\n");</code>	Prints the message <code>"Port City Expansion : Pettah and Maradana values +25%%.\n"</code> .
132	<code>addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL);</code>	<code>addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
133	<code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL); /* Colombo Fort Station */</code>	<code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL); /* Colombo Fort Station */</code>
134	<code>break;</code>	Breaks out of the current loop or switch.
135		Blank line.
136	<code>case 2:</code>	case 2:
137	<code>printf("IT Industry Growth : Maharagama, Nugegoda and "</code>	Prints the message <code>"IT Industry Growth : Maharagama, Nugegoda and "</code> .
138	<code>"Kottawa values +20%%.\n");</code>	<code>"Kottawa values +20%%.\n");</code>
139	<code>addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL);</code>	<code>addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
140	<code>break;</code>	Breaks out of the current loop or switch.
141		Blank line.
142	<code>case 3:</code>	case 3:
143	<code>printf("Northern Development Programme : Jaffna Town, Nallur "</code>	Prints the message <code>"Northern Development Programme : Jaffna Town, Nallur "</code> .
144	<code>"and Trincomalee values +30%%.\n");</code>	<code>"and Trincomalee values +30%%.\n");</code>
145	<code>addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL);</code>	<code>addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
146	<code>break;</code>	Breaks out of the current loop or switch.
147		Blank line.
148	<code>case 4:</code>	case 4:
149	<code>printf("Tea Export Boom : Nuwara Eliya value +35%%.\n");</code>	Prints the message <code>"Tea Export Boom : Nuwara Eliya value +35%%.\n"</code> .

150	<code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
151	<code>break;</code>	Breaks out of the current loop or switch.
152		Blank line.
153	<code>case 5:</code>	<code>case 5:</code>
154	<code>printf("Airport Expansion : Negombo, Katunayake and "</code>	Prints the message "Airport Expansion : Negombo, Katunayake and ".
155	<code>"Ja-Ela rents +30%%.\n");</code>	<code>"Ja-Ela rents +30%%.\n");</code>
156	<code>addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
157	<code>break;</code>	Breaks out of the current loop or switch.
158		Blank line.
159	<code>case 6:</code>	<code>case 6:</code>
160	<code>printf("University City Growth : Peradeniya and "</code>	Prints the message "University City Growth : Peradeniya and ".
161	<code>"Kandy City values +20%%.\n");</code>	<code>"Kandy City values +20%%.\n");</code>
162	<code>addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
163	<code>break;</code>	Breaks out of the current loop or switch.
164		Blank line.
165	<code>case 7:</code>	<code>case 7:</code>
166	<code>printf("Beach Pollution : Southern coastal rents -30%%.\n");</code>	Prints the message "Beach Pollution : Southern coastal rents -30%%.\n".
167	<code>addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
168	<code>break;</code>	Breaks out of the current loop or switch.
169		Blank line.
170	<code>case 8:</code>	<code>case 8:</code>
171	<code>printf("Flood Damage : Low-lying coastal properties lose 20%% value.\n");</code>	Prints the message "Flood Damage : Low-lying coastal properties lose 20%% value.".
172	<code>addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
173	<code>break;</code>	Breaks out of the current loop or switch.
174		Blank line.
175	<code>case 9:</code>	<code>case 9:</code>
176	<code>printf("Transport Strike : Railway revenue reduced by 40%%.\n");</code>	Prints the message "Transport Strike : Railway revenue reduced by 40%%.\n".
177	<code>addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
178	<code>break;</code>	Breaks out of the current loop or switch.
179		Blank line.
180	<code>case 10:</code>	<code>case 10:</code>
181	<code>printf("Electricity Tariff Increase : Utility rent +25%%.\n");</code>	Prints the message "Electricity Tariff Increase : Utility rent +25%%.\n".
182	<code>addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
183	<code>break;</code>	Breaks out of the current loop or switch.
184		Blank line.
185	<code>case 11:</code>	<code>case 11:</code>
186	<code>printf("Water Shortage : Utility revenue +20%%.\n");</code>	Prints the message "Water Shortage : Utility revenue +20%%.\n".

187	<code>addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
188	<code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
189	<code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL);</code>	Calls <code>addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL)</code> . (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting)
190	<code>break;</code>	Breaks out of the current loop or switch.
191		Blank line.
192	<code>default:</code>	The fallback switch branch when no case matched.
193	<code>break;</code>	Breaks out of the current loop or switch.
194	<code>}</code>	Closes the current block.
195	<code>}</code>	Closes the current block.

Nested functions inside `drawRegionalCard()` - study these

- `drawRegionalCard()` (Table 4: one of 12 regional development cards every 15 rounds, active for 15, all as `SRC_R`)
- `addSourcedModifier()` (Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting.)

MAIN PURPOSE

Table 4: one of 12 regional development cards every 15 rounds, active for 15, all as `SRC_REGIONAL` modifiers.

12.6 `printSectionHeader()`

```
void printSectionHeader(char title[])
```

Parameters: `title` | Returns: `void`.

The function, verbatim from the source (starts at line 196)

```
197  /* Prints a section title followed by a separator line. */
198  void printSectionHeader(char title[])
199  {
200      printf("%s\n", title);
201      printf("-----\n");
202  }
```

Line-by-line explanation

Line	Code	Explanation
196		Blank line.
197	<code>/* Prints a section title followed by a separator line. */</code>	Comment: <code>/* Prints a section title followed by a separator line. */</code>
198	<code>void printSectionHeader(char title[])</code>	<code>void printSectionHeader(char title[])</code>
199	<code>{</code>	Opens a new block.
200	<code>printf("%s\n", title);</code>	Prints the message <code>"%s\n"</code> .
201	<code>printf("-----\n");</code>	Prints the message <code>"-----\n"</code> .
202	<code>}</code>	Closes the current block.

Nested functions inside `printSectionHeader()` - study these

- `printSectionHeader()` (Prints a report section title and separator.)

MAIN PURPOSE

Prints a report section title and separator.

12.7 `printBoomDecline()`

```
void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
```

Parameters: `title`, `groupNameBuffer`, `percent`, `roundsLeft` | Returns: `void`.

The function, verbatim from the source (starts at line 203)

```
204  /* Prints a Market Boom or Market Decline block. */
205  void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
206  {
207      printSectionHeader(title);
208      printf("%s (%+d%)\n", groupNameBuffer, percent - 100);
209      printf("Rounds Remaining : %d\n", roundsLeft);
210  }
```

Line-by-line explanation

Line	Code	Explanation
203		Blank line.
204	/* Prints a Market Boom or Market Decline block. */	Comment: /* Prints a Market Boom or Market Decline block. */
205	void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)	void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
206	{	Opens a new block.
207	printSectionHeader(title);	Calls printSectionHeader(title). (Prints a report section title and separator)
208	printf("%s (%+d%)\n", groupNameBuffer, percent - 100);	Prints the message "%s (%+d%)\n".
209	printf("Rounds Remaining : %d\n", roundsLeft);	Prints the message "Rounds Remaining : %d\n".
210	}	Closes the current block.

Nested functions inside printBoomDecline() - study these

- printBoomDecline() (Prints one Market Boom / Market Decline block.)
- printSectionHeader() (Prints a report section title and separator.)

MAIN PURPOSE

Prints one Market Boom / Market Decline block.

12.8 describeRegionalCondition()

```
int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
```

Parameters: mod, out | Returns: 1 = wrote a line, 0 = not a printable type.

The function, verbatim from the source (starts at line 211)

```
212  /* Writes one Regional Development card line into out. Returns 1 if
213  the modifier has a line to show, 0 otherwise. */
214  int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
215  {
216      char name[48];
217
218      if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
219      {
220          if(mod.group >= 0 && mod.group < NO_GROUP)
221              groupName((PropertyGroup)mod.group, name);
222          else
223              strcpy(name, "Unknown");
224
225          if(mod.type == MOD_GROUP_VALUE)
226              sprintf(out, "%s (%+d%)", name, mod.percent - 100);
227          else
228              sprintf(out, "%s rents (%+d%)", name, mod.percent - 100);
229      }
230      else if(mod.type == MOD_INDEX_VALUE)
231      {
232          sprintf(out, "%s (%+d%)",
233                  game[0].board[mod.index].name, mod.percent - 100);
234      }
235      else if(mod.type == MOD_RAIL_RENT)
236      {
237          sprintf(out, "Railway rents (%+d%)", mod.percent - 100);
238      }
```

```

239     else if(mod.type == MOD_UTIL_RENT)
240     {
241         sprintf(out, "Utility rents (%+d%%)", mod.percent - 100);
242     }
243     else
244     {
245         return 0;
246     }

248     return 1;
249 }

```

Line-by-line explanation

Line	Code	Explanation
211		Blank line.
212	/* Writes one Regional Development card line into out. Returns 1 if	Comment: /* Writes one Regional Development card line into out. Returns 1 if
213	the modifier has a line to show, 0 otherwise. */	the modifier has a line to show, 0 otherwise. */
214	int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])	int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
215	{	Opens a new block.
216	char name[48];	Declares char name[48].
217		Blank line.
218	if(mod.type == MOD_GROUP_VALUE mod.type == MOD_GROUP_RENT)	Condition: mod.type == MOD_GROUP_VALUE mod.type == MOD_GROUP_RENT -> run the next block if true.
219	{	Opens a new block.
220	if(mod.group >= 0 && mod.group < NO_GROUP)	Condition: mod.group >= 0 && mod.group < NO_GROUP -> run the next block if true.
221	groupName((PropertyGroup)mod.group, name);	Calls groupName((PropertyGroup)mod.group, name). (Writes the display name of a colour group)
222	else	The alternative branch when the condition above is false.
223	strcpy(name, "Unknown");	Calls strcpy(name, "Unknown").
224		Blank line.
225	if(mod.type == MOD_GROUP_VALUE)	Condition: mod.type == MOD_GROUP_VALUE -> run the next block if true.
226	sprintf(out, "%s (%+d%%)", name, mod.percent - 100);	Calls sprintf(out, "%s (%+d%%)", name, mod.percent - 100).
227	else	The alternative branch when the condition above is false.
228	sprintf(out, "%s rents (%+d%%)", name, mod.percent - 100);	Calls sprintf(out, "%s rents (%+d%%)", name, mod.percent - 100).
229	}	Closes the current block.
230	else if(mod.type == MOD_INDEX_VALUE)	A further condition checked only if the previous one failed.
231	{	Opens a new block.
232	sprintf(out, "%s (%+d%%)",	sprintf(out, "%s (%+d%%)",
233	game[0].board[mod.index].name, mod.percent - 100);	game[0].board[mod.index].name, mod.percent - 100);
234	}	Closes the current block.
235	else if(mod.type == MOD_RAIL_RENT)	A further condition checked only if the previous one failed.
236	{	Opens a new block.
237	sprintf(out, "Railway rents (%+d%%)", mod.percent - 100);	Calls sprintf(out, "Railway rents (%+d%%)", mod.percent - 100).
238	}	Closes the current block.
239	else if(mod.type == MOD_UTIL_RENT)	A further condition checked only if the previous one failed.
240	{	Opens a new block.
241	sprintf(out, "Utility rents (%+d%%)", mod.percent - 100);	Calls sprintf(out, "Utility rents (%+d%%)", mod.percent - 100).
242	}	Closes the current block.
243	else	The alternative branch when the condition above is false.
244	{	Opens a new block.
245	return 0;	Returns 0 immediately.
246	}	Closes the current block.

247		Blank line.
248	return 1;	Returns 1 immediately.
249	}	Closes the current block.

Nested functions inside describeRegionalCondition() - study these

- describeRegionalCondition() (Writes one regional card effect line; returns 1 if a line was produced.)
- groupName() (Writes the display name of a colour group.)

MAIN PURPOSE

Writes one regional card effect line; returns 1 if a line was produced.

12.9 printRegionalSection()

```
int printRegionalSection(GameState game[])
```

Parameters: none | Returns: 1 if anything was printed.

The function, verbatim from the source (starts at line 250)

```

251  /* Prints the "Regional Development" section. Returns 1 if at least
252     one card effect was shown. */
253  int printRegionalSection(GameState game[])
254  {
255      int i;
256      int shown;

258      shown = 0;

260      for(i = 0; i < game[0].economy.modifierCount; i++)
261      {
262          ActiveModifier mod;
263          char line[100];

265          mod = game[0].economy.modifiers[i];

267          if(mod.source != SRC_REGIONAL)
268              continue;

270          if(!describeRegionalCondition(game, mod, line))
271              continue;

273          if(!shown)
274          {
275              printSectionHeader("Regional Development");
276              shown = 1;
277          }

279          printf("%s\n", line);
280          printf("Rounds Remaining : %d\n", mod.roundsLeft);
281      }

283      return shown;
284  }
```

Line-by-line explanation

Line	Code	Explanation
250		Blank line.
251	/* Prints the "Regional Development" section. Returns 1 if at least	Comment: /* Prints the "Regional Development" section. Returns 1 if at least
252	one card effect was shown. */	one card effect was shown. */
253	int printRegionalSection(GameState game[])	int printRegionalSection(GameState game[])
254	{	Opens a new block.
255	int i;	Declares int i.
256	int shown;	Declares int shown.
257		Blank line.

258	shown = 0;	shown = 0;
259		Blank line.
260	for(i = 0; i < game[0].economy.modifierCount; i++)	Loop over all entries: init i = 0; run while i < game[0].economy.modifierCount; every loop i++.
261	{	Opens a new block.
262	ActiveModifier mod;	ActiveModifier mod;
263	char line[100];	Declares char line[100].
264		Blank line.
265	mod = game[0].economy.modifiers[i];	mod = game[0].economy.modifiers[i];
266		Blank line.
267	if(mod.source != SRC_REGIONAL)	Condition: mod.source != SRC_REGIONAL -> run the next block if true.
268	continue;	Skips to the next loop iteration.
269		Blank line.
270	if(!describeRegionalCondition(game, mod, line))	Condition: !describeRegionalCondition(game, mod, line) -> run the next block if true.
271	continue;	Skips to the next loop iteration.
272		Blank line.
273	if(!shown)	Condition: !shown -> run the next block if true.
274	{	Opens a new block.
275	printSectionHeader("Regional Development");	Calls printSectionHeader("Regional Development"). (Prints a report section title and separator)
276	shown = 1;	shown = 1;
277	}	Closes the current block.
278		Blank line.
279	printf("%s\n", line);	Prints the message "%s\n".
280	printf("Rounds Remaining : %d\n", mod.roundsLeft);	Prints the message "Rounds Remaining : %d\n".
281	}	Closes the current block.
282		Blank line.
283	return shown;	Returns shown immediately.
284	}	Closes the current block.

Nested functions inside printRegionalSection() - study these

- printRegionalSection() (Prints all active Regional Development effects under one header.)
- describeRegionalCondition() (Writes one regional card effect line; returns 1 if a line was produced.)
- printSectionHeader() (Prints a report section title and separator.)

MAIN PURPOSE

Prints all active Regional Development effects under one header.

12.10 describeOtherCondition()

```
int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
```

Parameters: mod, out | Returns: 1 = wrote a line, 0 = not printable.

The function, verbatim from the source (starts at line 285)

```
286  /* Writes one "Other Active Conditions" line into out. Returns 1 if
287     the modifier has a line to show, 0 otherwise. */
288  int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
289  {
290      char name[32];

292      if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
293      {
294          groupName((PropertyGroup)mod.group, name);

296          if(mod.type == MOD_GROUP_VALUE)
297              sprintf(out, "%s : values %d%% (%d rounds remaining)",
298                     name, mod.percent, mod.roundsLeft);
299          else
300              sprintf(out, "%s : rents %d%% (%d rounds remaining)",
```

```

301         name, mod.percent, mod.roundsLeft);
302     }
303     else if(mod.type == MOD_INDEX_VALUE)
304     {
305         sprintf(out, "%s : value %d%% (%d rounds remaining)",
306             game[0].board[mod.index].name, mod.percent, mod.roundsLeft);
307     }
308     else if(mod.type == MOD_RAIL_VALUE)
309     {
310         sprintf(out, "Railway values : %d%% (%d rounds remaining)",
311             mod.percent, mod.roundsLeft);
312     }
313     else if(mod.type == MOD_RAIL_RENT)
314     {
315         sprintf(out, "Railway rents : %d%% (%d rounds remaining)",
316             mod.percent, mod.roundsLeft);
317     }
318     else if(mod.type == MOD_UTIL_RENT)
319     {
320         sprintf(out, "Utility rents : %d%% (%d rounds remaining)",
321             mod.percent, mod.roundsLeft);
322     }
323     else
324     {
325         return 0;
326     }
327
328     return 1;
329 }

```

Line-by-line explanation

Line	Code	Explanation
285		Blank line.
286	/* Writes one "Other Active Conditions" line into out. Returns 1 if	Comment: /* Writes one "Other Active Conditions" line into out. Returns 1 if
287	the modifier has a line to show, 0 otherwise. */	the modifier has a line to show, 0 otherwise. */
288	int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])	int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
289	{	Opens a new block.
290	char name[32];	Declares char name[32].
291		Blank line.
292	if(mod.type == MOD_GROUP_VALUE mod.type == MOD_GROUP_RENT)	Condition: mod.type == MOD_GROUP_VALUE mod.type == MOD_GROUP_RENT -> run the next block if true.
293	{	Opens a new block.
294	groupName((PropertyGroup)mod.group, name);	Calls groupName((PropertyGroup)mod.group, name). (Writes the display name of a colour group)
295		Blank line.
296	if(mod.type == MOD_GROUP_VALUE)	Condition: mod.type == MOD_GROUP_VALUE -> run the next block if true.
297	sprintf(out, "%s : values %d%% (%d rounds remaining)",	sprintf(out, "%s : values %d%% (%d rounds remaining)",
298	name, mod.percent, mod.roundsLeft);	name, mod.percent, mod.roundsLeft);
299	else	The alternative branch when the condition above is false.
300	sprintf(out, "%s : rents %d%% (%d rounds remaining)",	sprintf(out, "%s : rents %d%% (%d rounds remaining)",
301	name, mod.percent, mod.roundsLeft);	name, mod.percent, mod.roundsLeft);
302	}	Closes the current block.
303	else if(mod.type == MOD_INDEX_VALUE)	A further condition checked only if the previous one failed.
304	{	Opens a new block.
305	sprintf(out, "%s : value %d%% (%d rounds remaining)",	sprintf(out, "%s : value %d%% (%d rounds remaining)",

306	game[0].board[mod.index].name, mod.percent, mod.roundsLeft);	game[0].board[mod.index].name, mod.percent, mod.roundsLeft);
307	}	Closes the current block.
308	else if(mod.type == MOD_RAIL_VALUE)	A further condition checked only if the previous one failed.
309	{	Opens a new block.
310	sprintf(out, "Railway values : x%d%% (%d rounds remaining)",	sprintf(out, "Railway values : x%d%% (%d rounds remaining)",
311	mod.percent, mod.roundsLeft);	mod.percent, mod.roundsLeft);
312	}	Closes the current block.
313	else if(mod.type == MOD_RAIL_RENT)	A further condition checked only if the previous one failed.
314	{	Opens a new block.
315	sprintf(out, "Railway rents : x%d%% (%d rounds remaining)",	sprintf(out, "Railway rents : x%d%% (%d rounds remaining)",
316	mod.percent, mod.roundsLeft);	mod.percent, mod.roundsLeft);
317	}	Closes the current block.
318	else if(mod.type == MOD_UTIL_RENT)	A further condition checked only if the previous one failed.
319	{	Opens a new block.
320	sprintf(out, "Utility rents : x%d%% (%d rounds remaining)",	sprintf(out, "Utility rents : x%d%% (%d rounds remaining)",
321	mod.percent, mod.roundsLeft);	mod.percent, mod.roundsLeft);
322	}	Closes the current block.
323	else	The alternative branch when the condition above is false.
324	{	Opens a new block.
325	return 0;	Returns 0 immediately.
326	}	Closes the current block.
327		Blank line.
328	return 1;	Returns 1 immediately.
329	}	Closes the current block.

Nested functions inside describeOtherCondition() - study these

- describeOtherCondition() (Writes one 'Other Active Conditions' line; returns 1 if written.)
- groupName() (Writes the display name of a colour group.)

MAIN PURPOSE

Writes one 'Other Active Conditions' line; returns 1 if written.

12.11 printOtherConditions()

```
void printOtherConditions(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 330)

```

331  /* Prints the "Other Active Conditions" section (events and
332     government regulations still running). */
333  void printOtherConditions(GameState game[])
334  {
335      int i;
336      int shown;

338      shown = 0;

340      for(i = 0; i < game[0].economy.modifierCount; i++)
341      {
342          ActiveModifier mod;
343          char line[100];

345          mod = game[0].economy.modifiers[i];

347          if(mod.source != SRC_GENERAL)
348              continue;

```

```

350         if(!describeOtherCondition(game, mod, line))
351             continue;

353         if(!shown)
354         {
355             printSectionHeader("Other Active Conditions");
356             shown = 1;
357         }

359         printf("%s\n", line);
360     }
361 }

```

Line-by-line explanation

Line	Code	Explanation
330		Blank line.
331	/* Prints the "Other Active Conditions" section (events and	Comment: /* Prints the "Other Active Conditions" section (events and
332	government regulations still running). */	government regulations still running). */
333	void printOtherConditions(GameState game[])	void printOtherConditions(GameState game[])
334	{	Opens a new block.
335	int i;	Declares int i.
336	int shown;	Declares int shown.
337		Blank line.
338	shown = 0;	shown = 0;
339		Blank line.
340	for(i = 0; i < game[0].economy.modifierCount; i++)	Loop over all entries: init i = 0; run while i < game[0].economy.modifierCount; every loop i++.
341	{	Opens a new block.
342	ActiveModifier mod;	ActiveModifier mod;
343	char line[100];	Declares char line[100].
344		Blank line.
345	mod = game[0].economy.modifiers[i];	mod = game[0].economy.modifiers[i];
346		Blank line.
347	if(mod.source != SRC_GENERAL)	Condition: mod.source != SRC_GENERAL -> run the next block if true.
348	continue;	Skips to the next loop iteration.
349		Blank line.
350	if(!describeOtherCondition(game, mod, line))	Condition: !describeOtherCondition(game, mod, line) -> run the next block if true.
351	continue;	Skips to the next loop iteration.
352		Blank line.
353	if(!shown)	Condition: !shown -> run the next block if true.
354	{	Opens a new block.
355	printSectionHeader("Other Active Conditions");	Calls printSectionHeader("Other Active Conditions"). (Prints a report section title and separator)
356	shown = 1;	shown = 1;
357	}	Closes the current block.
358		Blank line.
359	printf("%s\n", line);	Prints the message "%s\n".
360	}	Closes the current block.
361	}	Closes the current block.

Nested functions inside printOtherConditions() - study these

- printOtherConditions() (Prints still-running event/regulation (SRC_GENERAL) modifiers.)
- describeOtherCondition() (Writes one 'Other Active Conditions' line; returns 1 if written.)
- printSectionHeader() (Prints a report section title and separator.)

MAIN PURPOSE

Prints still-running event/regulation (SRC_GENERAL) modifiers.

12.12 displayMarketConditions()

```
void displayMarketConditions(GameState game[])
```

Parameters: none | Returns: void.

The function, verbatim from the source (starts at line 362)

```
363  /* Rule-LK 36: shows the currently active market conditions at the
364      end of every round, in the Section 5 format. */
365  void displayMarketConditions(GameState game[])
366  {
367      int i;
368      int shown;

370      printf("\n=====\\n");
371      printf("Current Market Conditions\\n");
372      printf("=====\\n");

374      shown = 0;

376      /* Market Boom (Rule-LK 31) */
377      for(i = 0; i < game[0].economy.modifierCount; i++)
378      {
379          char name[32];

381          if(game[0].economy.modifiers[i].source != SRC_BOOM ||
382             game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
383              continue;

385          groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
386          printBoomDecline("Market Boom", name,
387                          game[0].economy.modifiers[i].percent,
388                          game[0].economy.modifiers[i].roundsLeft);
389          shown = 1;
390          break;
391      }

393      /* Market Decline (Rule-LK 32) */
394      for(i = 0; i < game[0].economy.modifierCount; i++)
395      {
396          char name[32];

398          if(game[0].economy.modifiers[i].source != SRC_DECLINE ||
399             game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
400              continue;

402          groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
403          printBoomDecline("Market Decline", name,
404                          game[0].economy.modifiers[i].percent,
405                          game[0].economy.modifiers[i].roundsLeft);
406          shown = 1;
407          break;
408      }

410      /* Regional Development Card effects (Table 4) */
411      if(printRegionalSection(game))
412          shown = 1;

414      if(!shown)
415          printf("No active market booms, declines or regional conditions right now.\\n");

417      /* Other active event/regulation effects */
418      printOtherConditions(game);
```

```

420     printSectionHeader("Inflation");
421     printf("%d%%\n", game[0].economy.inflationRate);

423     printSectionHeader("Current Loan Interest");
424     printf("%d%%\n", game[0].economy.loanInterestRate);

426     printSectionHeader("Current Income Tax Rate");
427     printf("%d%%\n", game[0].economy.incomeTaxRate);

429     printf("=====\n");
430 }

```

Line-by-line explanation

Line	Code	Explanation
362		Blank line.
363	/* Rule-LK 36: shows the currently active market conditions at the	Comment: /* Rule-LK 36: shows the currently active market conditions at the
364	end of every round, in the Section 5 format. */	end of every round, in the Section 5 format. */
365	void displayMarketConditions(GameState game[])	void displayMarketConditions(GameState game[])
366	{	Opens a new block.
367	int i;	Declares int i.
368	int shown;	Declares int shown.
369		Blank line.
370	printf("\n=====\n");	Prints the message "\n=====\n".
371	printf("Current Market Conditions\n");	Prints the message "Current Market Conditions\n".
372	printf("=====\n");	Prints the message "=====\n".
373		Blank line.
374	shown = 0;	shown = 0;
375		Blank line.
376	/* Market Boom (Rule-LK 31) */	Comment: /* Market Boom (Rule-LK 31) */
377	for(i = 0; i < game[0].economy.modifierCount; i++)	Loop over all entries: init i = 0; run while i < game[0].economy.modifierCount; every loop i++.
378	{	Opens a new block.
379	char name[32];	Declares char name[32].
380		Blank line.
381	if(game[0].economy.modifiers[i].source != SRC_BOOM	Condition: -> run the next block if true.
382	game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)	game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
383	continue;	Skips to the next loop iteration.
384		Blank line.
385	groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);	Calls groupName((PropertyGroup)game[0].economy.modifiers[i].group, name). (Writes the display name of a colour group)
386	printBoomDecline("Market Boom", name,	printBoomDecline("Market Boom", name,
387	game[0].economy.modifiers[i].percent,	game[0].economy.modifiers[i].percent,
388	game[0].economy.modifiers[i].roundsLeft);	game[0].economy.modifiers[i].roundsLeft);
389	shown = 1;	shown = 1;
390	break;	Breaks out of the current loop or switch.
391	}	Closes the current block.
392		Blank line.
393	/* Market Decline (Rule-LK 32) */	Comment: /* Market Decline (Rule-LK 32) */
394	for(i = 0; i < game[0].economy.modifierCount; i++)	Loop over all entries: init i = 0; run while i < game[0].economy.modifierCount; every loop i++.
395	{	Opens a new block.
396	char name[32];	Declares char name[32].
397		Blank line.

398	if(game[0].economy.modifiers[i].source != SRC_DECLINE	Condition: -> run the next block if true.
399	game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)	game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
400	continue;	Skips to the next loop iteration.
401		Blank line.
402	groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);	Calls groupName((PropertyGroup)game[0].economy.modifiers[i].group, name). (Writes the display name of a colour group)
403	printBoomDecline("Market Decline", name,	printBoomDecline("Market Decline", name,
404	game[0].economy.modifiers[i].percent,	game[0].economy.modifiers[i].percent,
405	game[0].economy.modifiers[i].roundsLeft);	game[0].economy.modifiers[i].roundsLeft);
406	shown = 1;	shown = 1;
407	break;	Breaks out of the current loop or switch.
408	}	Closes the current block.
409		Blank line.
410	/* Regional Development Card effects (Table 4) */	Comment: /* Regional Development Card effects (Table 4) */
411	if(printRegionalSection(game))	Condition: printRegionalSection(game) -> run the next block if true.
412	shown = 1;	shown = 1;
413		Blank line.
414	if(!shown)	Condition: !shown -> run the next block if true.
415	printf("No active market booms, declines or regional conditions right now.\n");	Prints the message "No active market booms, declines or regional conditions right now.\n".
416		Blank line.
417	/* Other active event/regulation effects */	Comment: /* Other active event/regulation effects */
418	printOtherConditions(game);	Calls printOtherConditions(game). (Prints still-running event/regulation (SRC_GENERAL) modifiers)
419		Blank line.
420	printSectionHeader("Inflation");	Calls printSectionHeader("Inflation"). (Prints a report section title and separator)
421	printf("%d%%\n", game[0].economy.inflationRate);	Prints the message "%d%%\n".
422		Blank line.
423	printSectionHeader("Current Loan Interest");	Calls printSectionHeader("Current Loan Interest"). (Prints a report section title and separator)
424	printf("%d%%\n", game[0].economy.loanInterestRate);	Prints the message "%d%%\n".
425		Blank line.
426	printSectionHeader("Current Income Tax Rate");	Calls printSectionHeader("Current Income Tax Rate"). (Prints a report section title and separator)
427	printf("%d%%\n", game[0].economy.incomeTaxRate);	Prints the message "%d%%\n".
428		Blank line.
429	printf("=====\n");	Prints the message "=====\n".
430	}	Closes the current block.

Nested functions inside displayMarketConditions() - study these

- displayMarketConditions() (Rule-LK 36: the end-of-round market report - boom, decline, regional development, other co)
- groupName() (Writes the display name of a colour group.)
- printBoomDecline() (Prints one Market Boom / Market Decline block.)
- printRegionalSection() (Prints all active Regional Development effects under one header.)
- printOtherConditions() (Prints still-running event/regulation (SRC_GENERAL) modifiers.)
- printSectionHeader() (Prints a report section title and separator.)

MAIN PURPOSE

Rule-LK 36: the end-of-round market report - boom, decline, regional development, other conditions, inflation, loan interest and tax rate.

Entry point. Creates the single GameState (array-of-one idiom) and starts the whole simulation.

The sub-topics below are the 1 functions implemented in main.c. Each one is shown with its original source line number, the exact code, a line-by-line explanation, the nested functions to study and its main purpose.

Index of sub-topics in main.c

13.1 main() (source line 5) p. 237

13.1 main()

```
int main(void)
```

Parameters: none | Returns: 0 on success.

The function, verbatim from the source (starts at line 5)

```
5 int main(void)
6 {
7     /* the only place a GameState is ever created */
8     GameState game[1] = {0};
9
10    printf("=====\n");
11    printf("    MONOPOLY-LK SIMULATION\n");
12    printf("=====\n\n");
13
14    startGame(game);
15
16    return 0;
17 }
```

Line-by-line explanation

Line	Code	Explanation
5	int main(void)	int main(void)
6	{	Opens a new block.
7	/* the only place a GameState is ever created */	Comment: /* the only place a GameState is ever created */
8	GameState game[1] = {0};	GameState game[1] = {0};
9		Blank line.
10	printf("=====\n");	Prints the message "=====\n".
11	printf(" MONOPOLY-LK SIMULATION\n");	Prints the message " MONOPOLY-LK SIMULATION\n".
12	printf("=====\n\n");	Prints the message "=====\n\n".
13		Blank line.
14	startGame(game);	Calls startGame(game). (Entry point driven by main)
15		Blank line.
16	return 0;	Returns 0 immediately.
17	}	Closes the current block.

Nested functions inside main() - study these

- main() (The only place a GameState is ever created (game[1] = {0}), then startGame runs the whole)
- startGame() (Entry point driven by main.c: seeds RNG, initialises everything, runs the game and prints)

MAIN PURPOSE

The only place a GameState is ever created (game[1] = {0}), then startGame runs the whole simulation.